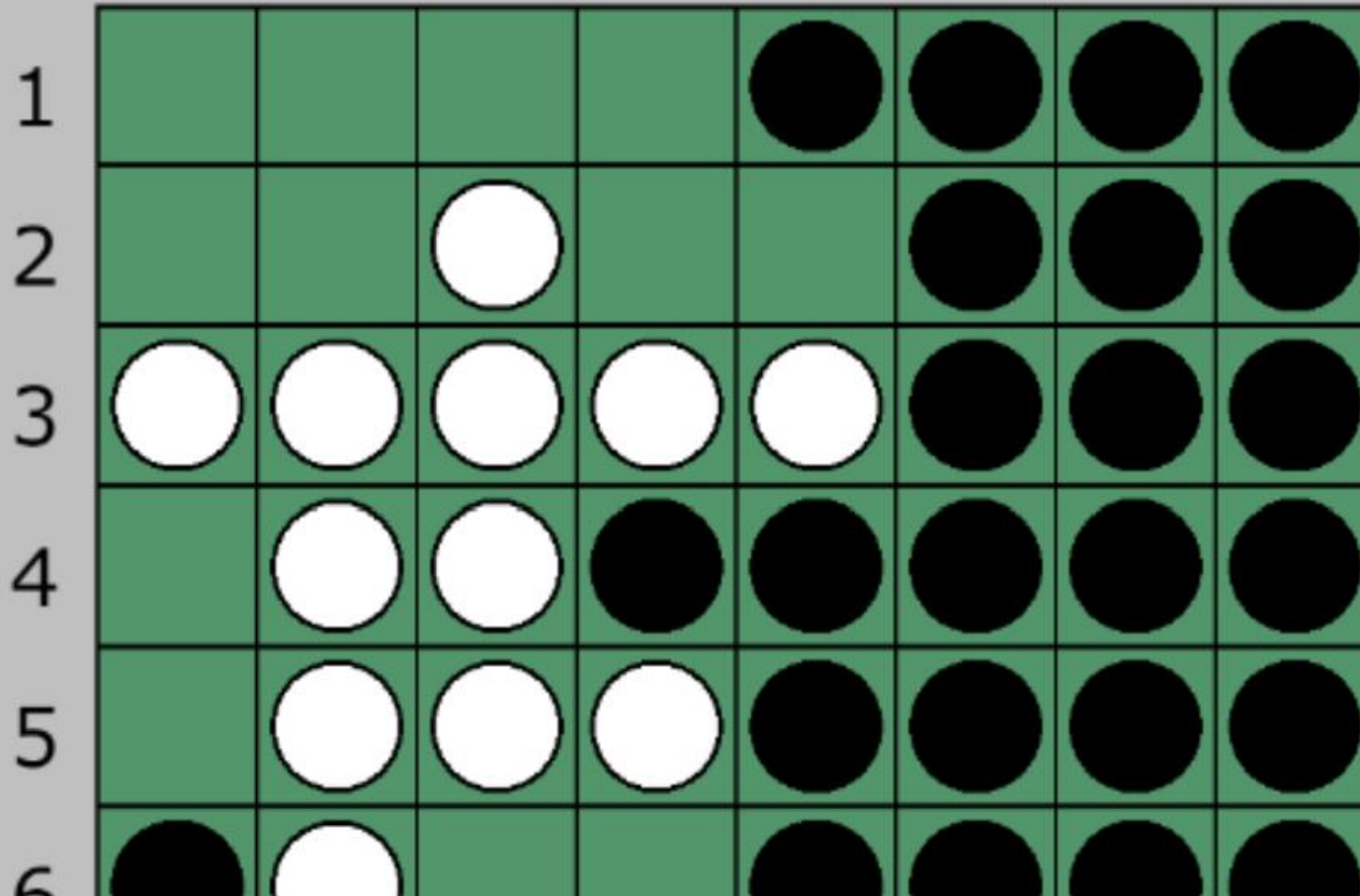


square1001



a8

36
+2



Default



b8

13
-1

問題 1「碁石ならべ 2」解説

Problème 1 "Disposer des Pierres 2"

解説: 米田 寛峻 (よねだ ひろたか) / square1001

1

問題の説明

N 個の碁石を左から順に並べます

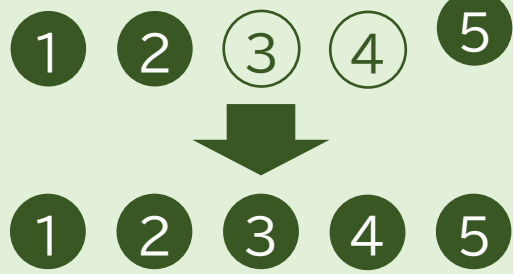
1



i 番目に置く石の色は A_i です

Diagram 1 shows a sequence of stones with colors red, blue, yellow, green, and an ellipsis. The text below states: i 番目に置く石の色は A_i です.

2



New!

同色の 2 つの石に挟まれる
と同じ色に変化します

Diagram 2 shows a sequence of stones with colors red, blue, yellow, green, and a new stone. An arrow points from the new stone to the yellow stone, indicating a color change. The text below states: 同色の 2 つの石に挟まれると同じ色に変化します.

3



最終的な各石の色を求めよ

Diagram 3 shows a sequence of five stones, each with a question mark. The text below states: 最終的な各石の色を求めよ.

1

制約

$$N \leq 2000$$
$$A_i \leq 10^9$$

小課題 1 (25 点)

$$N \leq 200\,000$$
$$A_i = 1, 2$$

小課題 2 (60 点)

$$N \leq 200\,000$$
$$A_i \leq 10^9$$

小課題 3 (100 点)

Chapter II

小課題 1 の解法

Sous-tâche 1

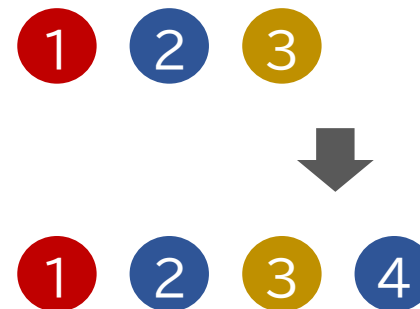
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



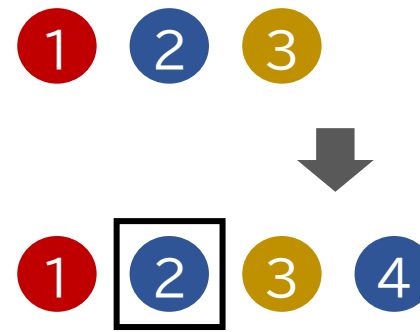
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



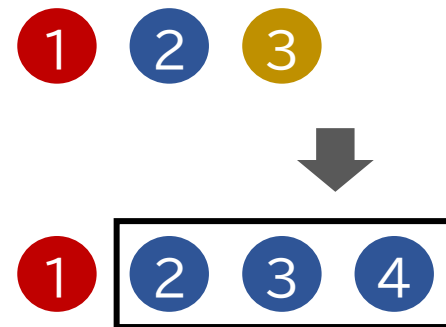
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



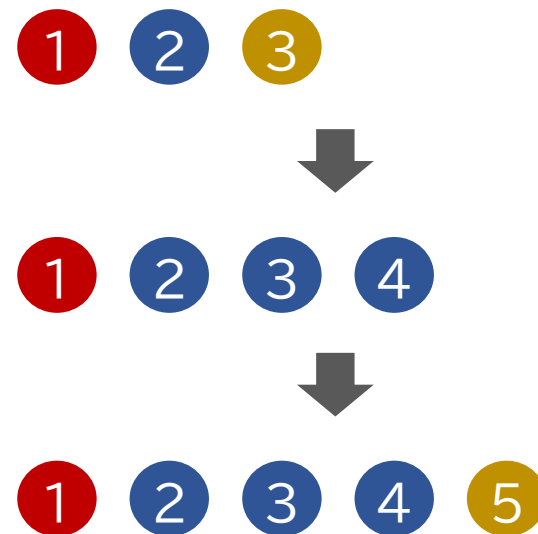
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



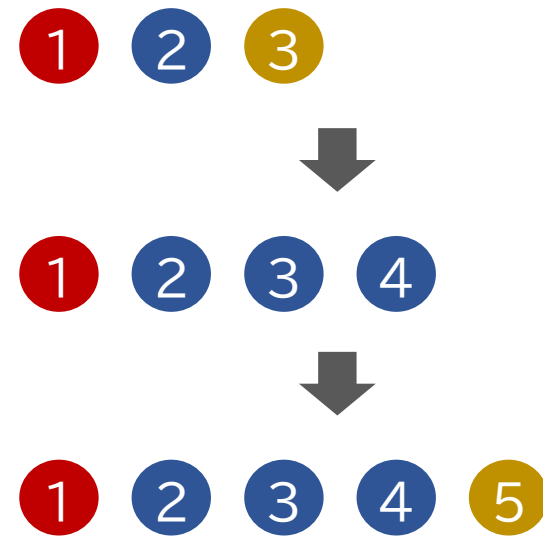
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



黄色の石は存在しない!

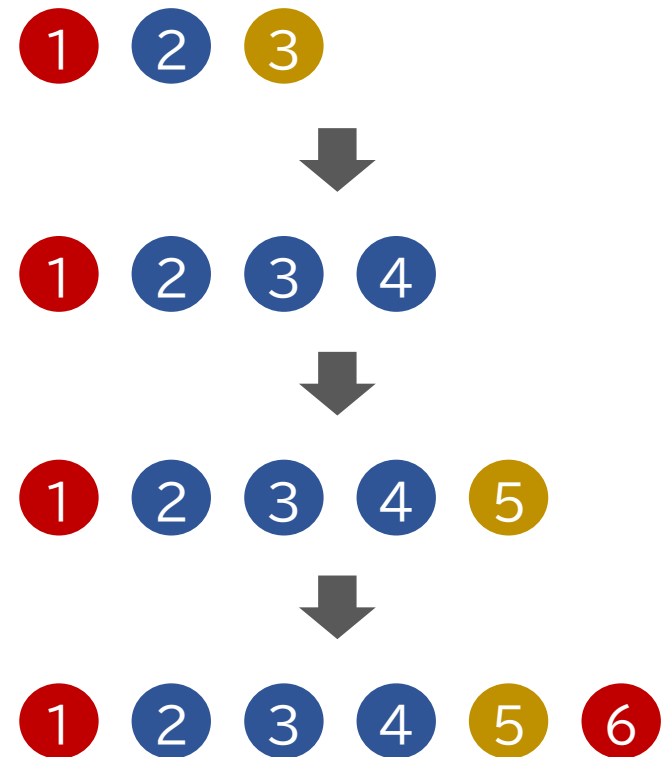
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



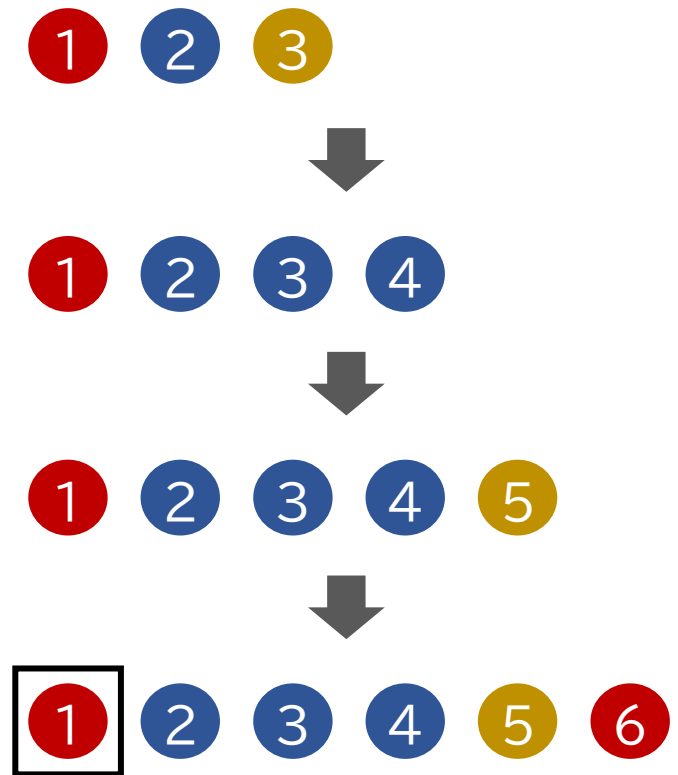
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



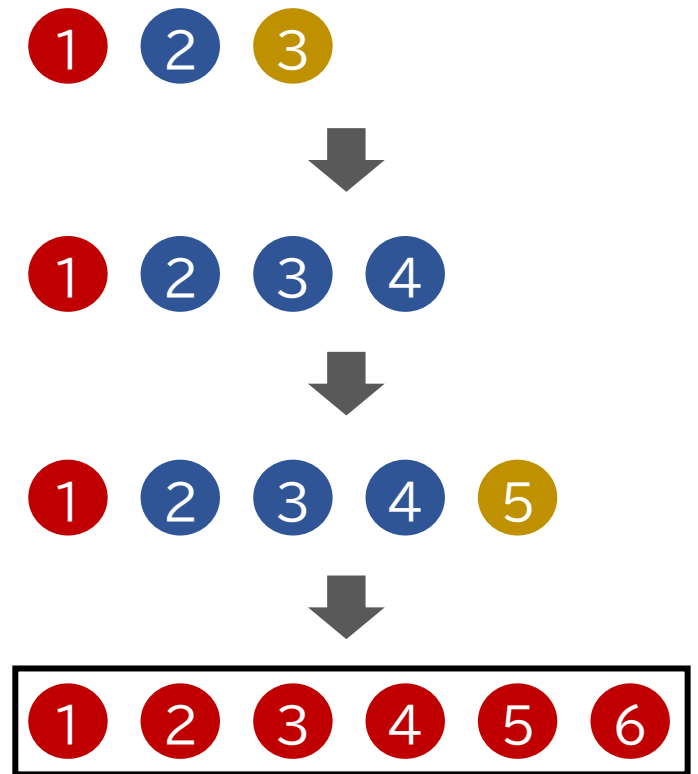
2 小課題 1 の解法

小課題 1 の制約は $N \leq 2000$ です

単純にシミュレーションすることを考えましょう

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える



2 小課題 1 の解法

解答例を右に示します

手順ごとに分けて考えると実装しやすいです

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える

```
int main() {  
    // step #1. read input  
    int N;  
    cin >> N;  
    vector<int> A(N);  
    for (int i = 0; i < N; i++) {  
        cin >> A[i];  
    }  
  
    // step #2. simulation  
    vector<int> stones;  
    for (int i = 0; i < N; i++) {  
        int pos = -1;  
        for (int j = 0; j < i; j++) {  
            if (stones[j] == A[i]) {  
                pos = max(pos, j);  
            }  
        }  
        if (pos != -1) {  
            for (int j = pos + 1; j < i; j++) {  
                stones[j] = A[i];  
            }  
        }  
        stones.push_back(A[i]);  
    }  
  
    // step #3. output  
    for (int i = 0; i < N; i++) {  
        cout << stones[i] << endl;  
    }  
  
    return 0;  
}
```

2 小課題 1 の解法

解答例を右に示します

手順ごとに分けて考えると実装しやすいです

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える

入力部分

```
int main() {  
    // step #1. read input  
    int N;  
    cin >> N;  
    vector<int> A(N);  
    for (int i = 0; i < N; i++) {  
        cin >> A[i];  
    }  
  
    // step #2. simulation  
    vector<int> stones;  
    for (int i = 0; i < N; i++) {  
        int pos = -1;  
        for (int j = 0; j < i; j++) {  
            if (stones[j] == A[i]) {  
                pos = max(pos, j);  
            }  
        }  
        if (pos != -1) {  
            for (int j = pos + 1; j < i; j++) {  
                stones[j] = A[i];  
            }  
        }  
        stones.push_back(A[i]);  
    }  
  
    // step #3. output  
    for (int i = 0; i < N; i++) {  
        cout << stones[i] << endl;  
    }  
  
    return 0;  
}
```

2 小課題 1 の解法

解答例を右に示します

手順ごとに分けて考えると実装しやすいです

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える

```
int main() {  
    // step #1. read input  
    int N;  
    cin >> N;  
    vector<int> A(N);  
    for (int i = 0; i < N; i++) {  
        cin >> A[i];  
    }
```

// step #2. simulation

```
vector<int> stones;  
for (int i = 0; i < N; i++) {
```

手順 1 の部分

```
    int pos = -1;  
    for (int j = 0; j < i; j++) {  
        if (stones[j] == A[i]) {  
            pos = max(pos, j);  
        }  
    }
```

```
    if (pos != -1) {  
        for (int j = pos + 1; j < i; j++) {  
            stones[j] = A[i];  
        }  
    }
```

```
    stones.push_back(A[i]);  
}
```

```
// step #3. output  
for (int i = 0; i < N; i++) {  
    cout << stones[i] << endl;  
}
```

```
return 0;  
}
```

2 小課題 1 の解法

解答例を右に示します

手順ごとに分けて考えると実装しやすいです

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える

```
int main() {  
    // step #1. read input  
    int N;  
    cin >> N;  
    vector<int> A(N);  
    for (int i = 0; i < N; i++) {  
        cin >> A[i];  
    }  
  
    // step #2. simulation  
    vector<int> stones;  
    for (int i = 0; i < N; i++) {  
        int pos = -1;  
        for (int j = 0; j < i; j++) {  
            if (stones[j] == A[i]) {  
                pos = max(pos, j);  
            }  
        }  
        if (pos != -1) {  
            for (int j = pos + 1; j < i; j++) {  
                stones[j] = A[i];  
            }  
        }  
        stones.push_back(A[i]);  
    }  
  
    // step #3. output  
    for (int i = 0; i < N; i++) {  
        cout << stones[i] << endl;  
    }  
  
    return 0;  
}
```

手順 2 の部分

2 小課題 1 の解法

解答例を右に示します

手順ごとに分けて考えると実装しやすいです

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える

```
int main() {  
    // step #1. read input  
    int N;  
    cin >> N;  
    vector<int> A(N);  
    for (int i = 0; i < N; i++) {  
        cin >> A[i];  
    }  
  
    // step #2. simulation  
    vector<int> stones;  
    for (int i = 0; i < N; i++) {  
        int pos = -1;  
        for (int j = 0; j < i; j++) {  
            if (stones[j] == A[i]) {  
                pos = max(pos, j);  
            }  
        }  
        if (pos != -1) {  
            for (int j = pos + 1; j < i; j++) {  
                stones[j] = A[i];  
            }  
        }  
        stones.push_back(A[i]);  
    }  
  
    // step #3. output  
    for (int i = 0; i < N; i++) {  
        cout << stones[i] << endl;  
    }  
  
    return 0;  
}
```

出力部分

2 小課題 1 の解法

解答例を右に示します

手順ごとに分けて考えると実装しやすいです

各ステップで行う操作

- 1 色 A_i の石の中で最も右のものを求める
- 2 1 で求めた場所より右を色 A_i で塗り替える

```
int main() {  
    // step #1. read input  
    int N;  
    cin >> N;  
    vector<int> A(N);  
    for (int i = 0; i < N; i++) {  
        cin >> A[i];  
    }  
    // step #2. simulation  
    vector<int> stones;  
    for (int i = 0; i < N; i++) {  
        int pos = -1;  
        for (int j = pos + 1; j < i; j++) {  
            if (A[j] == A[i]) {  
                pos = max(pos, j);  
            }  
        }  
        stones.push_back(A[i]);  
    }  
    // step #3. output  
    for (int i = 0; i < N; i++) {  
        cout << stones[i] << endl;  
    }  
    return 0;  
}
```

出力部分

Chapter III

小課題 2 の解法

Sous-tâche 2

3 小課題 2 の解法

小課題 2 の制約は「 $A_i = 1, 2$ 」です

つまり、囲碁やオセロのように「黒と白しかない」ということです



3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

1

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

① ② ③ ④

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

① ② ③ ④

① ② ③ ④ ⑤

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

① ② ③ ④

① ② ③ ④ ⑤

① ② ③ ④ ⑤ ⑥

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

① ② ③ ④

① ② ③ ④ ⑤

① ② ③ ④ ⑤ ⑥

① ② ③ ④ ⑤ ⑥ ⑦

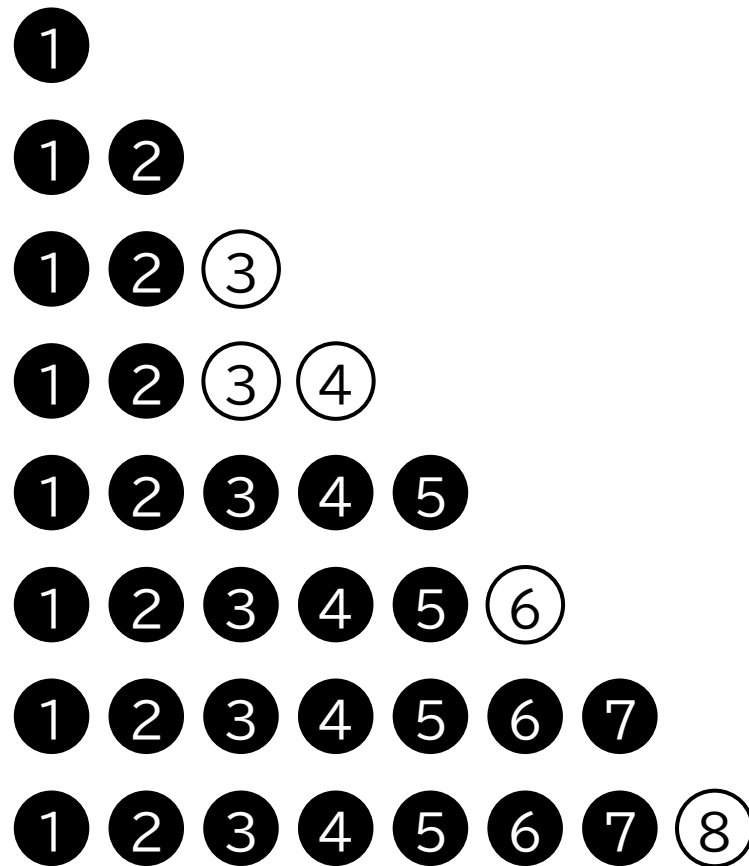
3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合



3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう



例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

① ② ③ ④

① ② ③ ④ ⑤

① ② ③ ④ ⑤ ⑥

① ② ③ ④ ⑤ ⑥ ⑦

① ② ③ ④ ⑤ ⑥ ⑦ ⑧

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨

3 小課題 2 の解法

どのような性質があるでしょうか？

これを探るため、小さいケースで実験してみよう

結論

どんな場合でも「黒, 黒, ..., 黒 の後に
白, 白, ..., 白」という状態になる！

例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合

①

① ②

① ② ③

① ② ③ ④

① ② ③ ④ ⑤

① ② ③ ④ ⑤ ⑥

① ② ③ ④ ⑤ ⑥ ⑦

① ② ③ ④ ⑤ ⑥ ⑦ ⑧

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨

3 小課題 2 の解法

結論

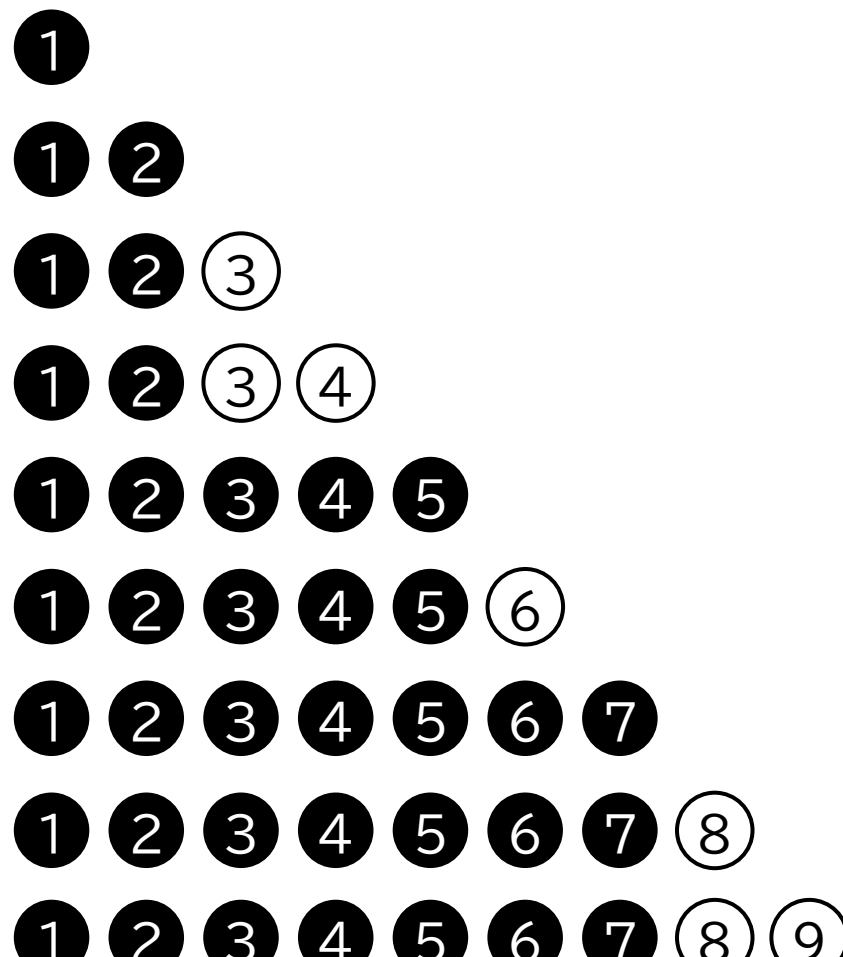
どんな場合でも「黒, 黒, ..., 黒 の後に
白, 白, ..., 白」という状態になる!

どうしてそんなことになるのか?

k 個石を置いたとき「黒, ..., 黒, 白, ..., 白」となっていると仮定する

- ① もし白が置かれると「黒, ..., 黒, 白, ..., 白, 白」となる
- ② もし黒が置かれると「黒, ..., 黒, 黒, ..., 黒, 黒」となる
→ $k + 1$ 個でも「黒, ..., 黒, 白, ..., 白」に!

例: $A = (1, 1, 2, 2, 1, 2, 1, 2, 2)$ の場合



3 小課題 2 の解法

解法

一番左が黒であれば、一番右の黒まで全部黒になる（それより右は全部白）

入力: ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨



出力: ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨

3 小課題 2 の解法

解法

一番左が黒であれば、一番右の黒まで全部黒になる（それより右は全部白）

入力: ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨



出力: ① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨

実際にプログラムを書くときは...

$A_1 = A_i$ となる最大の i まで、
全部色 A_1 で塗られる

ということになります

当然、計算量は $O(N)$

3 小課題 2 の解法

解法

一番左が黒であれば、一番右の黒まで全部黒になる（それより右は全部白）

実際にプログラムを書くときは…

1 となる最大の i まで、 i 番目まで塗られるということになります

当然、計算量は $O(N)$

60点獲得!

入力: 1 2 3 4 5 6 7 8 9



出力: 1 2 3 4 5 6 7 8 9

Chapter IV

小課題 3 (満点) の解法

Sous-tâche 3

4

満点解法

実は、一般の場合でも、小課題 2 と同じような性質がある

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

入力: ① ② ③ ④ ⑤ ⑥

①

① ②

① ② ③

① ② ③ ④

① ② ③ ④ ⑤

① ② ③ ④ ⑤ ⑥

4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

この性質を使って、答えを求めよう!



4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

この性質を使って、答えを求めよう！



最も右の赤色はここなので…

4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

この性質を使って、答えを求めよう！



これより左が全部赤とわかる！

4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

この性質を使って、答えを求めよう！



これより左が全部赤とわかる！

Question. 残りの部分はどうやって求める？

4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

一旦 5 番目までの石の存在を忘れることにしよう（後の結果には関係ない）



4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

一旦 5 番目までの石の存在を忘れることにしよう（後の結果には関係ない）



最も右の黄色はここなので…

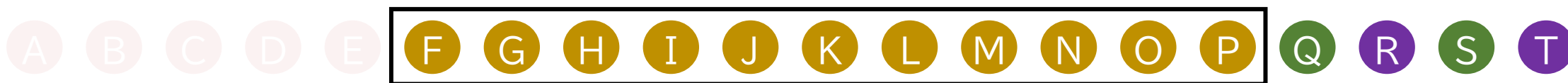
4

満点解法

性質

最も右の色 A_1 の石までが、全部色 A_1 に塗り替わる

一旦 5 番目までの石の存在を忘れることにしよう（後の結果には関係ない）

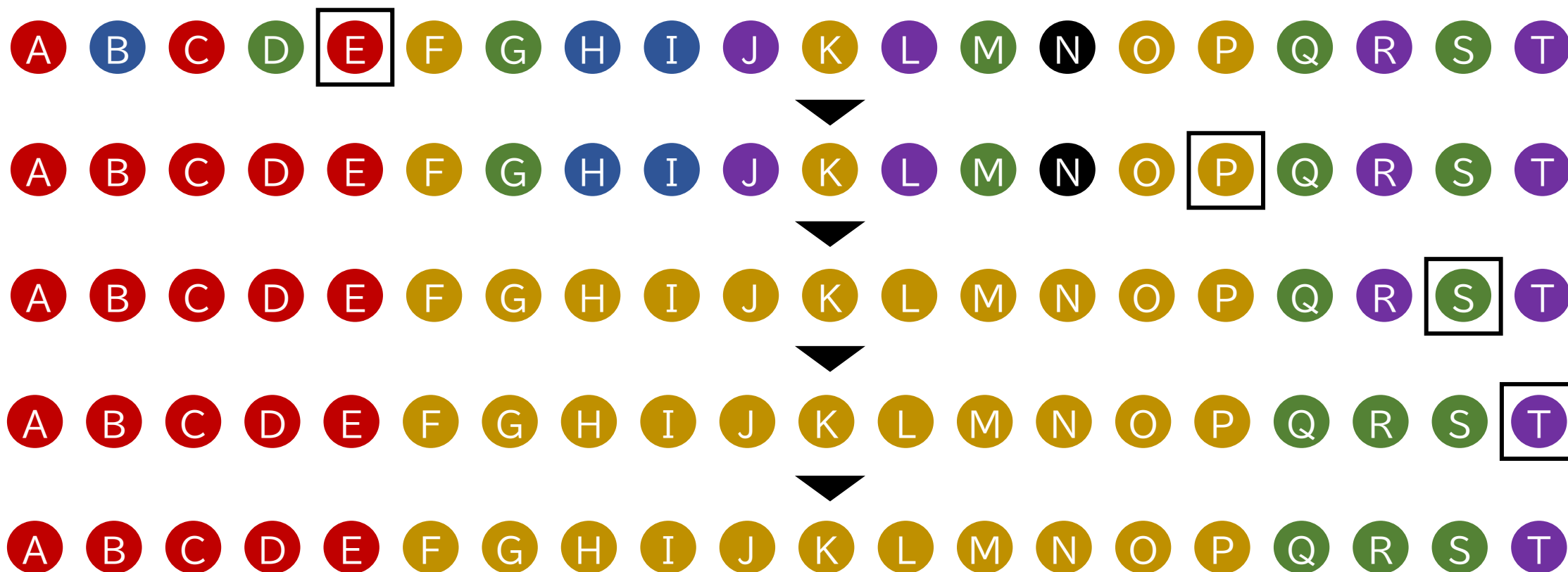


これより左が全部黄色とわかる!

4

満点解法

残りも同様にやっていくと、次のようになります



4

満点解法

残りも同様にやっていくと、次のようになります

しかし、最悪ケースでは N ステップかかる

計算量 $O(N^2)$ になってしまう！

A B C D E F G H I J K L M N O P Q R S T

4

満点解法

Question. 各ステップで「どこまで塗り替わるか」は、どうやって高速計算できるか？

4

満点解法

Question. 各ステップで「どこまで塗り替わるか」は、どうやって高速計算できるか？

アイデア

各色に対して、最も右のものを前計算する



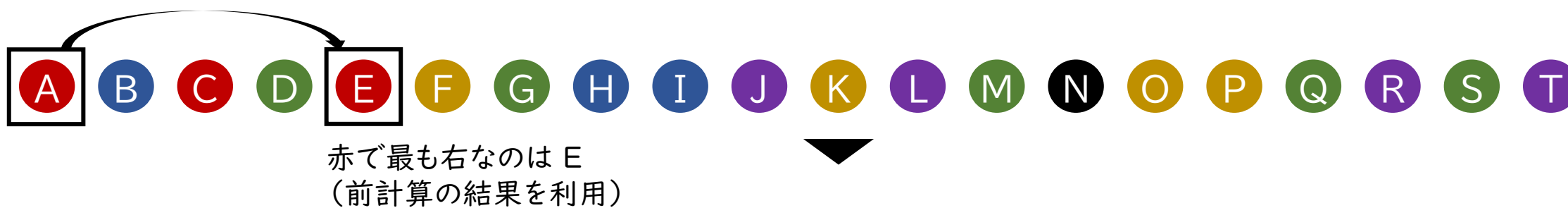
(この例だと、赤: E、青: I、黄: P、緑: S、紫: T、黒: N)

C++ では map を使って実装すると便利

4

満点解法

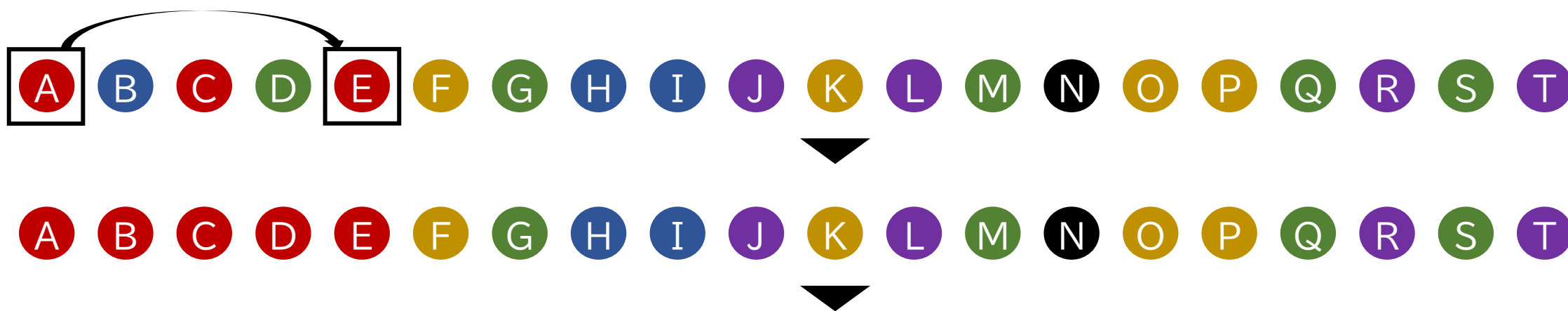
すると…? (最も右は 赤: E、青: I、黄: P、緑: S、紫: T、黒: N)



4

満点解法

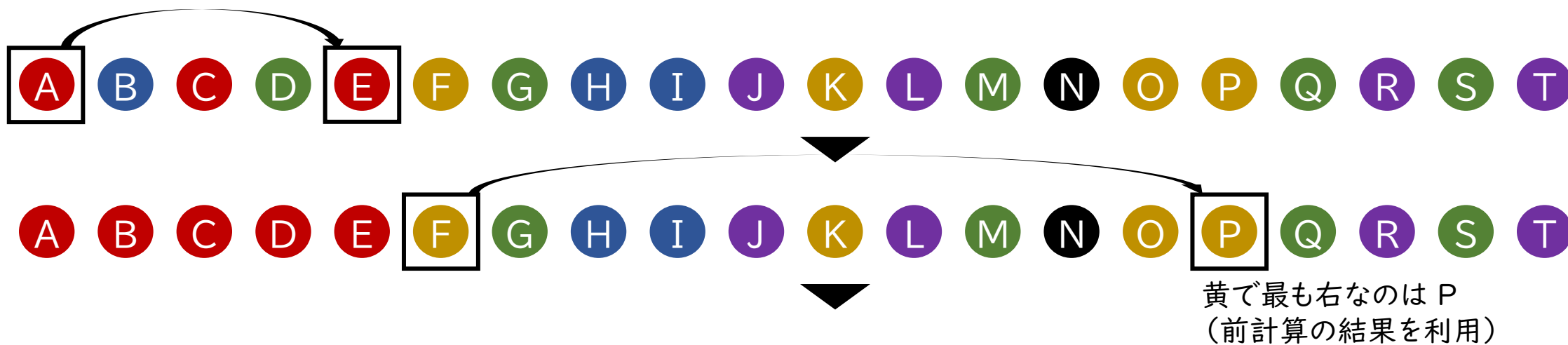
すると…? (最も右は 赤: E、青: I、黄: P、緑: S、紫: T、黒: N)



4

満点解法

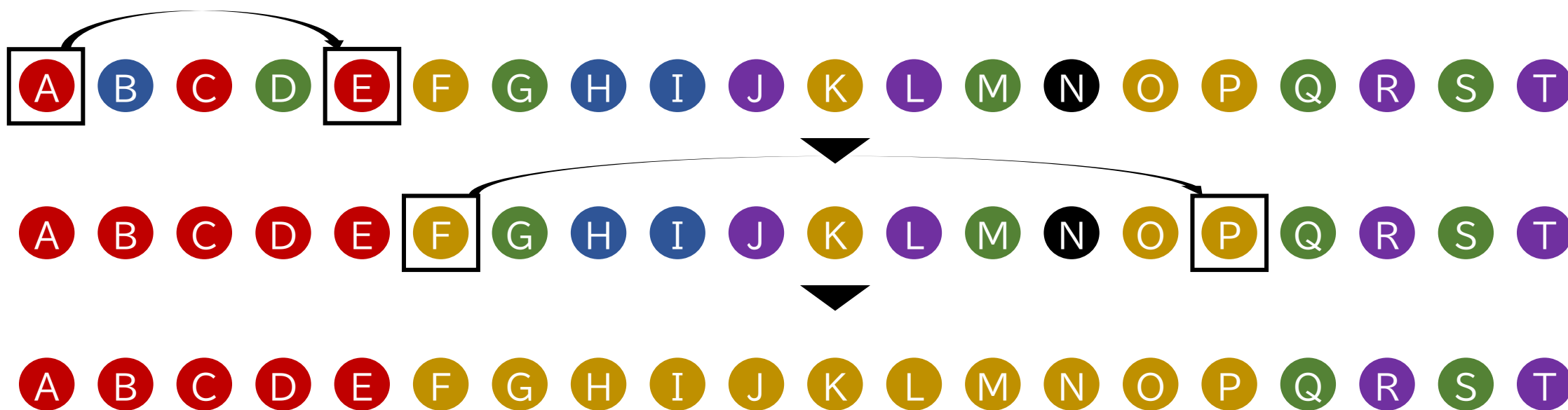
すると…? (最も右は 赤: E、青: I、黄: P、緑: S、紫: T、黒: N)



4

満点解法

すると…? (最も右は 赤: E、青: I、黄: P、緑: S、紫: T、黒: N)



4

満点解法

すると…？（最も右は 赤: E、青: I、黄: P、緑: S、紫: T、黒: N）

前計算の結果を利用すれば

1 ステップをすぐに終わらせる！

4

満点解法

計算量について

- map を使うとアクセスに $O(\log n)$ かかります
- そのため、全体計算量 $O(N \log N)$ で解けます

別解について

- ハッシュマップ (unordered_map) を使うと計算量 $O(N)$ になります
- 圧縮を使って同じ計算量で解く解法もあります

4

満点解法

計算量について

- map を使うとアクセスに $O(\log n)$ かかります

- そのため、全体計算量 $O(N \log V)$ で解けます

別解について

- ハッシュマップ (unordered_map) を使うと計算量 $O(N)$ になります
- 圧縮を使って同じ計算量で解く解法もあります

100点獲得！

Chapter V

得点分布

Répartition des scores

5

得点分布

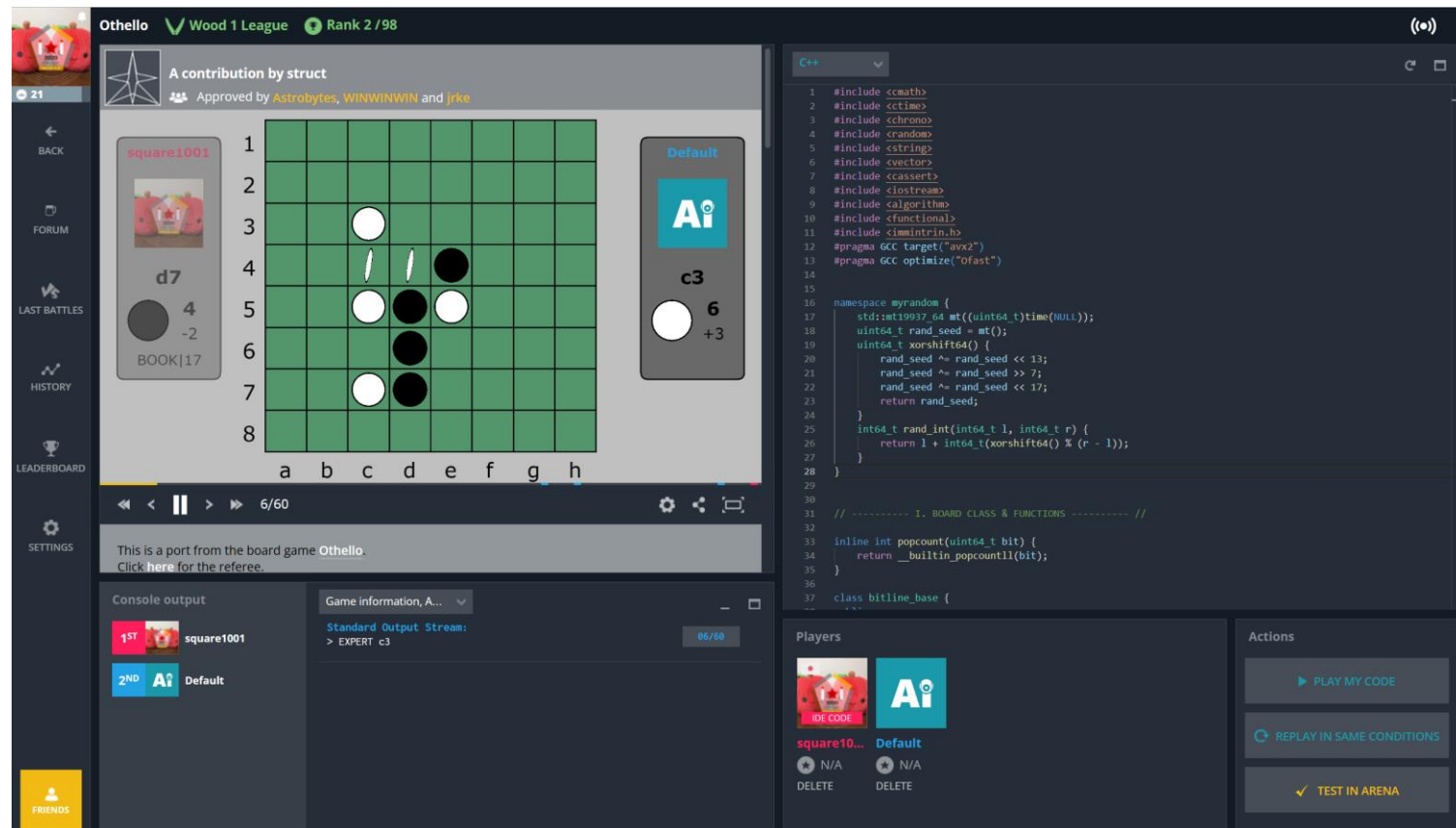
得点分布

- 100 点：135 人
- 60 点（小課題 1+2）：25 人
- 35 点（小課題 2）：3 人
- 25 点（小課題 2）：6 人

5 宣伝

CodinGame というサイトでオセロ AI を作って遊べます

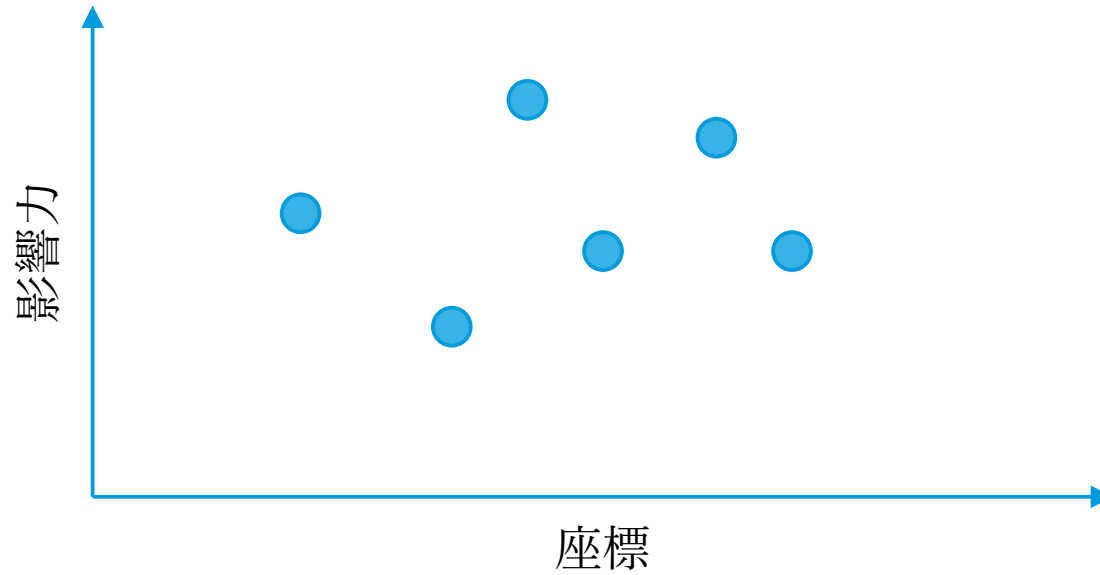
- 打倒 square1001 !



本選第 2 問 宣伝 2 (Advertisement 2) 解説

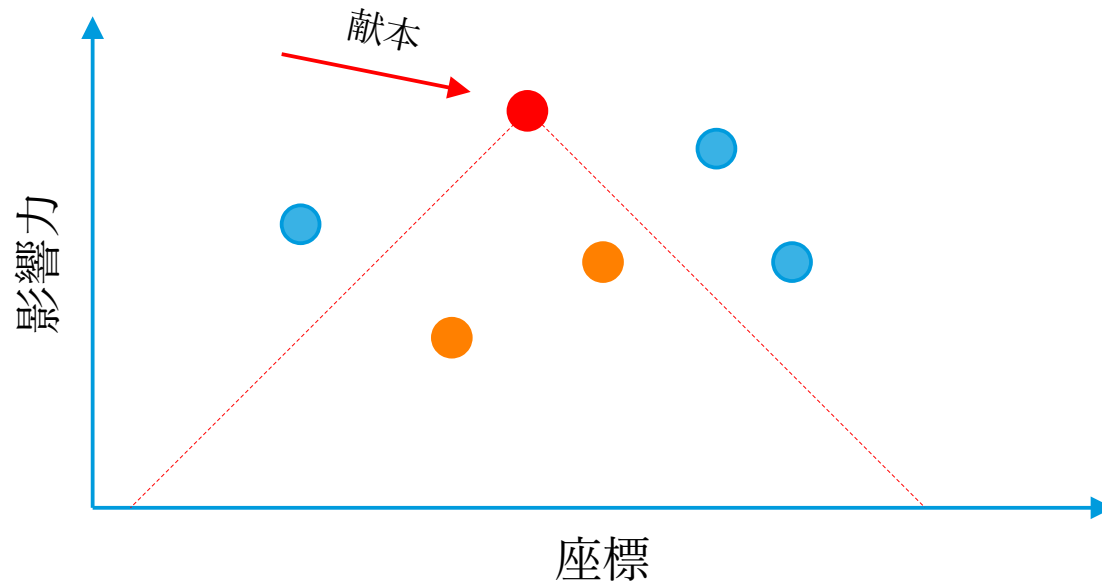
松尾 凜太郎

問題イメージ



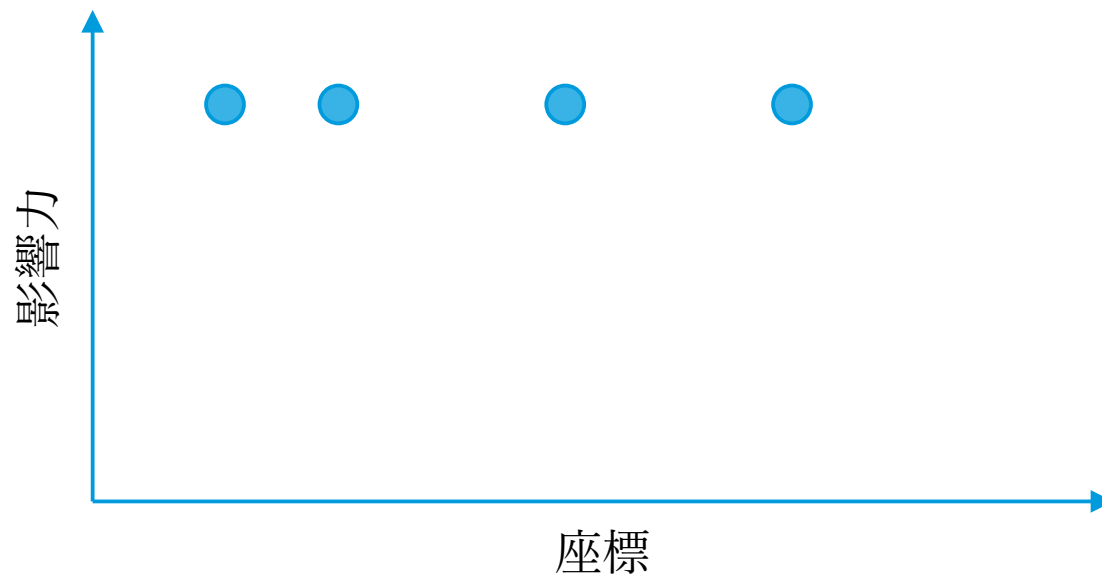
問題イメージ

住人 i と住人 j との数直線上での距離が $E_i - E_j$ 以下である.



小課題 1 (10点)

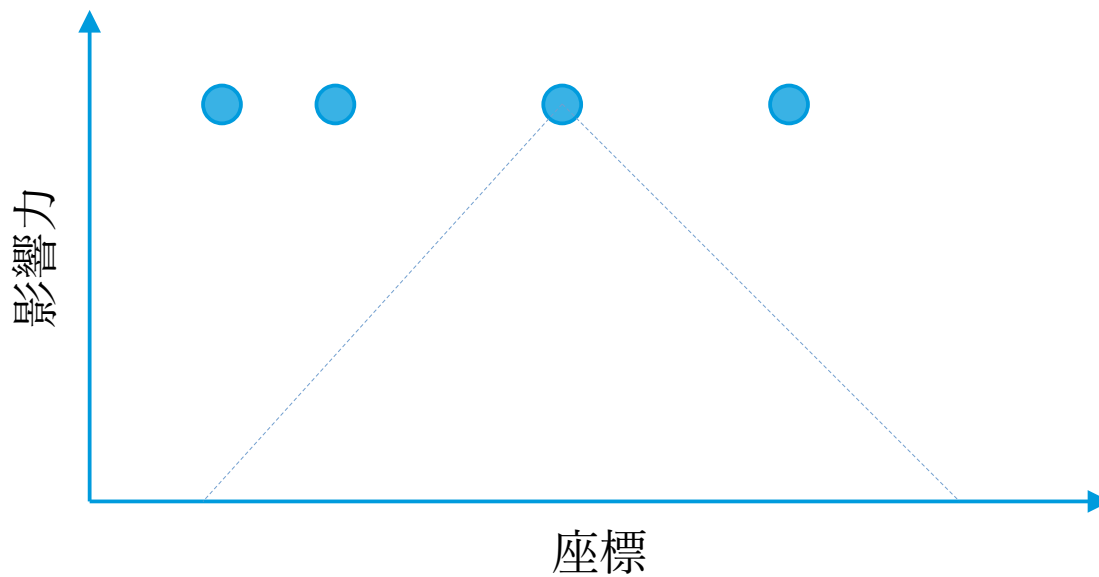
小課題 1 : 影響力が全員等しい



小課題 1 (10点)

小課題 1: 影響力が全員等しい

→ 基本的に、献本しても本を手に入れるのは 1 人だけ



小課題 1 (10点)

小課題 1 : 影響力が全員等しい

→ 基本的に、献本しても本を手に入れるのは 1 人だけ

同じ座標 (かつ同じ影響力) の住人が複数人いる場合だけは、
そのうち 1 人だけに献本すればよい

→ 座標の種類数が答え

小課題 1 (10点)

小課題 1: 影響力が全員等しい

→ 基本的に、献本しても本を手に入れるのは 1 人だけ

同じ座標 (かつ同じ影響力) の住人が複数人いる場合だけは、そのうち 1 人だけに献本すればよい

→ 座標の種類数が答え

- `std::sort` + `std::unique` (定数倍速い)
- `std::set` に全部突っ込む (実装楽)

$O(N \log(N))$

`std::unordered_set` なら平均 $O(N)$ (hack には注意)

小課題 2 (23点)

小課題 2 : $N \leq 16$

考察 : 誰に献本するかだけ決めれば、全員に本が行き渡るかは決まる

→ 2^{16} 全探索

bit 全探索等で実装するとよい
 $O(2^N \times N^2)$

小課題 3 (36点)

小課題 3 : $N \leq 1000$

住人 i への献本により住人 j が本を買う条件

$$|X[i] - X[j]| \leq E[i] - E[j]$$

$$X[i] - X[j] \leq E[i] - E[j] \text{ かつ } -(X[i] - X[j]) \leq E[i] - E[j]$$

$$X[i] - E[i] \leq X[j] - E[j] \text{ かつ } X[i] + E[i] \geq X[j] + E[j]$$

$$a[i] = X[i] - E[i]$$

$$b[i] = X[i] + E[i]$$

とすると

$$a[i] \leq a[j] \text{ かつ } b[i] \geq b[j]$$

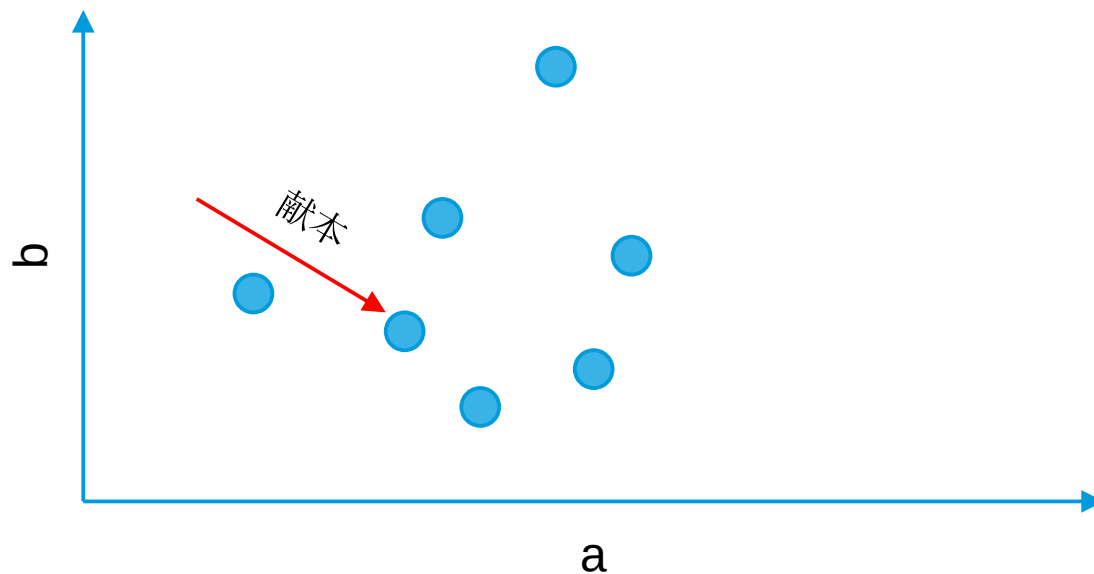
小課題 3 (36点)

$$a[i] = X[i] - E[i]$$

$$b[i] = X[i] + E[i]$$

とすると

$$a[i] \leq a[j] \text{ かつ } b[i] \geq b[j]$$



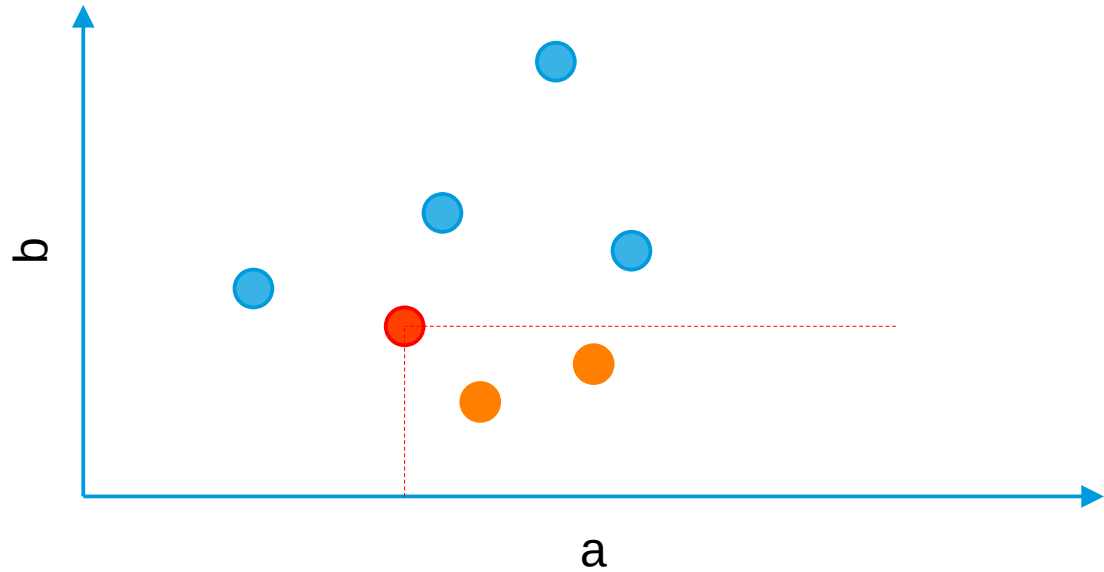
小課題 3 (36点)

$$a[i] = X[i] - E[i]$$

$$b[i] = X[i] + E[i]$$

とすると

$$a[i] \leq a[j] \text{ かつ } b[i] \geq b[j]$$



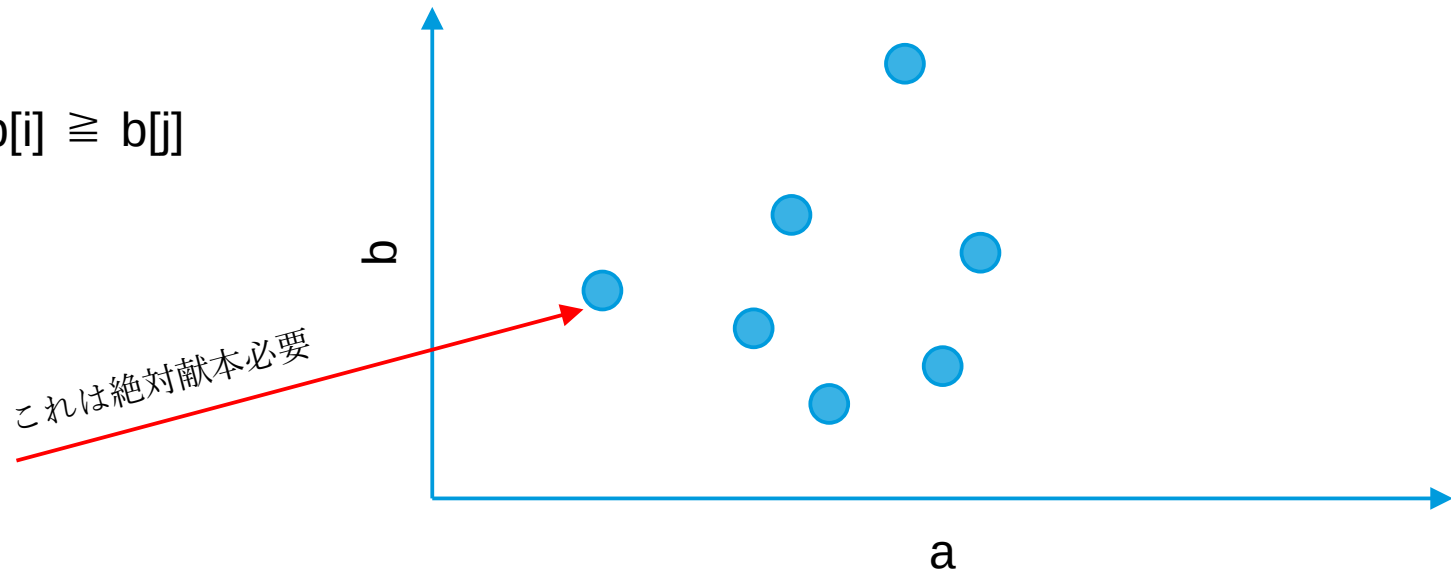
小課題 3 (36点)

$$a[i] = X[i] - E[i]$$

$$b[i] = X[i] + E[i]$$

とすると

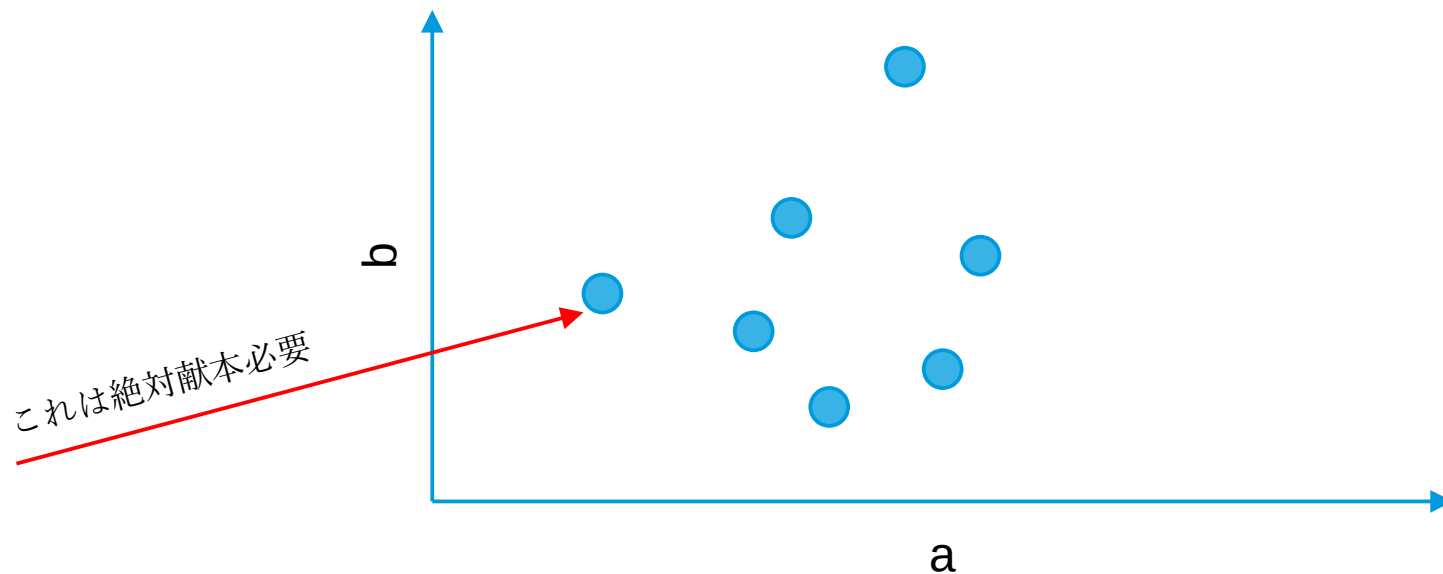
$$a[i] \leq a[j] \text{ かつ } b[i] \geq b[j]$$



小課題 3 (36点)

$a[i]$ 最小、同じなら $b[i]$ 最大のものは献本必要

→ その人への献本で本を手に入れた人を除いて同じことを繰り返す



小課題 3 (36点)

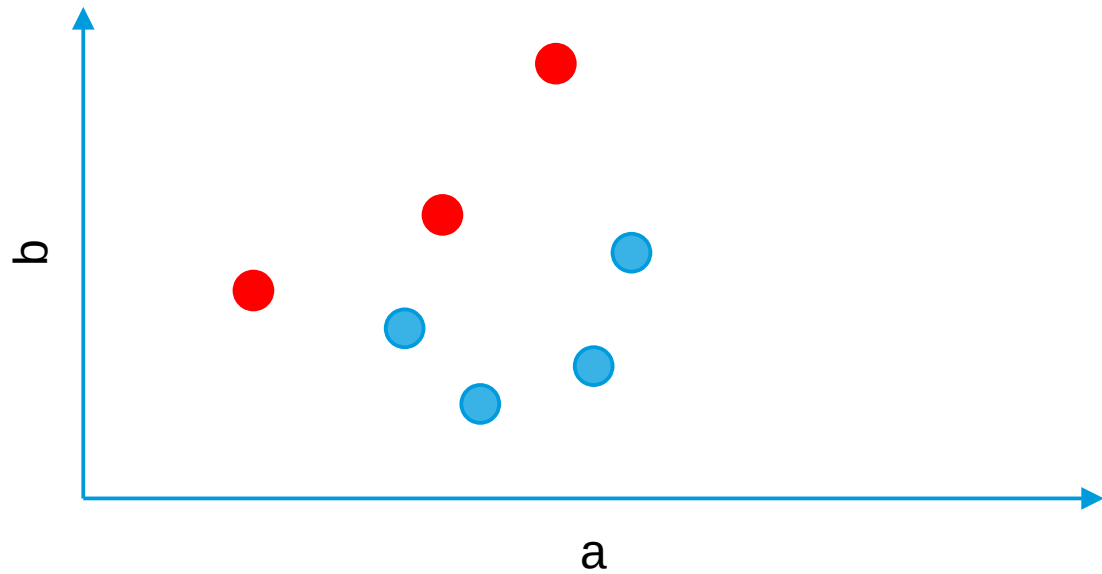
$a[i]$ 最小、同じなら $b[i]$ 最大のものは献本必要

→ その人への献本で本を手に入れた人を除いて同じことを繰り返す

$O(N^2)$

小課題 4 (31点)

左から見ていって、 b の最大値が更新される人に献本すれば必要十分



小課題 4 (31点)

左から見ていって、 b の最大値が更新される人に献本すれば必要十分

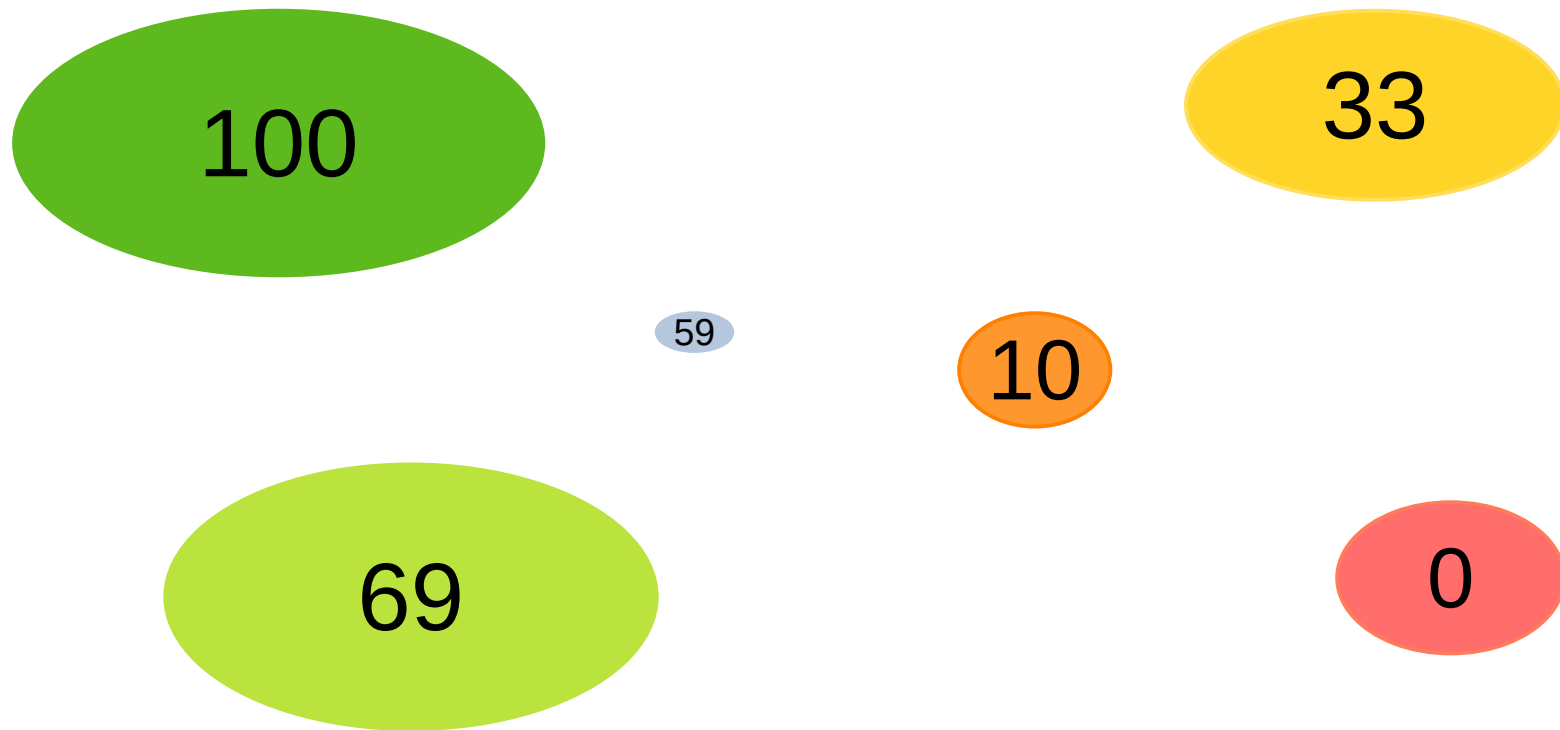
ソート + $O(N)$

別解

影響力の一番大きい住人を 1 人選ぶと、その人には献本するとしてよい
→ 満点もとれるが、結局 $X[i] - E[i]$ と $X[i] + E[i]$ を使うことになる可能性が高い

$X[i]$ の昇順に見て、今まで見た中で献本が必要な住人のリストを更新していく
いわゆる「**stack**に出し入れ」

得点分布



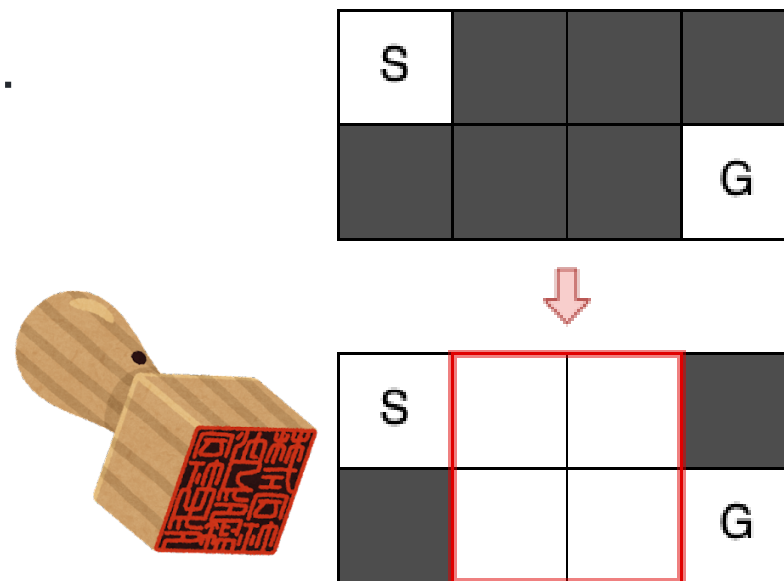
JOI 2022/2023 本選 問題 3

迷路 (Maze) 解説

解説担当 : tatyam

問題概要

- $R \times C$ マスのマス目があり、通れるマスと通れないマスがある.
- $N \times N$ マスの領域を選び、通れるようにする操作を何回でも行える.
- マス (S_r, S_c) とマス (G_r, G_c) を連結にしたい.
- 操作回数の最小値は？
- $R \times C \leq 6 \times 10^6$



小課題

1. (8 点) $N = 1$, $R \times C \leq 1.5 \times 10^6$.
2. (19 点) $R \times C \leq 10^3$.
3. (16 点) 答えは 10 以下である, $R \times C \leq 1.5 \times 10^6$.
4. (19 点) $R \times C \leq 6 \times 10^4$.
5. (5 点) $R \times C \leq 1.5 \times 10^5$.
6. (19 点) $R \times C \leq 1.5 \times 10^6$.
7. (8 点) $R \times C \leq 3 \times 10^6$.
8. (6 点) $R \times C \leq 6 \times 10^6$.

小課題 1 (8 点)

- $N = 1$
- 1×1 マスの領域を選び, 通れるようにする操作を何回でも行える.

これは... ?

C - 器物損壊！高橋君

解説

実行時間制限: 2 sec / メモリ制限: 64 MB

問題文

良く見てみるとカードの有効期限が切れていたので、高橋君は諦めて魚屋に直接うなぎを買いに行くことにしました。
彼の住む街は長方形の形をしており、格子状の区画に区切られています。区画は道または塀のどちらかであり、高橋君は道を東西南北に移動できますが斜めには移動できません。また、塀の区画は通ることができません。高橋君の家から魚屋までの道のりは非常に複雑なため、単純に歩くだけでは辿り着くことは困難です。
しかし、高橋君は腕力には自信があるので道に上下左右で面している塀を 2 回までなら壊して道にすることができます。

高橋君が魚屋に辿り着くことができるかどうか教えてください。

入力

入力は以下の形式で標準入力から与えられる。

```
H W
C(0,0)C(0,1) ... C(0,W-1)
C(1,0)C(1,1) ... C(1,W-1)
:
:
C(H-1,0)C(H-1,1) ... C(H-1,W-1)
```

- 入力は $H + 1$ 行ある。

考察① 重みつきグラフへの変換

- 黒いマスに 1 個通るのに 1 回操作が必要
- 同じ黒マスを複数回通る必要はない

考察① 重みつきグラフへの変換

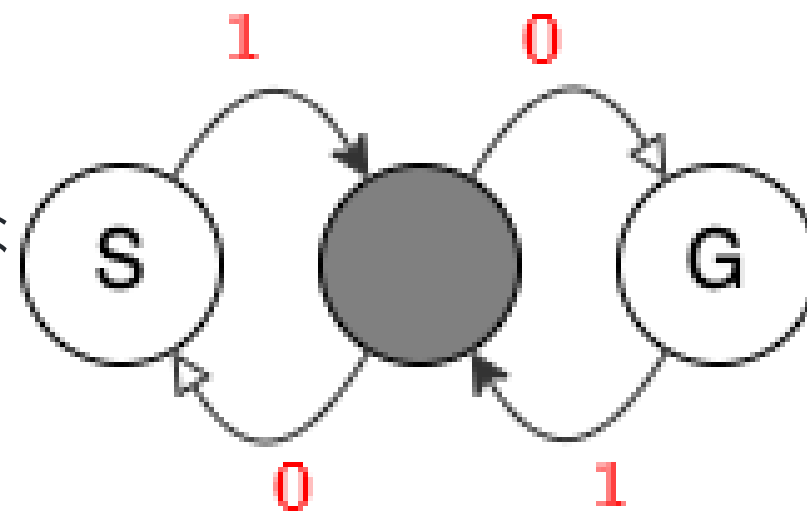
- 黒いマスを通るのに 1 回操作が必要
- 同じ黒マスを複数回通る必要はない

→ 黒マスを通る回数が最小となる S から G への経路を見つける問題

考察① 重みつきグラフへの変換

- 黒いマスに入る辺のコストを 1
- 白いマスに入る辺のコストを 0

とすると，頂点 S から頂点 G への最短経路問題になる



小課題 1 の解法

- 重み付きグラフの最短路問題

→ ダイクストラ法で $O(RC \log(RC))$ だ！

それでもいいが...？

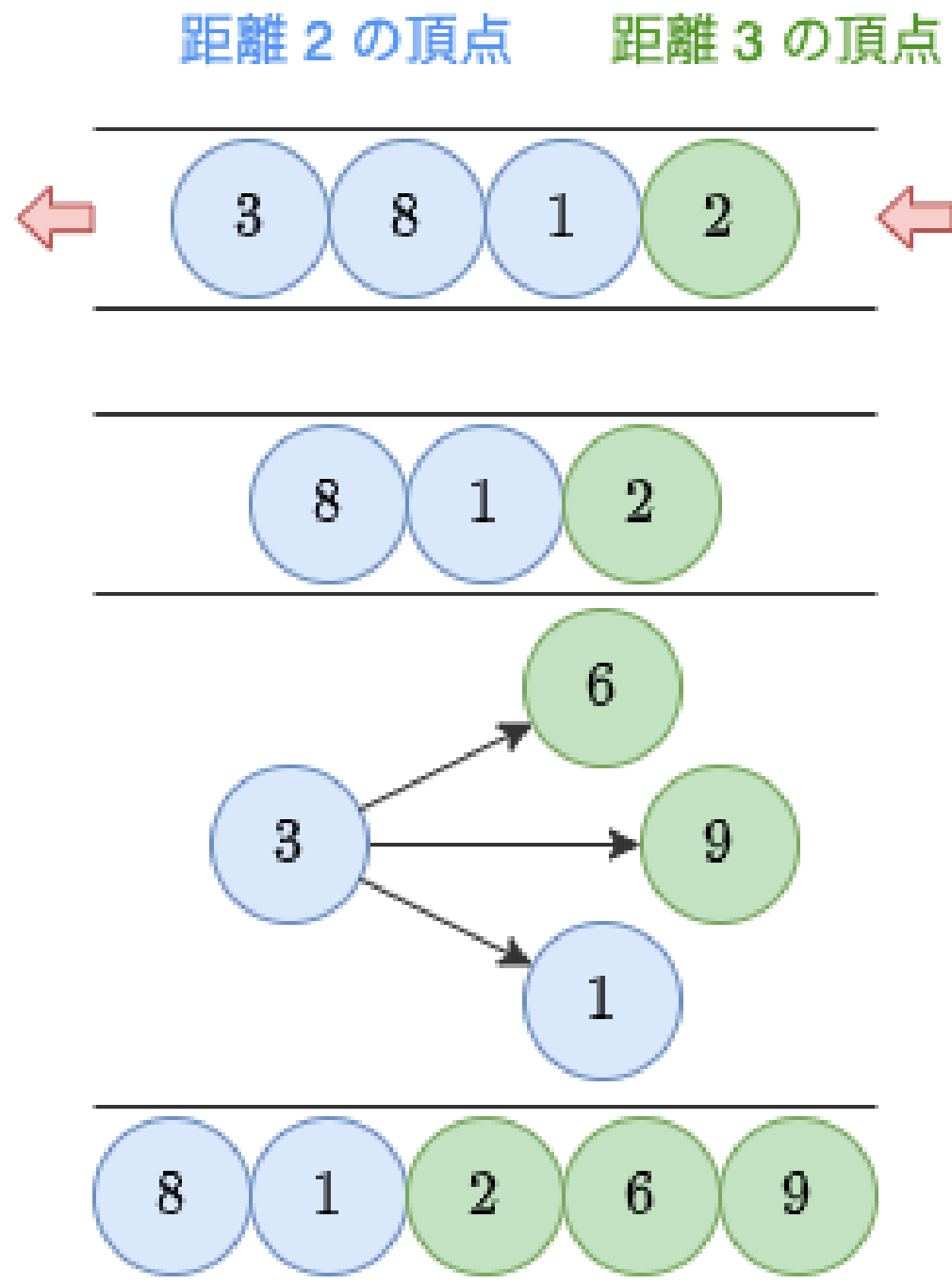
小課題 1 の解法

01 BFS

- 辺の重みが 0, 1 のみである場合に使える BFS の亜種
- BFS では探索する頂点をキュー (queue) に入れていた
- 01-BFS では探索する頂点を両終端キュー (deque) に入れる

BFS の場合

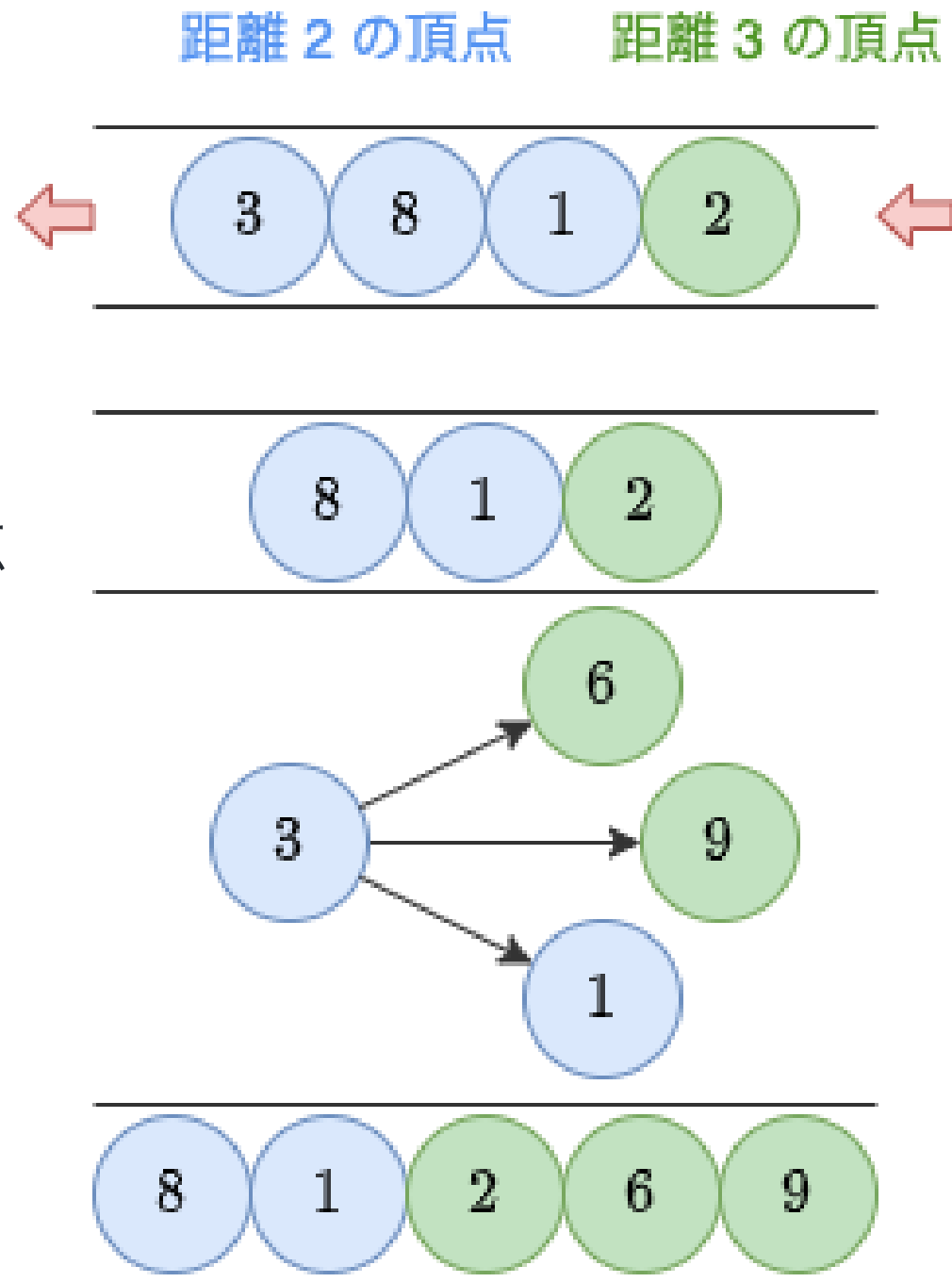
- 探索する頂点を **queue** に入れる
- queue の内部に境界がある. 境界の前には始点から距離 x の頂点が, 境界の後ろには始点から距離 $x + 1$ の頂点が入っている... (★)



BFS の場合

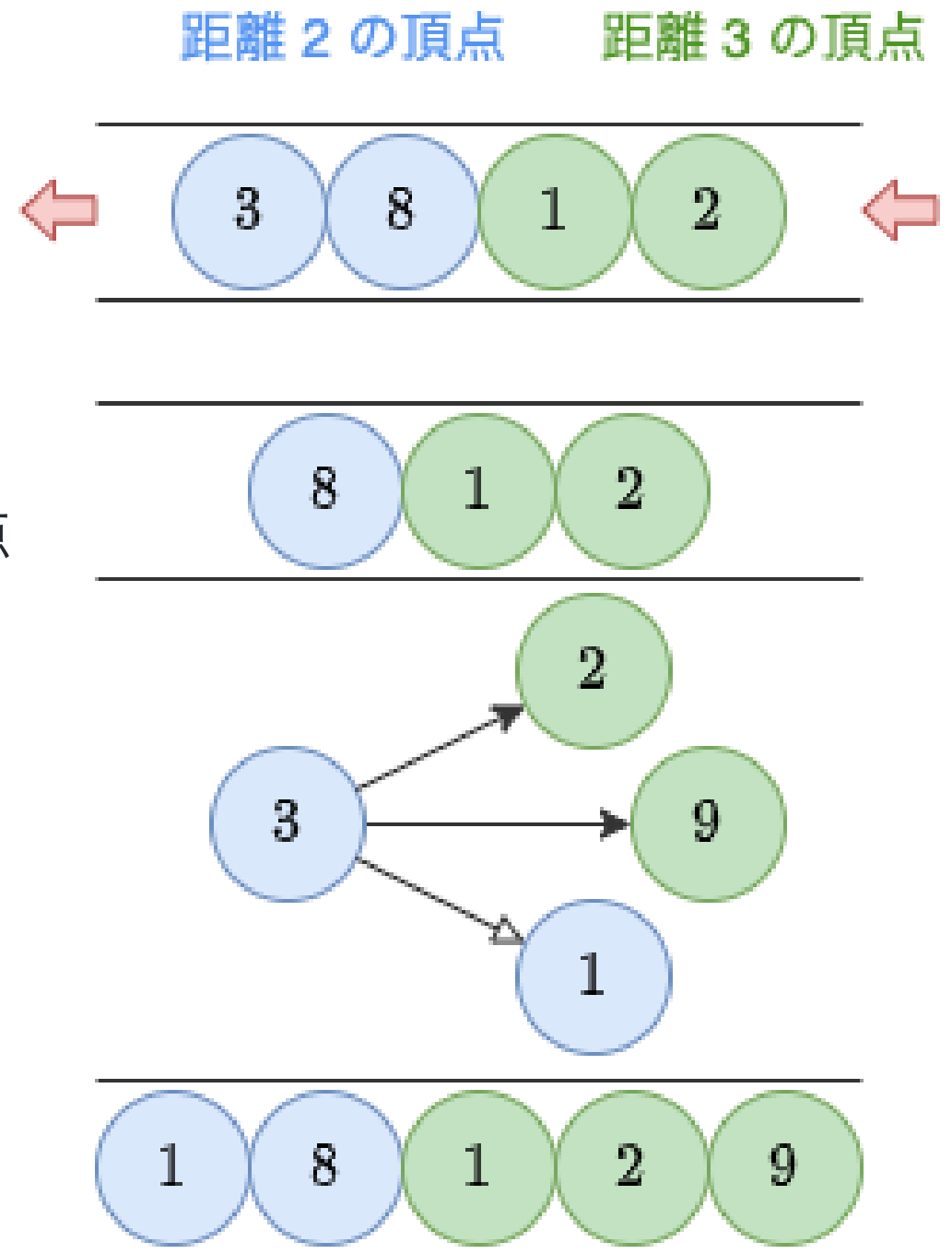
- 探索する頂点を **queue** に入れる
- queue の内部に境界がある・・・(★)
- 前から頂点を取り出して、隣接する頂点を距離 $x + 1$ で更新する
- 更新された頂点を queue の後ろに入れる
- これらの操作を行なっても (★) は満たされたまま

→ 距離の小さい順に探索できる！



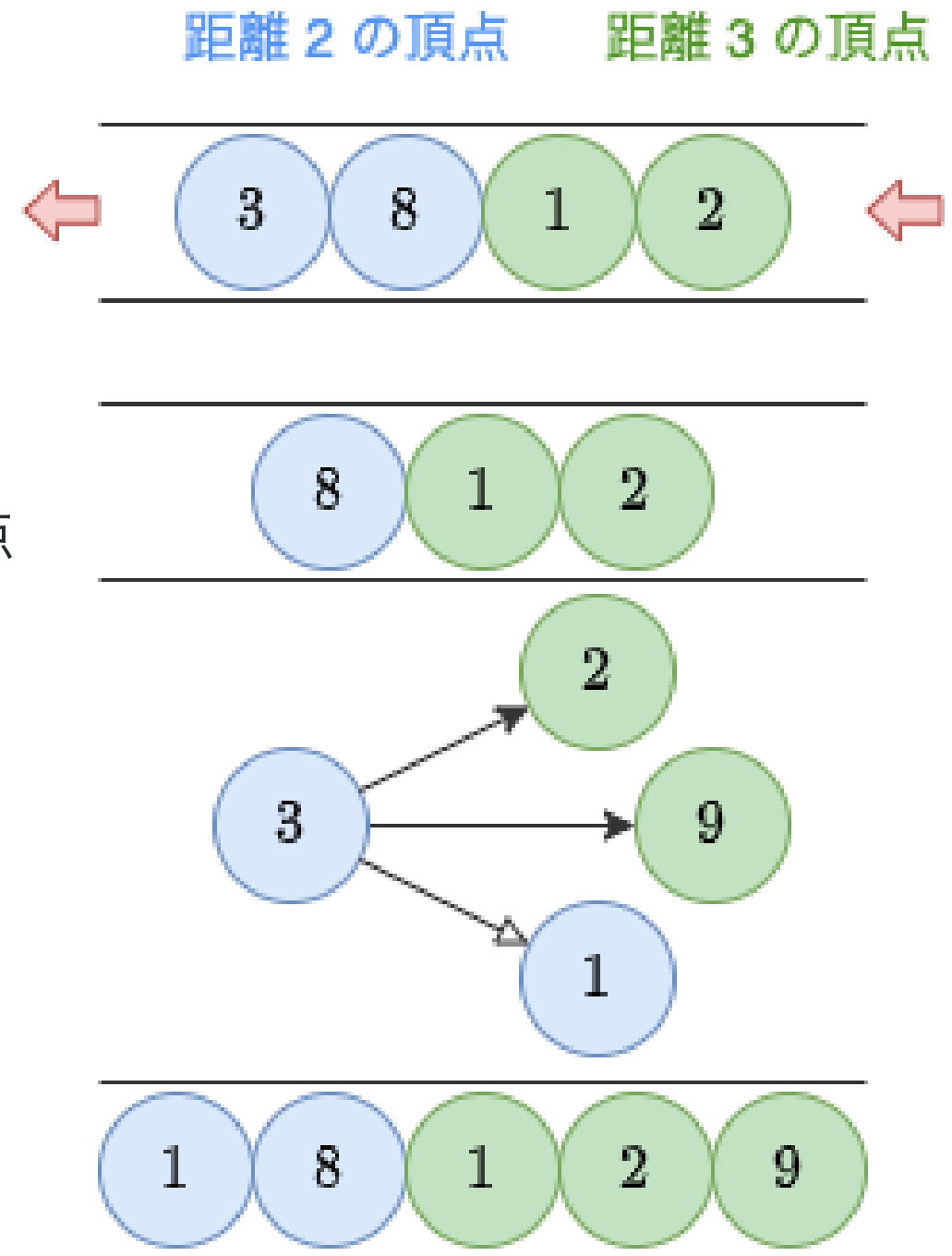
01-BFS の場合

- 探索する頂点を **deque** に入れる
- **deque** の内部に境界がある・・・(★)
- 前から頂点を取り出して、隣接する頂点を距離 x または距離 $x + 1$ で更新する



01-BFS の場合

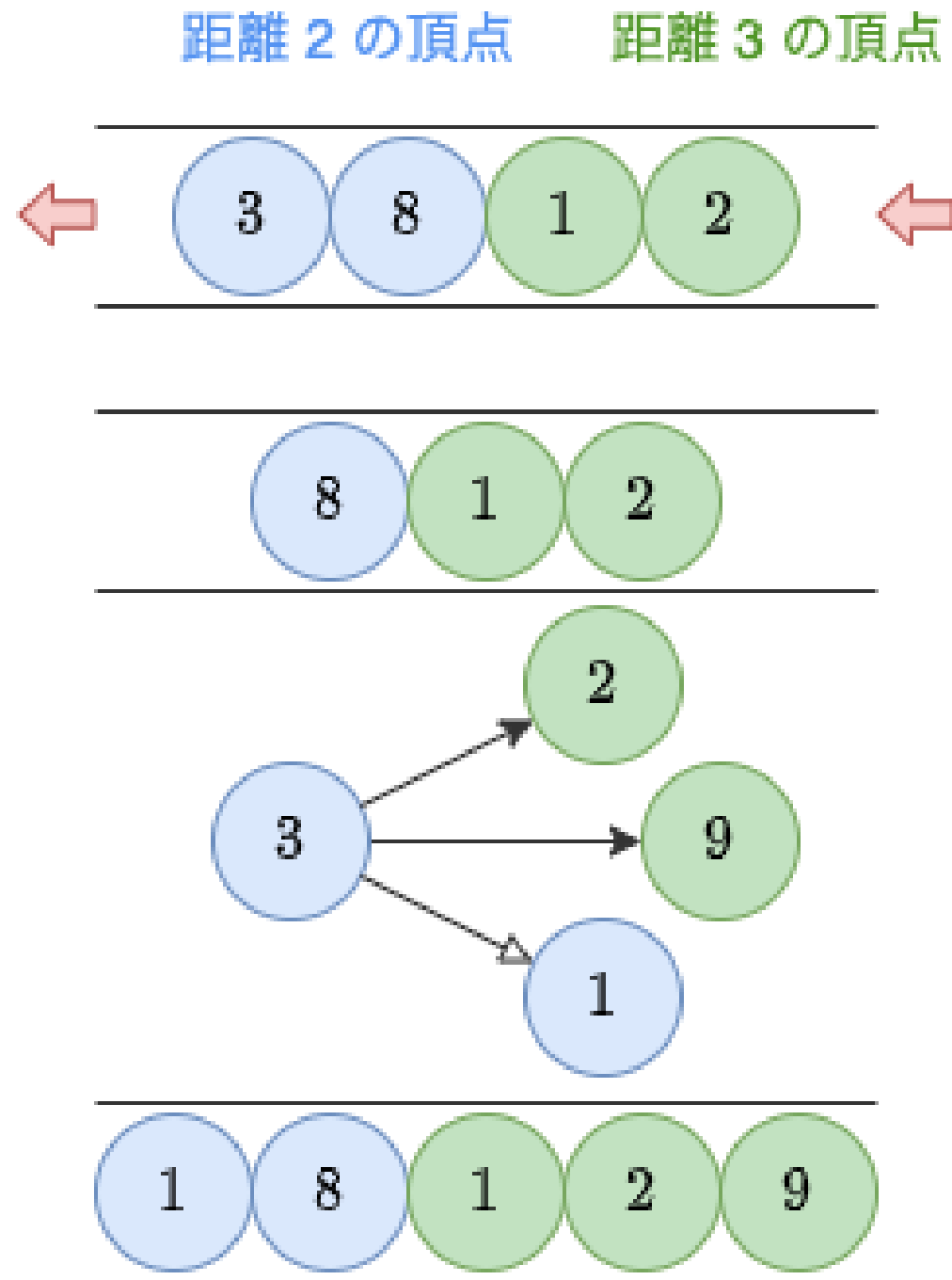
- 探索する頂点を **deque** に入れる
- **deque** の内部に境界がある... (★)
- 前から頂点を取り出して、隣接する頂点を距離 x または距離 $x + 1$ で更新する
- 距離 x で更新された頂点を **deque** の前に入れる
- 距離 $x + 1$ で更新された頂点を **deque** の後ろに入れる



01-BFS の場合

- 探索する頂点を **deque** に入れる
- **deque** の内部に境界がある・・・(★)
- 前から頂点を取り出して、隣接する頂点を **距離 x**
または距離 $x + 1$ で更新する
- **距離 x** で更新された頂点を **deque の前**に入れる
- **距離 $x + 1$** で更新された頂点を **deque の後ろ**に入れる
- これらの操作を行なっても (★) は満たされたまま

→ 距離の小さい順に探索できる！



小課題 1 の解法 まとめ

- マス目を辺の重みが 0 または 1 である重み付き有向グラフに変換する
- RC 頂点, $4RC - 2(R + C)$ 辺のグラフができる
- 01-BFS で S から G への最短経路を $O(RC)$ 時間で求める

豆知識 : `vector` のメモリ再確保

`vector<vector<pair<int, int>>>` で RC 頂点の重み付きグラフを作って持つとかなり遅い

- `vector` は 連続したメモリ領域 を使用することが保証されている. 確保したメモリ領域が足りなくなった場合は, 長さを 2 倍程度にして **メモリを再確保** する. このメモリ再確保はそれなりの時間がかかる.
- `vector` を RC 個作って `push_back` していくと, メモリ再確保が RC 回以上起こり, かなり遅くなる.

豆知識 : vector のメモリ再確保

`vector<vector<pair<int, int>>>` で RC 頂点の重み付きグラフを作って持つとかなり遅い

- `vector` は 連続したメモリ領域 を使用することが保証されている。確保したメモリ領域が足りなくなった場合は、長さを 2 倍程度にして メモリを再確保 する。このメモリ再確保はそれなりの時間がかかる。
- `vector` を RC 個作って `push_back` していくと、メモリ再確保が RC 回以上起こり、かなり遅くなる。

→ グラフを作らずにその都度 4 方向を調べるようにすると、速い

より一般的な方法 : 全部まとめて 1 つの `vector` に入れる (**CSR format**)

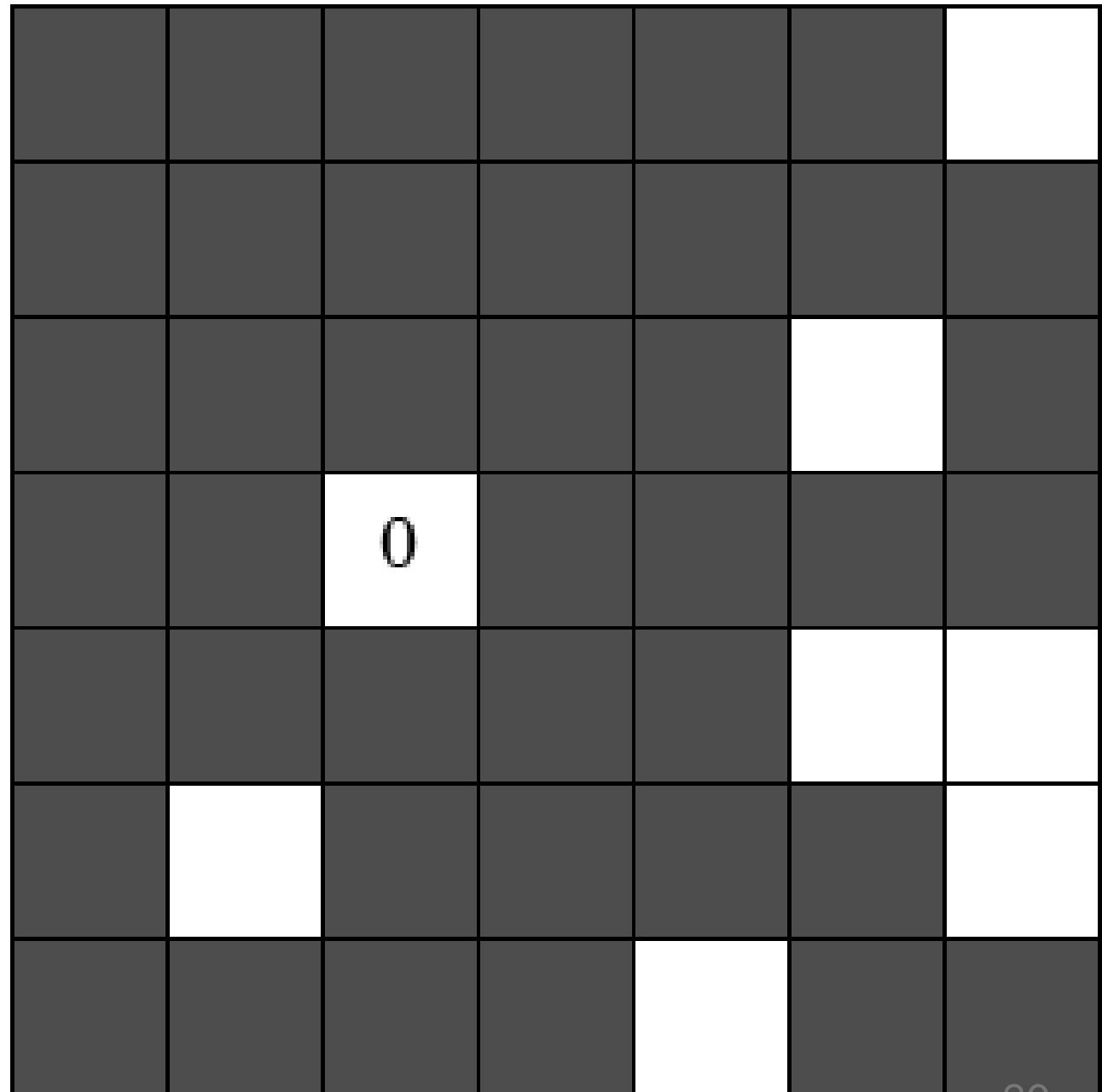
小課題 2 (19 点)

- $R \times C \leq 10^3$
- $O((RC)^2)$ などの計算量で解けば良い

考察② 1 手で行ける範囲

$N = 2$ とする.

4 方向を黒マスで塞がれているとき、1 手でどこまで行けるか？



		1	1			
		1	1			
		0				

	1	1	1			
	1	1	1			
		0				

	1	1	1			
	1	1	1	1	1	
		0	1	1		

	1	1	1			
	1	1	1	1	1	
		0	1	1		
			1	1	1	1
						1

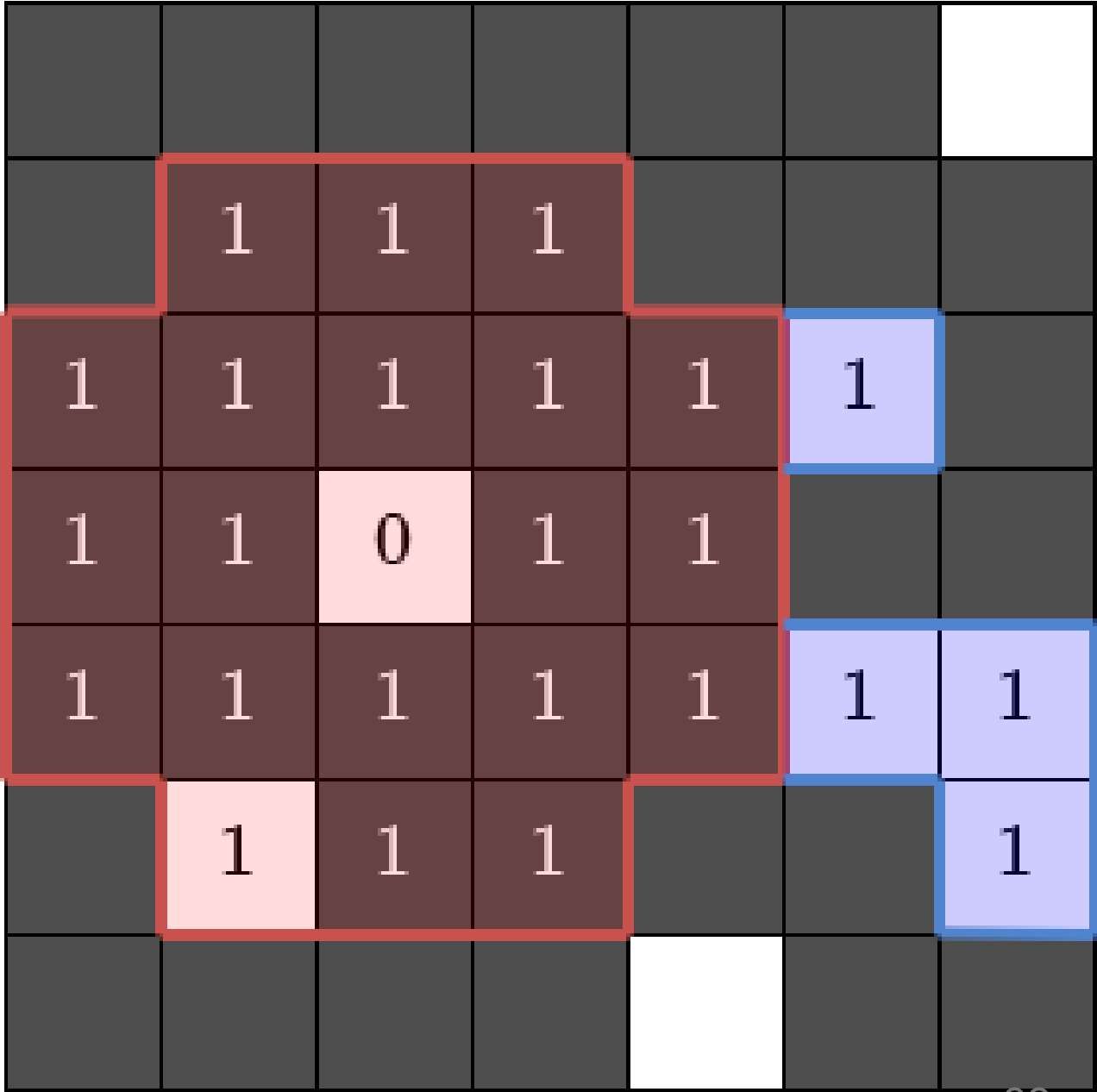
	1	1	1			
	1	1	1	1	1	
		0	1	1		
		1	1	1	1	1
	1	1	1			1

	1	1	1			
	1	1	1	1	1	
		0	1	1		
	1	1	1	1	1	1
	1	1	1			1

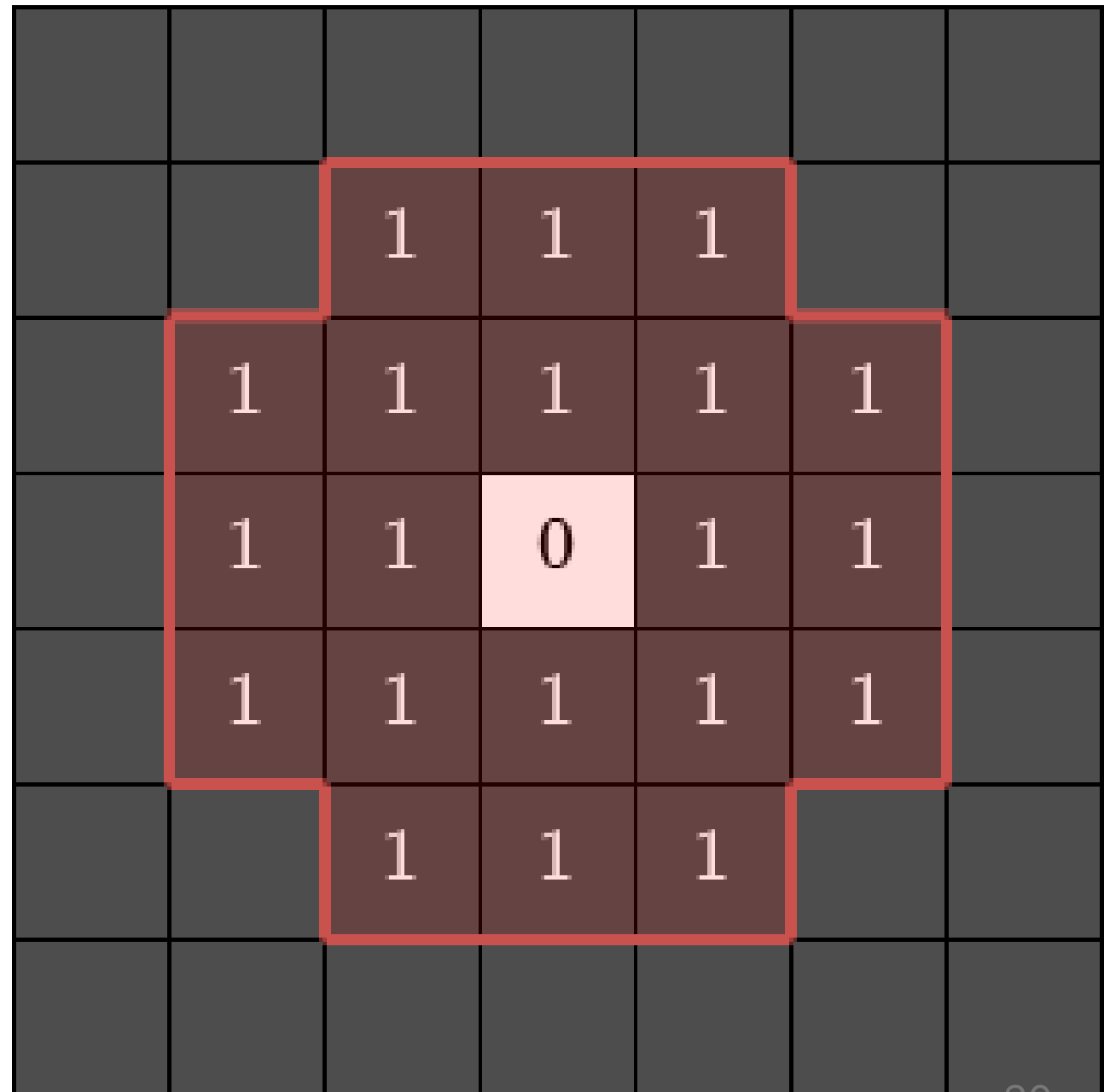
	1	1	1			
	1	1	1	1	1	
1	1	0	1	1		
1	1	1	1	1	1	1
	1	1	1			1

	1	1	1			
1	1	1	1	1	1	
1	1	0	1	1		
1	1	1	1	1	1	1
	1	1	1			1

(1 手で行ける範囲)
= (ハンコで塗れる範囲) +
(そこから白いマスで移動できる範囲)

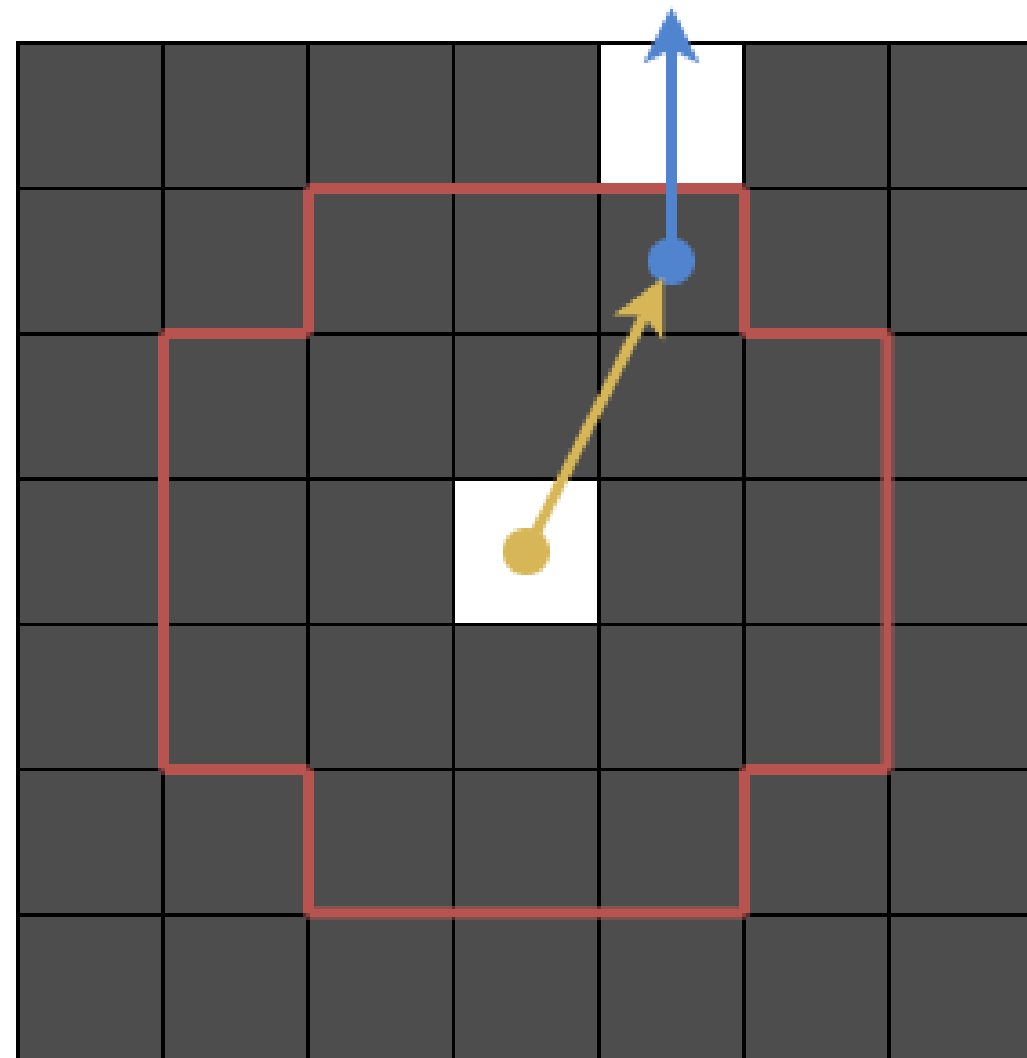


ハンコで塗れる範囲は
 $(2N + 1) \times (2N + 1)$ マスの正
方形の領域から四隅を取り除いた
形



考察③ ワープへの言い換え

「 $N \times N$ のハンコを押して移動する」操作は, 「ハンコで塗れる範囲のマスに色を無視してワープする」操作に言い換えることができる.

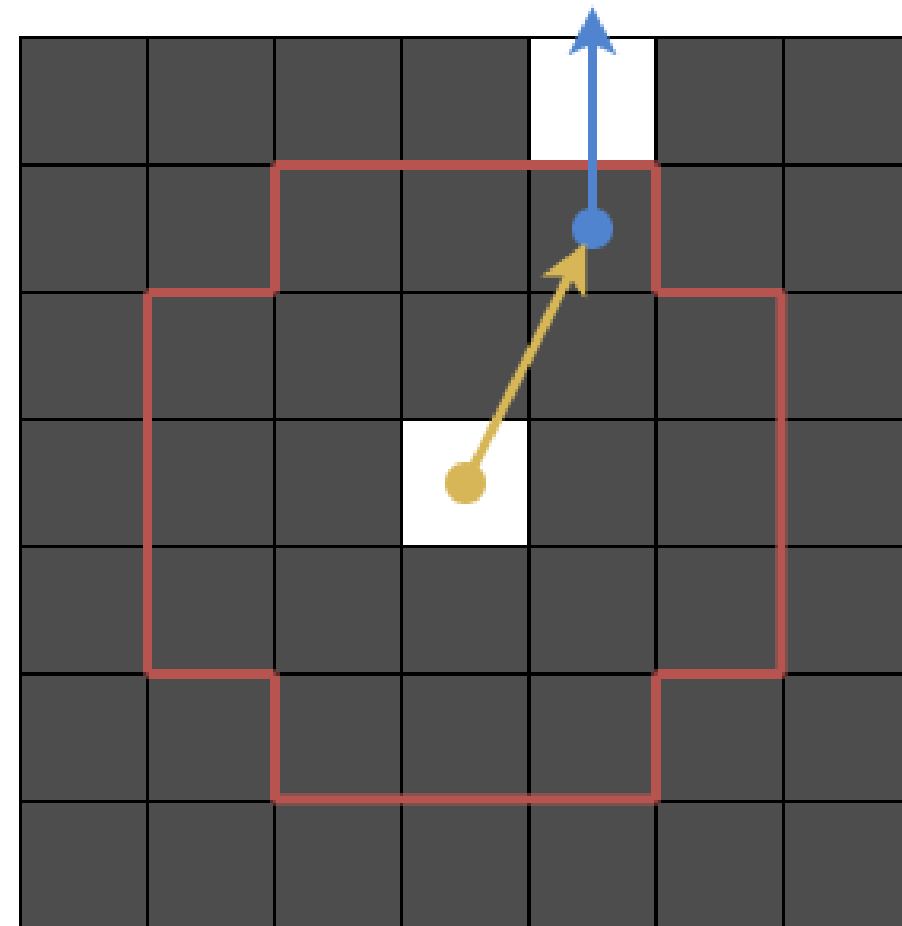


考察③ ワープへの言い換え

- ワープする操作は白マスが残らないため、
ハンコを押す操作より弱い操作

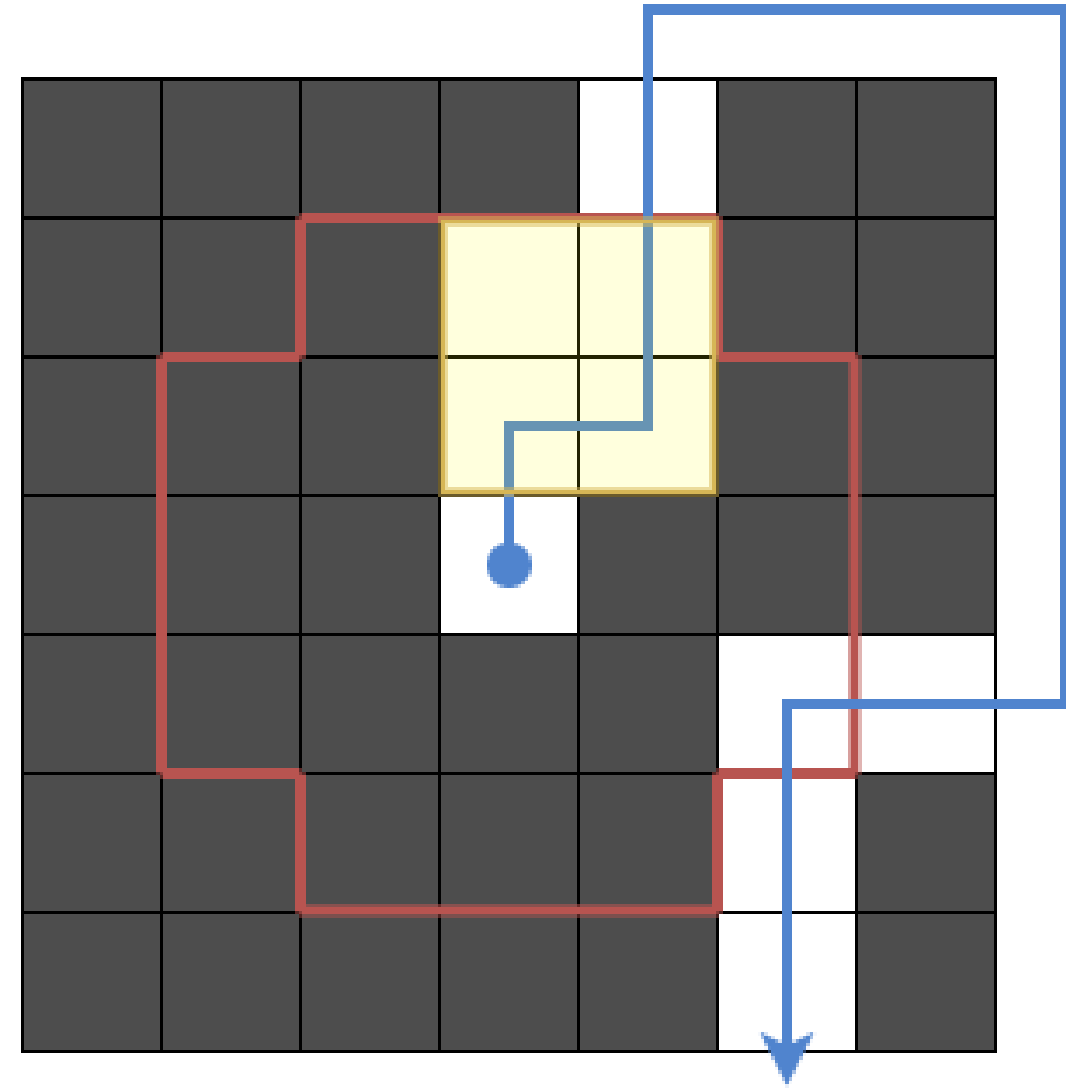
→ (ハンコの場合の最小の操作回数) $\leq$ (ワープの場合の最小の操作回数) は明らか.

(ハンコの場合の最小の操作回数) $\geq$ (ワープの場合の最小の操作回数) を示す.



補題

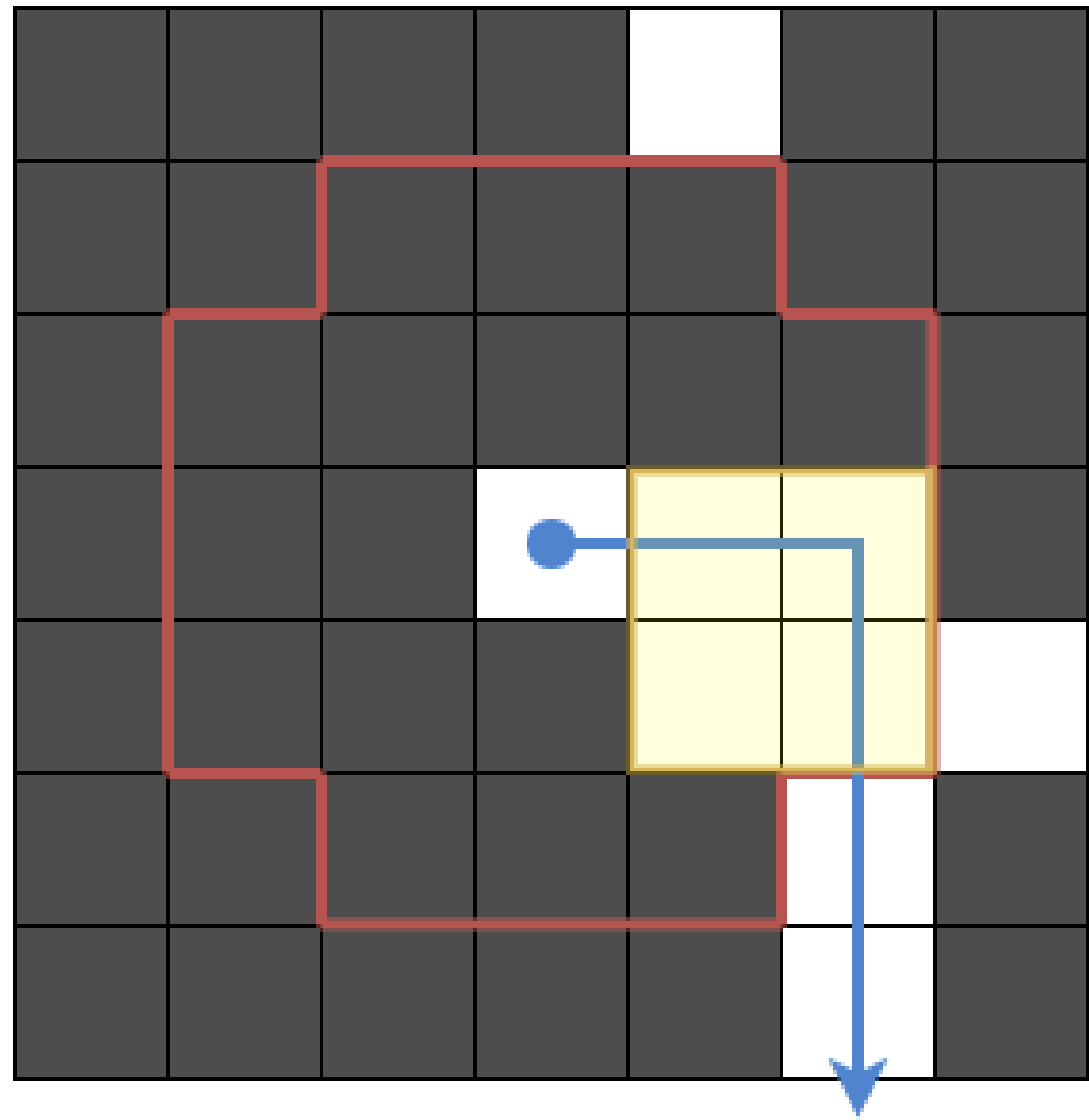
ハンコを押した後 **ハンコで塗れる範囲** を
出たら、 **ハンコで塗れる範囲** には戻って
こないとして良い (最小の操作回数は変わ
らない)



補題

ハンコを押した後 **ハンコで塗れる範囲** を出たら、**ハンコで塗れる範囲** には戻ってこないとして良い (最小の操作回数は変わらない)

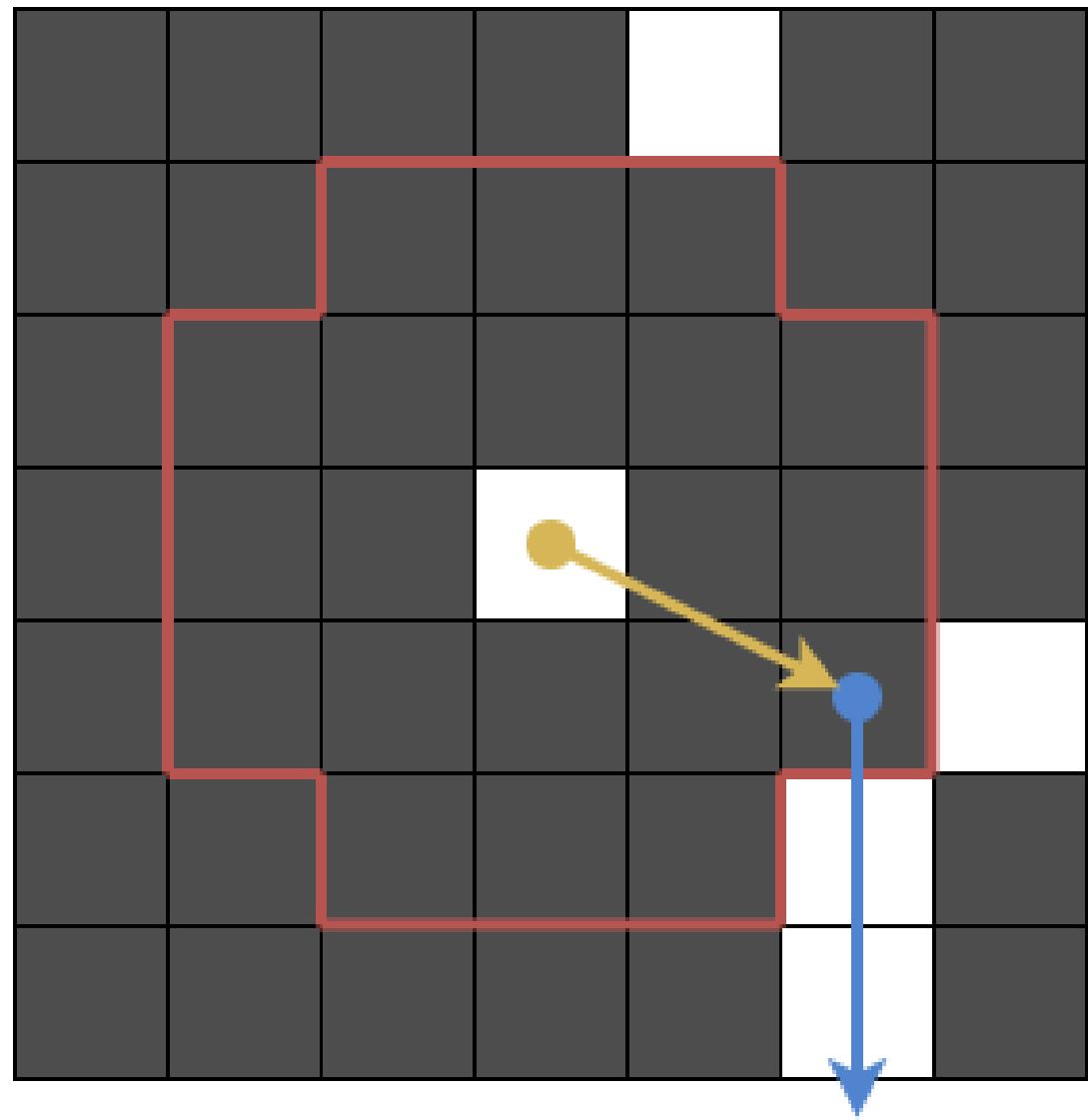
- 最初から 2 回目の方へ向かえば良い



考察③ ワープへの言い換え

ハンコで塗れる範囲を 1 回しか出ないのであれば、ハンコをワープで代用できる

→ ワープなら 01-BFS で扱える！



小課題 2 の解法 まとめ

- 小課題 1 と同様の 01-BFS
- 各マスから **ハンコで塗れる範囲** へコスト 1 の辺を張る
- 各マスから隣接する白マスへコスト 0 の辺を張る
- 辺の数は $O(RCN^2)$ 本なので, 01-BFS で $O(RCN^2)$

豆知識 : push 関数を作ろう

グリッド BFS を書くとき...

```
while(q.size()) {  
    auto [r, c] = q.front();  
    q.pop();  
    if(r > 0 && chmin(cost[r - 1][c], cost[r][c] + 1))  
        q.emplace(r - 1, c);  
    if(r + 1 < R && chmin(cost[r + 1][c], cost[r][c] + 1))  
        q.emplace(r + 1, c);  
    if(c > 0 && chmin(cost[r][c - 1], cost[r][c] + 1))  
        q.emplace(r, c - 1);  
    if(c + 1 < C && chmin(cost[r][c + 1], cost[r][c] + 1))  
        q.emplace(r, c + 1);  
}
```

同じことを何度も書きすぎ！！

```
auto push = [&](int r, int c, int x) {  
    if(cost[r][c] <= x) return;  
    cost[r][c] = x;  
    q.emplace(r, c);  
};  
while(q.size()) {  
    auto [r, c] = q.front();  
    q.pop();  
    const int x = cost[r][c] + 1;  
    if(r > 0) push(r - 1, c, x);  
    if(r + 1 < R) push(r + 1, c, x);  
    if(c > 0) push(r, c - 1, x);  
    if(c + 1 < C) push(r, c + 1, x);  
}
```

豆知識: push 関数を作る
う

関数を使ってまとめよう

```
auto push = [&](int r, int c, int x) {  
    if(r < 0 || r >= R || c < 0 || c >= C) return;  
    if(cost[r][c] <= x) return;  
    cost[r][c] = x;  
    q.emplace(r, c);  
};  
while(q.size()) {  
    auto [r, c] = q.front();  
    q.pop();  
    const int x = cost[r][c] + 1;  
    push(r - 1, c, x);  
    push(r + 1, c, x);  
    push(r, c - 1, x);  
    push(r, c + 1, x);  
}
```

push 関数の中で境界判定をやっても良いですね

小課題 3 (16 点)

- 答えは 10 以下である.
- 答えを ANS として, $O(RCANS)$ くらいの計算量で解けば良い

考察④ x 手で行ける範囲

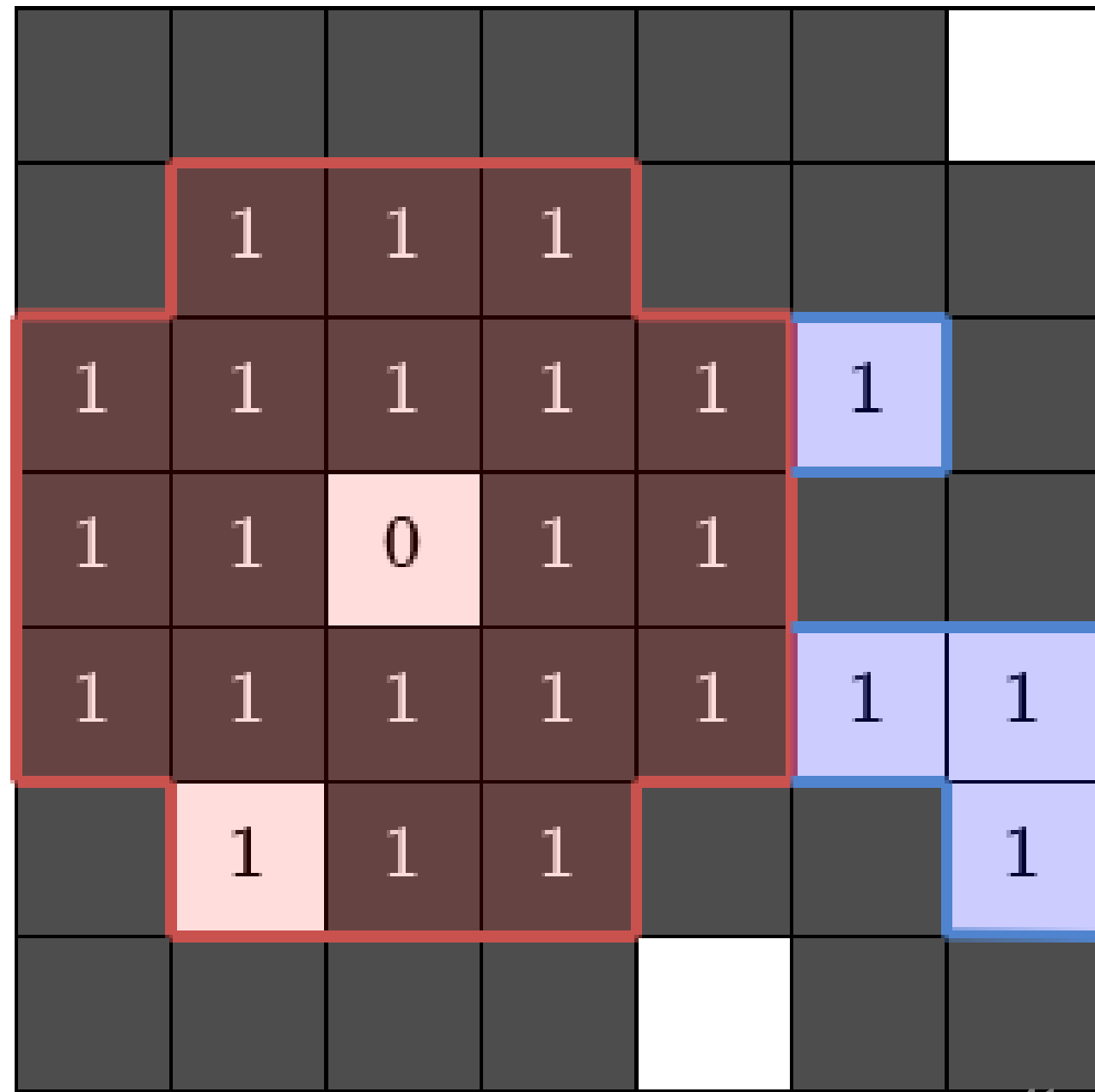
周囲を黒マスで囲まれているとき,

(1 手で行ける範囲)

= (ハンコで塗れる範囲) +

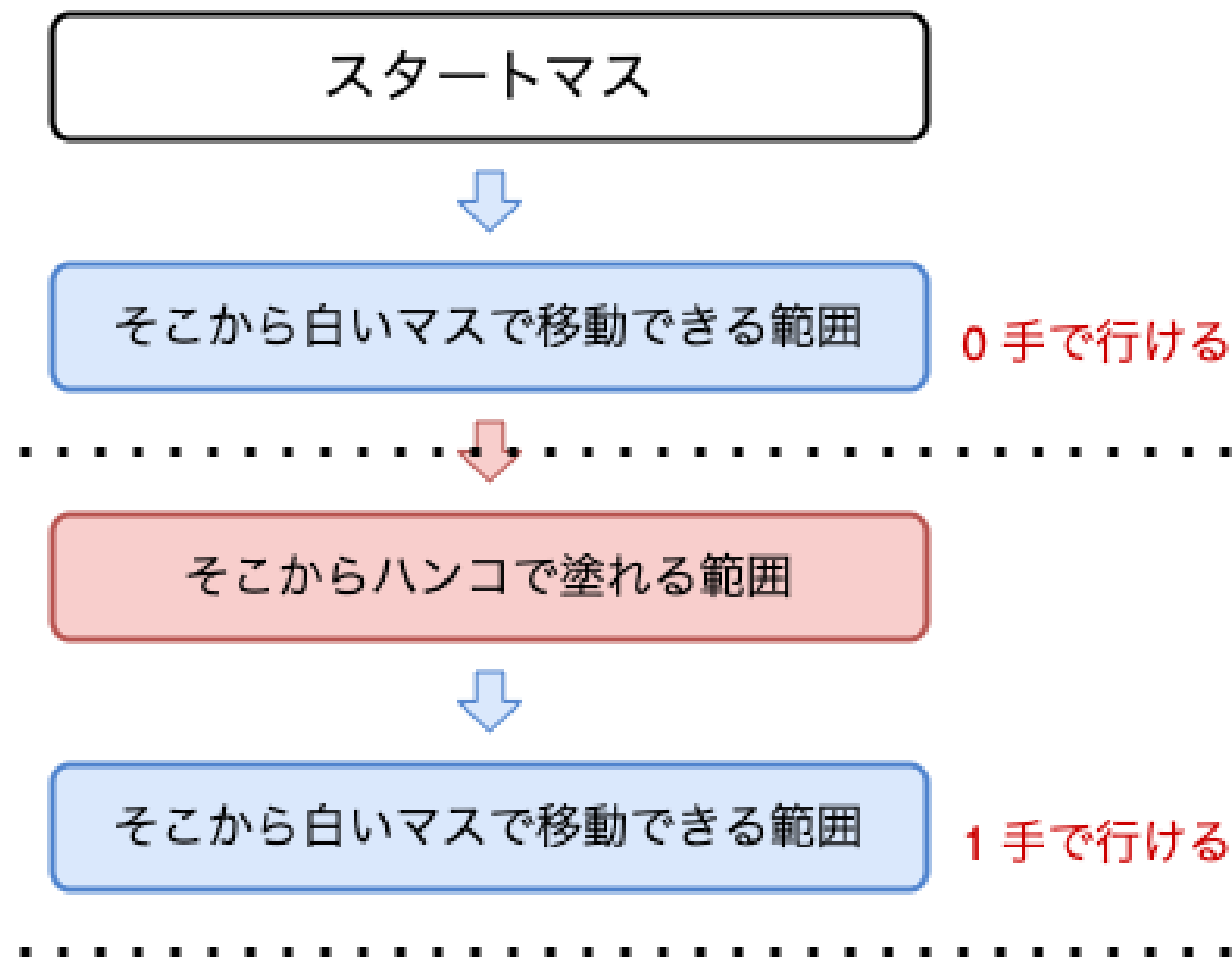
(そこから白いマスで移動できる範囲)

だった.



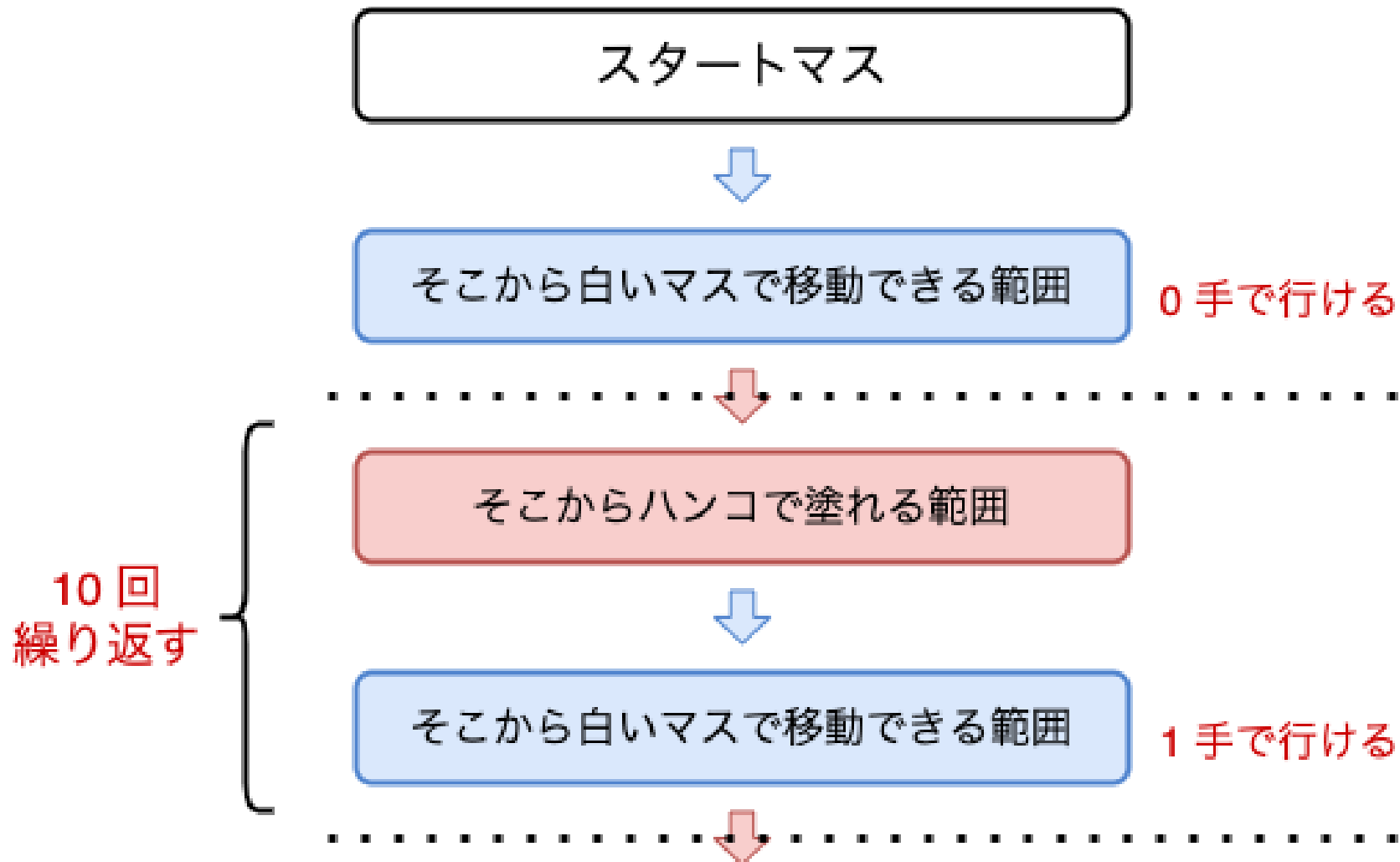
考察④ x 手で行ける範囲

図で整理するところ



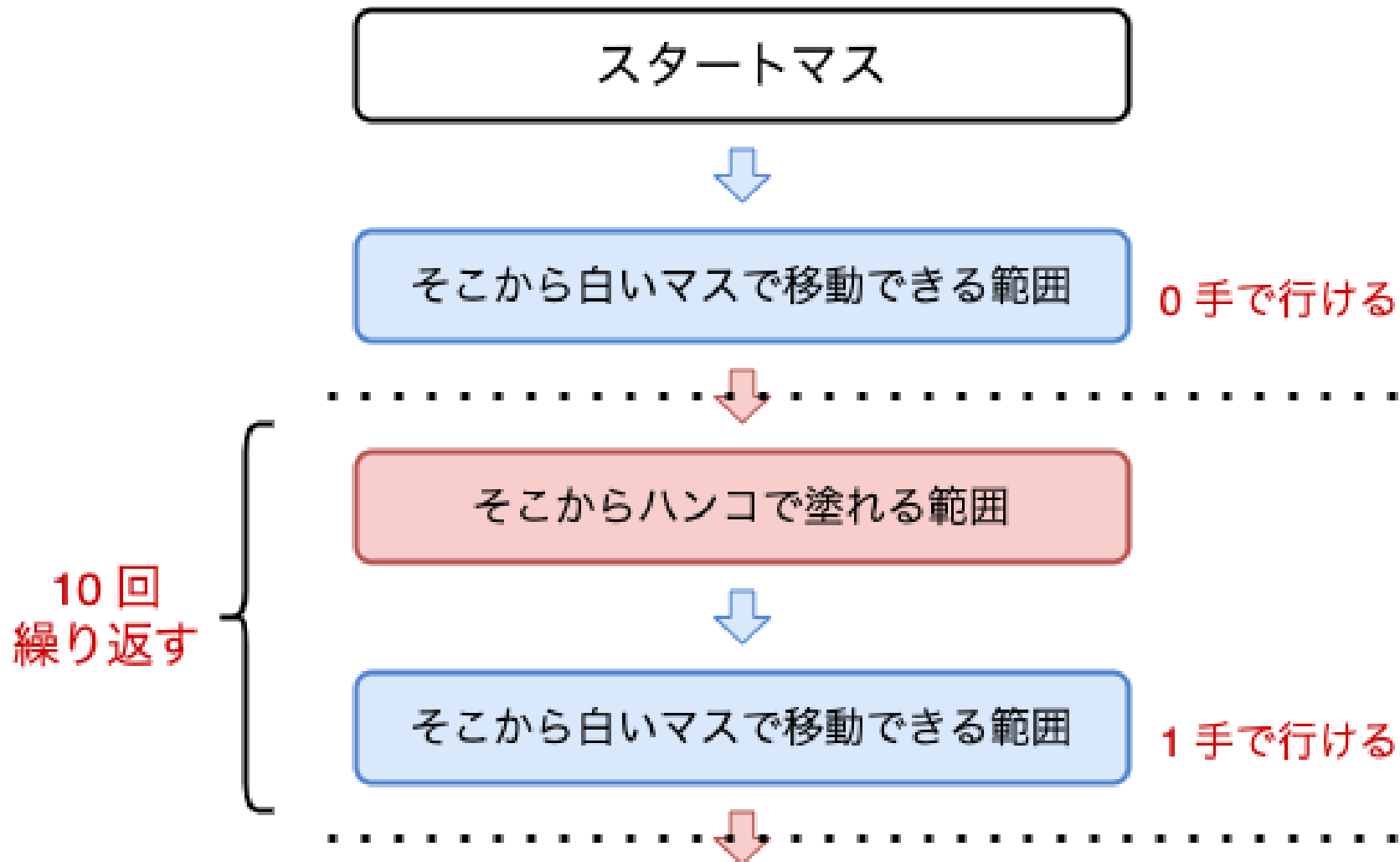
考察④ x 手で 行ける範囲

$ANS \leq 10$ なら、
10 回繰り返せば答
えがわかる



考察④ x 手で 行ける範囲

$ANS \leq 10$ なら、
10 回繰り返せば答
えがわかる



考察④ x 手で行ける範囲

そこから白いマスで移動できる範囲は **BFS** で求められる.

そこからハンコで塗れる範囲は全探索だと遅いが、どうやって求める？

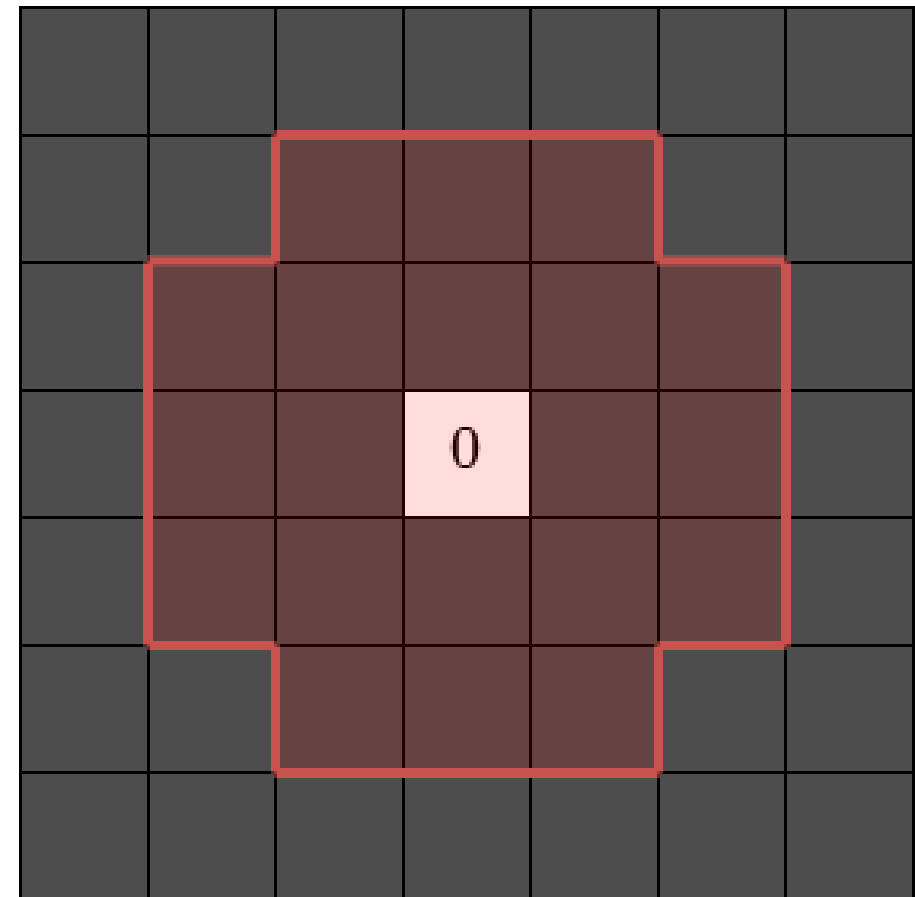


そこからハンコで塗れる範囲を求める

- 1 つ前の範囲に入っている各マスから右図の形を伸ばして、それらの和集合を取りたい.

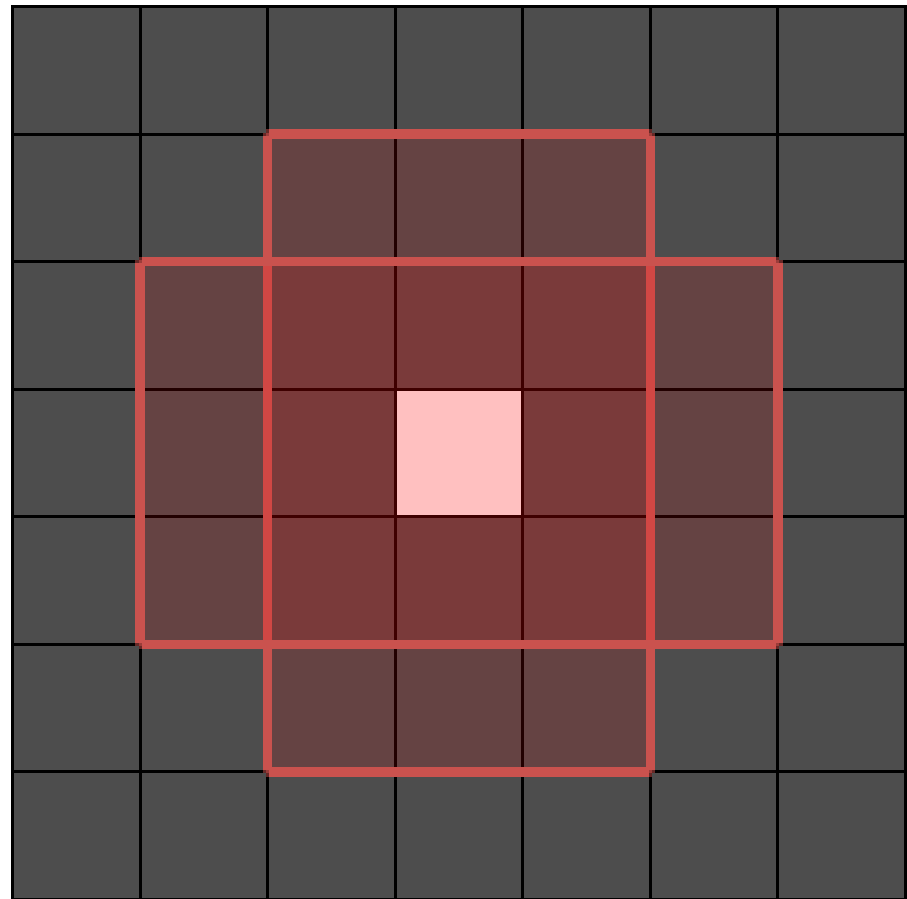
範囲加算・1 点取得 これは...

いわゆる いもす法！



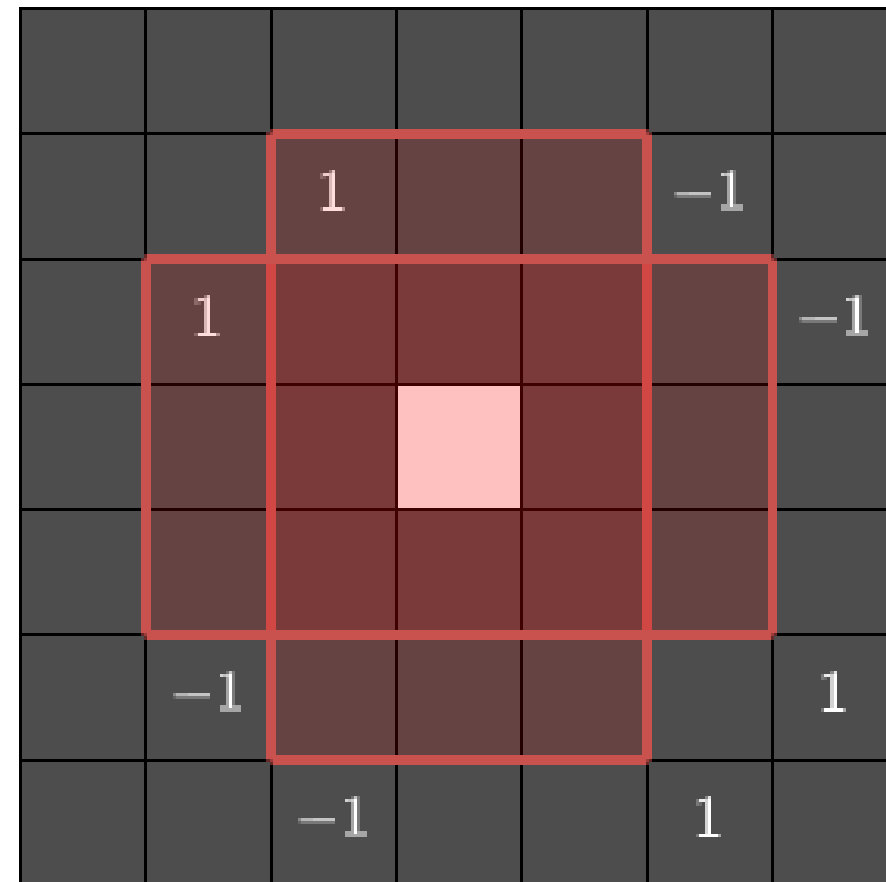
いもす法

- 正方形から四隅を抜いた形 を
 $(2N + 1) \times (2N - 1)$ マスの長方形と
 $(2N - 1) \times (2N + 1)$ マスの長方形に分
解
- 和集合を取るなので, 多く加算しても良い



いもす法

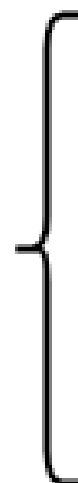
- 正方形から四隅を抜いた形 を
 $(2N + 1) \times (2N - 1)$ マスの長方形と
 $(2N - 1) \times (2N + 1)$ マスの長方形に分
 解
- 和集合を取るので、多く加算しても良い
- ± 1 を配置し、後から 2 次元累積和
- 値が正になった部分が、**そこからハンコで塗れる範囲** である.



小課題 3 の解法 まとめ

$O(RCANS)$ で解
ける.

10 回
繰り返す



スタートマス

↓ BFS

そこから白いマスで移動できる範囲

0 手で行ける

↓ 2 次元いもす法

そこからハンコで塗れる範囲

↓ BFS

そこから白いマスで移動できる範囲

1 手で行ける

↓ 2 次元いもす法

豆知識 : queue は空でも 512 Byte のメモリを食う

- `std::queue` や `std::stack` は別のコンテナ型を用いて動く型である。デフォルトでは `std::deque` を用いる。
- GCC では, `std::deque` は 512 Byte ごとのブロックでメモリ確保を行う。空でも 1 ブロック確保するため, `std::deque` は空でも 512 Byte のメモリを食う。

→ queue は `std::vector` で代用できる。

先頭要素を削除する代わりに 先頭の **index** を管理 する変数 `s` を持ち, `s++` で `pop` を行う。

注意

- ここから先は**考察要素の組み合わせ**によって様々な計算量が考えられます.
- **実装方針**による定数倍も様々あり, 計算量が同じでも様々な点数になることが予想されます.
 - 実際にグラフを構築して遅くなる
 - 01-BFS の代わりにダイクストラ法を使って $\log$ をつける
- したがって, **考察要素のみ**を紹介します.

小課題 4 (19 点)

- $R \times C \leq 6 \times 10^4$
- $N \leq R \leq C$ に注意すると, $N \leq \sqrt{RC} \leq \sqrt{6 \times 10^4} < 245$
- $O(RCN \log N)$ などの計算量で解けば良い.

小課題 5 (5 点)

- $R \times C \leq 1.5 \times 10^5$
- $O(RCN)$ や $O((RC)^{3/2})$ などの計算量で解けば良い.

小課題 6 (19 点)

- $R \times C \leq 1.5 \times 10^6$
- $O(RC(\log N)^2)$ や $O((RC)^{4/3})$ などの計算量で解けば良い.

小課題 7 (8 点)

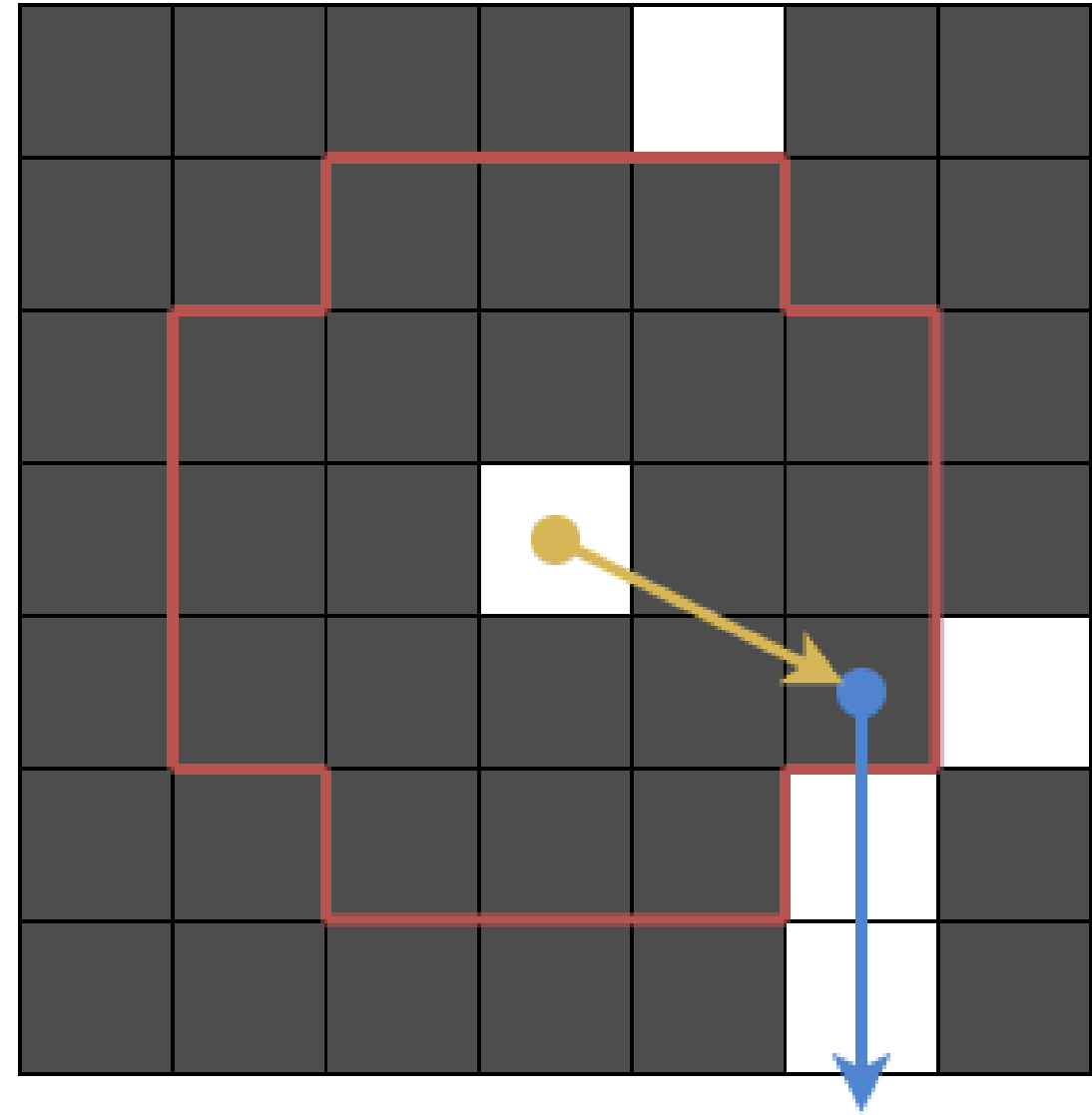
- $R \times C \leq 3 \times 10^6$
- $O(RC \log N)$ などの計算量で解けば良い.

小課題 8 (6 点)

- $R \times C \leq 6 \times 10^6$
- $O(RC)$ などの計算量で解けば良い.

考察⑤ 外側しか遷移しなくて良い

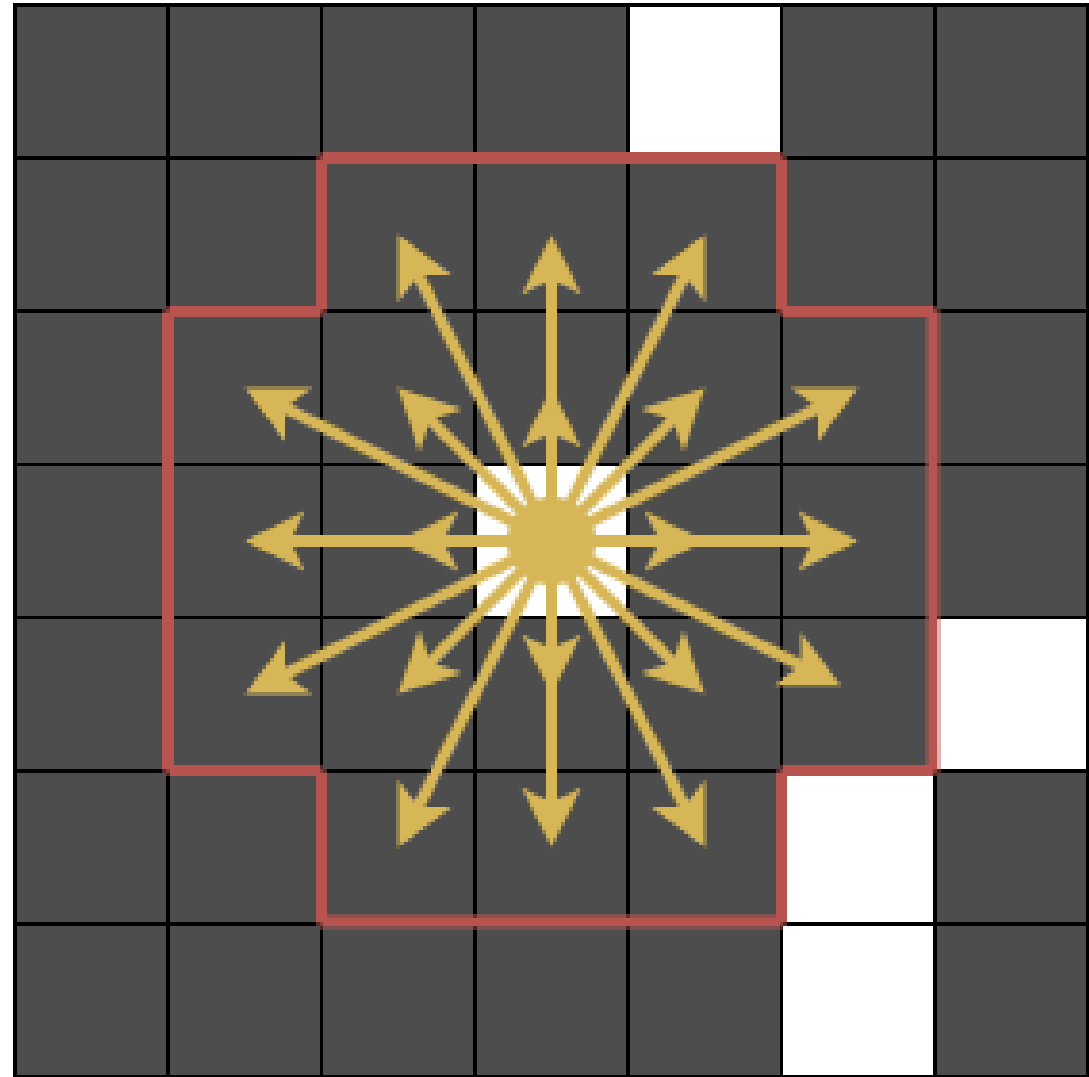
小課題 2 では、**ハンコで塗れる範囲** を 1 回までしか出ないので、ハンコをワープに言い換えていた.



考察⑤ 外側しか遷移しなくて良い

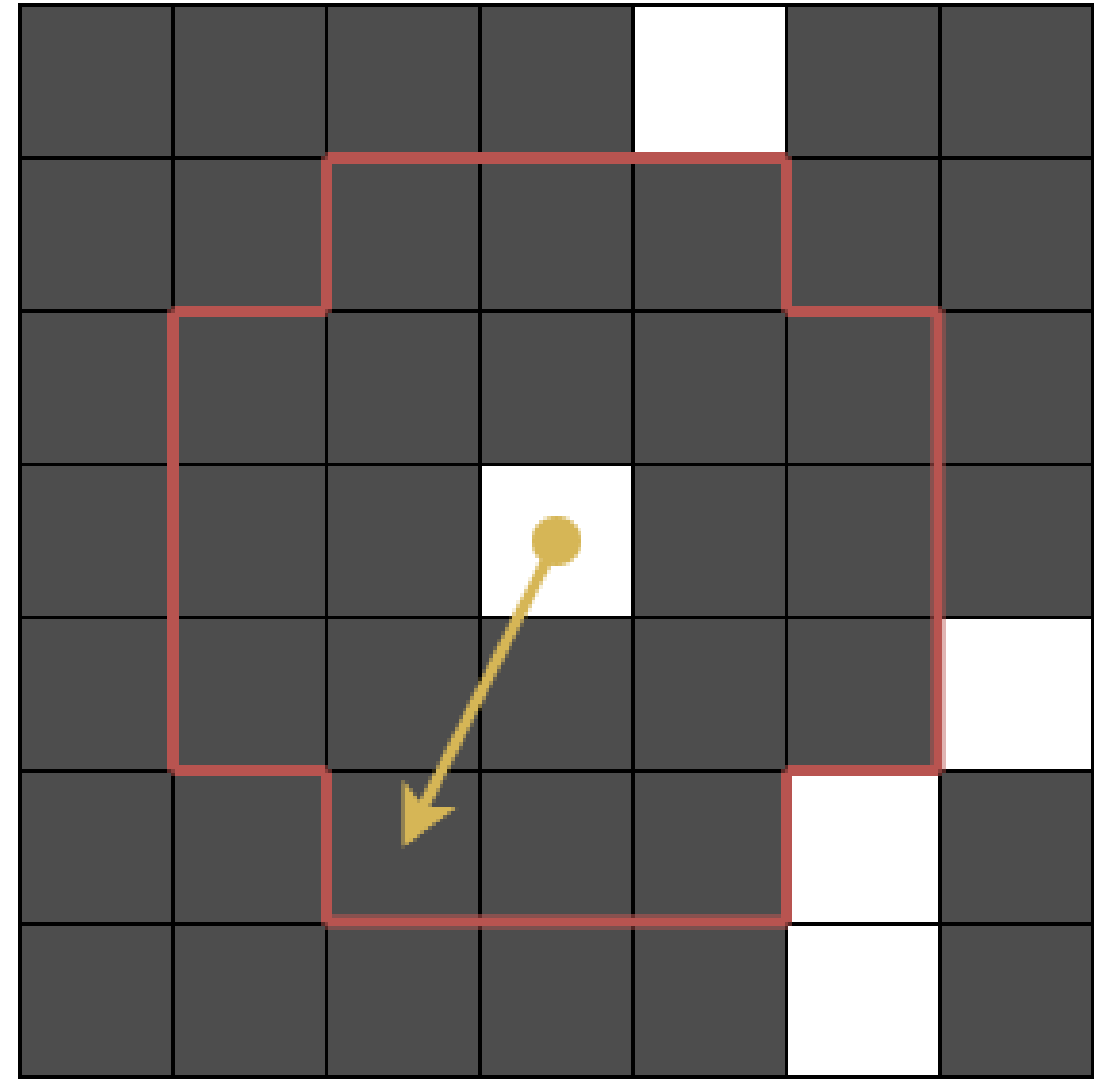
小課題 2 では、**ハンコで塗れる範囲** を 1 回までしか出ないので、ハンコをワープに言い換えていた.

また、各マスから **ハンコで塗れる範囲** 全てにコスト 1 の辺を張っていた.



考察⑤ 外側しか遷移しなくて良い

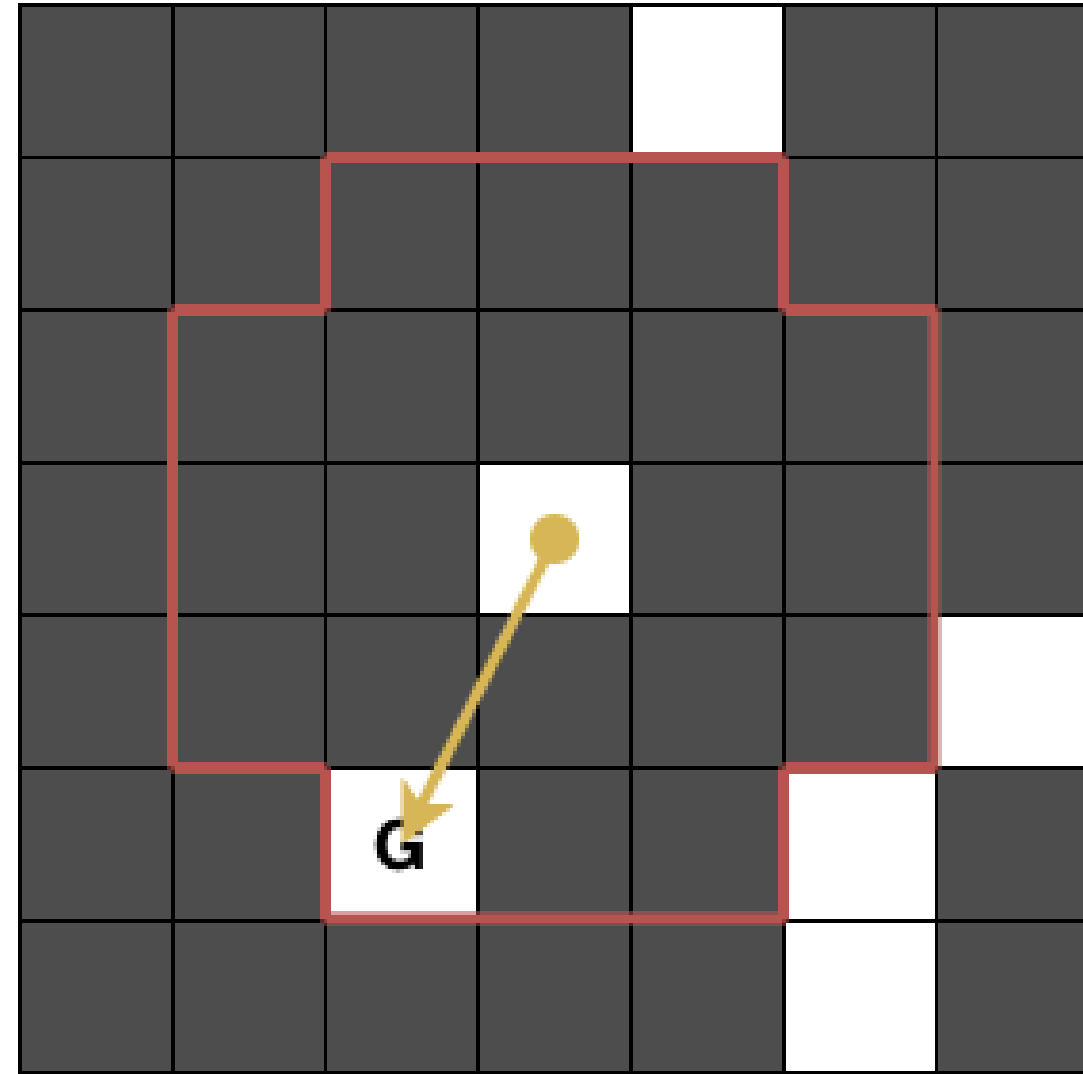
Q. ハンコで塗れる範囲を 1 回も出ないと
きってどんなとき？



考察⑤ 外側しか遷移しなくて良い

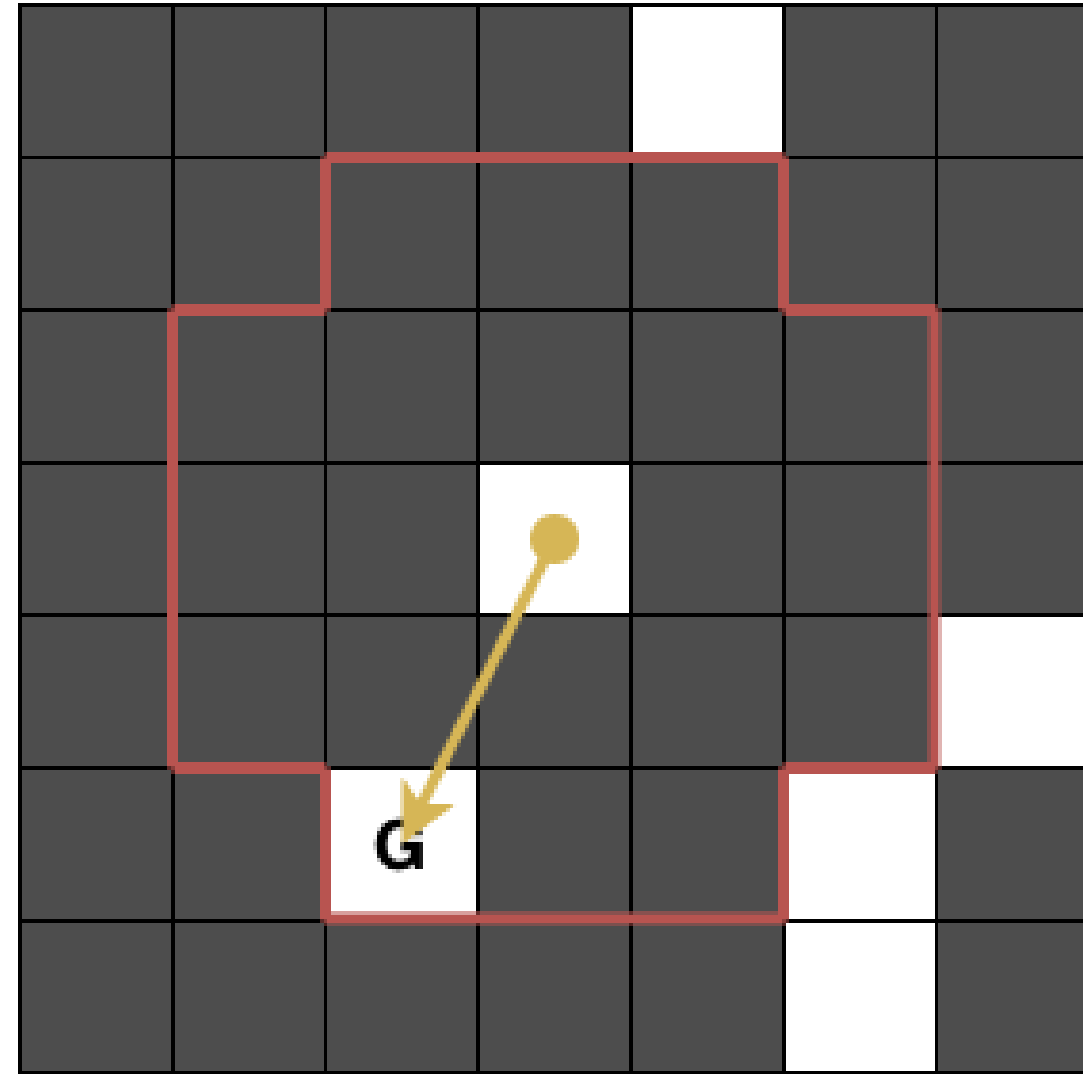
Q. ハンコで塗れる範囲を 1 回も出ないときってどんなとき？

A. ハンコで塗れる範囲の中にゴールがあるとき



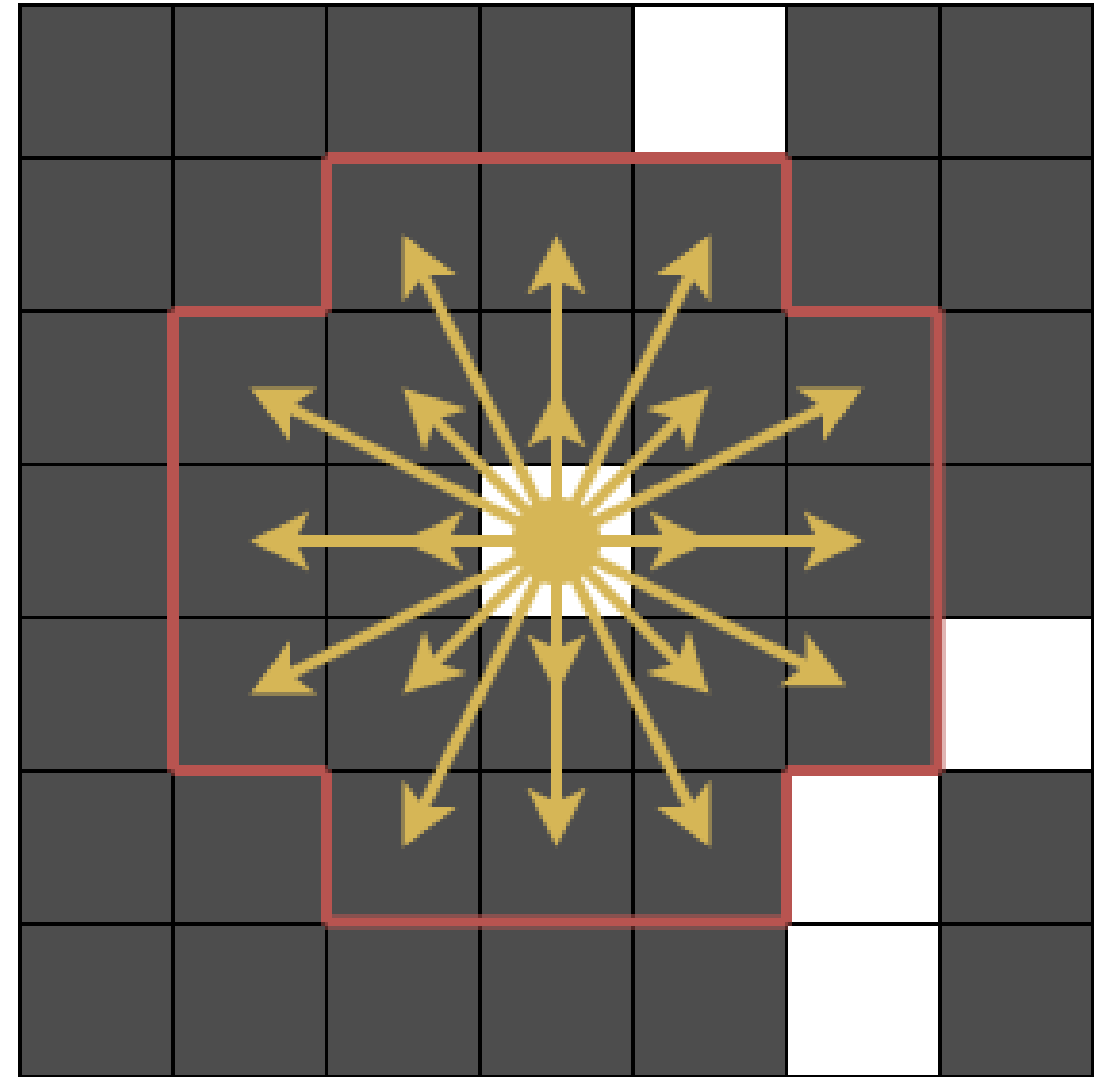
考察⑤ 外側しか遷移しなくて良い

- ハンコで塗れる範囲の中にゴールがあるとき、ゴールにワープできて、終了
- ハンコで塗れる範囲の外にゴールがあるとき、絶対にハンコで塗れる範囲を出なければならない



考察⑤ 外側しか遷移しなくて良い

ということは、**ハンコで塗れる範囲** 全部
に辺を張る必要はなくて...

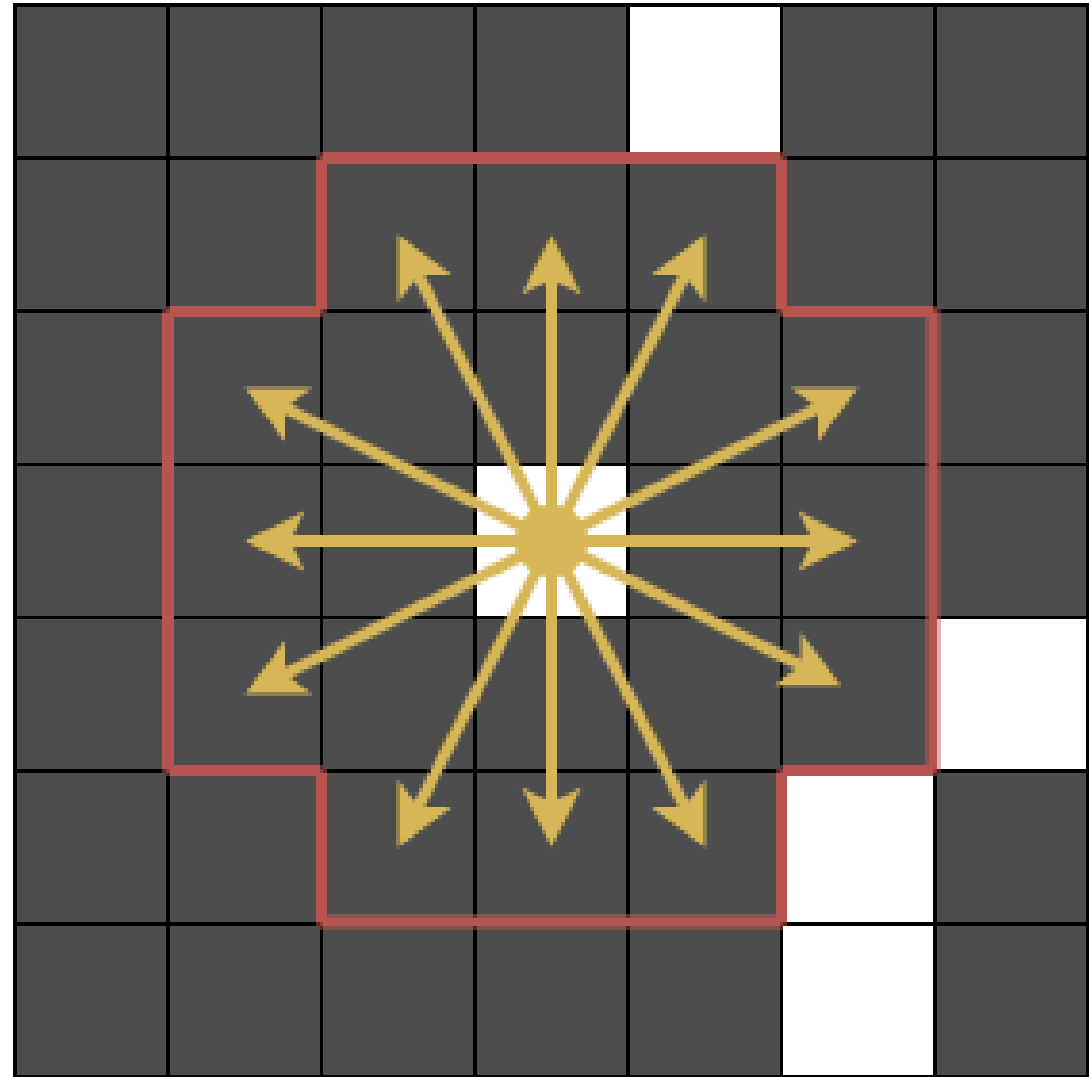


考察⑤ 外側しか遷移しなくて良い

ということは、**ハンコで塗れる範囲** 全部に辺を張る必要はなく、

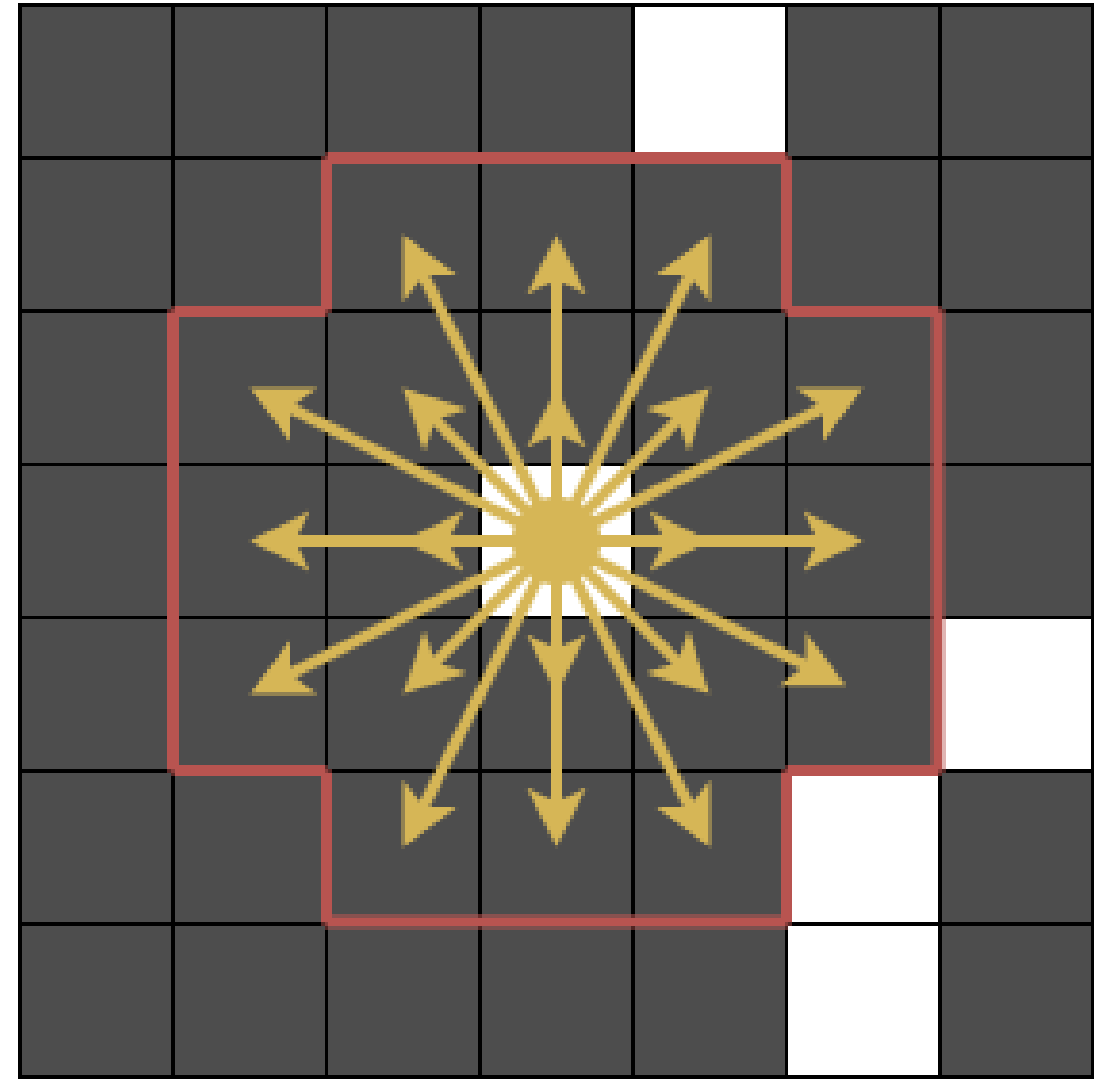
最も外側の $4 \times (2N - 1)$ マスにのみ辺を張れば良い！

※ **範囲内** にゴールがあればゴールへの辺も必要



考察⑤ 外側しか遷移しなくて良い

- 辺の個数が $O(RCN)$ 本になったので, 01-BFS で $O(RCN)$ 時間



考察⑥ 解法を組み合わせる

- 小課題 3 の計算量は $O(RCANS)$ だった.
- N が大きくなるほど, 1 度に広い範囲を塗れるので, ANS は小さくなる
- 具体的には,

$$ANS \leq \lceil \max(\frac{R+C}{2N-1}, \frac{C}{N}) \rceil \leq \frac{2C}{N} + 1$$



考察⑥ 解法を組み合わせる

- ANS は N に反比例して小さくなる
- 一方, $O(RCN^2)$ や $O(RCN)$ などの解法は N に比例して時間がかかる

→ 解法を組み合わせよう！

考察⑥ 解法を組み合わせる

- $O(RCN^2)$ が間に合うときは小課題 2 の解法 $O(RCN^2)$ で、そうでないときは小課題 3 の解法 $O(RCANS)$ で解く
- $O(RC \min(N^2, ANS))$ 時間
- $ANS \leq \frac{2C}{N} + 1$ に注意すると, $O((RC)^{3/2})$ 時間であることが分かる

考察⑥ 解法を組み合わせる

- $O(RCN)$ が間に合うときは外側しか遷移しない解法 $O(RCN)$ で、そうでないときは小課題 3 の解法 $O(RCANS)$ で解く
- $O(RC \min(N, ANS))$ 時間
- $ANS \leq \frac{2C}{N} + 1$ に注意すると, $O((RC)^{4/3})$ 時間であることが分かる



セグ木の形にして区間に辺を張るテク

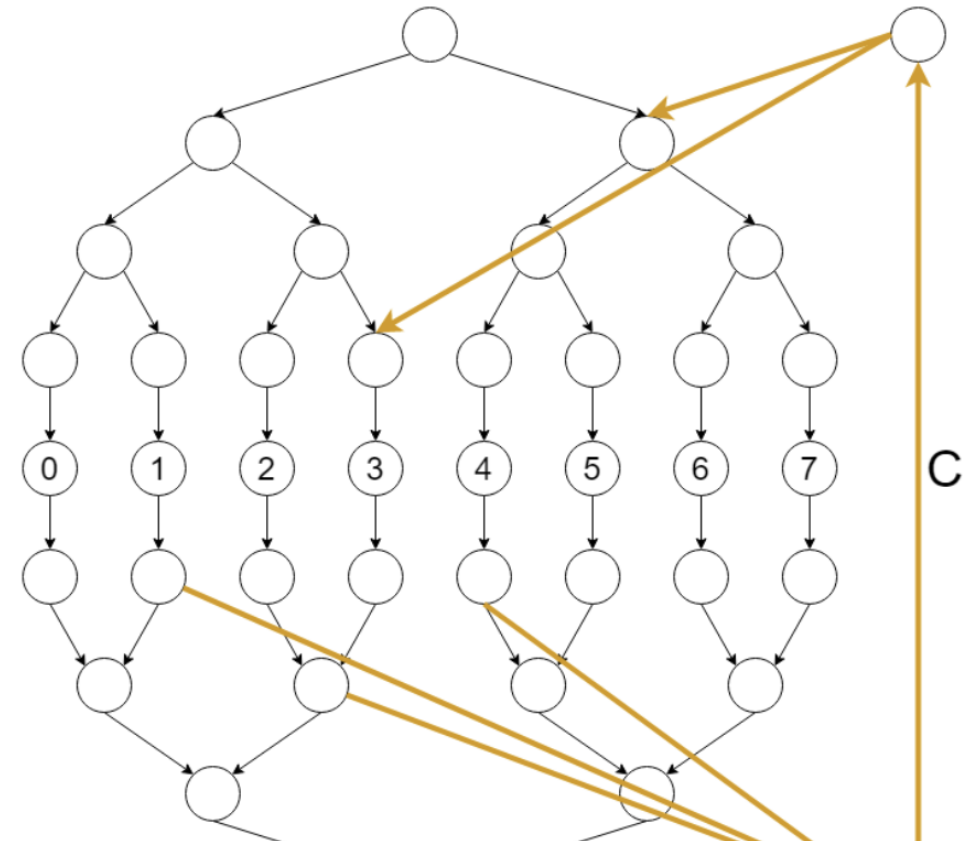
頂点 $+N$ 個

辺 $+N + E \log N$ 個

区間に辺を張るテクニック

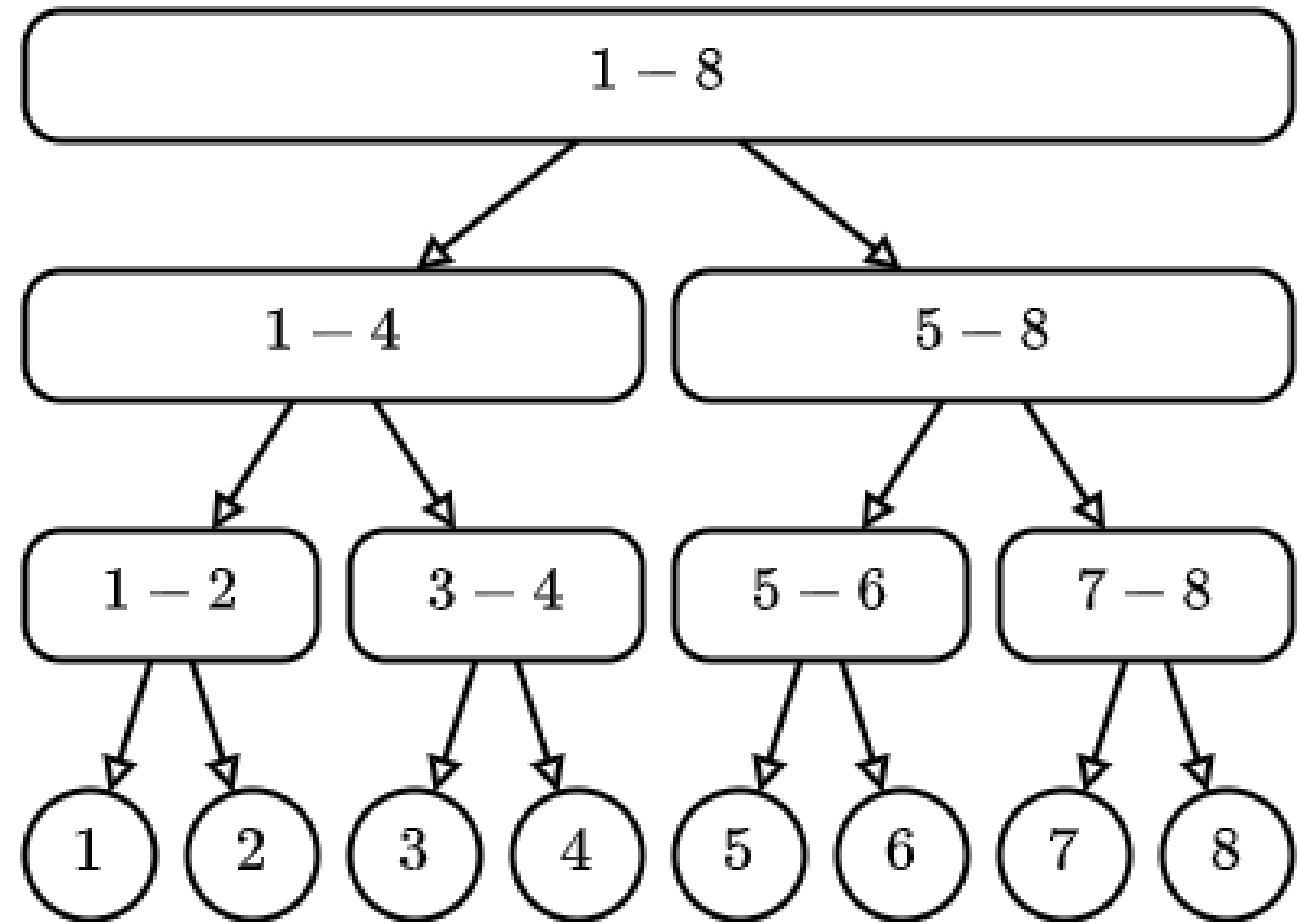
セグメント木状に頂点を増やしておく
と、長さ $2N + 1$ の区間に辺を張るのに
 $2N + 1$ 本必要だったのが、 $2 \log_2 N$
本くらいで張れるようになる

[1, 4] から [3, 7] にコスト C の辺を張る



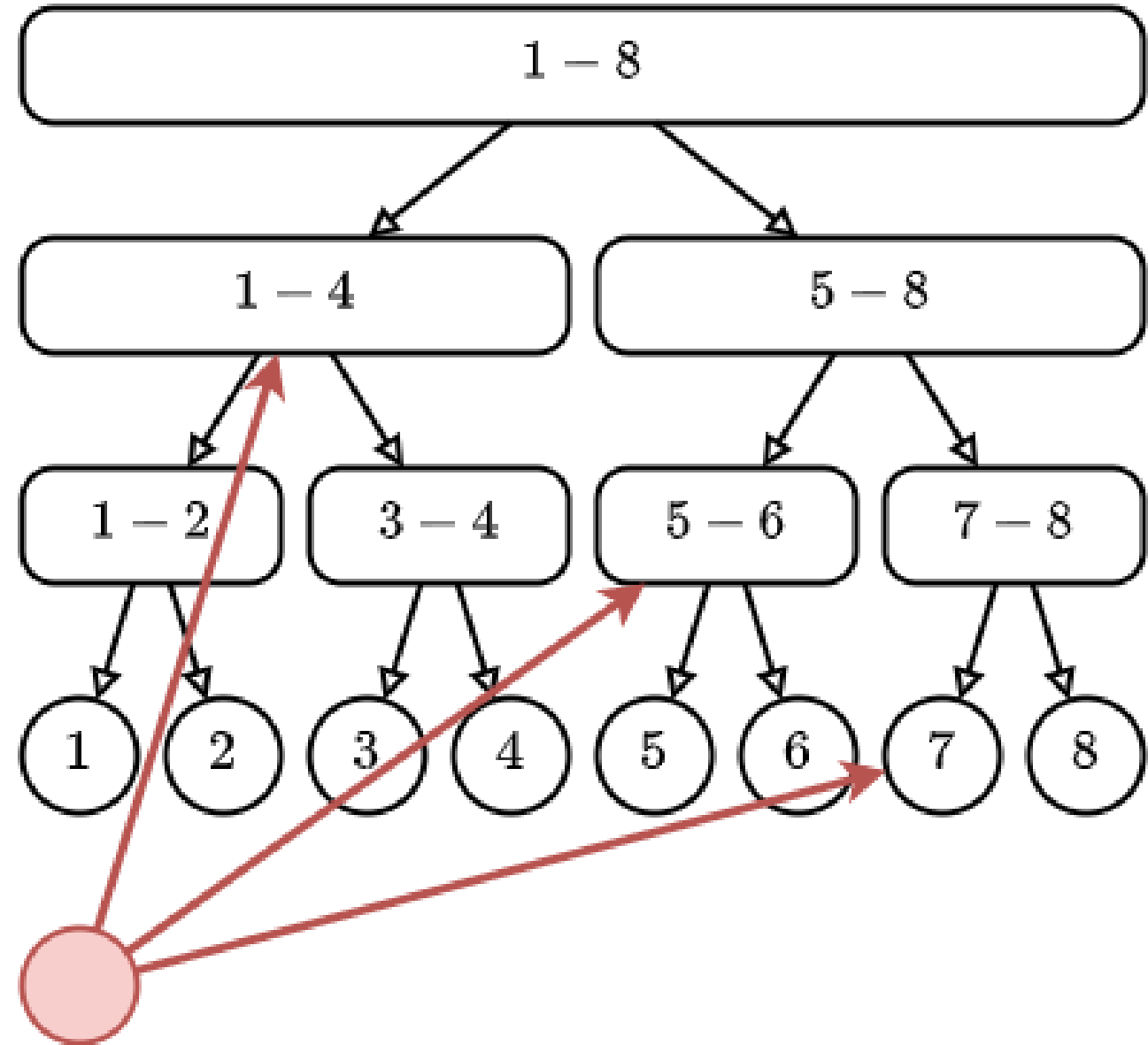
区間に辺を張るテクニック

- セグメント木状に頂点を用意し，下方方向にコスト 0 の辺を張る



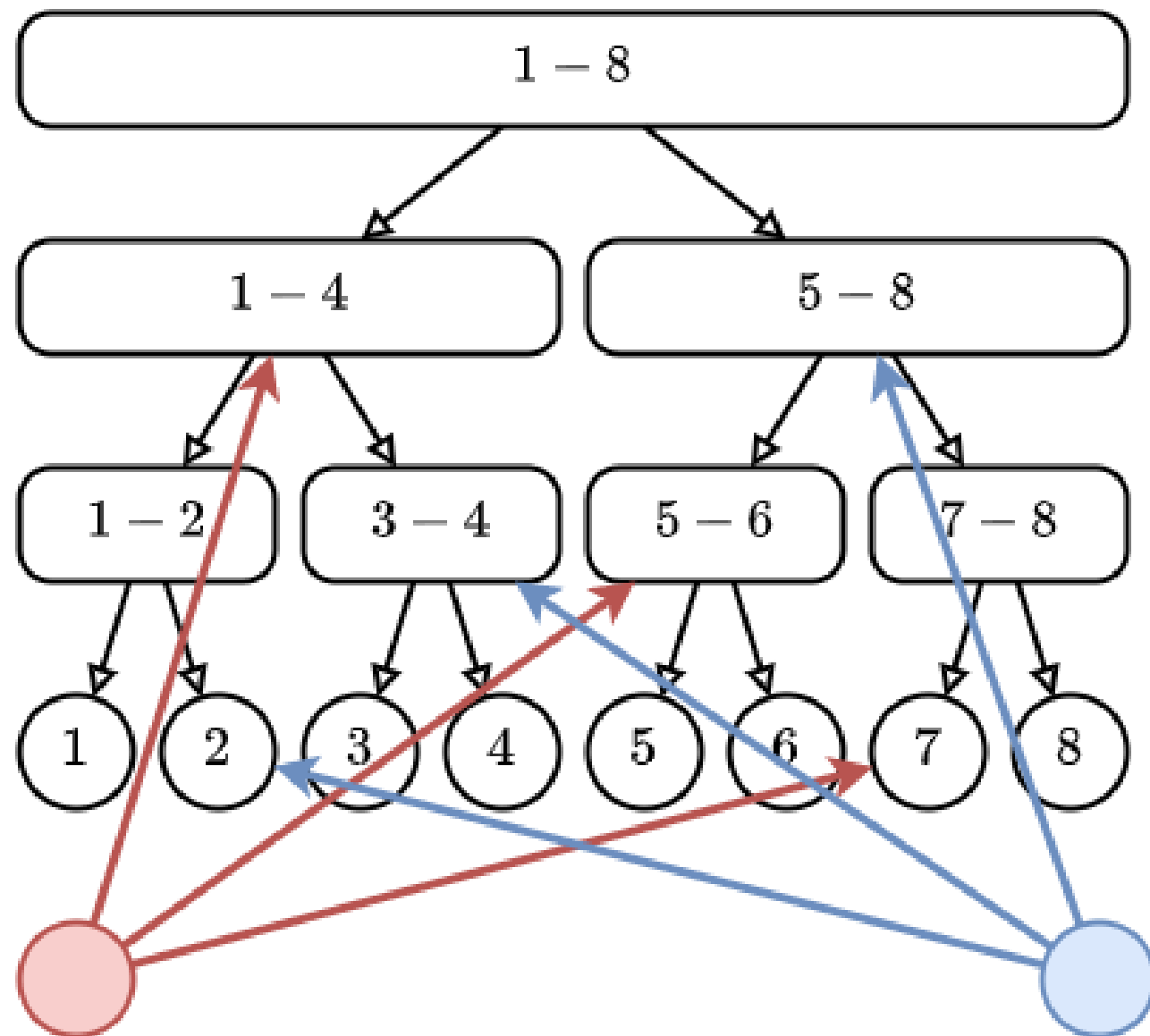
区間に辺を張るテクニック

- セグメント木状に頂点を用意し，下方方向にコスト 0 の辺を張る
- 区間 $[1, 7]$ にコスト 1 の辺を張る



区間に辺を張るテクニック

- セグメント木状に頂点を用意し，下方方向にコスト 0 の辺を張る
- 区間 $[1, 7]$ にコスト 1 の辺を張る
- 区間 $[2, 8]$ にコスト 1 の辺を張る



区間に辺を張るテクニック

- 小課題 2 の $O(RCN^2)$ 時間
→ $O(RCN \log N)$ 時間に
- 外側にしか辺を張らなくていい $O(RCN)$ 時間
→ $O(RC \log N)$ 時間に (定数倍によっては満点を取れる)

注意: グラフを実際に構築すると遅い

豆知識：簡単なセグ木の作り方

長さ $2n$ のセグ木にする
とよい

出典：非再帰セグ木
サイコー！ 一番す
きなセグ木です
p.35

別の話
○○○○○

入門編
○○○○○○○○○○○○○○●○○○

応用編
○○○○○○○○○○○○○○○

おわり
○○○

驚愕の事実？

上のコードは要素数 n を二ベキにしなくても動く。
 $O(\log n)$ 個の完全二分木のセグ木が存在していると思わせる。

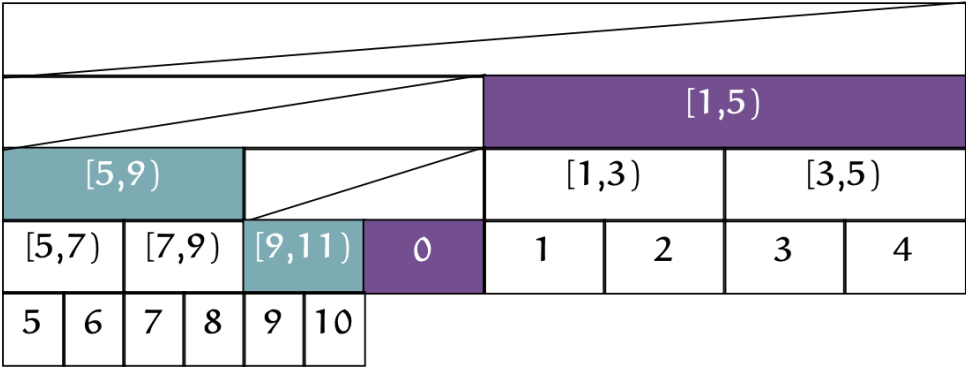
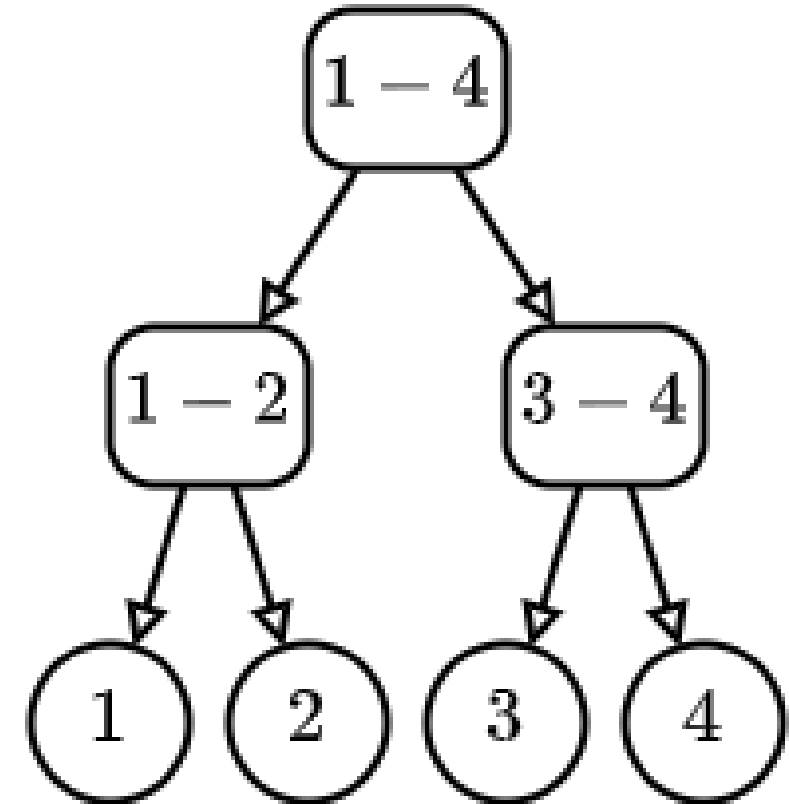


Figure: 11 要素の配列に対応するセグ木

区間に辺を張るテクニック②

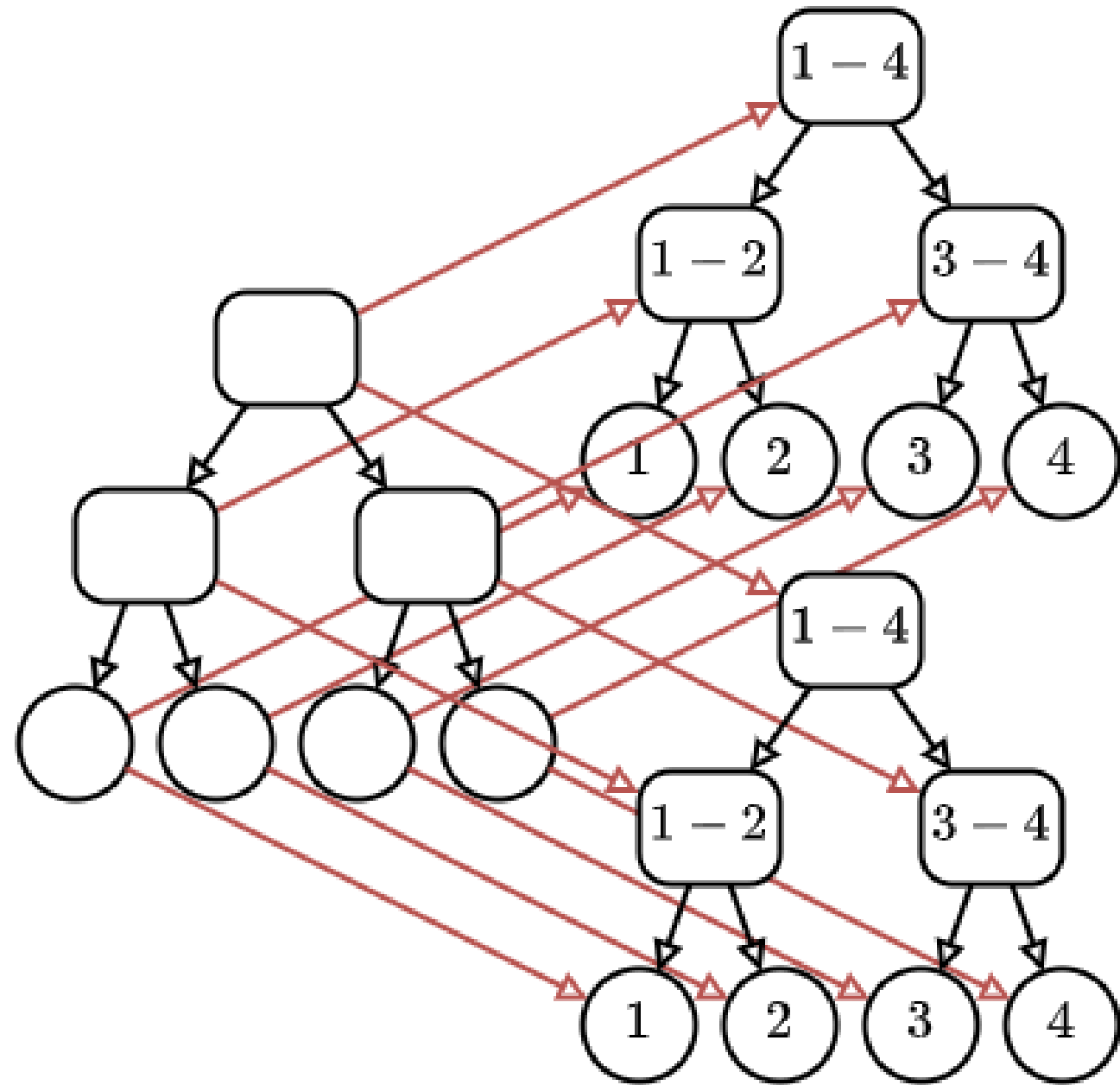
- 1次元セグ木でできるなら, 2次元セグ木でもできるよね



区間に辺を張るテクニック

②

- 1次元セグ木でできるなら, 2次元セグ木でもできるよね



区間に辺を張るテクニック②

- 2次元セグ木を使って長方形に辺を張る
- 小課題 2 の $O(RCN^2)$ 時間
→ $O(RC(\log N)^2)$ 時間に (小課題 6 まで取れる)

注意: グラフを実際に構築すると遅い

区間に辺を張るテクニク③

- 列を N 個くらいのブロックに区切って、各ブロック内で右側方向に繋げる頂点と、左側方向に繋げる頂点を作る.

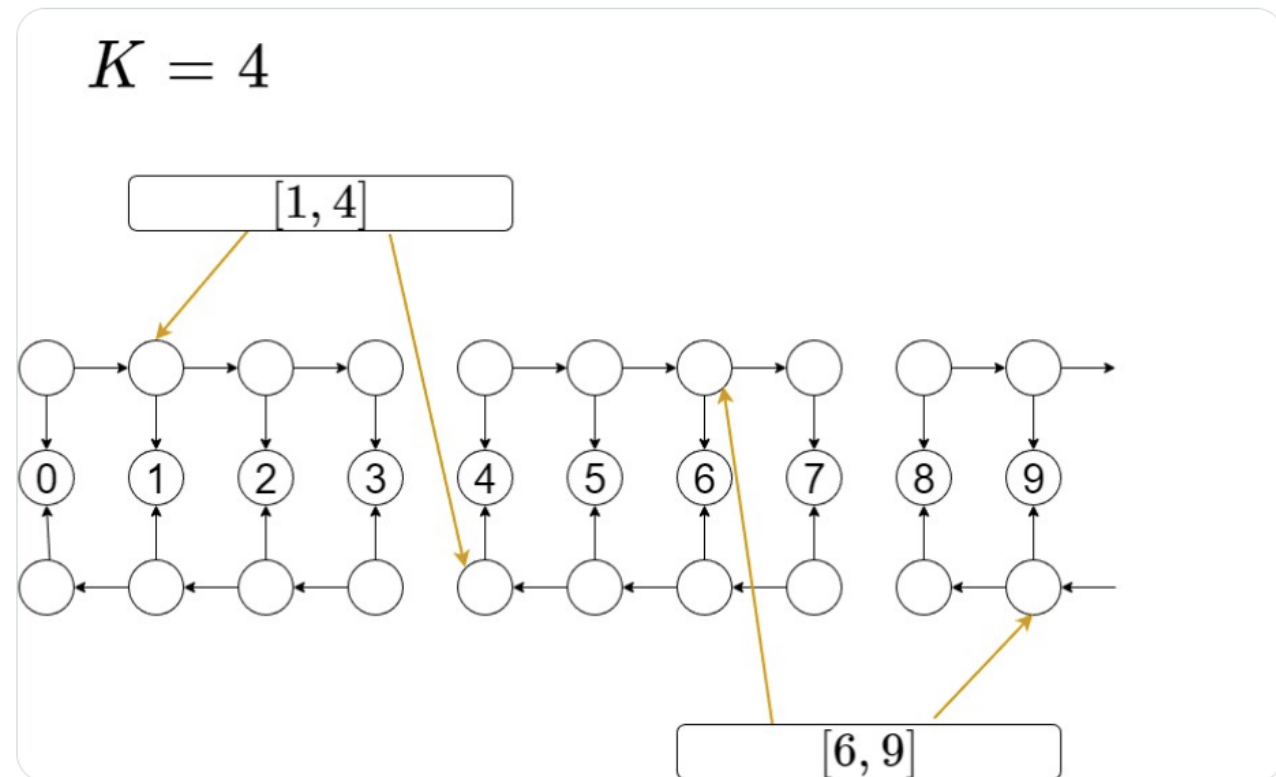


熨斗袋

@noshi91 · Follow

区間に辺を張るテク②

区間の幅が K で固定なとき、 $\log(N)$ を付けずに張ることが出来る



1:22 PM · Jun 15, 2020



区間に辺を張るテクニック③

- 列を N 個くらいのブロックに区切って、各ブロック内で右側方向に繋げる頂点と、左側方向に繋げる頂点を作る.
- 小課題 2 の $O(RCN^2)$ 時間
→ $O(RCN)$ 時間に
- 外側にしか辺を張らなくていい $O(RCN)$ 時間
→ $O(RC)$ 時間に

区間に辺を張るテクニク④

- こっちも同様に 2 次元にできる
- $N \times N$ マスくらいのブロックに区切って、各ブロック内で右上方向、右下方向、左上方向、左下方向に繋げる頂点を作る

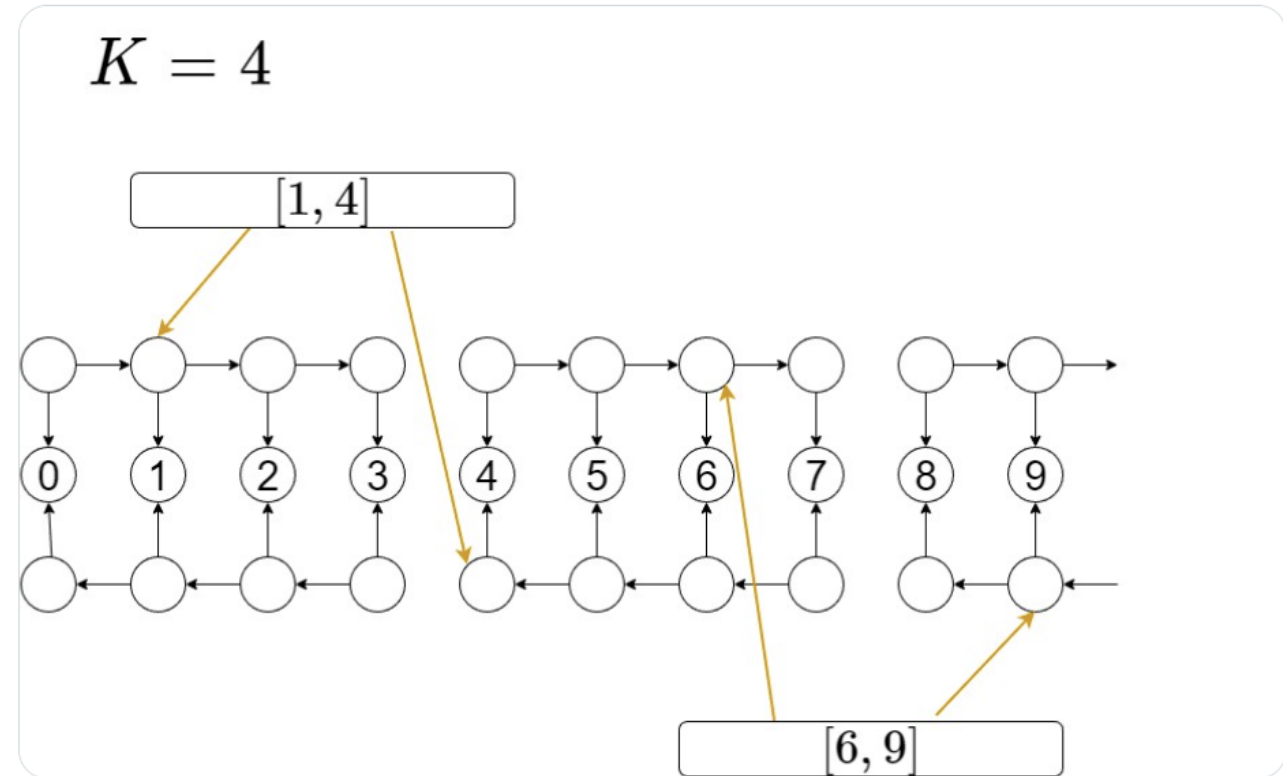


熨斗袋

@noshi91 · Follow

区間に辺を張るテク②

区間の幅が K で固定なとき、 $\log(N)$ を付けずに張ることが出来る



1:22 PM · Jun 15, 2020



区間に辺を張るテクニック④

- $N \times N$ マスくらいのブロックに区切って、各ブロック内で右上方向、右下方向、左上方向、左下方向に繋げる頂点を作る
- 小課題 2 の $O(RCN^2)$ 時間
→ $O(RC)$ 時間に (定数倍が重いので、工夫をしないと小課題 7 までかも)

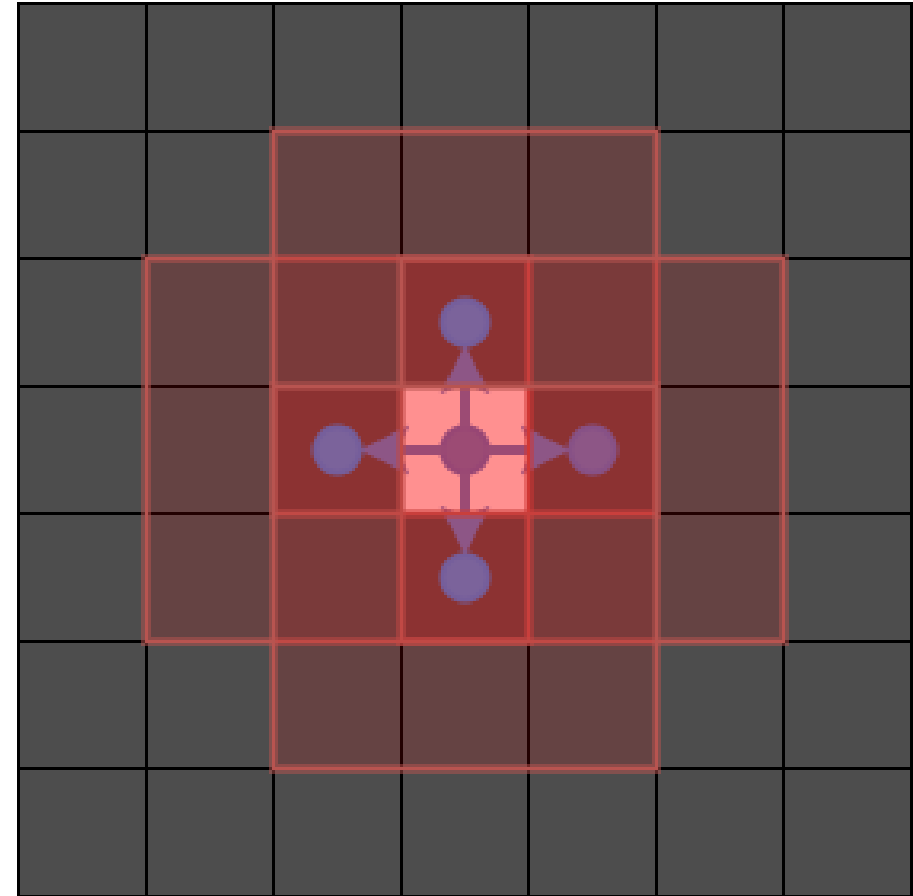
考察⑦ 3 段階に分ける

- 小課題 3 の解法は, $\Theta(RC)$ 時間の 2 次元いもす法を *ANS* 回繰り返すのがボトルネック
- これを全体で $\Theta(RC)$ にしたい.



考察⑦ 3 段階に分ける

(ハンコで塗れる範囲) = (隣接する黒マスを中心に $(2N - 1) \times (2N - 1)$ マスの範囲)



考察⑦ 3 段階に分ける

周囲 $(2N - 1) \times (2N - 1)$ マス
に広げた範囲を計算するには？

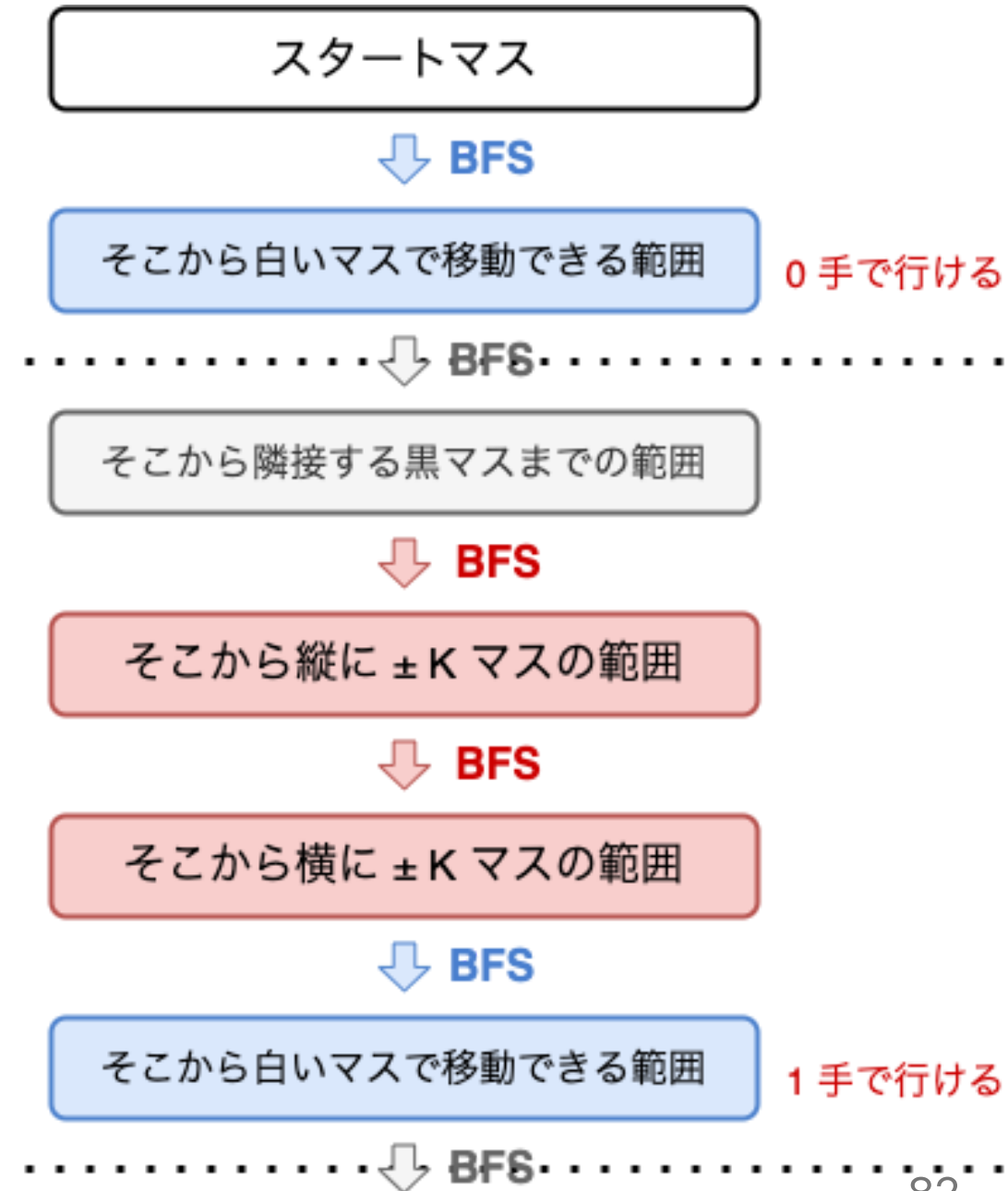


考察⑦ 3 段階に分ける

周囲 $(2N - 1) \times (2N - 1)$ マスに広げた範囲を計算するには？

→ 縦と横が独立なので、分けてやる

全部 BFS でできるので、 $O(RC)$ 時間で解けた！ 100



得点情報

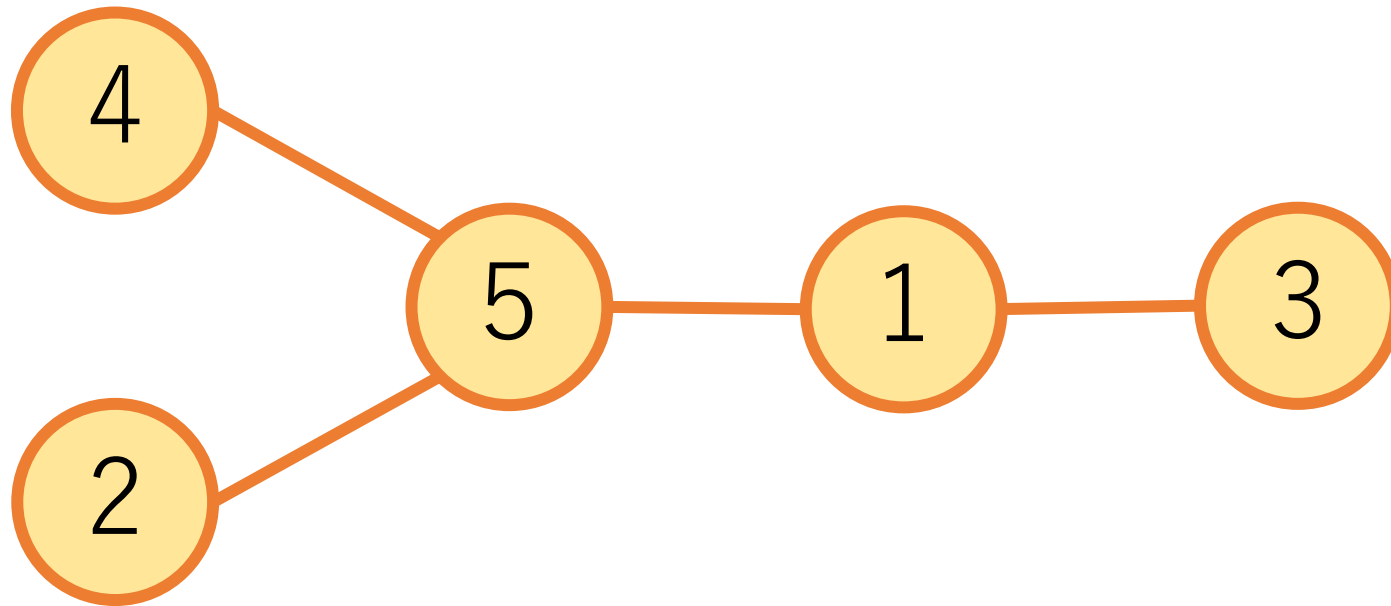
得点	小課題	人数	累積人数
<u>100</u>	12345678	8	8
94	1234567	4	12
86	123456	3	15
67	12345	2	17
62	1234	1	18
51	12 45	7	25
46	12 4	3	28
43	123	1	29
27	12	25	54
8	1	32	86

本選4 Cat Exercise

米山 瑛士 (ynymxiaolongbao)

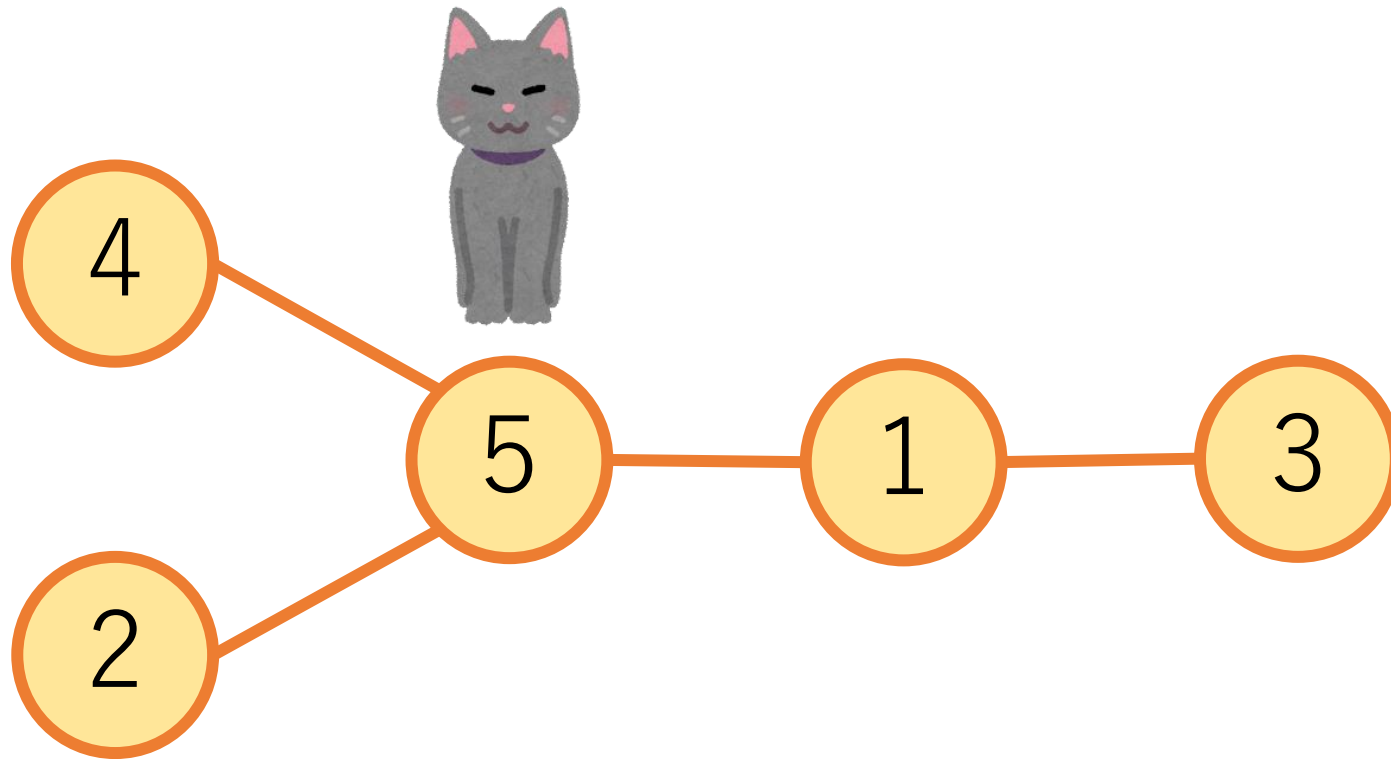
問題概要

- 木がある
- 各頂点は高さ $P[i]$



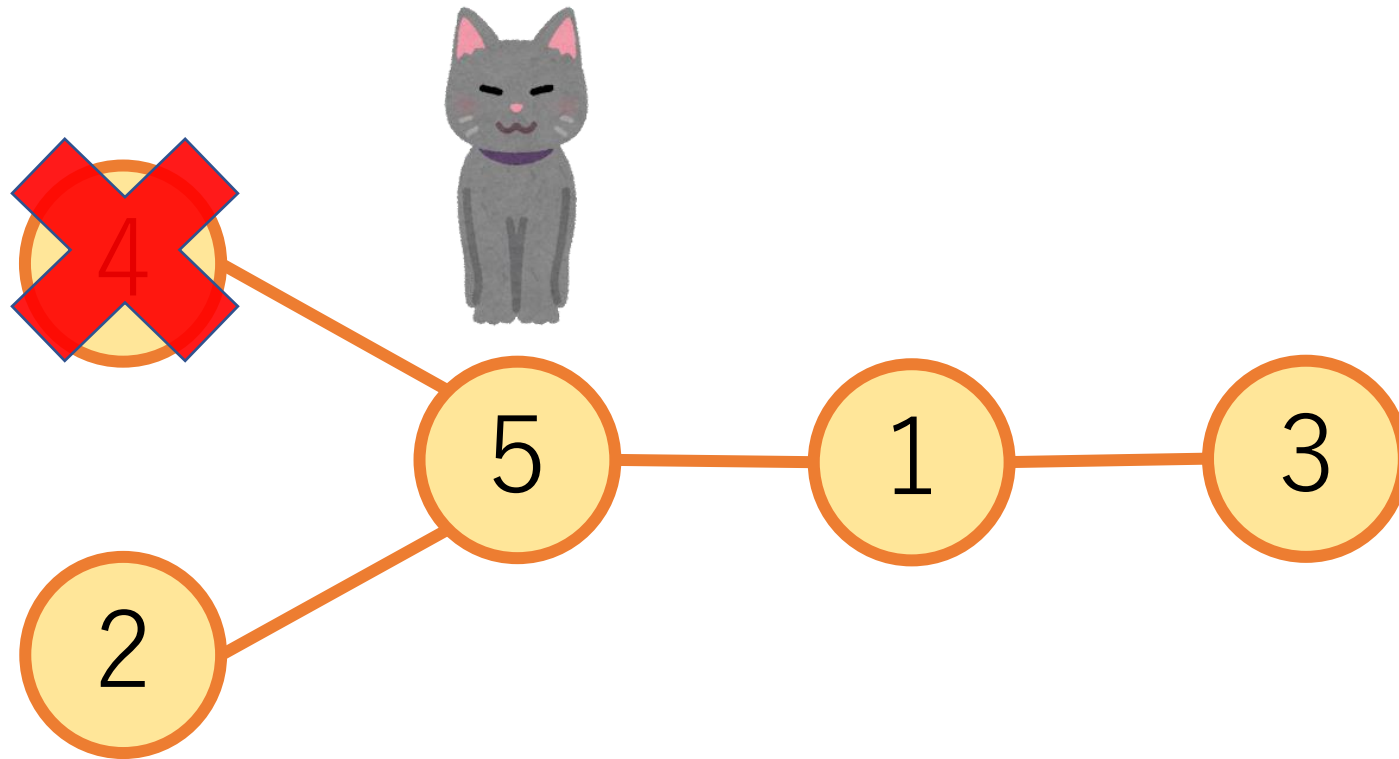
問題概要

- 猫がいる
- はじめ、最も高い頂点にいる



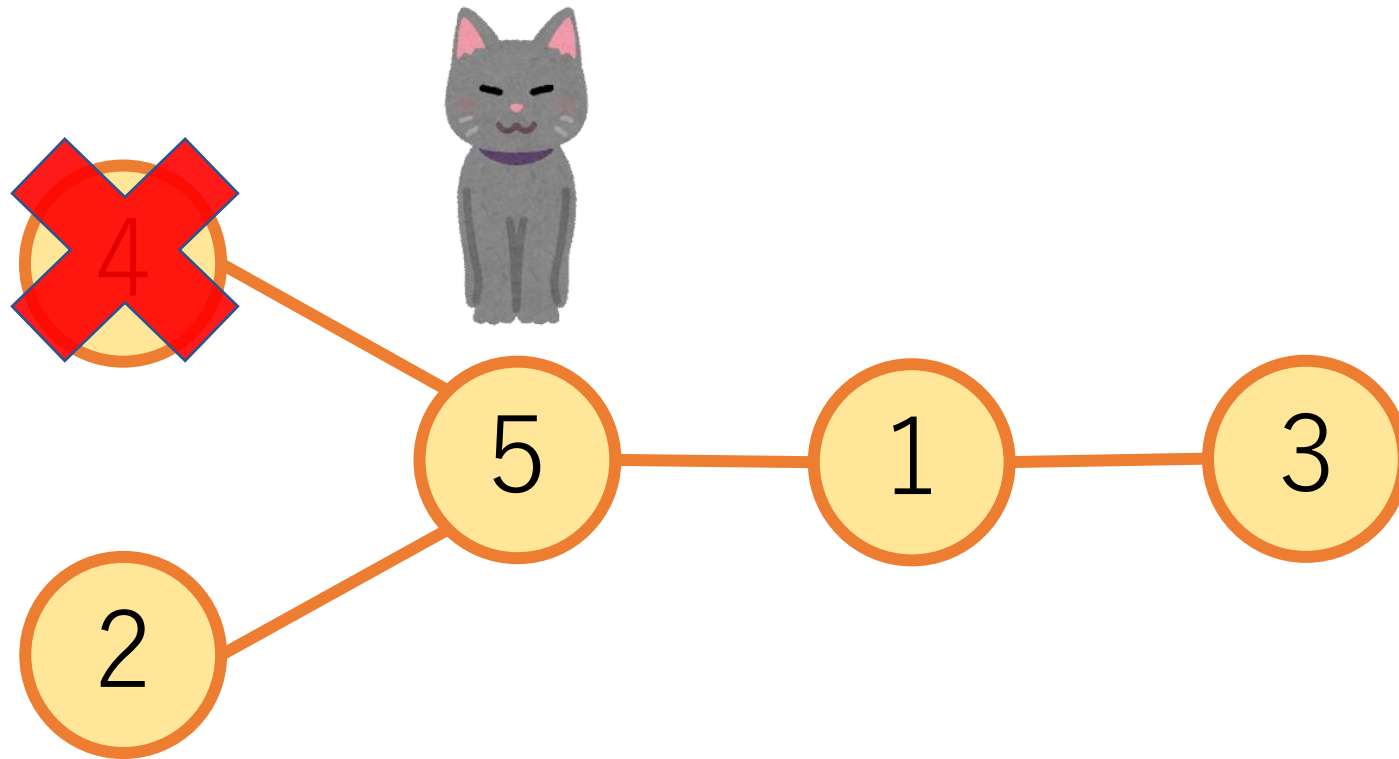
問題概要

- 一つの頂点を選んで障害物を置く



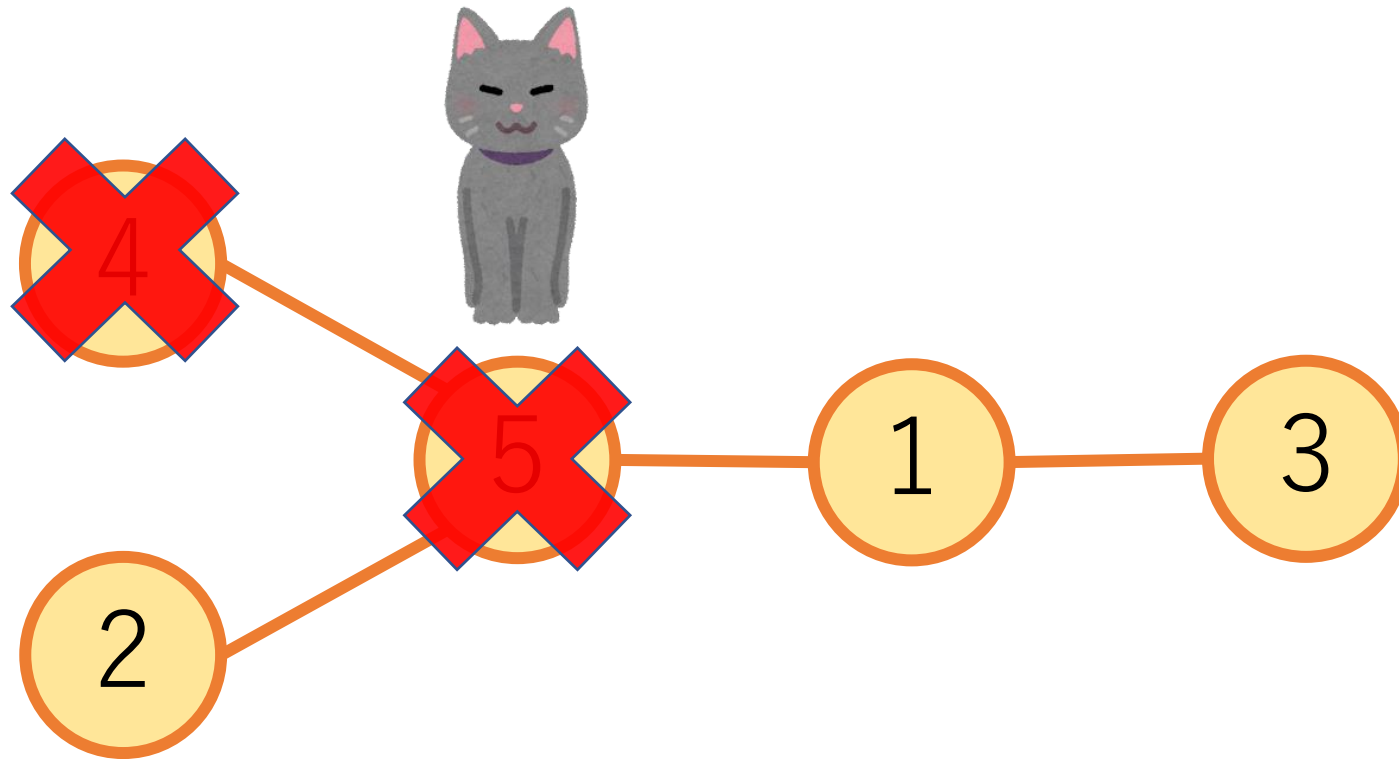
問題概要

- 選んだ頂点に猫がいなければ、何も起こらない



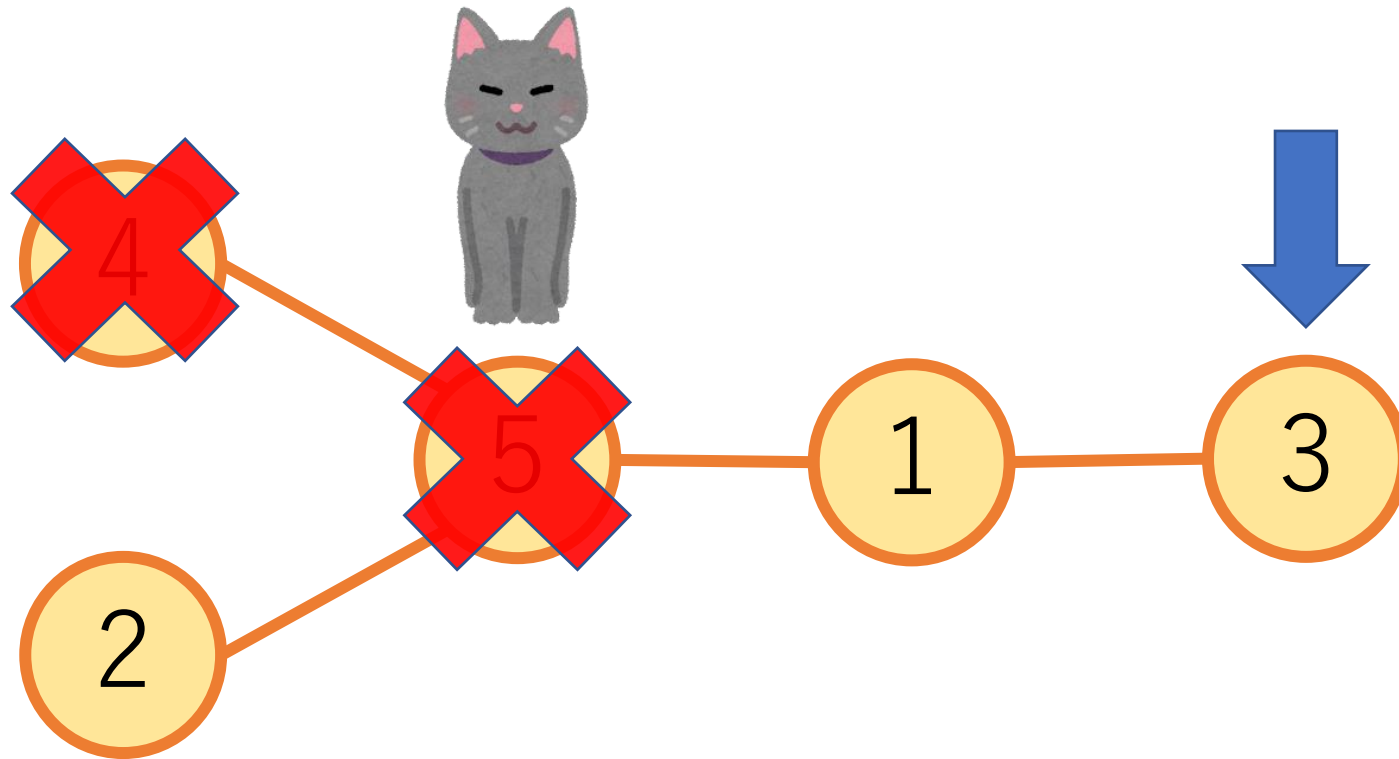
問題概要

- (再) 一つの頂点を選んで障害物を置く



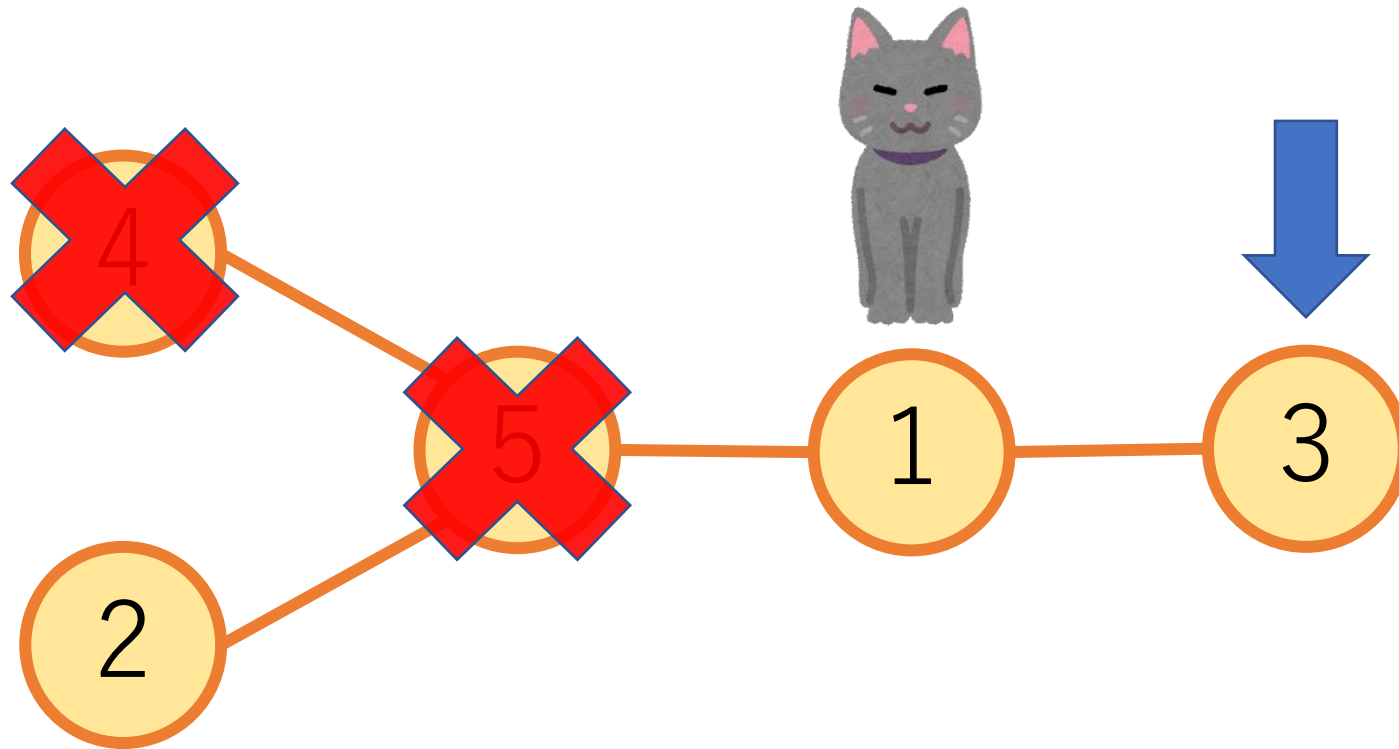
問題概要

- 選んだ頂点に猫がいた場合、その頂点から到達できる最も高い頂点に猫が移動する



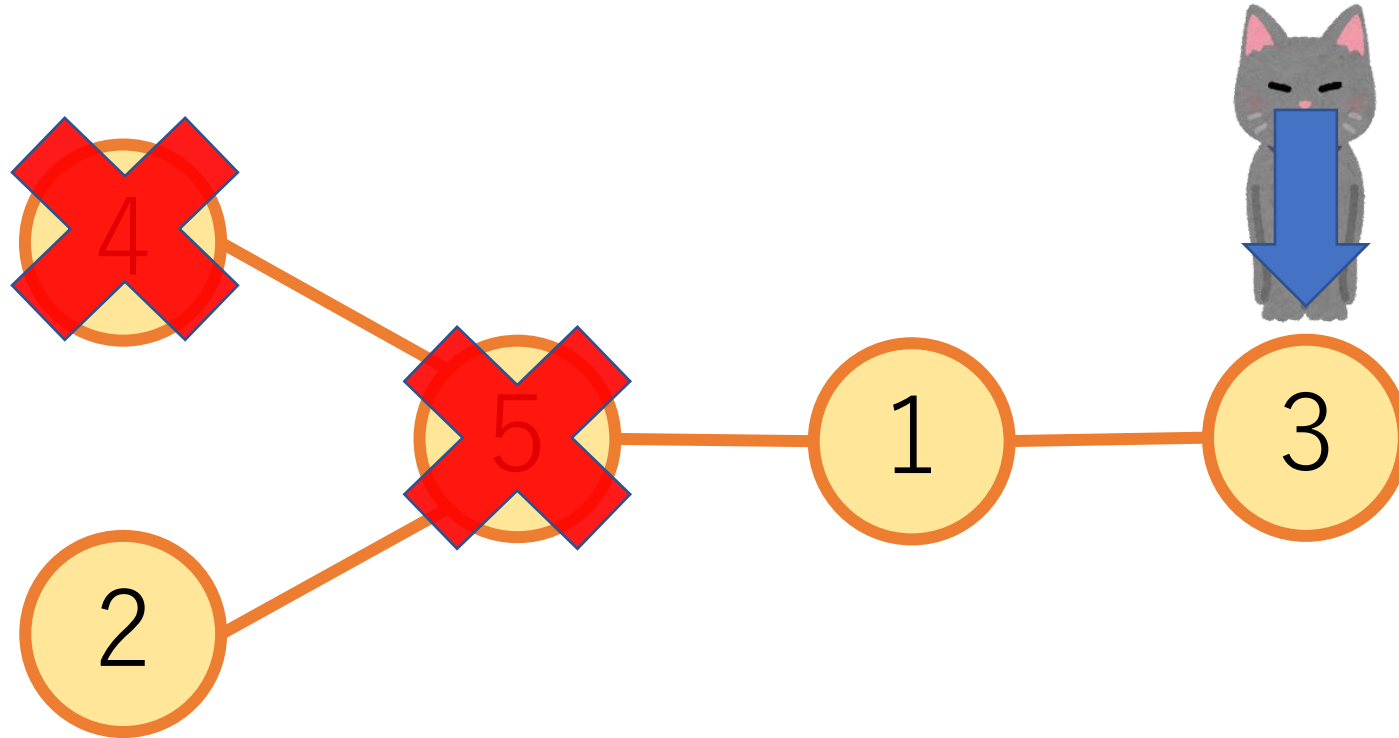
問題概要

- 選んだ頂点に猫がいた場合、その頂点から到達できる最も高い頂点に猫が移動する



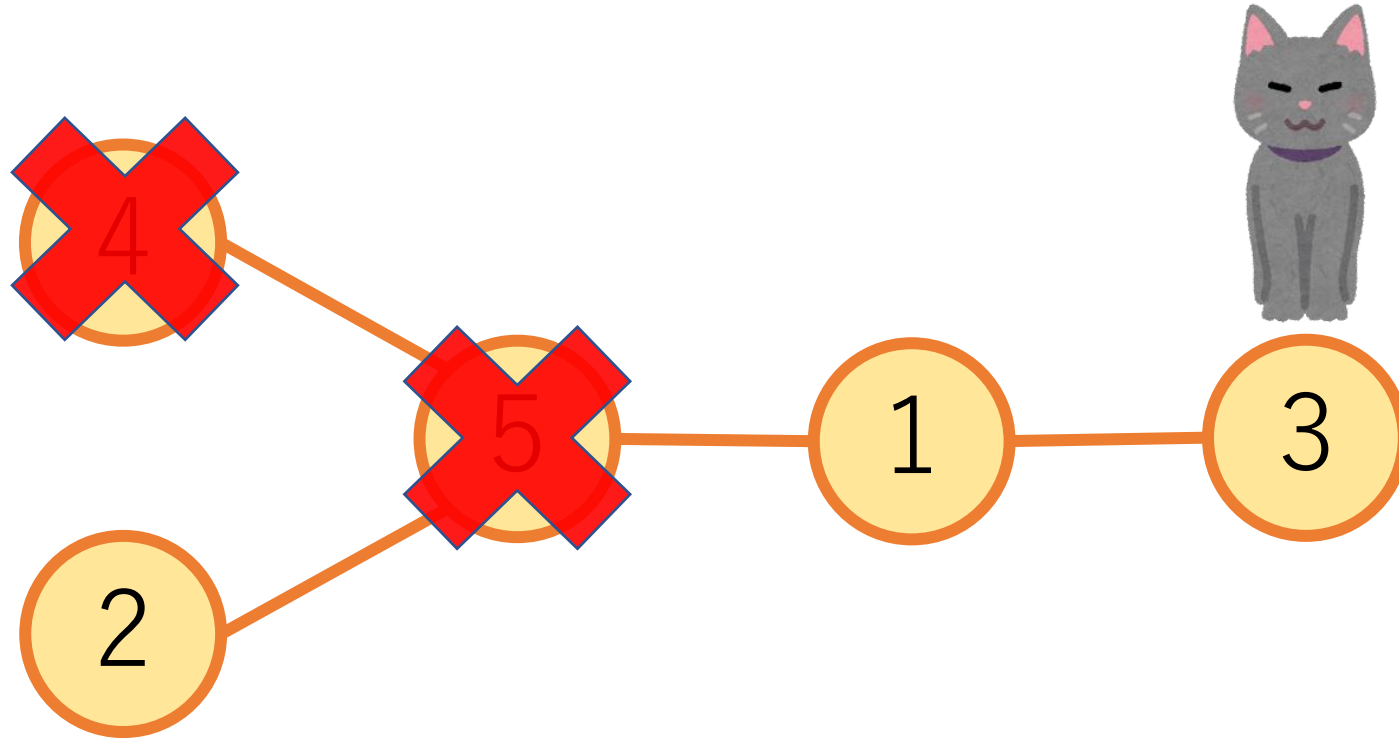
問題概要

- 選んだ頂点に猫がいた場合、その頂点から到達できる最も高い頂点に猫が移動する



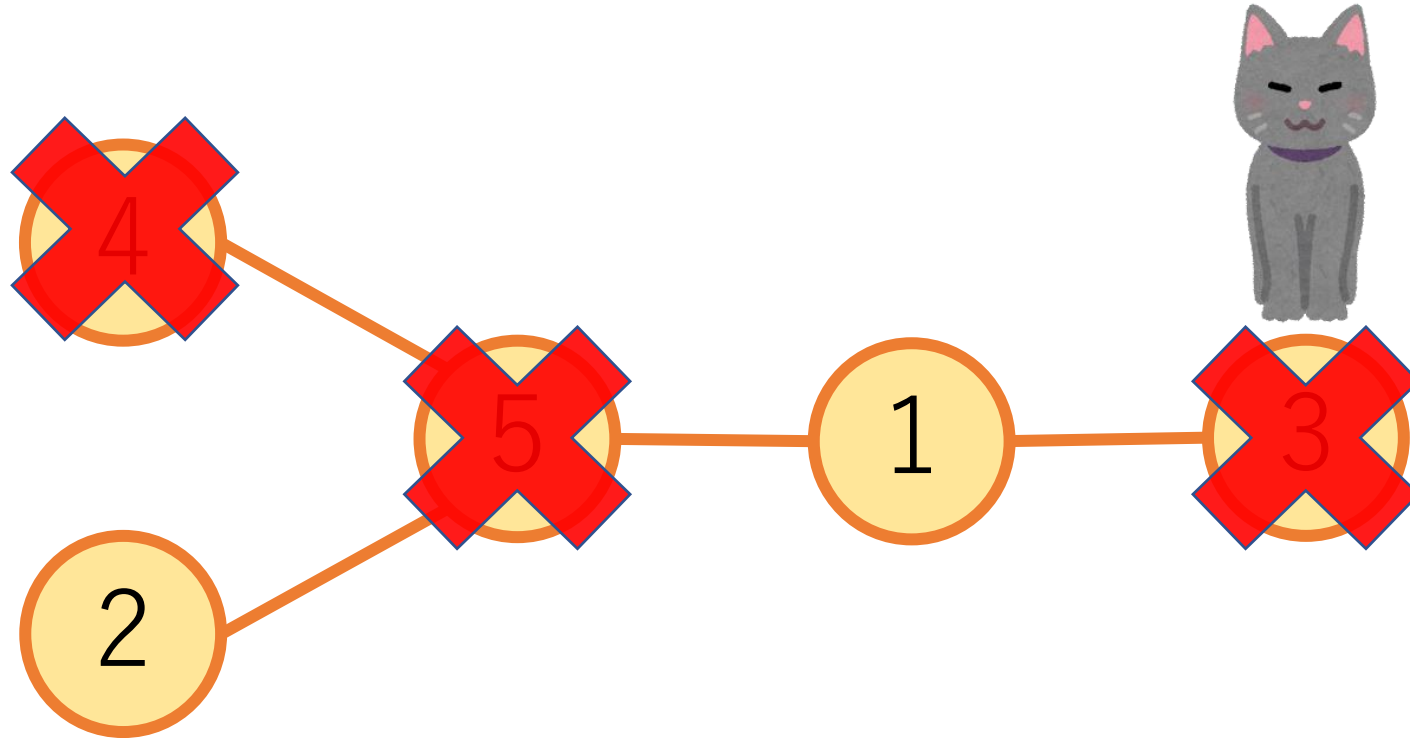
問題概要

- 猫は距離 2 だけ移動した



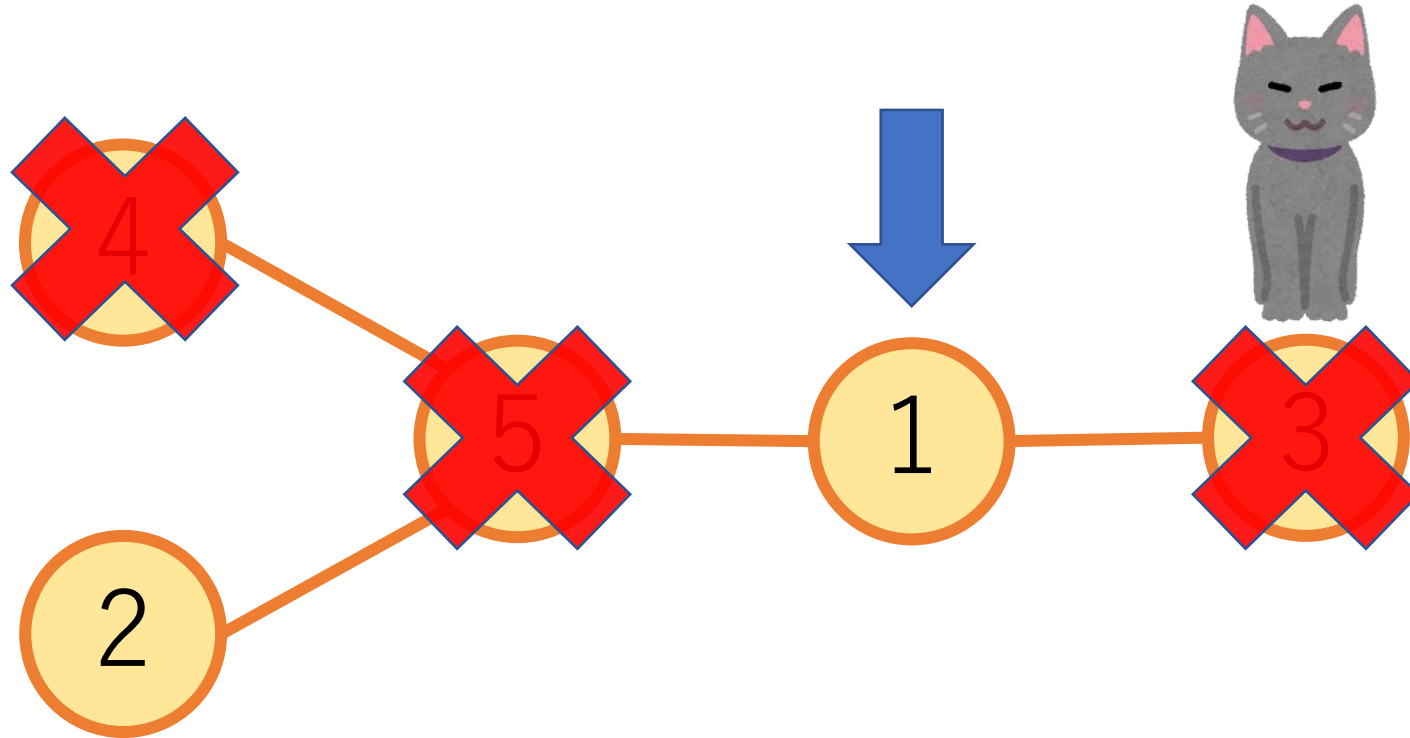
問題概要

- (再) 一つの頂点を選んで障害物を置く



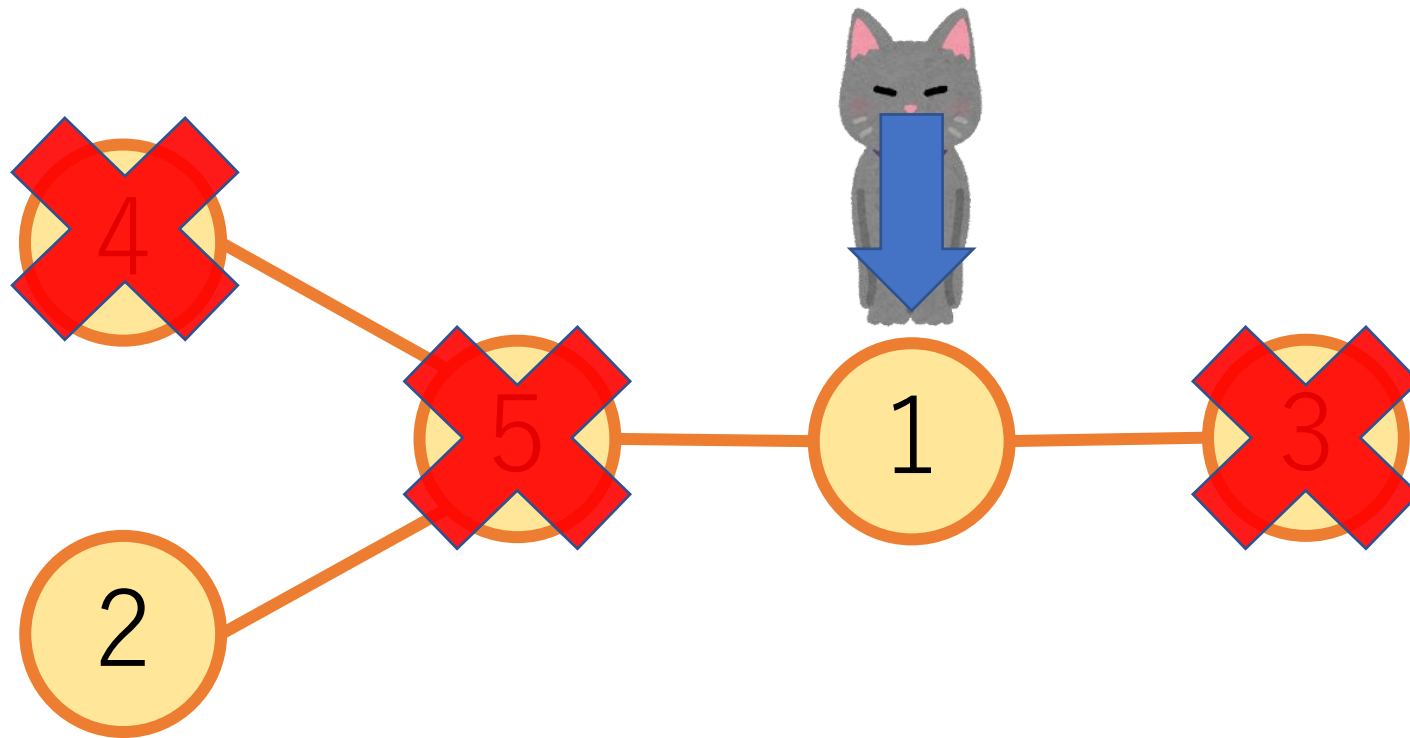
問題概要

- (再) 選んだ頂点に猫がいた場合、その頂点から到達できる最も高い頂点に猫が移動する



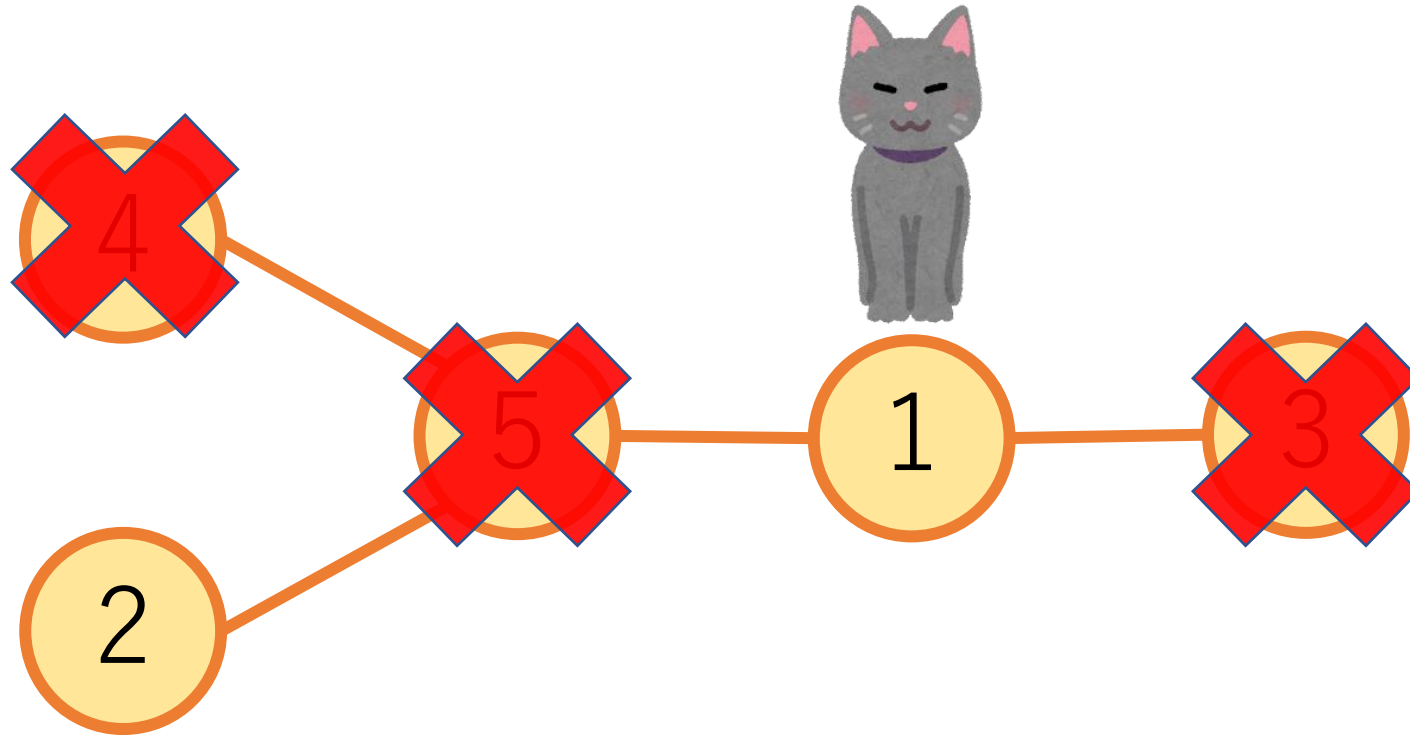
問題概要

- (再) 選んだ頂点に猫がいた場合、その頂点から到達できる最も高い頂点に猫が移動する



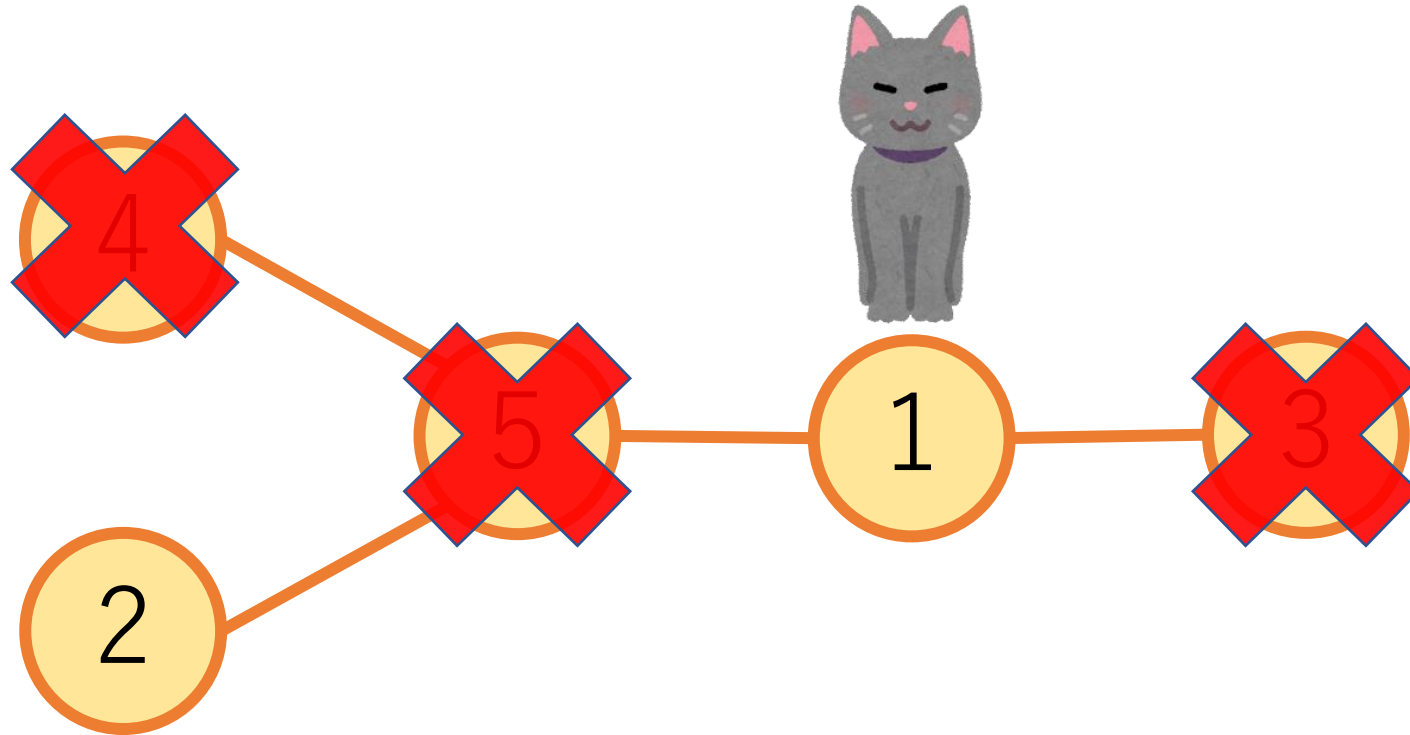
問題概要

- 猫は合計距離 3 だけ移動した



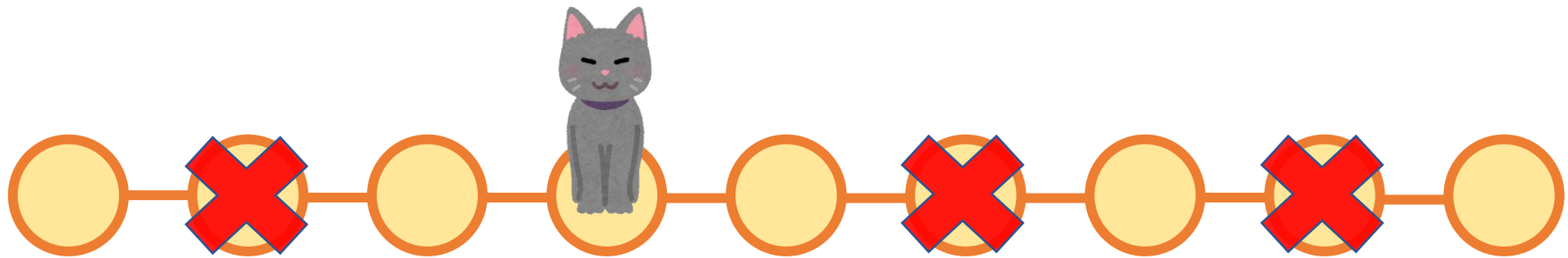
問題概要

- 頂点を選ぶ順番を工夫して、猫の移動距離の総和を最大化する



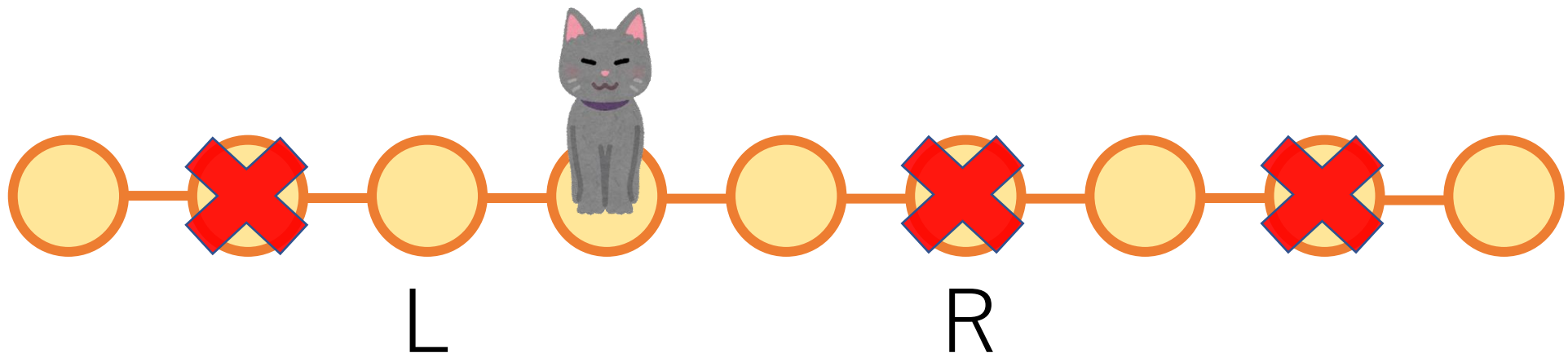
小課題 1 ($N \leq 16$, パス)

- bitDP
- $dp[S][x]$:= 頂点集合 S に障害物があり、猫が頂点 x にいるときの、これからの移動距離の最大値
- $O(2^N \cdot \text{poly}(N))$



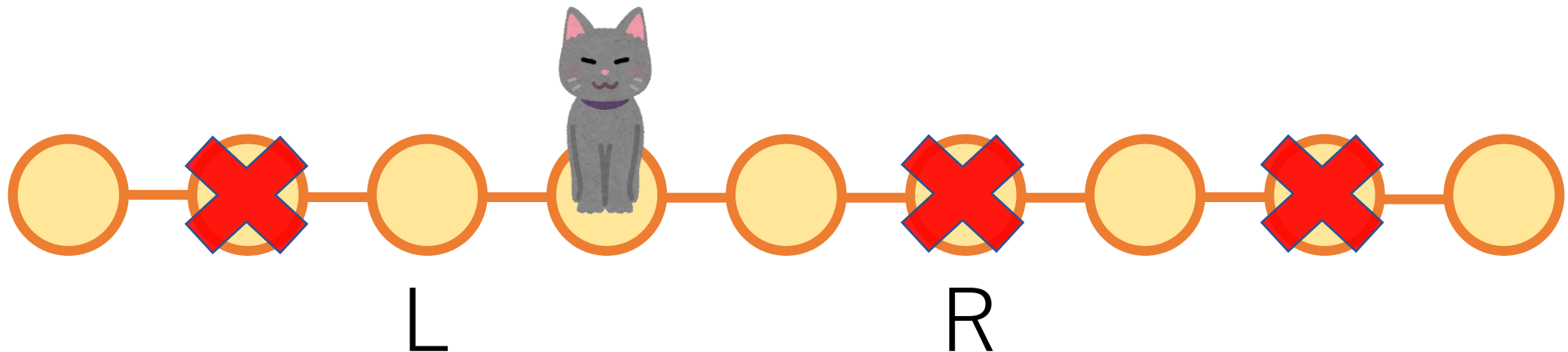
小課題 2 ($N \leq 300$, パス)

- もう少し情報を絞る
 - 猫がいる区間の外でどう障害物が置かれているかは関係ない
 - 猫は区間内で最も高い場所にいる
- 猫がいる区間だけで状態を区別



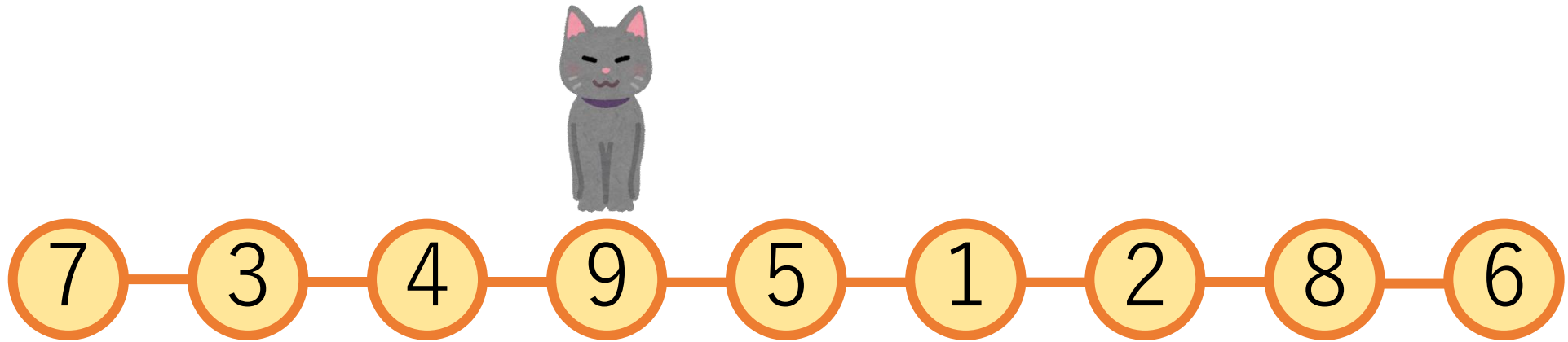
小課題 2 ($N \leq 300$, パス)

- 区間DP
- $dp[L][R]$:= 猫がいる区間が $[L, R)$ のときの, これからの移動距離の最大値
- 遷移: mid ($L \leq mid < R$) に障害物を置く
- $O(N^3)$



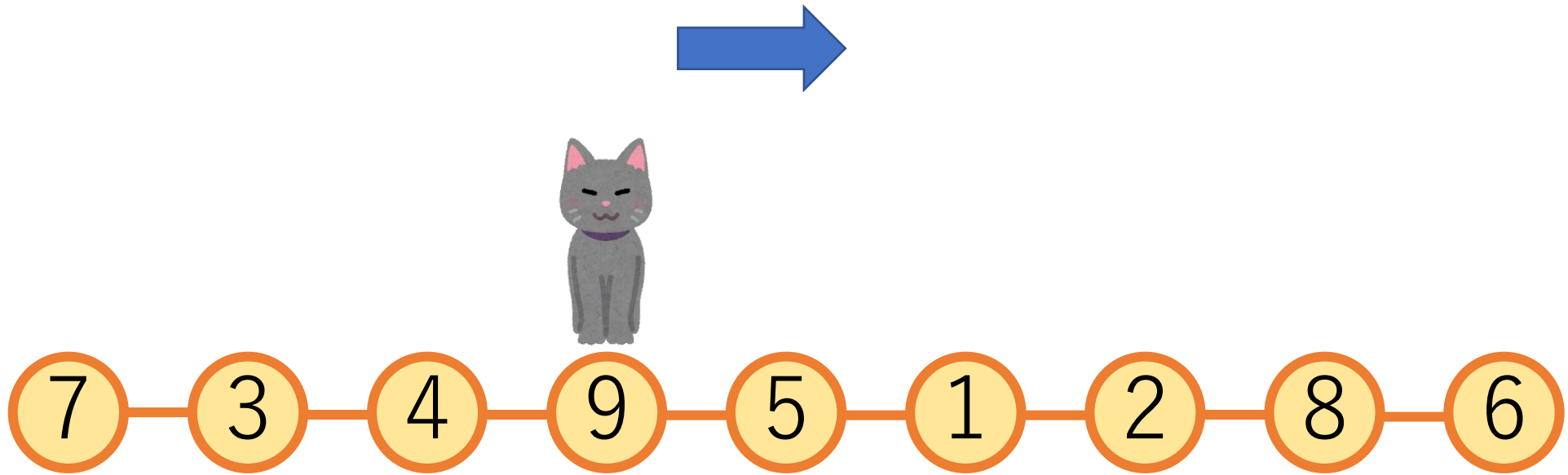
小課題3 ($N \leq 5000$, パス)

- 戦略を立てたい



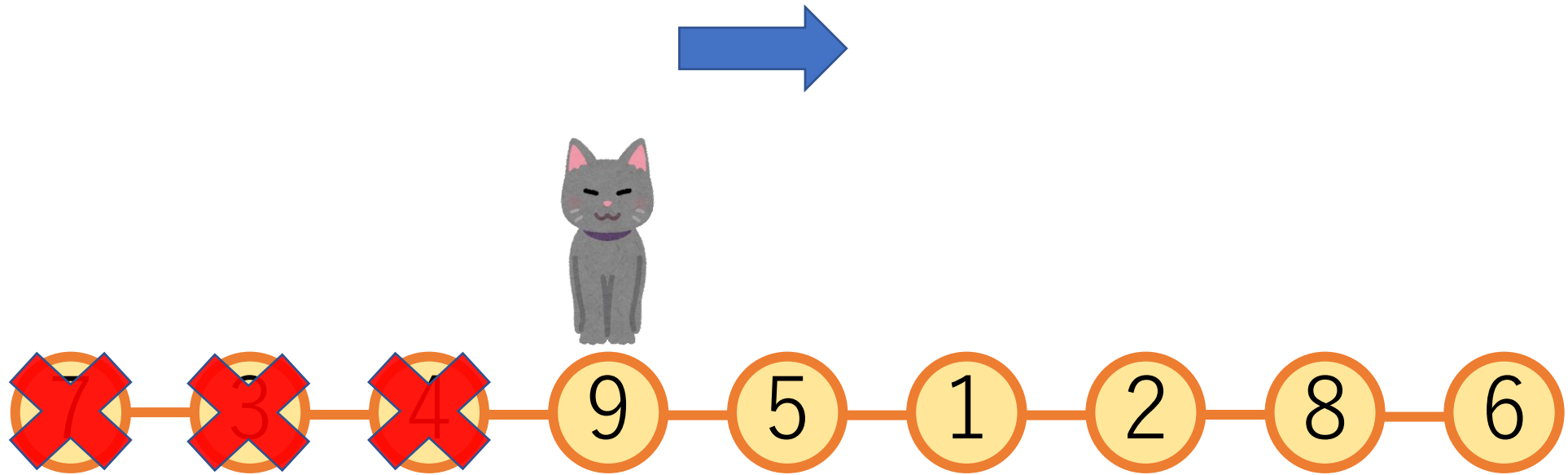
小課題3 ($N \leq 5000$, パス)

- 猫が次に右に移動するとする



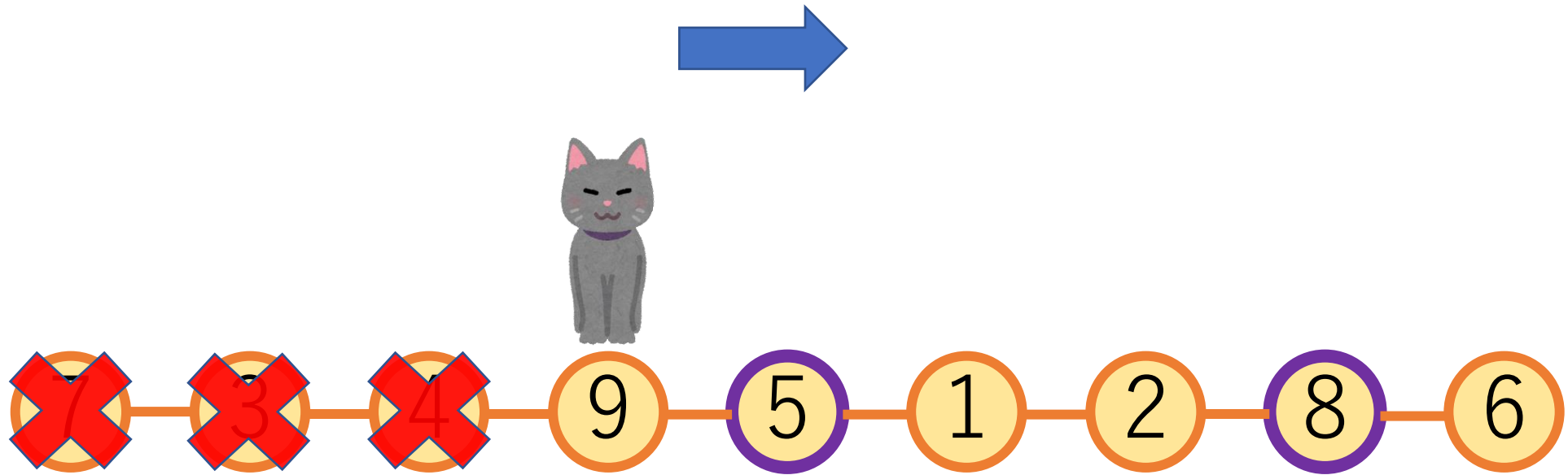
小課題3 ($N \leq 5000$, パス)

- 猫が次に右に移動するとする
- 左側の連結成分はすべて×にしてよい



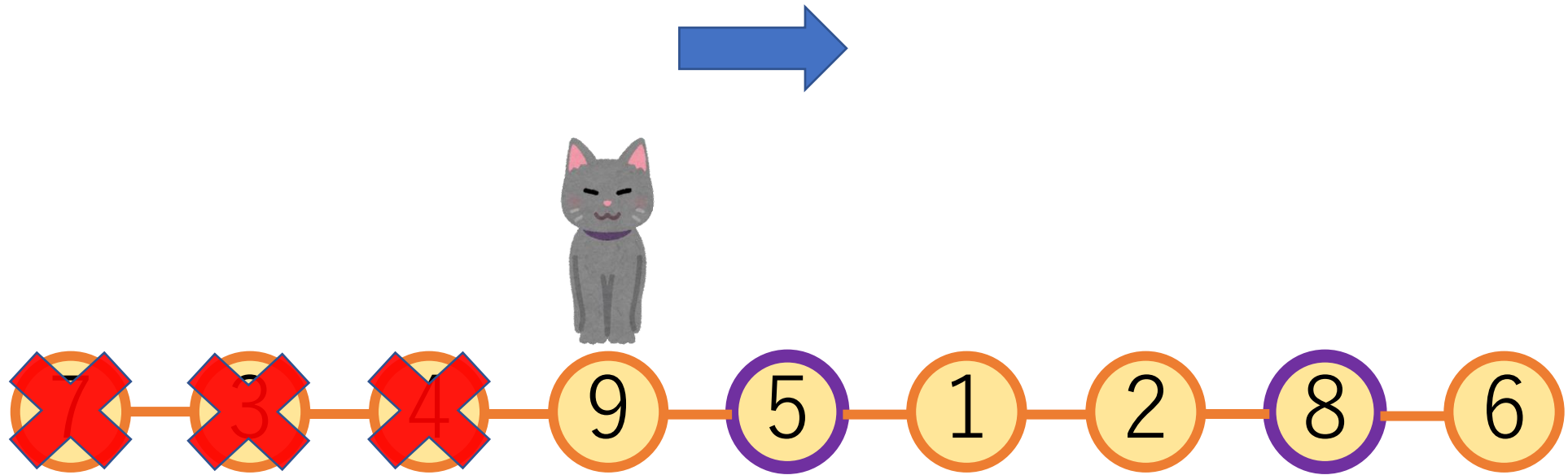
小課題3 ($N \leq 5000$, パス)

- 猫の移動先の候補は、左から見て最大値を更新する頂点



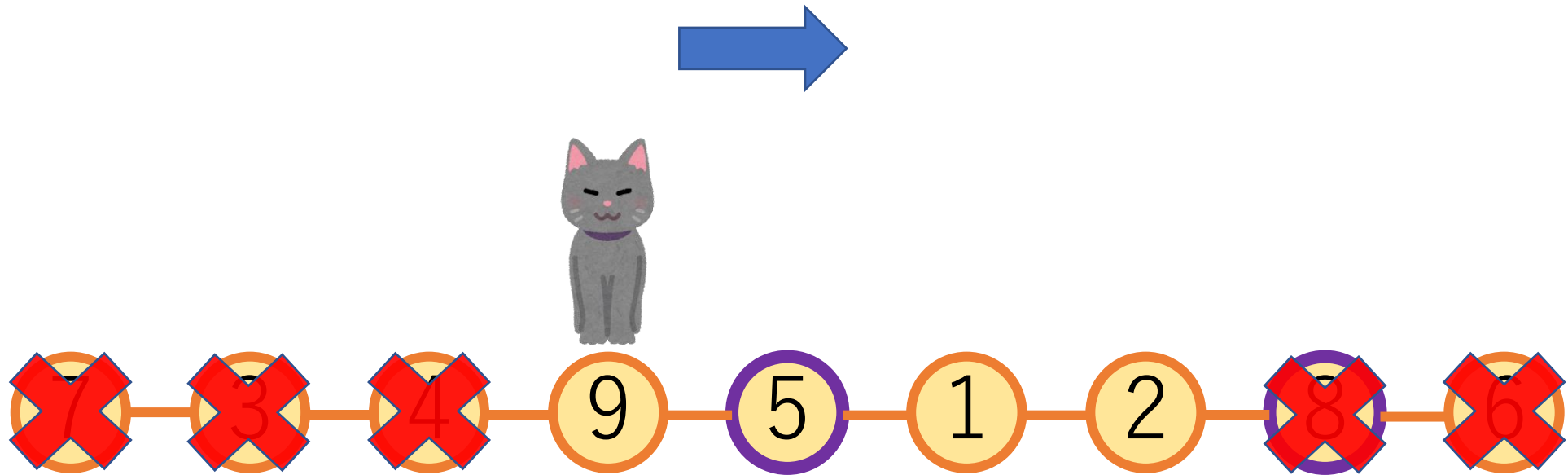
小課題3 ($N \leq 5000$, パス)

- 猫が手前の頂点（高さ 5）に移動するとする



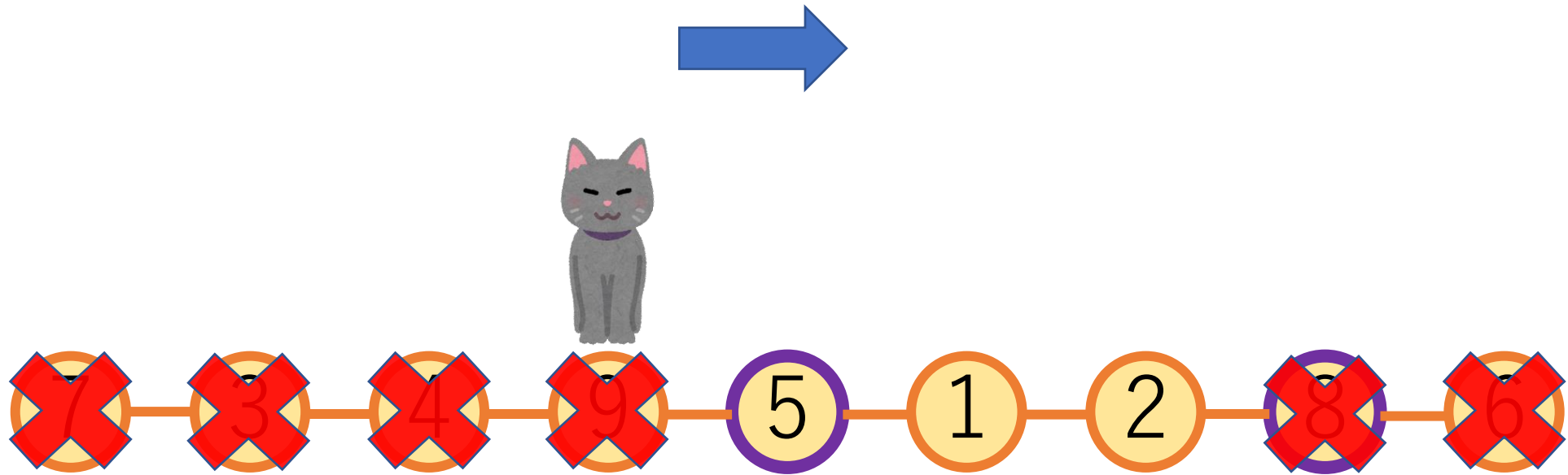
小課題3 ($N \leq 5000$, パス)

- 猫が手前の頂点（高さ 5）に移動するとする
- 次の候補を × にする必要がある



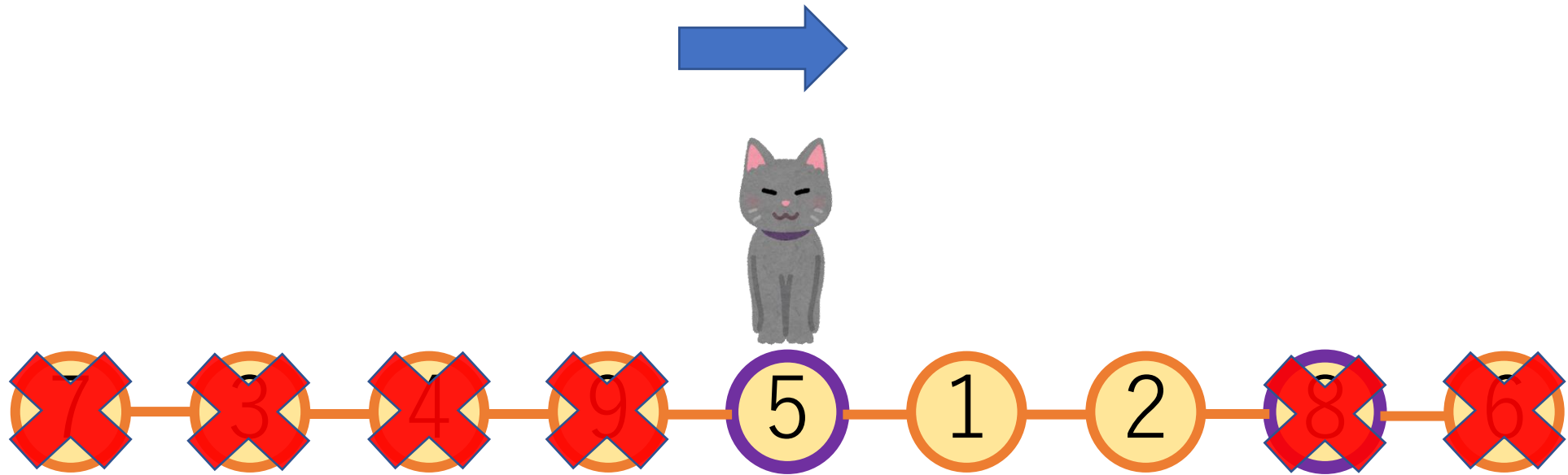
小課題3 ($N \leq 5000$, パス)

- 猫が手前の頂点（高さ 5）に移動するとする
- 次の候補を × にする必要がある
- 猫のいる頂点を × にして移動させる



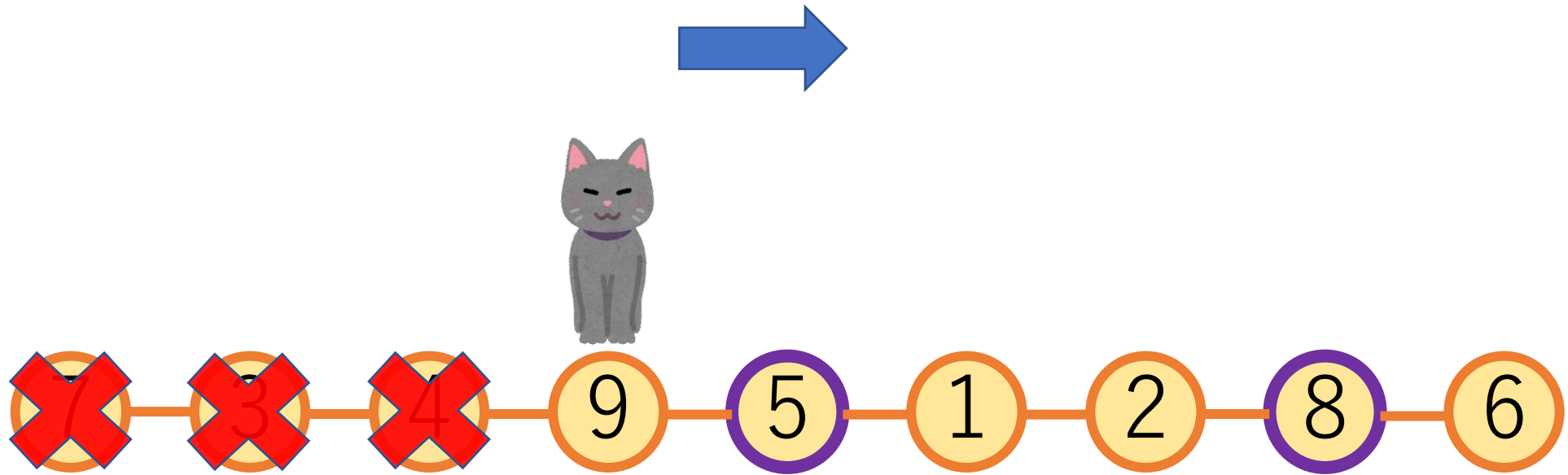
小課題3 ($N \leq 5000$, パス)

- 猫が手前の頂点（高さ 5）に移動するとする
- 次の候補を × にする必要がある
- 猫のいる頂点を × にして移動させる



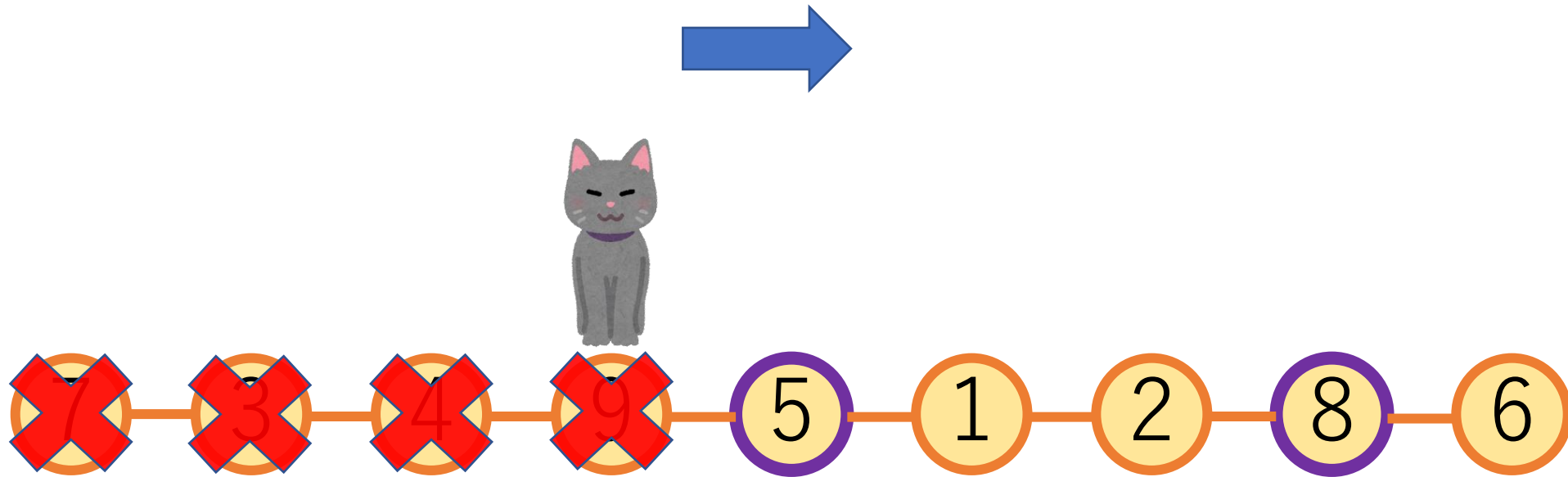
小課題3 ($N \leq 5000$, パス)

- 猫が奥の頂点（高さ 8）に移動するとする



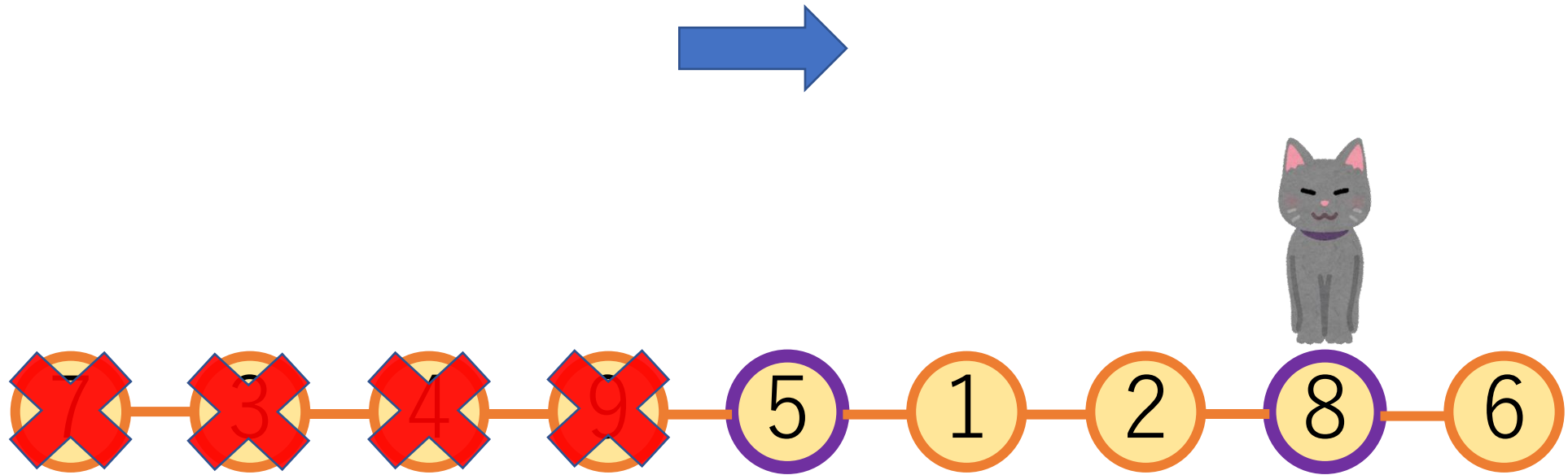
小課題3 ($N \leq 5000$, パス)

- 猫が奥の頂点（高さ 8）に移動するとする
- 猫のいる頂点を $\times$ にして移動させる



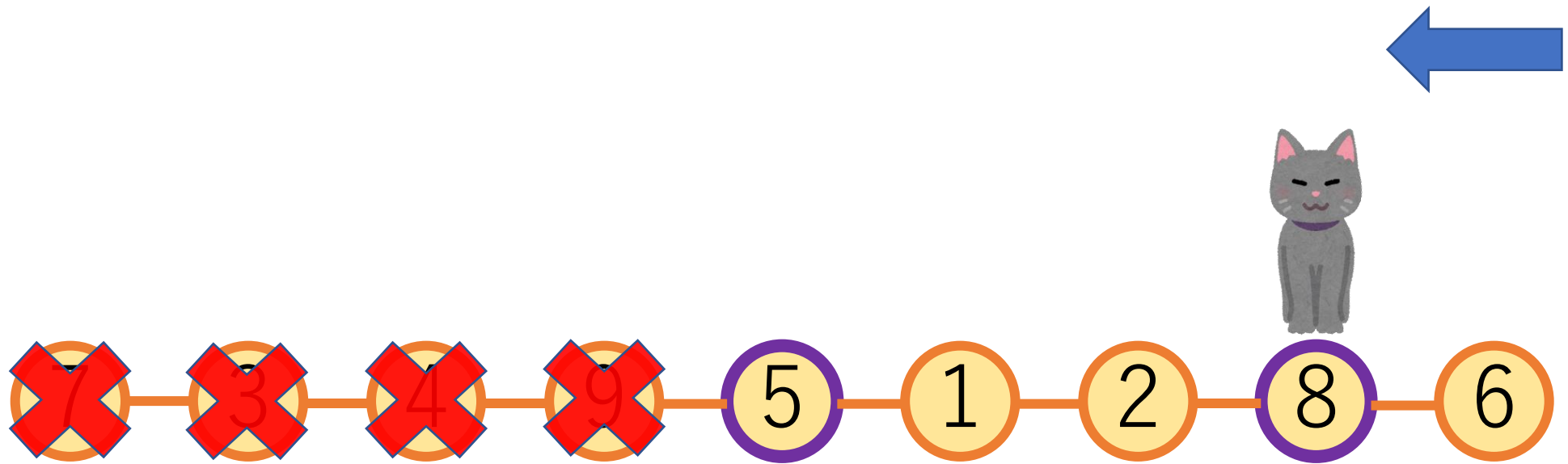
小課題3 ($N \leq 5000$, パス)

- 猫が奥の頂点（高さ 8）に移動するとする
- 猫のいる頂点を × にして移動させる



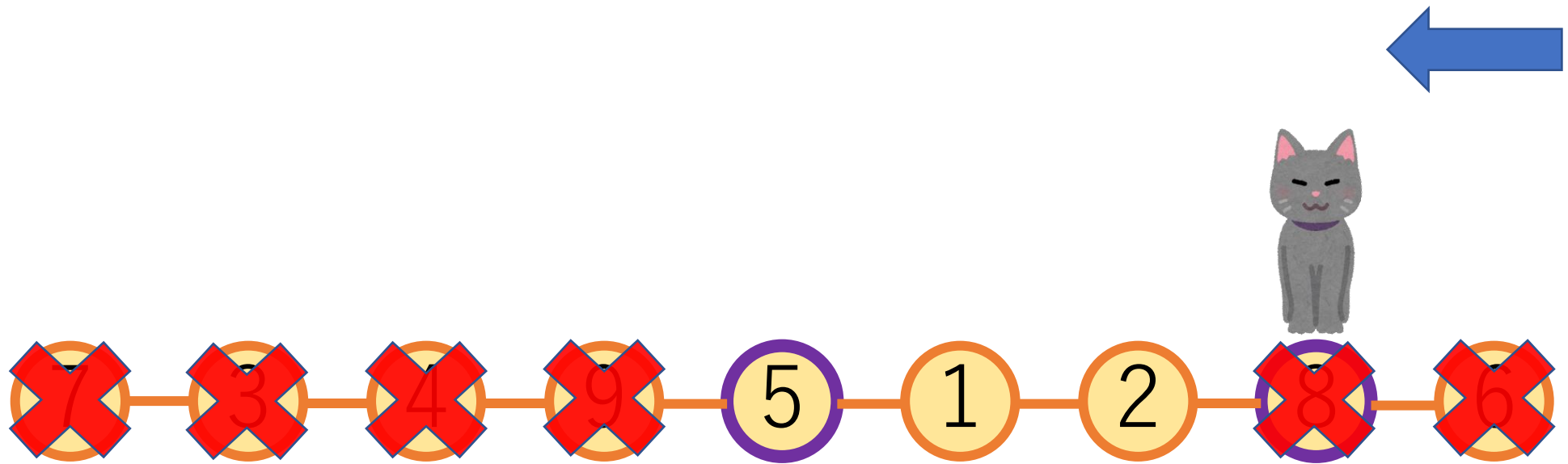
小課題3 ($N \leq 5000$, パス)

- 猫が奥の頂点（高さ 8）に移動するとする
- 猫のいる頂点を×にして移動させる
- 猫を右から追い詰めてみる



小課題3 ($N \leq 5000$, パス)

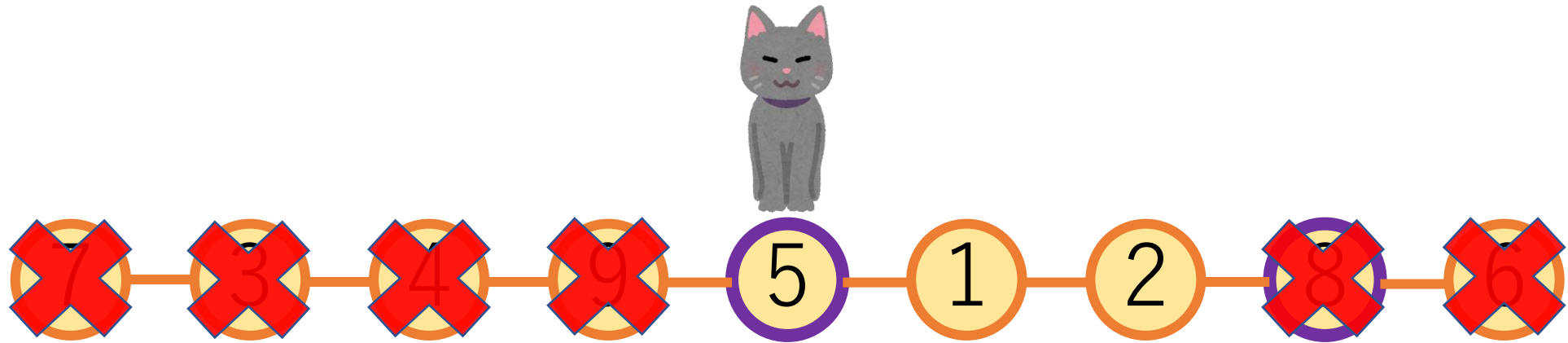
- 猫が奥の頂点（高さ 8）に移動するとする
- 猫のいる頂点を×にして移動させる
- 猫を右から追い詰めてみる



小課題3 ($N \leq 5000$, パス)

- 猫が奥の頂点（高さ 8）に移動するとする
- 猫のいる頂点を×にして移動させる
- 猫を右から追い詰めてみる

→手前の頂点（高さ 5）に移動した場合と同じ状態に！

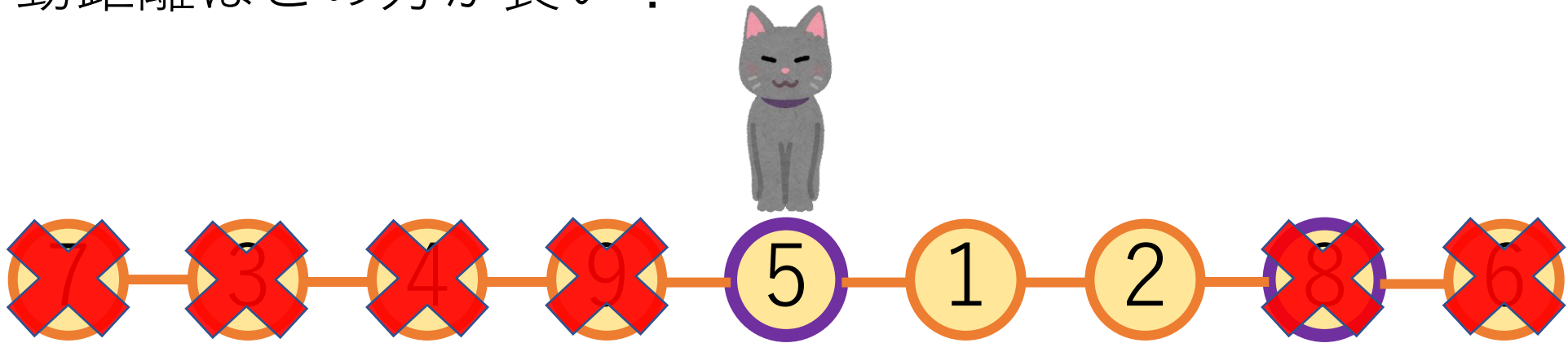


小課題3 ($N \leq 5000$, パス)

- 猫が奥の頂点（高さ 8）に移動するとする
- 猫のいる頂点を×にして移動させる
- 猫を右から追い詰めてみる

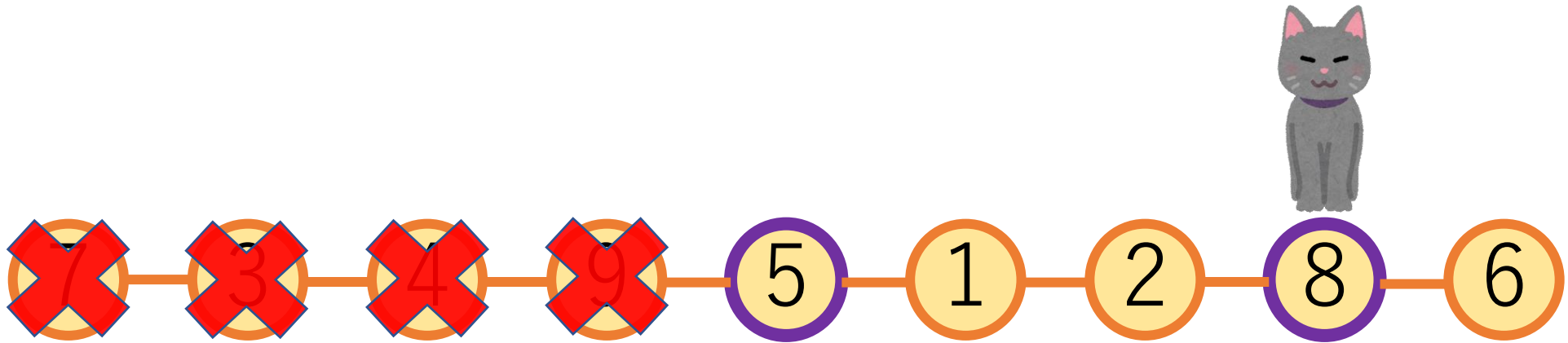
→手前の頂点（高さ 5）に移動した場合と同じ状態に！

移動距離はこの方が長い！



小課題3 ($N \leq 5000$, パス)

- 何ができるか？
- 左右それぞれで、最も高い頂点に移動するのがよい
- 移動した後は、元と同じ形の問題になる

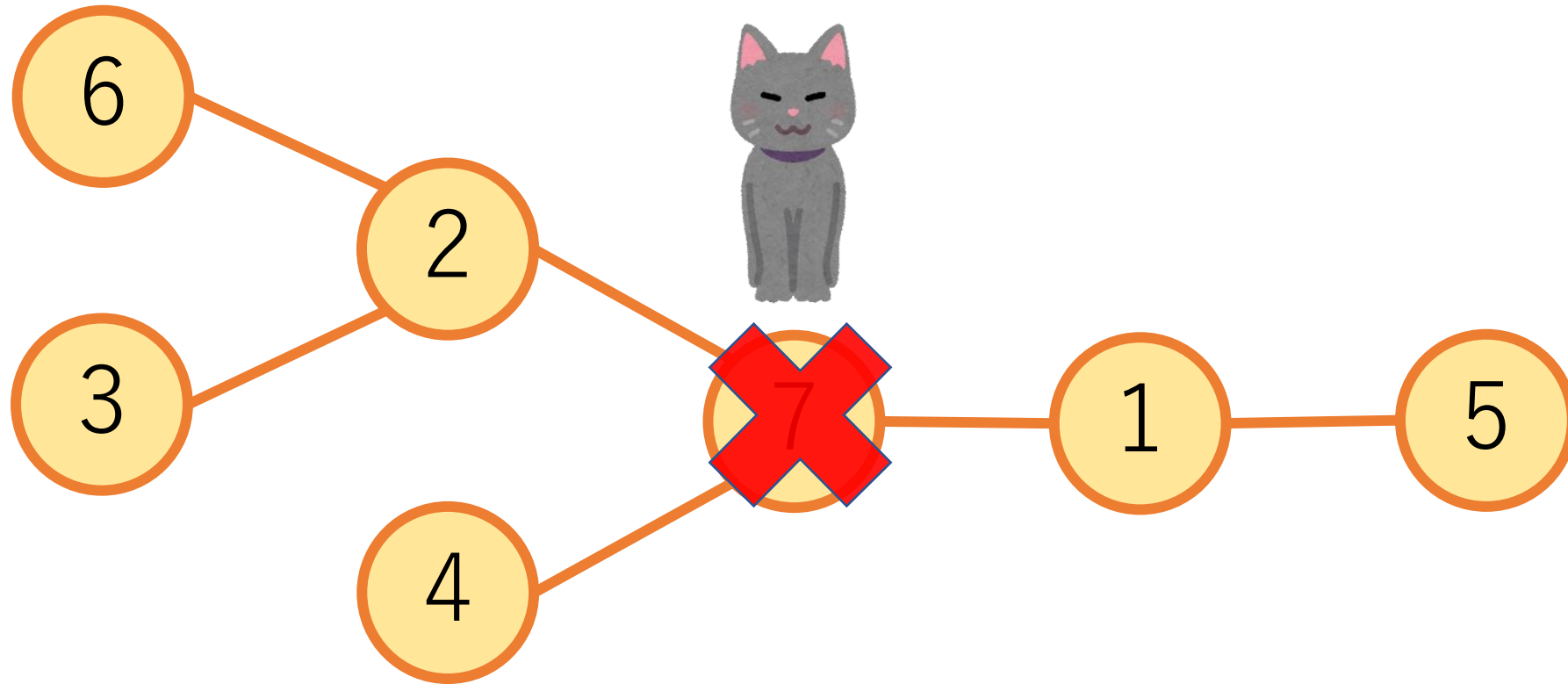


小課題3 ($N \leq 5000$, パス)

- $dp[v]$:= 猫がいる頂点が v のときの、これからの移動距離の最大値
- 頂点 v で左右に分けたときの、左右の各区間の最も高い頂点 $u1, u2$ を求める ← **for文で $O(N)$**
- $dp[u1], dp[u2]$ を再帰的に求める
- $dp[v] = \max\{ dp[ui] + \text{dist}(ui, v) \}$
- **$O(N^2)$**

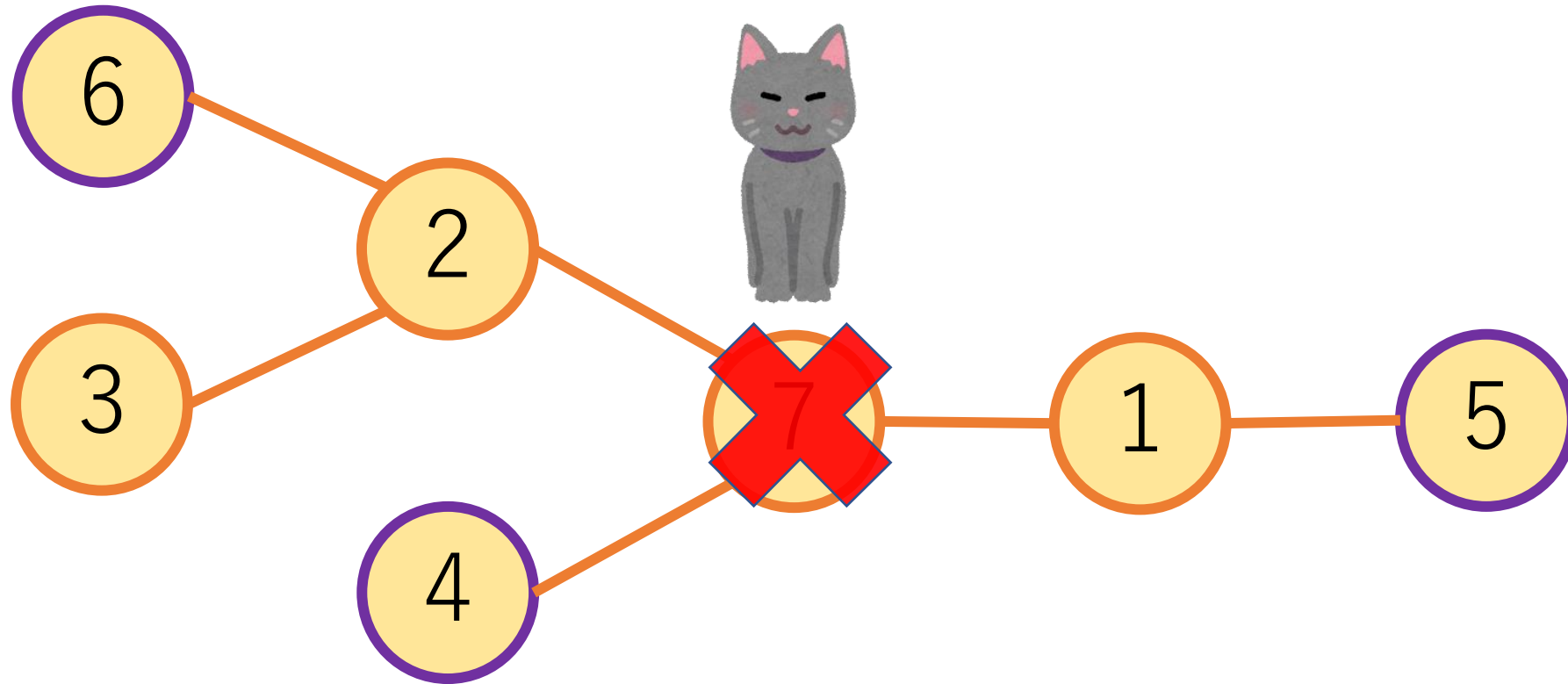
小課題4 ($N \leq 5000$, 木)

- 木の場合も、猫のいる頂点で分けたときの各連結成分で最も高い頂点に移動させるのが良い



小課題4 ($N \leq 5000$, 木)

- 木の場合も、猫のいる頂点で分けたときの各連結成分で最も高い頂点に移動させるのが良い



小課題4 ($N \leq 5000$, 木)

- $dp[v]$:= 猫がいる頂点が v のときの、これからの移動距離の最大値
- 頂点 v で分けたときの、各連結成分の最も高い頂点 $u_1, u_2, \dots$ を求める ← **DFSで $O(N)$**
- $dp[u_1], dp[u_2], \dots$ を再帰的に求める
- $dp[v] = \max\{ dp[u_i] + \text{dist}(u_i, v) \}$
- **$O(N^2)$**

小課題5 ($N \leq 200000$, パス)

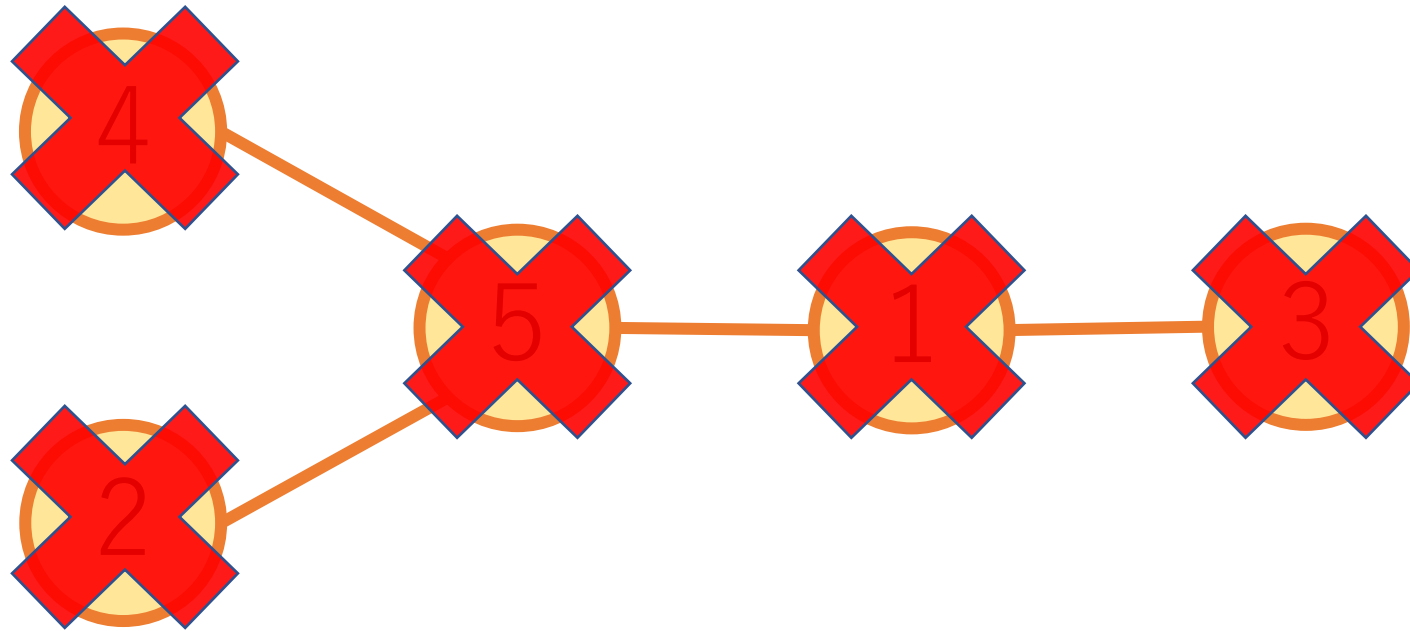
- $dp[v]$:= 猫がいる頂点が v のときの、これからの移動距離の最大値
- 頂点 v で左右に分けたときの、左右の各区間の最も高い頂点 $u1, u2$ を求める ← **SegmentTree** で $O(\log N)$
- $dp[u1], dp[u2]$ を再帰的に求める
- $dp[v] = \max\{ dp[ui] + \text{dist}(ui, v) \}$
- **$O(N \log N)$**

小課題6 ($N \leq 200000$, 完全二分木)

- 解法が色々ある
- 連結成分の最大値を速く求める
- 部分木の最大値 & 一点更新 がSegtreeもどきでできる
- 最も高い頂点の子の部分木の問題を解く $\rightarrow$ 頂点の値を $-\text{inf}$ にする $\rightarrow$ もう一度元の部分木の問題を解く (すべて $-\text{inf}$ になったら終了)
- 2 頂点間の距離...ビット演算で $O(1)$
- $O(N \log N)$

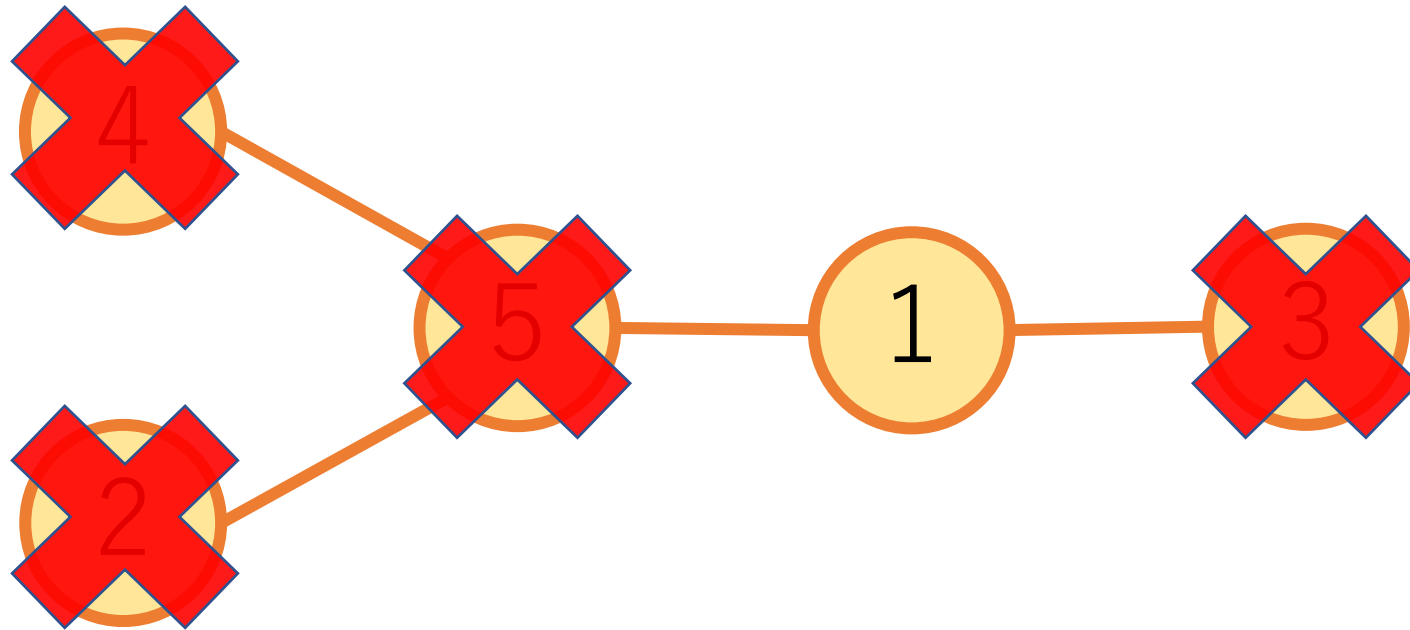
満点解法

- 方針：小課題4のDPを逆順に実行する（連結成分を切る方向ではなく、つなげる方向で考える）
- すべてに障害物を置いた状態から始める



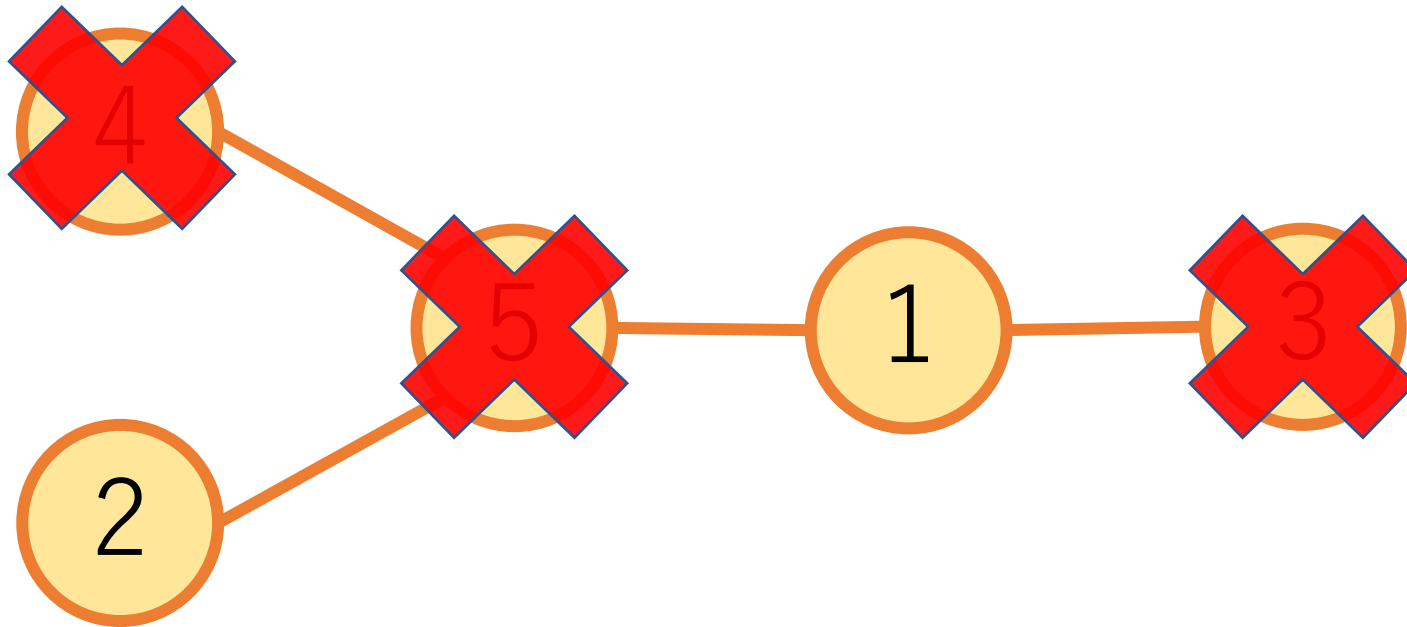
満点解法

- 最も低い頂点の障害物を取り除く



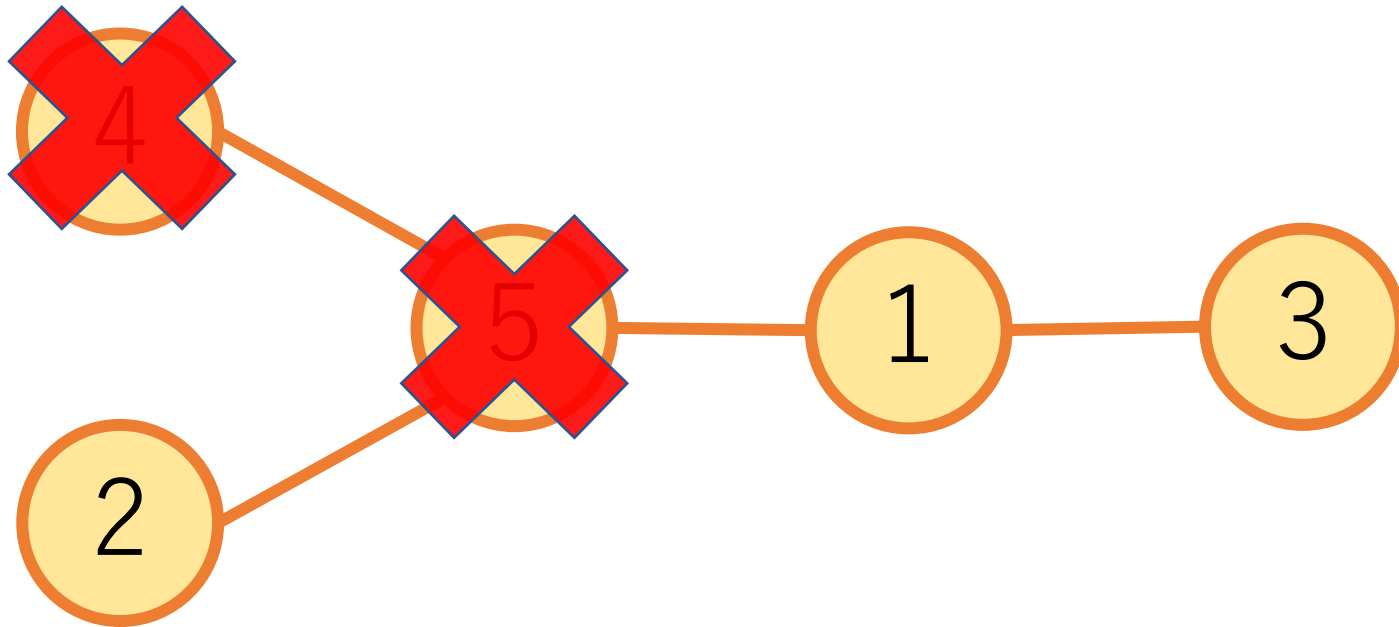
満点解法

- (再) 最も低い頂点の障害物を取り除く



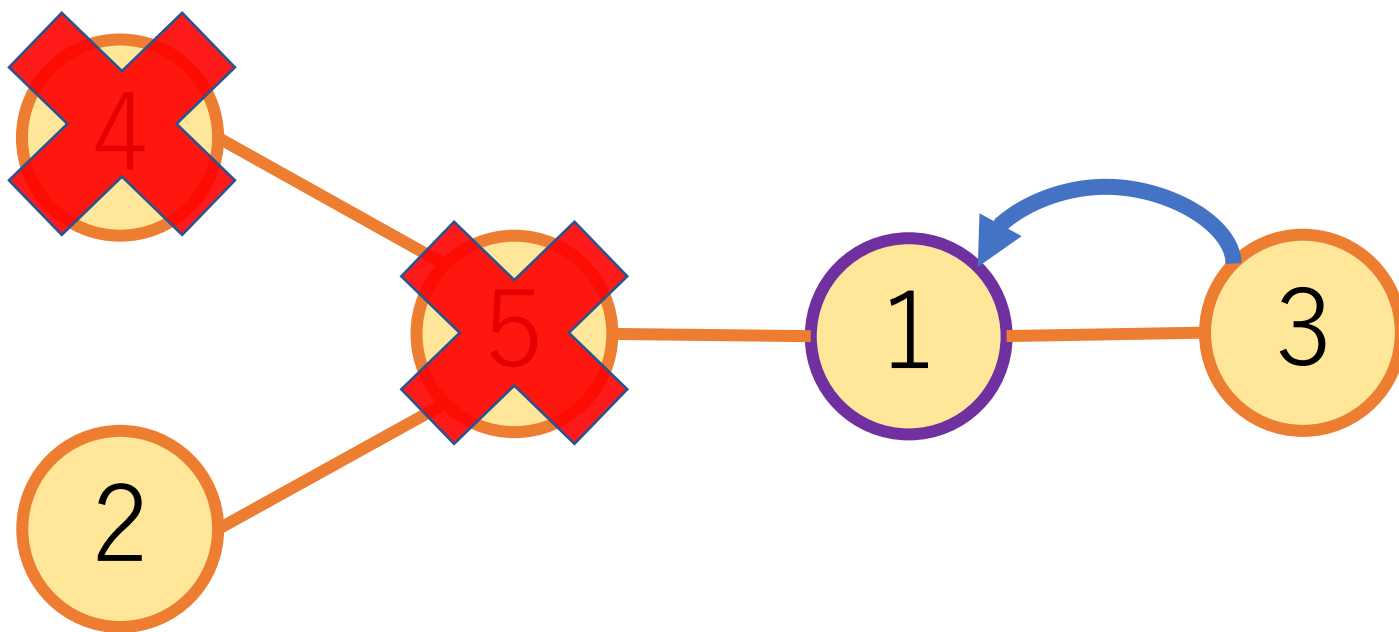
満点解法

- (再) 最も低い頂点の障害物を取り除く



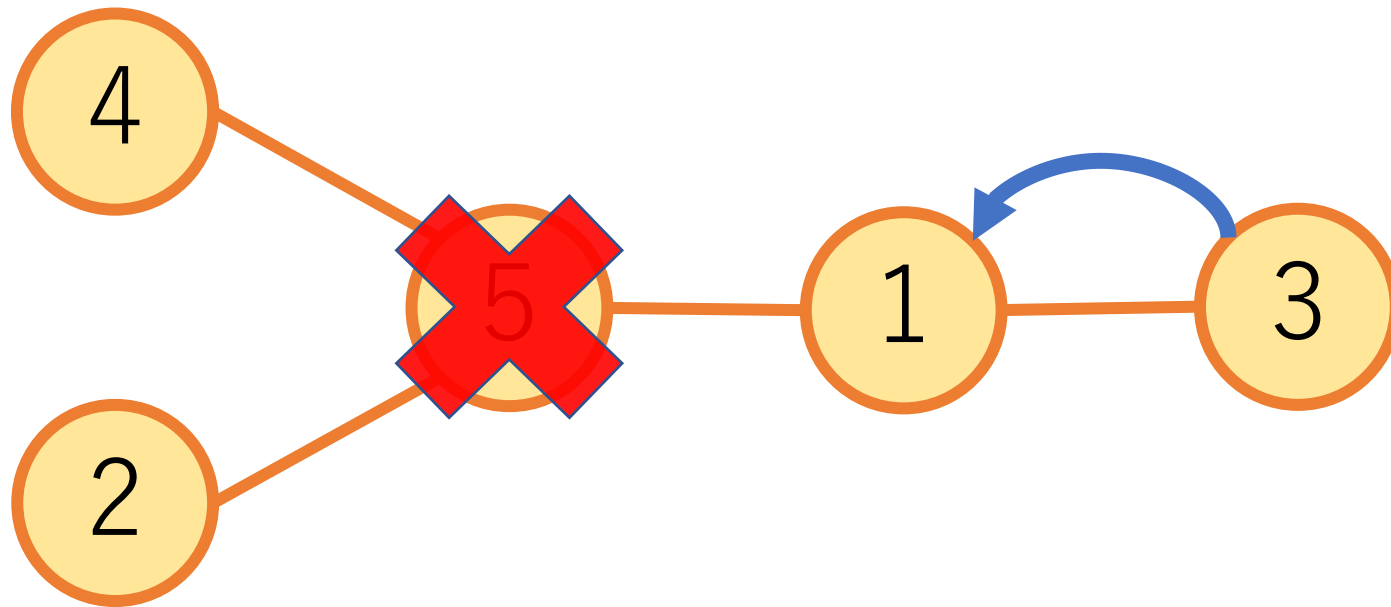
満点解法

- (再) 最も低い頂点の障害物を取り除く
- 連結した場合、元からあった各連結成分内で最も高い頂点へ辺を張る



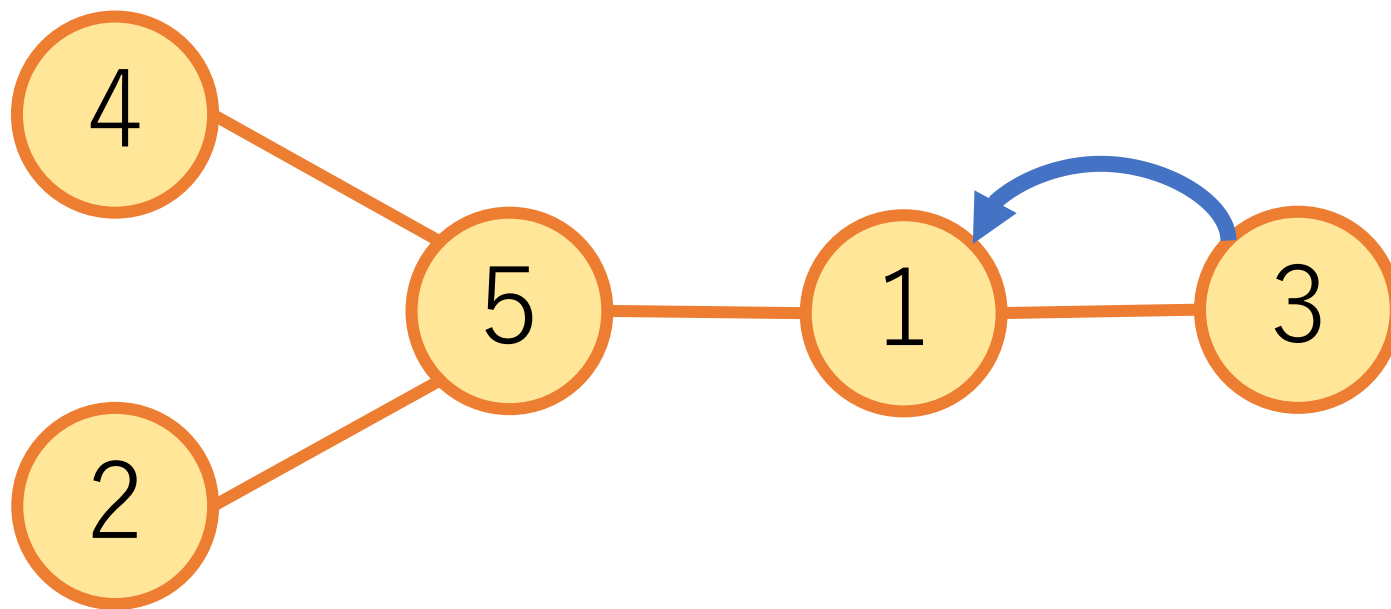
満点解法

- (再) 最も低い頂点の障害物を取り除く



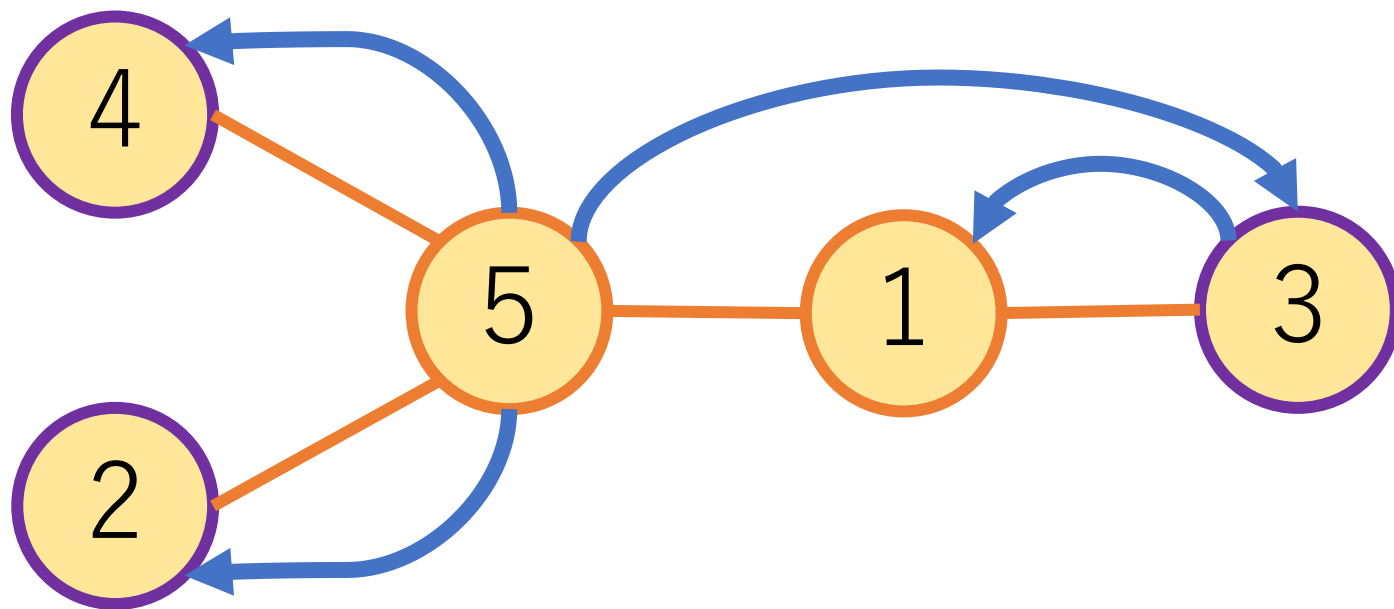
満点解法

- (再) 最も低い頂点の障害物を取り除く



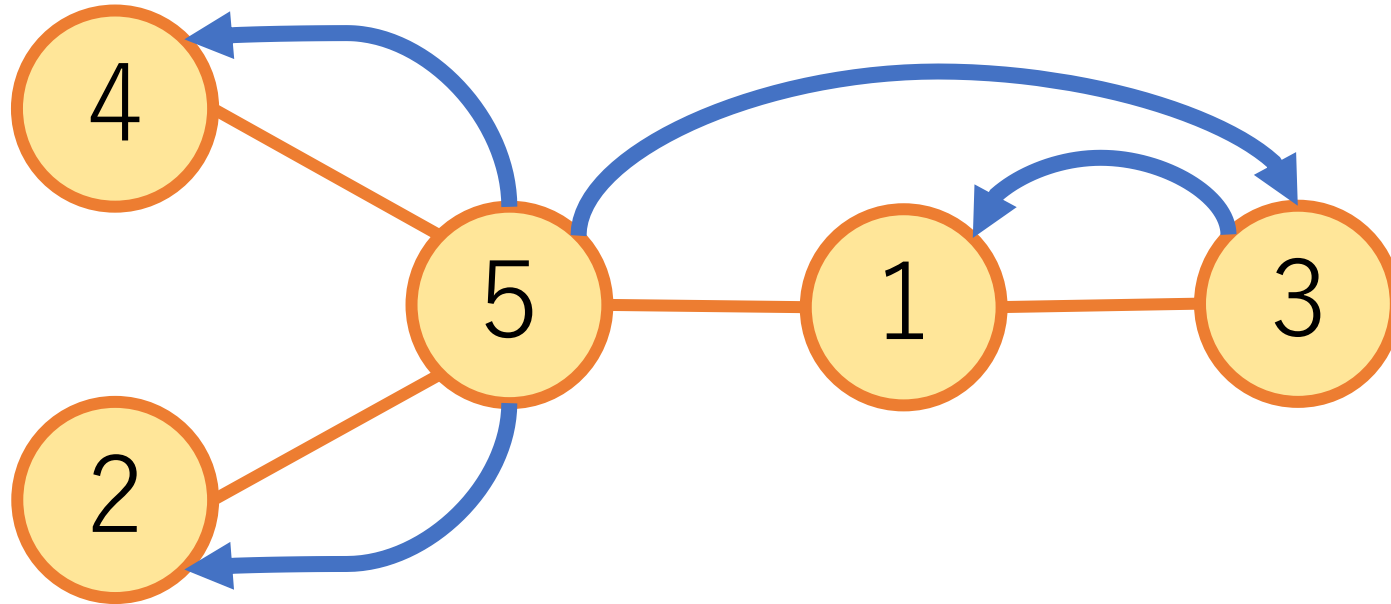
満点解法

- (再) 最も低い頂点の障害物を取り除く
- 連結した場合、元からあった各連結成分内で最も高い頂点へ辺を張る



満点解法

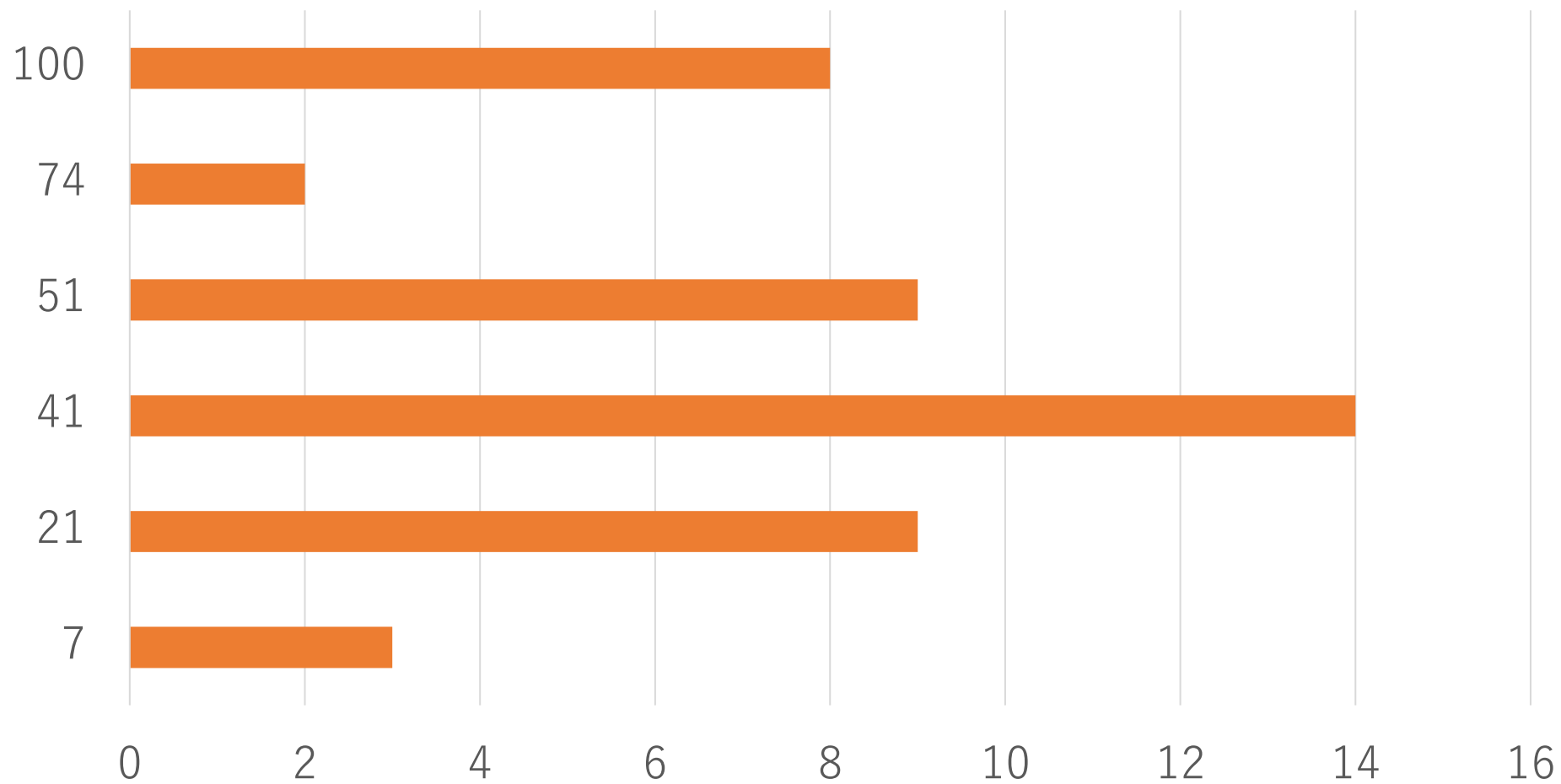
- (再) 最も低い頂点の障害物を取り除く
- 連結した場合、元からあった各連結成分内で最も高い頂点へ辺を張る
- こうしてできた木での、根からの移動距離の最大値が答え



満点解法

- UnionFindで連結成分を管理
- 連結成分内で最も高い頂点が分かるようにする（根にするなど）
- 木の二頂点間の距離 $\text{dist}(u,v)$ はLCAで一回 $O(\log N)$
- $O(N \log N)$

得点分布



- 0点：130人



現代的な機械 解説

米田 優峻 (E86120)

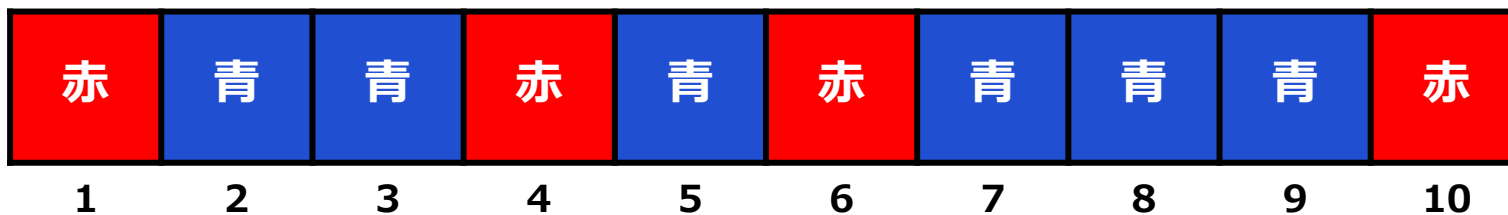
第22回情報オリンピック 本選5

- N 個のライトタイルと M 個のボタンからなる「JOI マシン」があります
- ボタン i を押すと、次のことが起こります
 1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
 2. ボールが外に出るまで、以下が繰り返される
 - 青色のタイルの上にボールがある場合：ボールが 1 つ左に移動し、タイルの色が赤になる
 - 赤色のタイルの上にボールがある場合：ボールが 1 つ右に移動し、タイルの色が青になる
- ボタン $L_i, L_i + 1, \dots, R_i$ をその順に押したときの最終的な赤タイルの個数を求めよ、という質問が Q 個与えられるので、それぞれ答えてください

問題設定が少し複雑なので、例を示します

例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

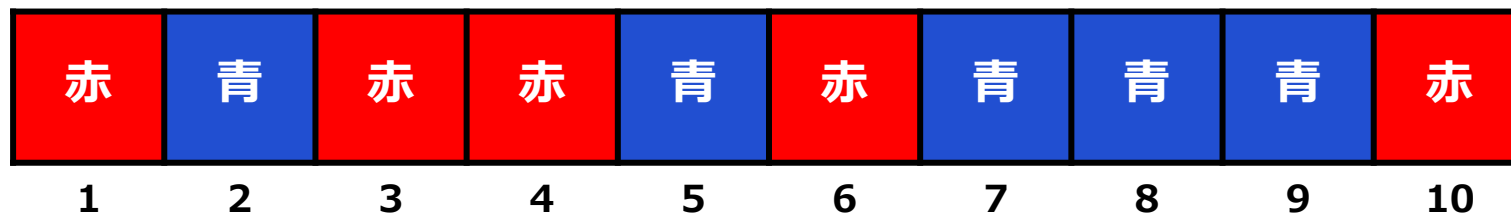
1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

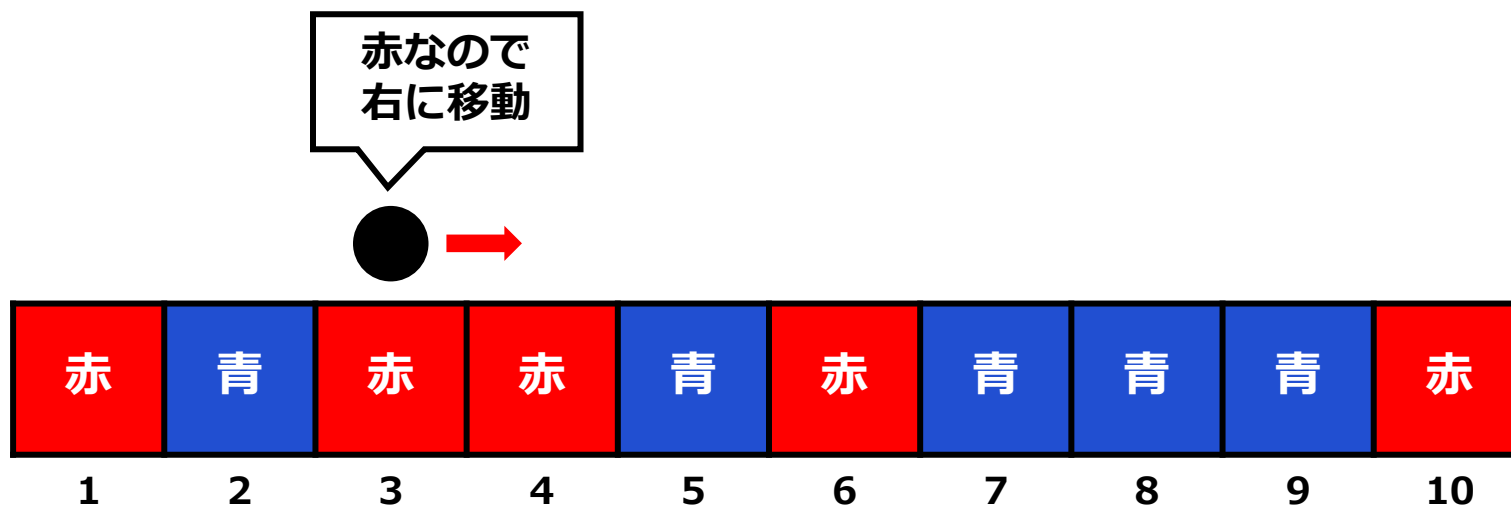
1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す

最初に、ボールの場所の
ライトタイルが赤に



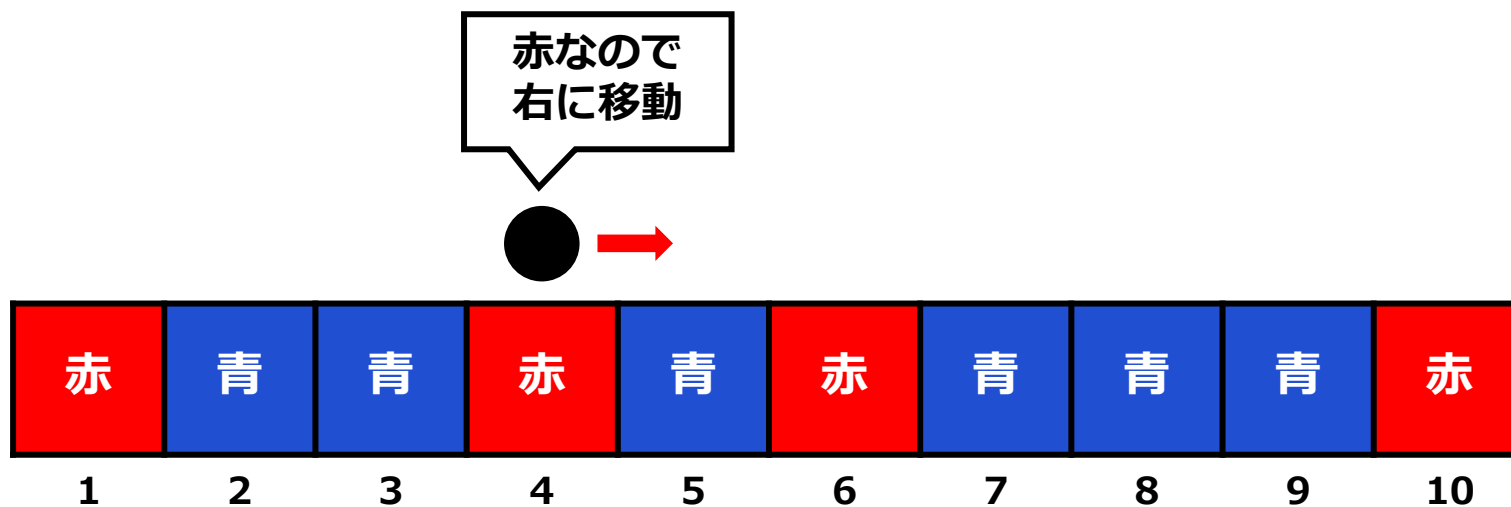
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



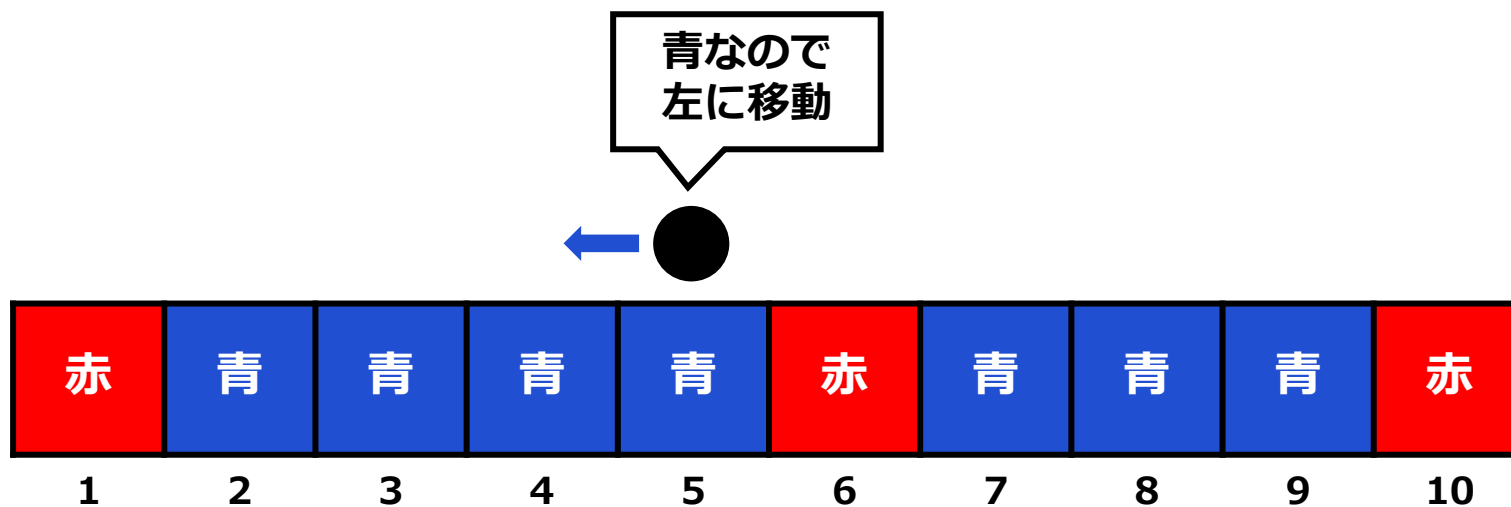
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



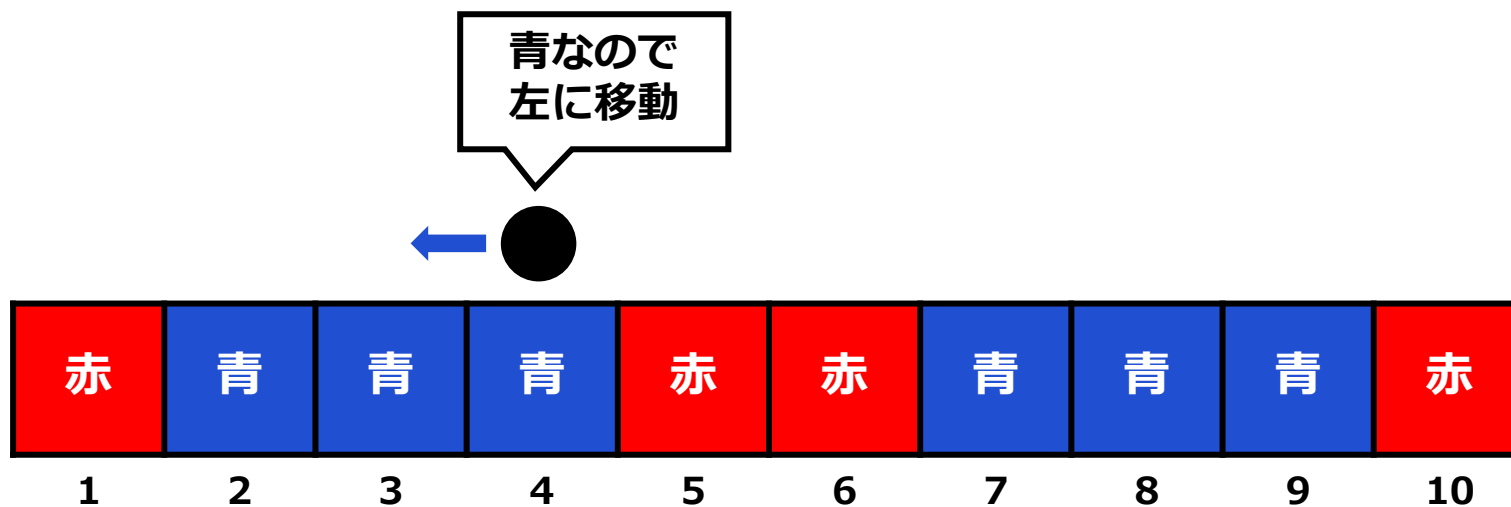
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



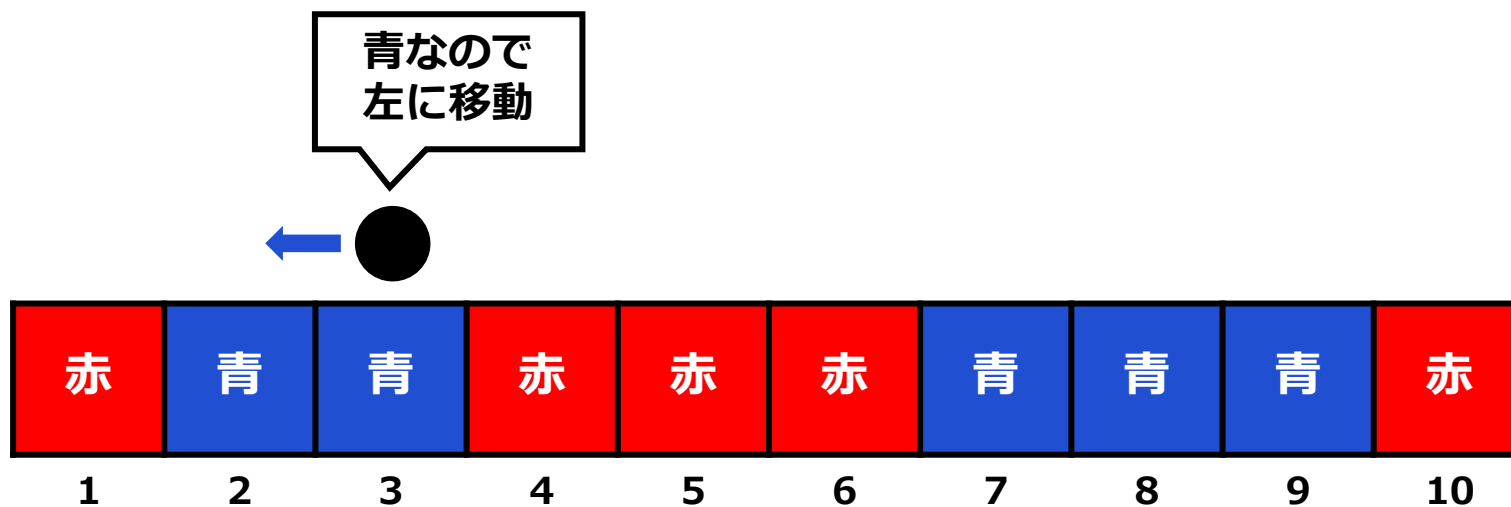
例：初期状態が「赤青青赤青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



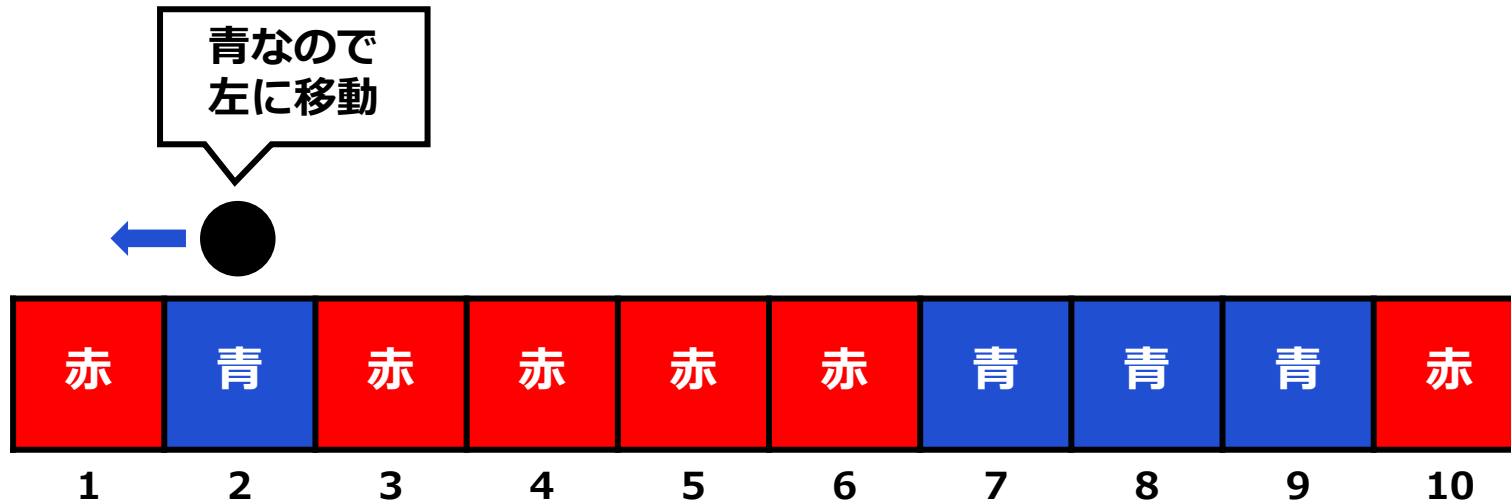
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



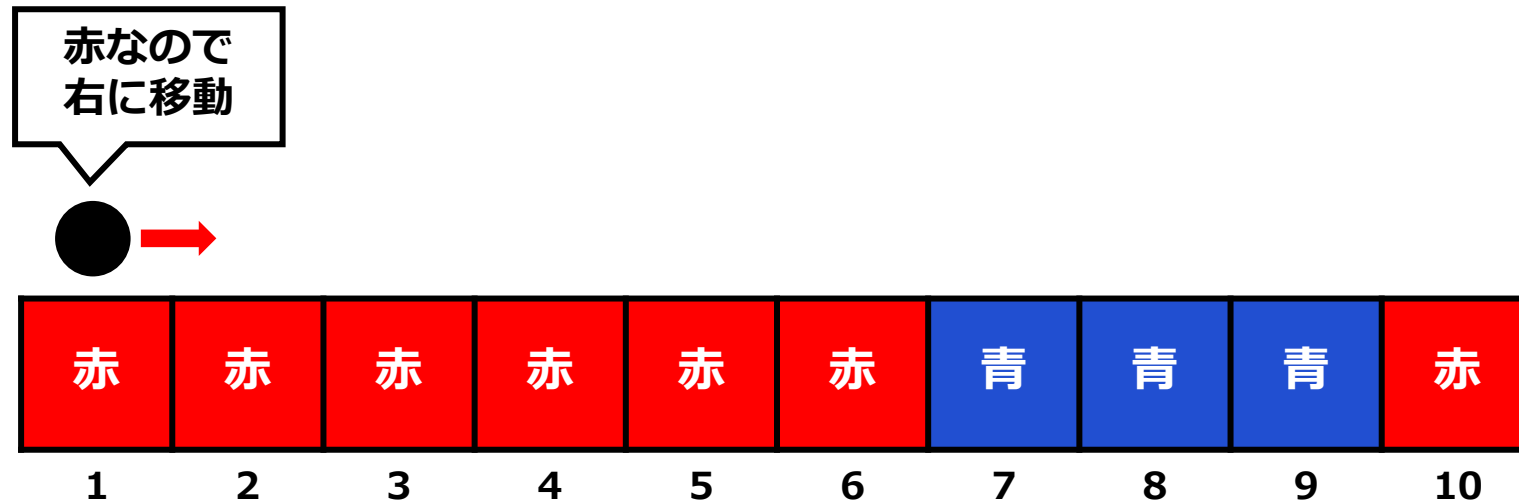
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



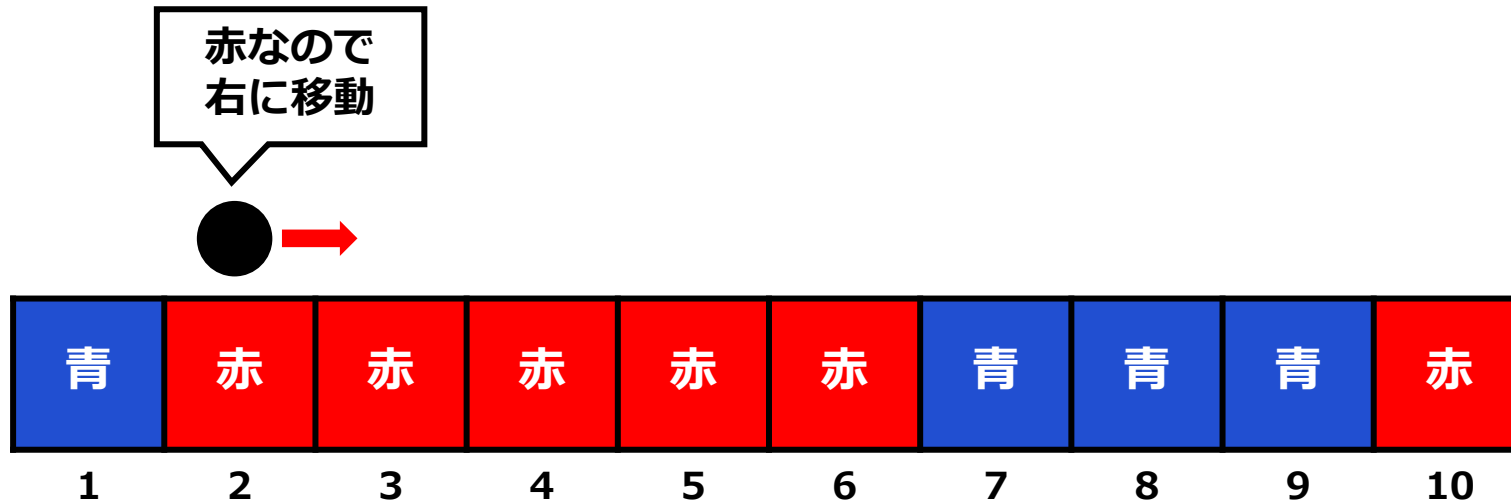
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



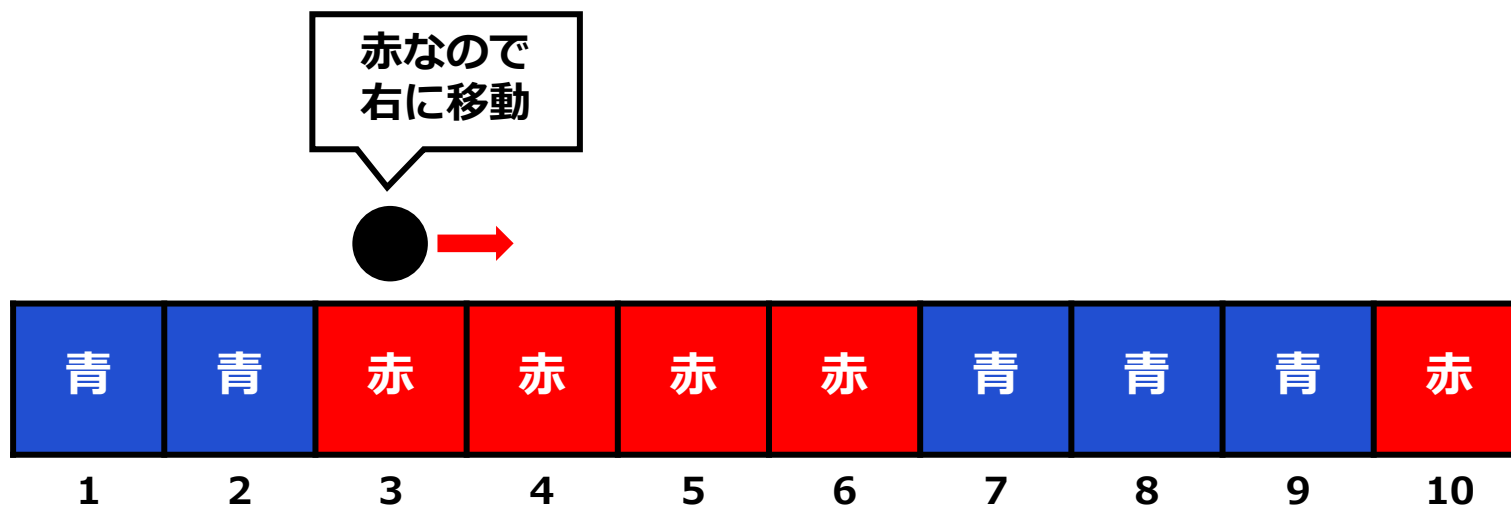
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



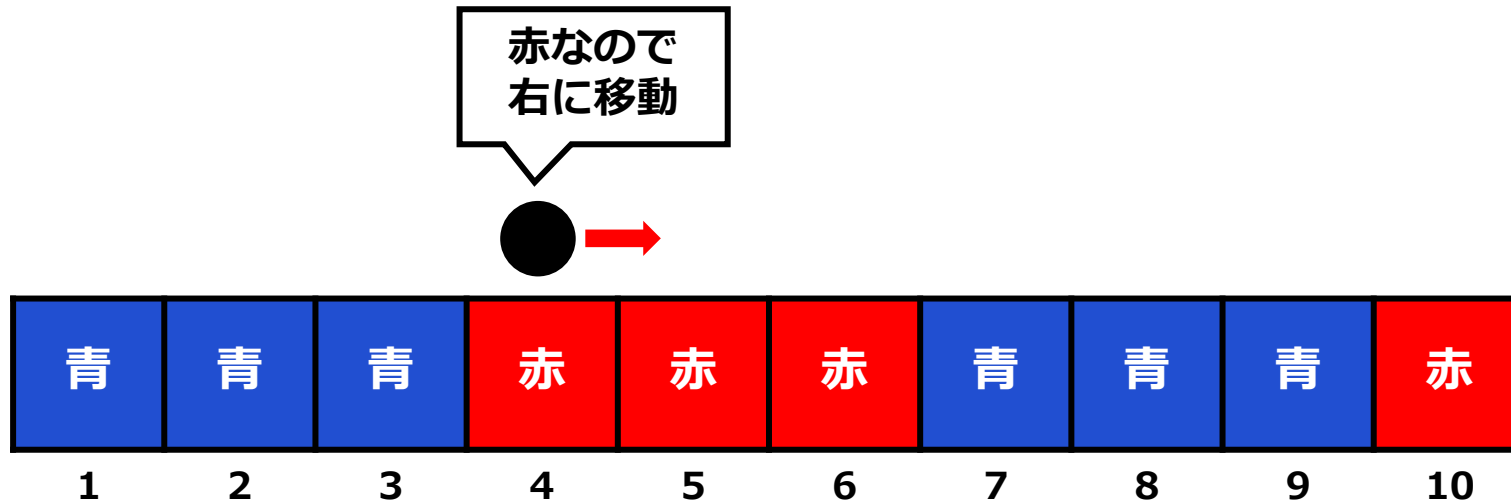
例：初期状態が「赤青青赤青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



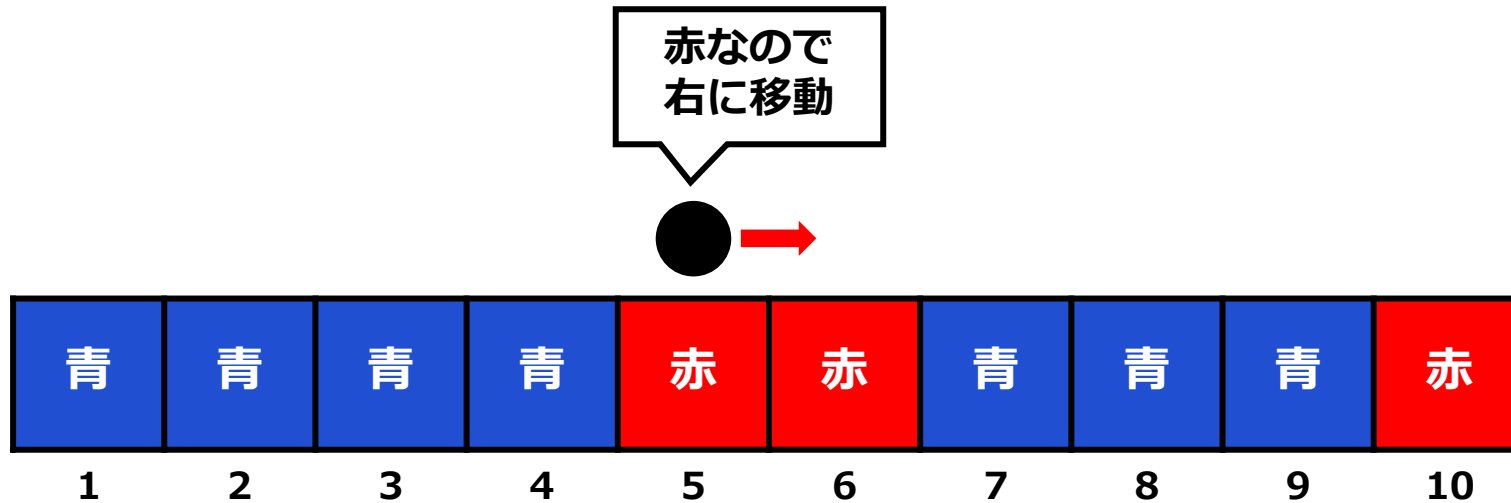
例：初期状態が「赤青青赤青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



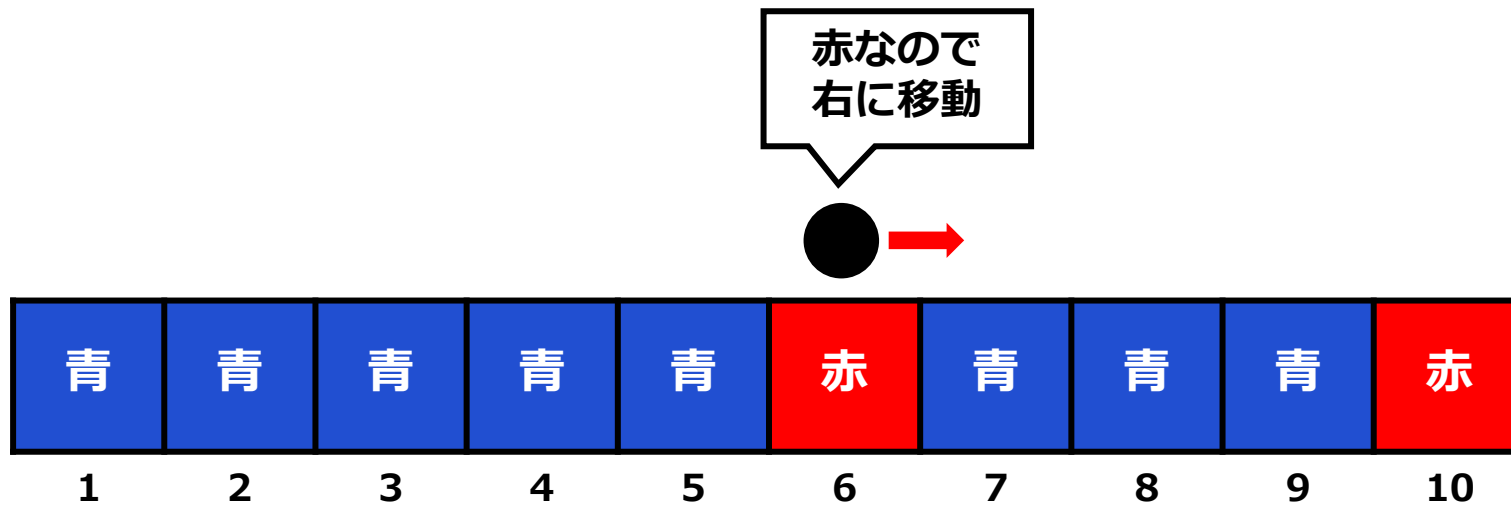
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



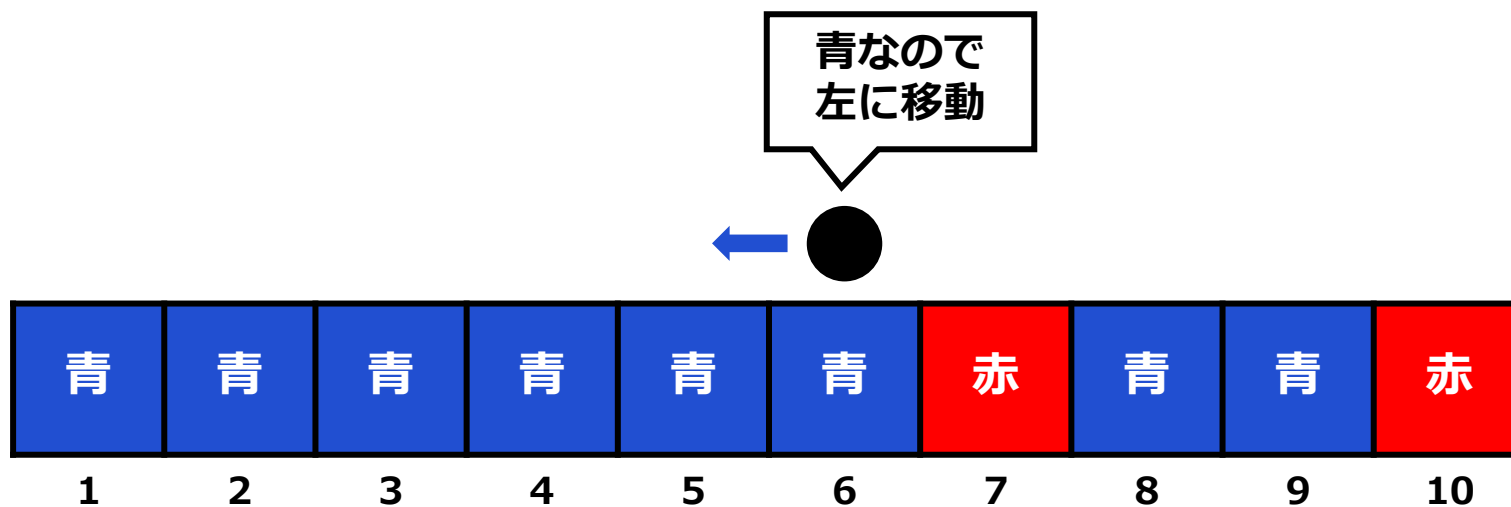
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



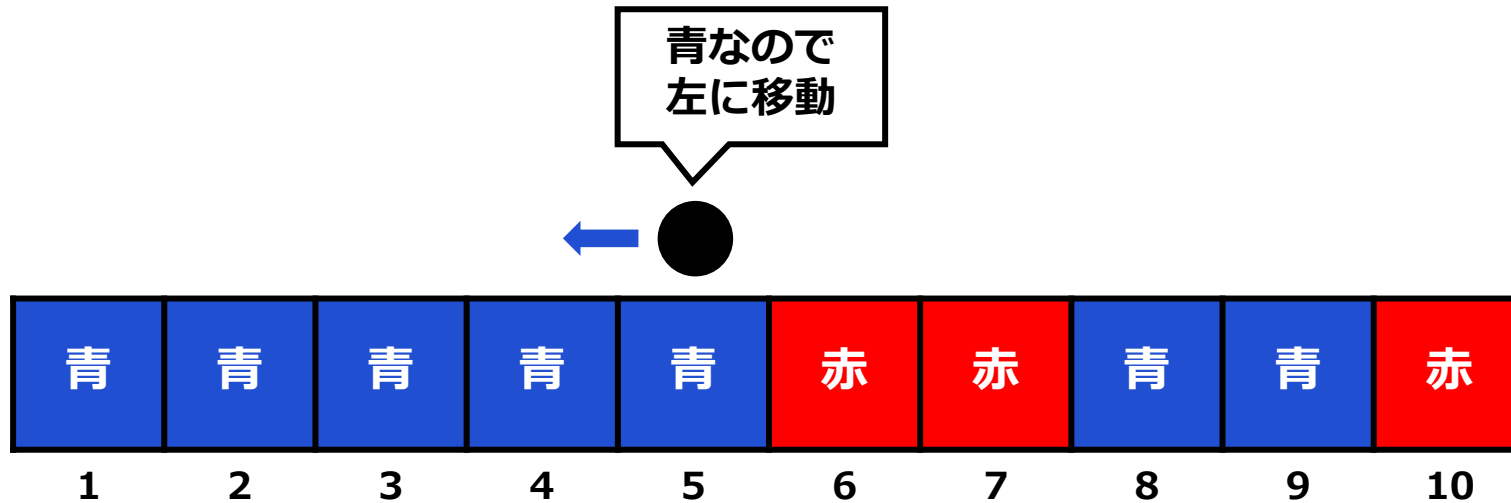
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



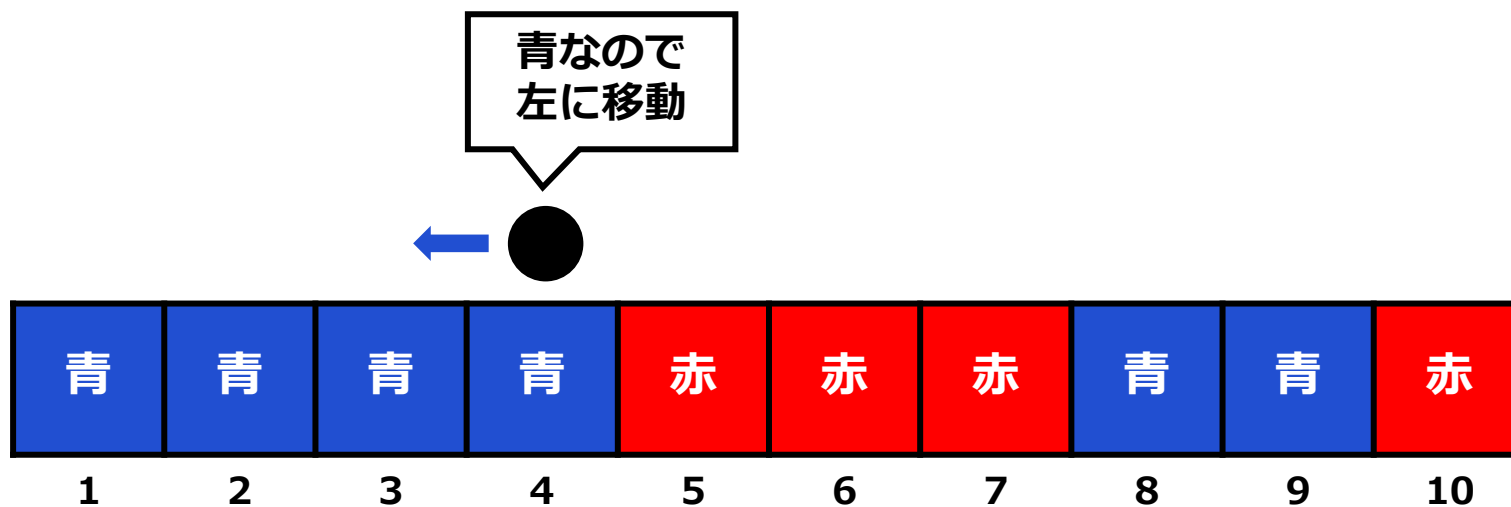
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



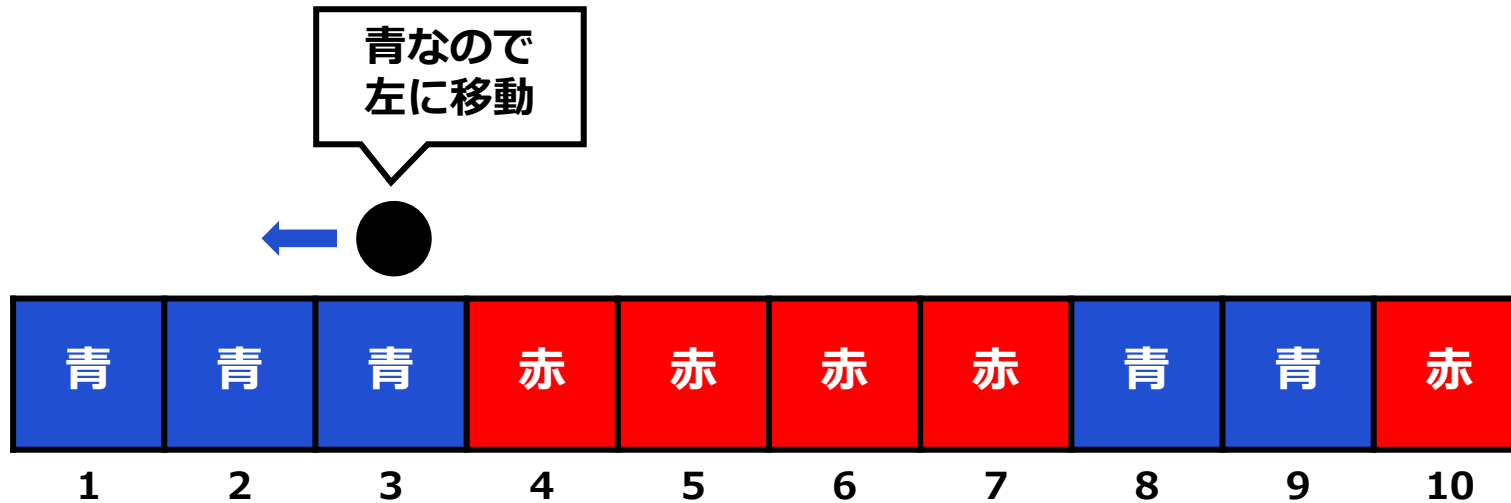
例：初期状態が「赤青青赤青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



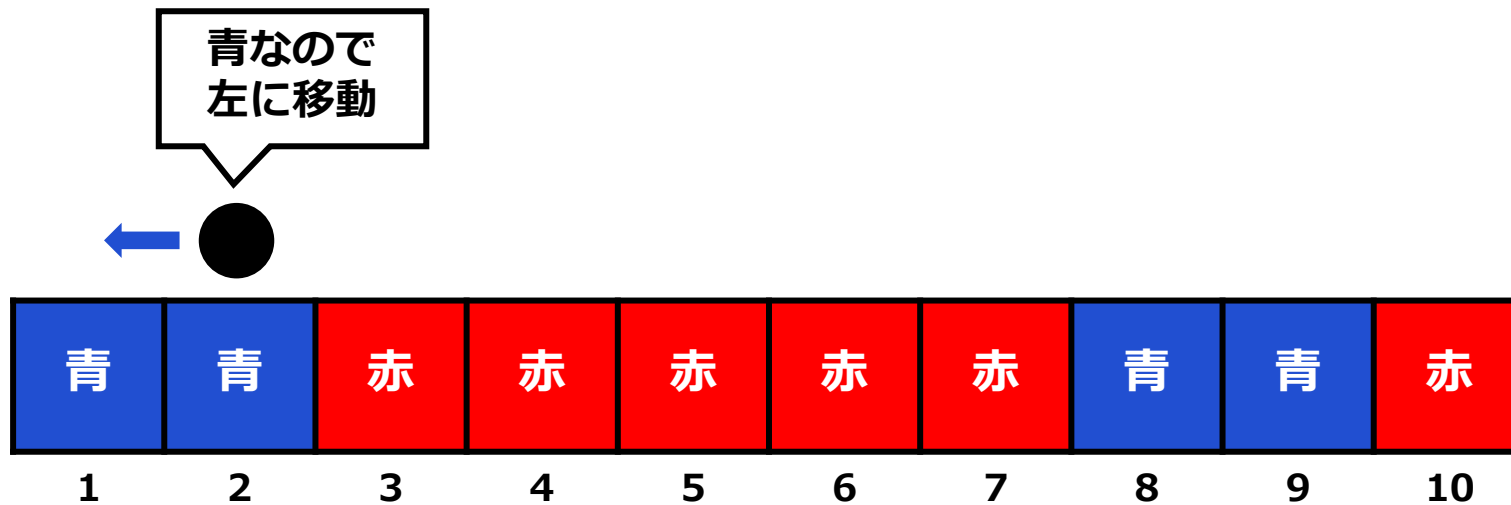
例：初期状態が「赤青青赤青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



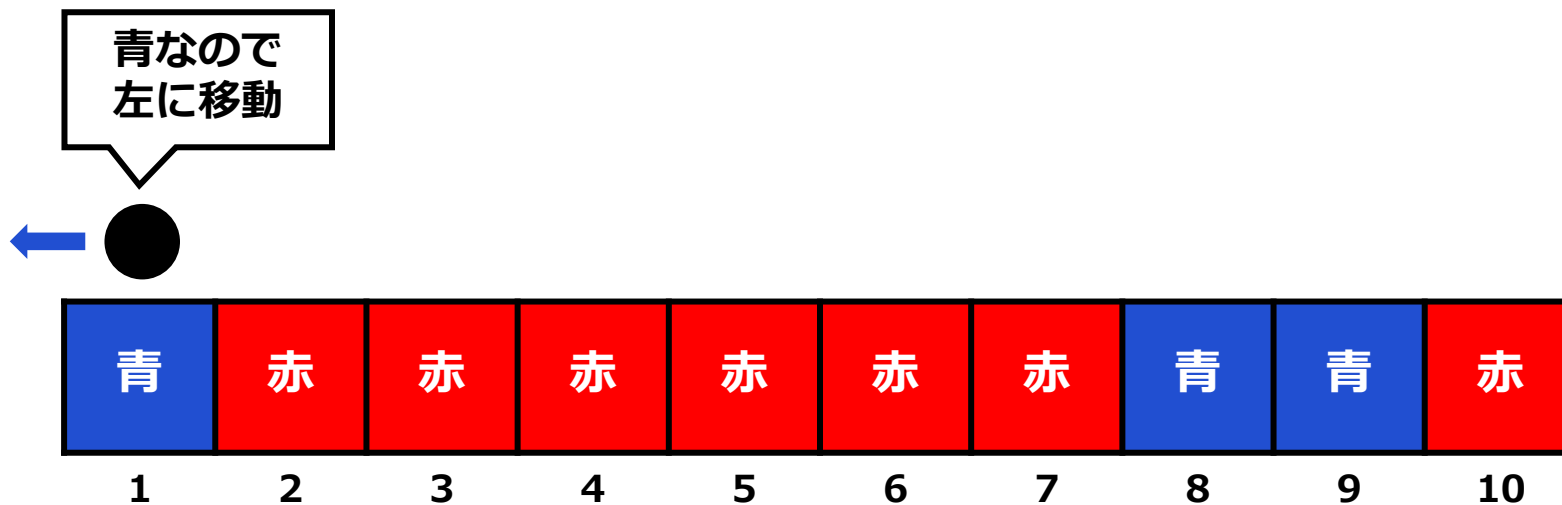
例：初期状態が「赤青青赤青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す



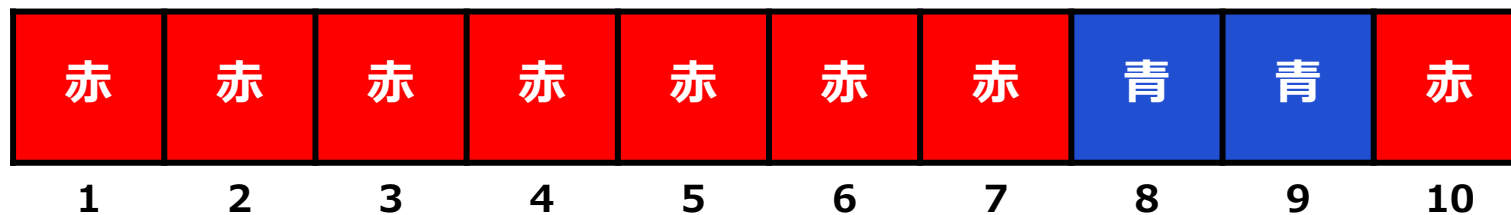
例：初期状態が「赤青青赤青青青青赤」で、ボールをタイル 3 の上に置いた場合どうなるか？

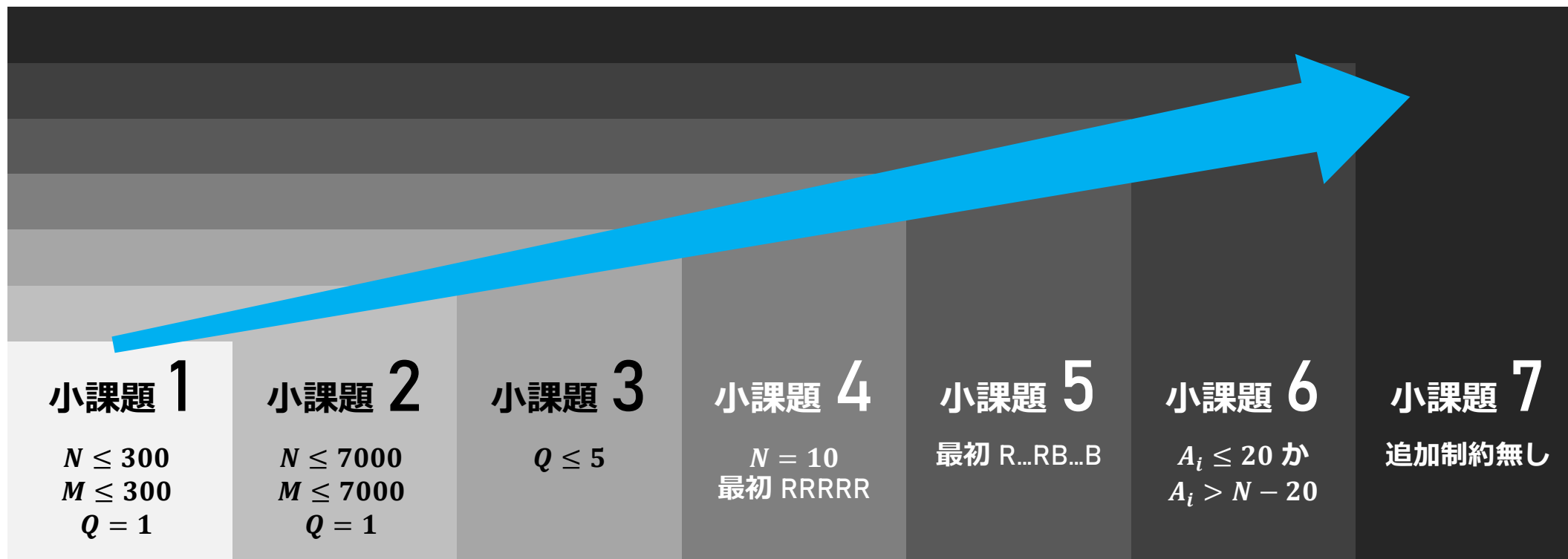
1. ボールがライトタイル A_i の上に置かれ、ライトタイル A_i の色が赤色に変わる
2. ボールが外に出るまで、移動／塗り替えを繰り返す

外に出たので
操作終了



赤のタイルは
8 枚





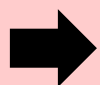
小課題 1

$$N \leq 300, M \leq 300, Q = 1$$

ボールの動きを直接シミュレーションすると、解ける

理由

- 1 回ボタンを押した後のステップ数は、 $O(N^2)$ 回以下になるため
- ボタンは高々 M 回押されるため



計算量は $O(N^2 M)$ なので、 $N \leq 300, M \leq 300$ で間にあう

3点

ボールの動きを直接シミュレーションすると、解ける

理由

- 1回ボタンを押した後のステップ数は、 $O(N^2)$ 回以下になるため
- ボタンは高々 M 回押されるため



計算量は $O(N^2)$

こういうケースで $\Theta(N^2)$ 回かかる



で間にある

3点

小課題 2

$N \leq 2000, M \leq 2000, Q = 1$

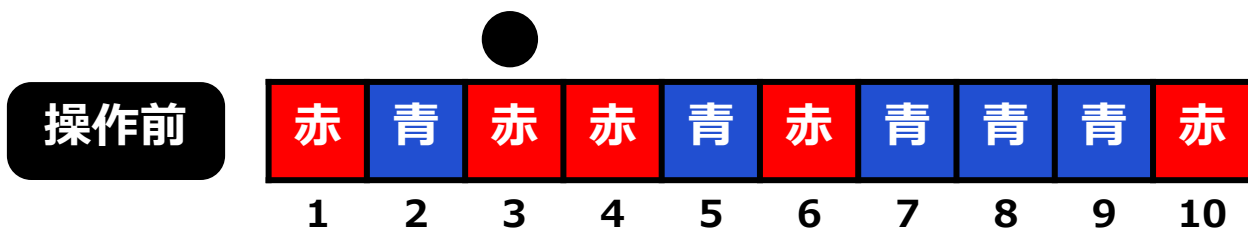
ボールをタイル x の上に置いたときの、操作後の盤面は…

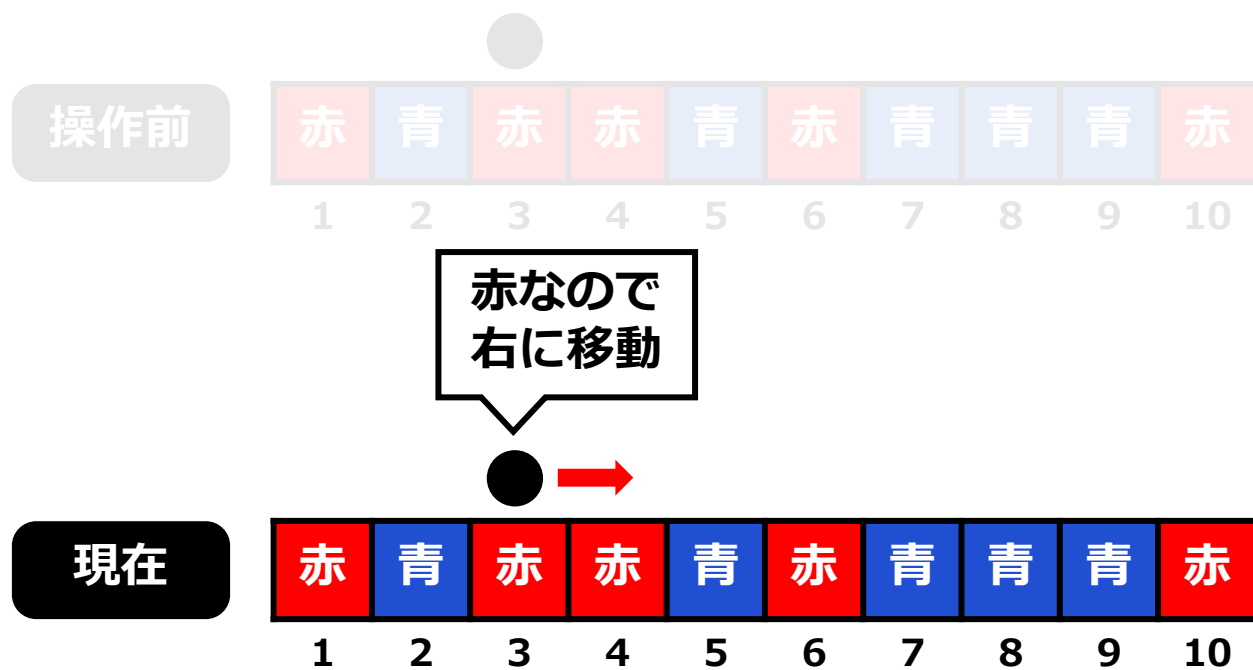
ボールが左から
出た場合

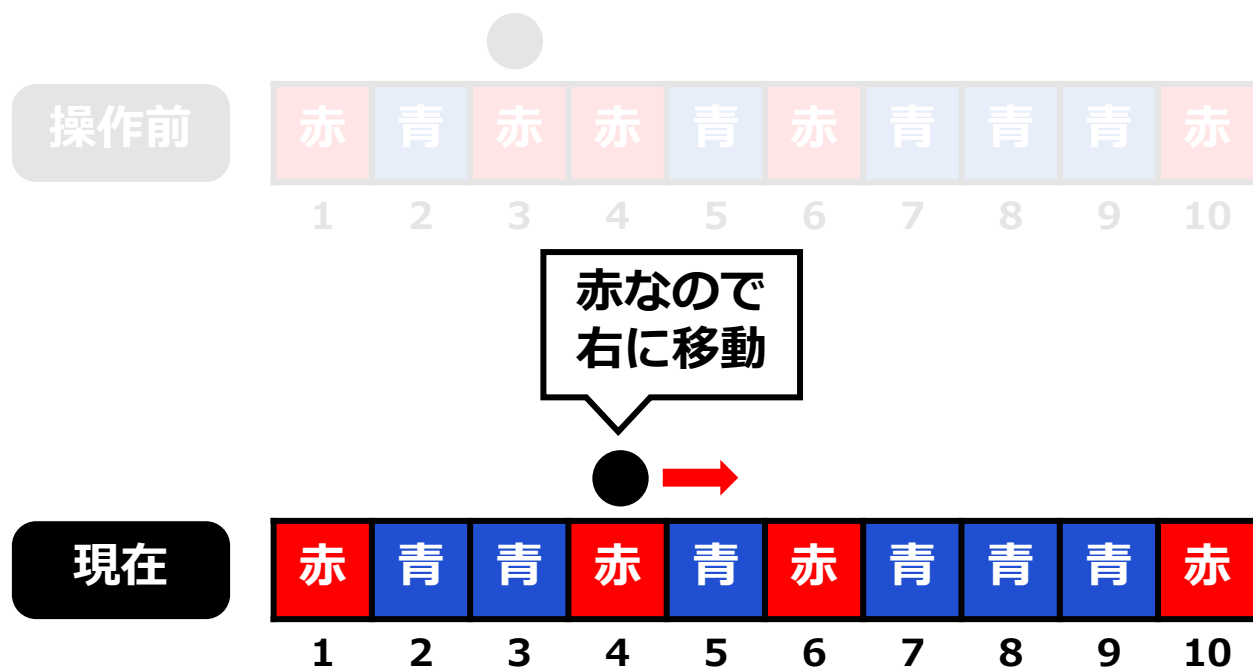
左から x 番目の赤タイルまでが青になる

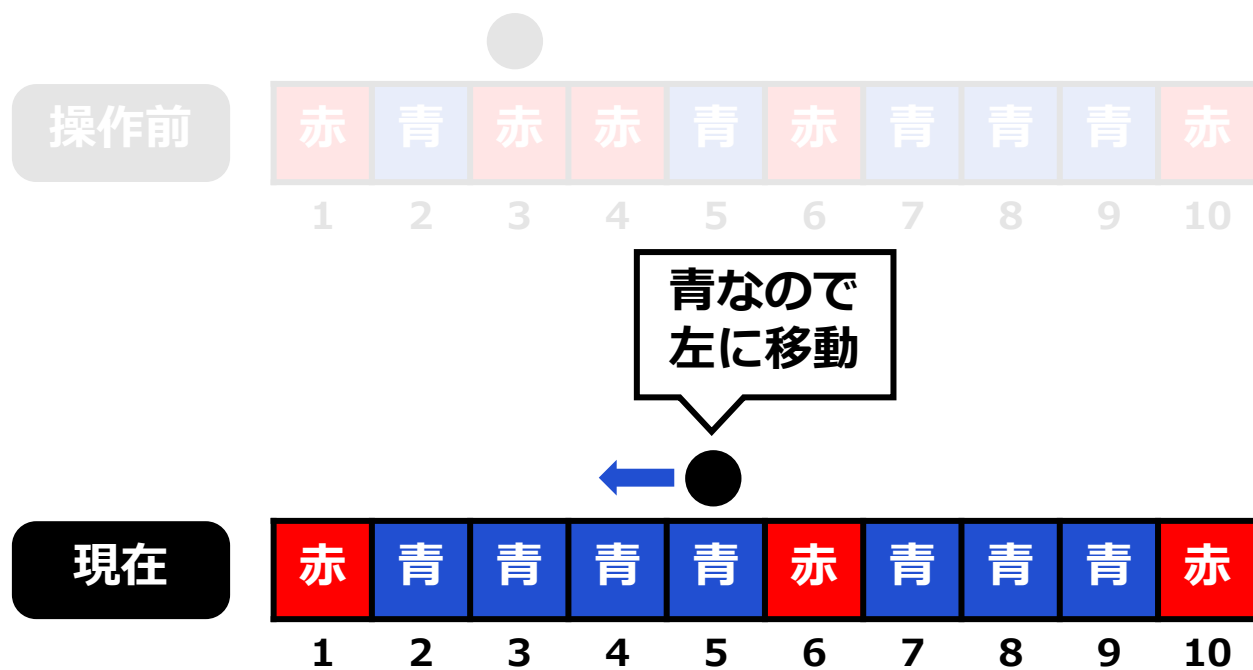
ボールが右から
出た場合

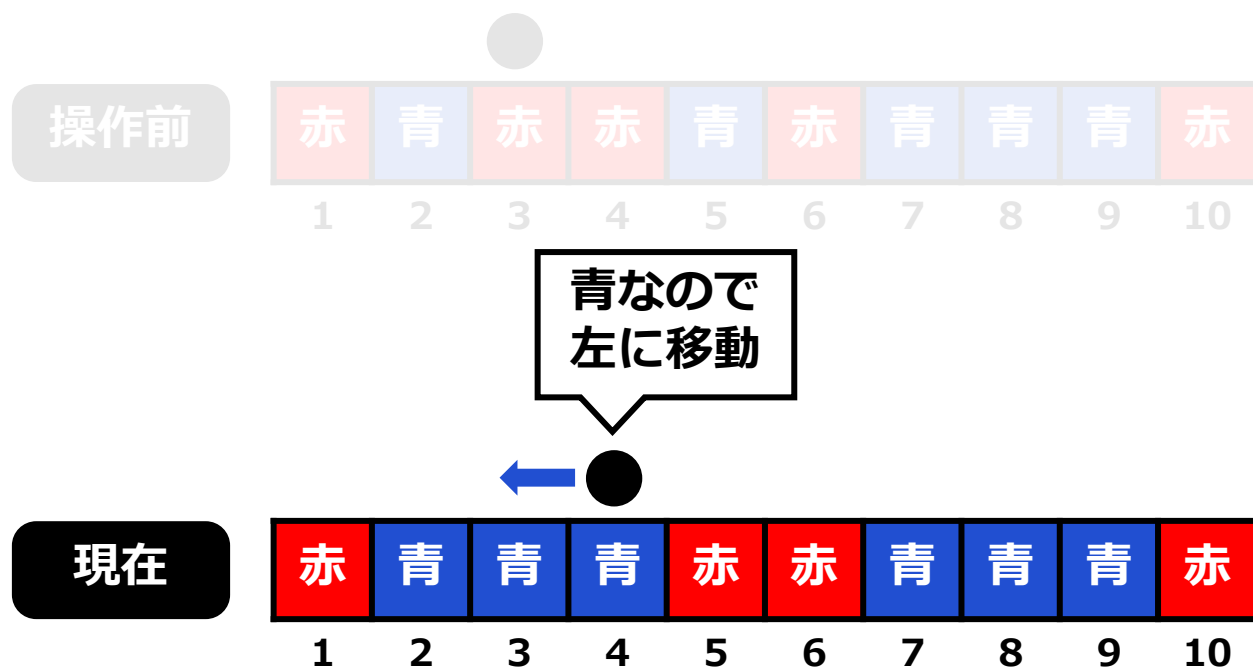
右から $N + 1 - x$ 番目の青タイルまでが赤になる

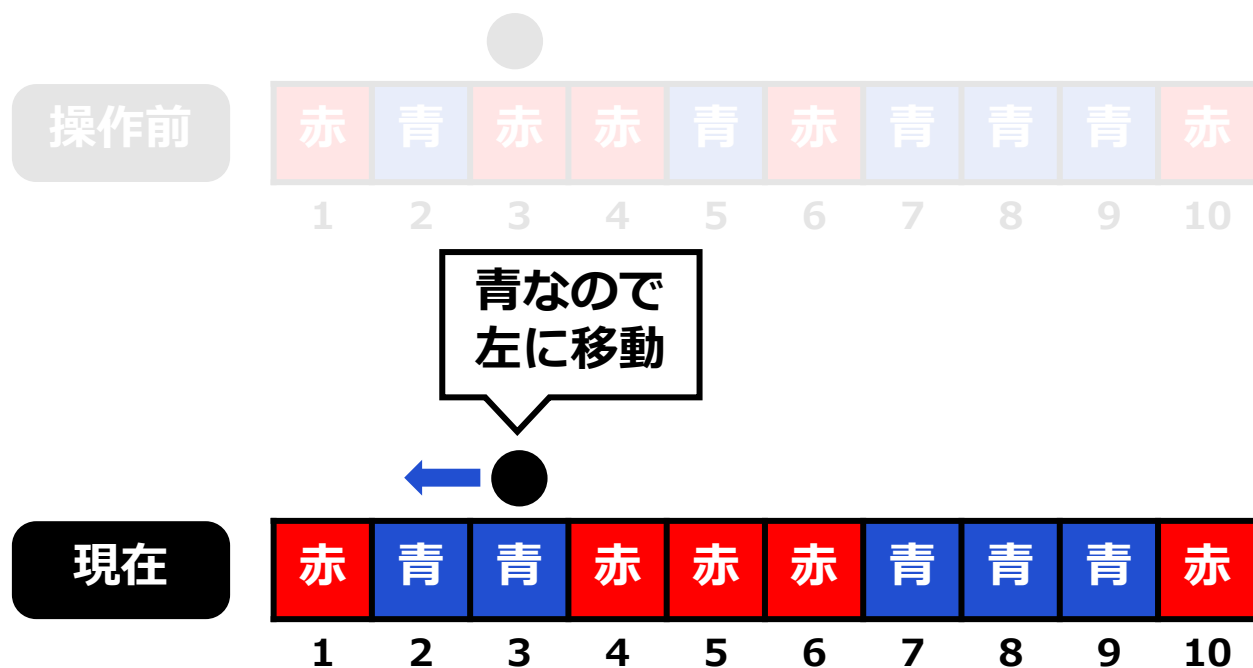


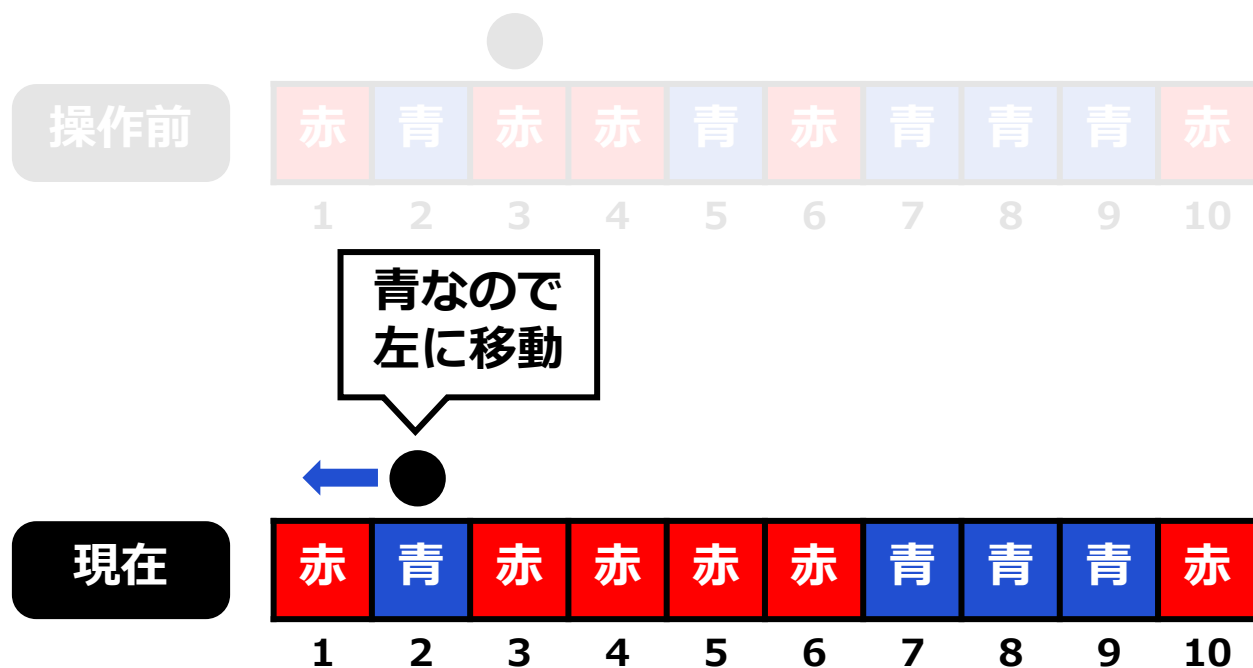


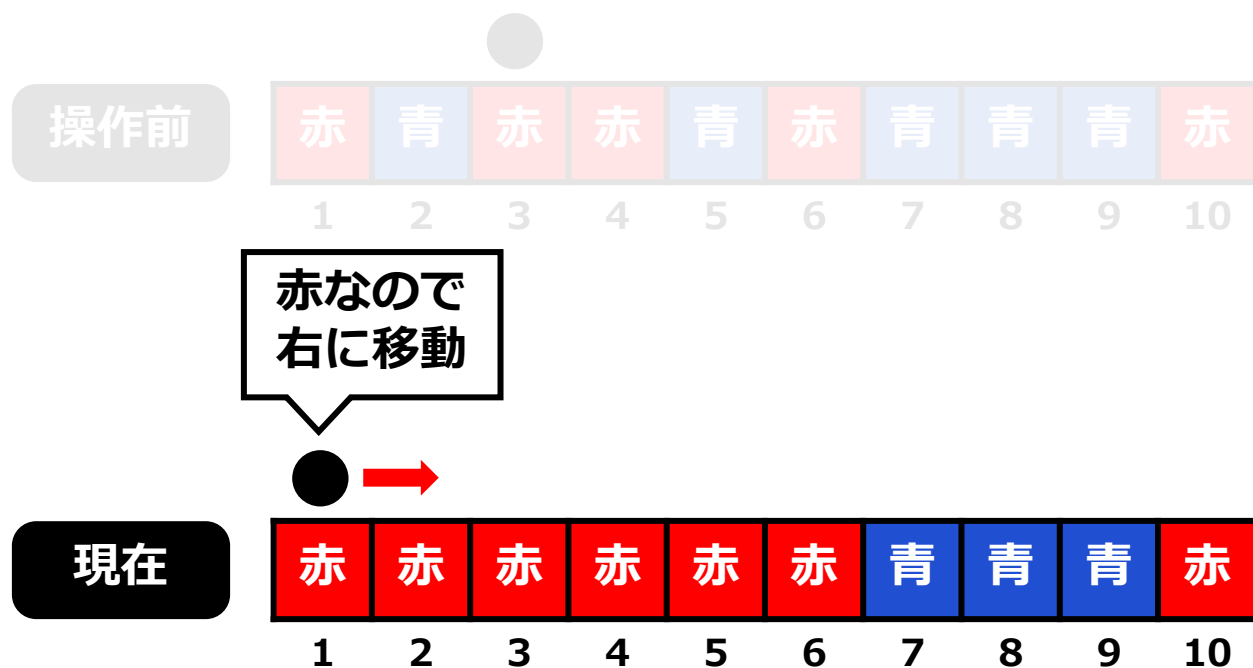


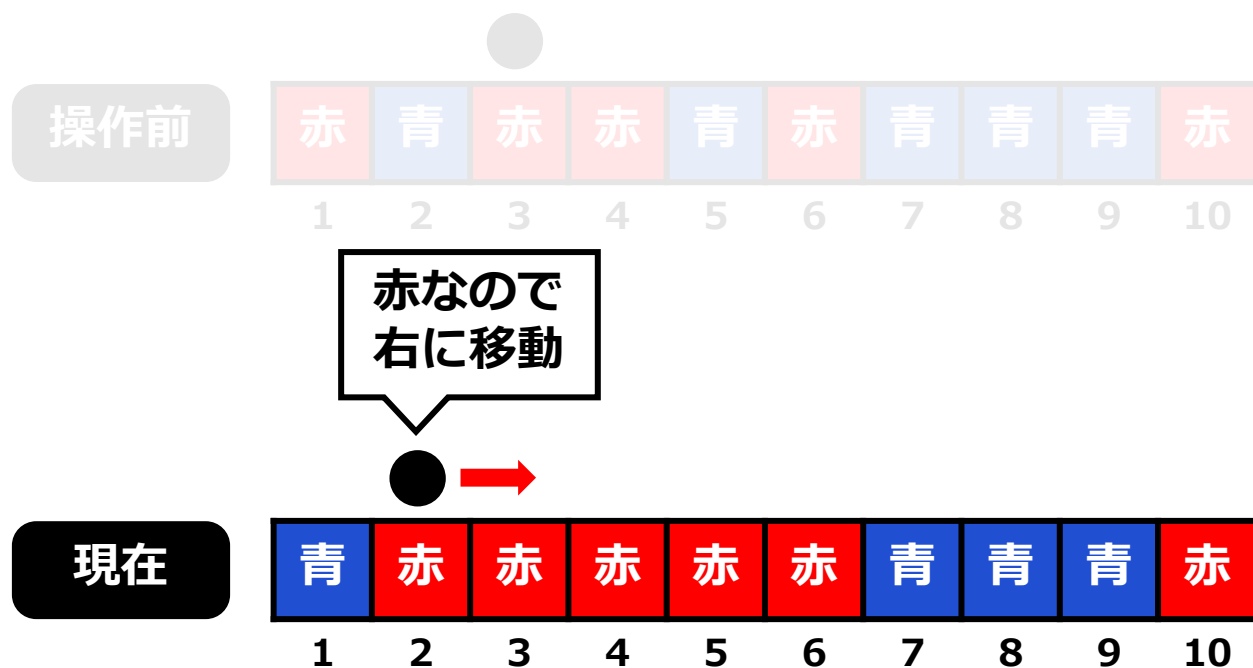


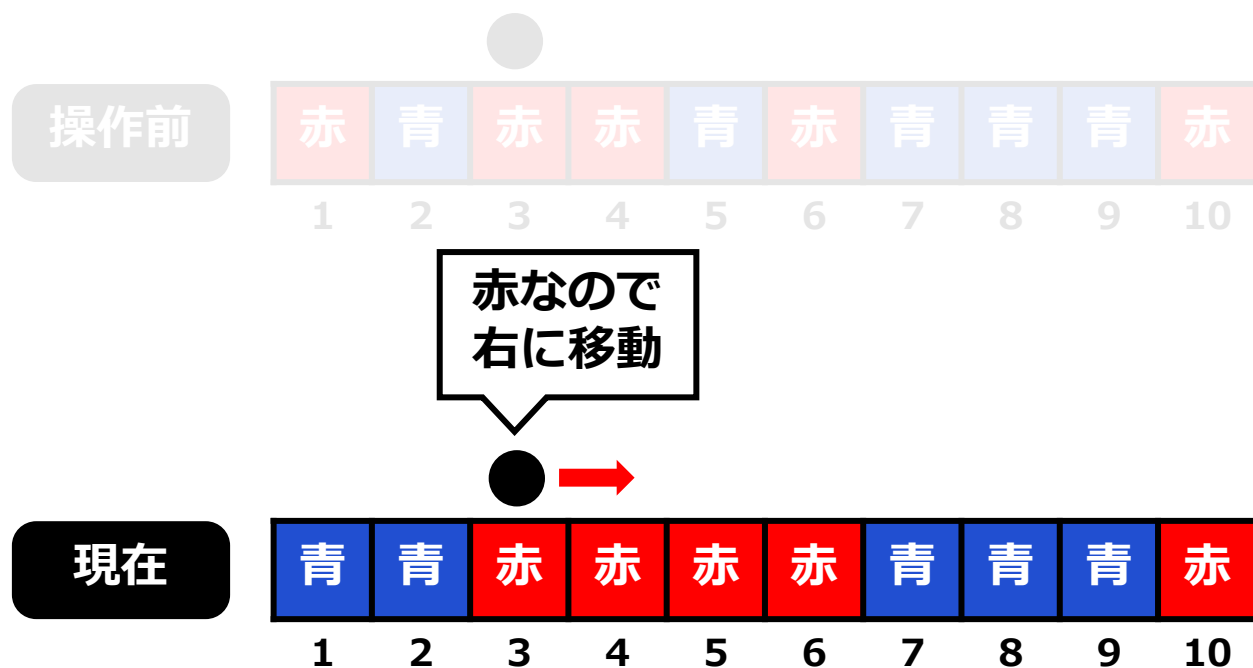


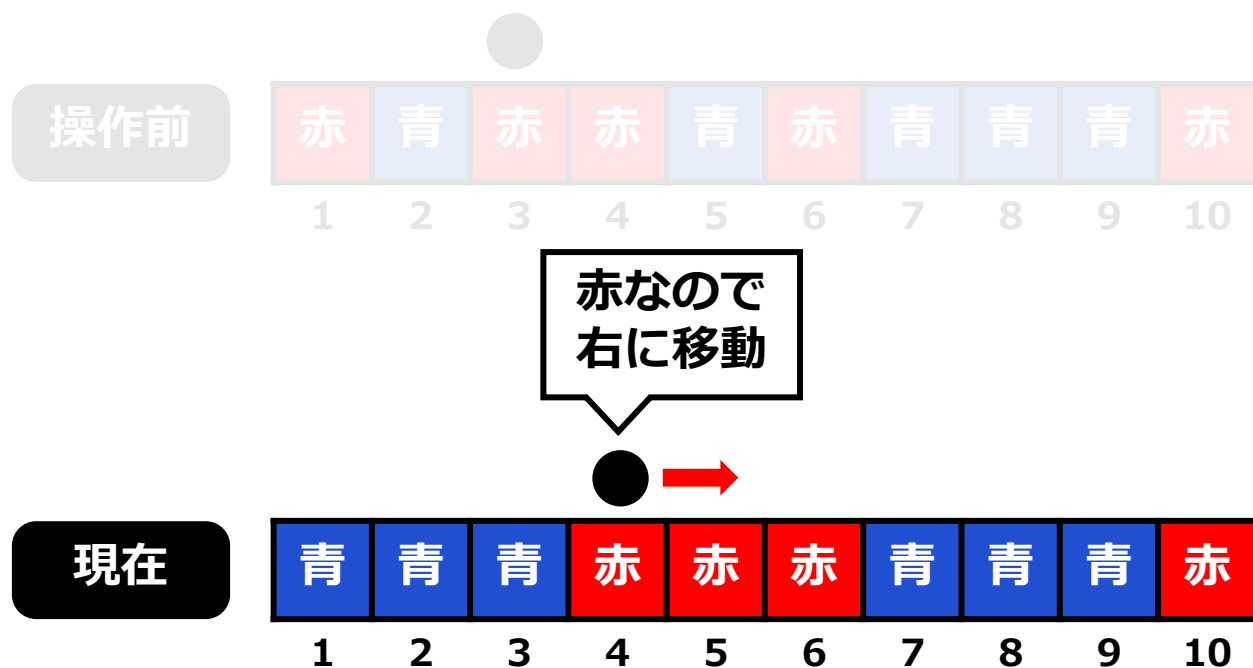


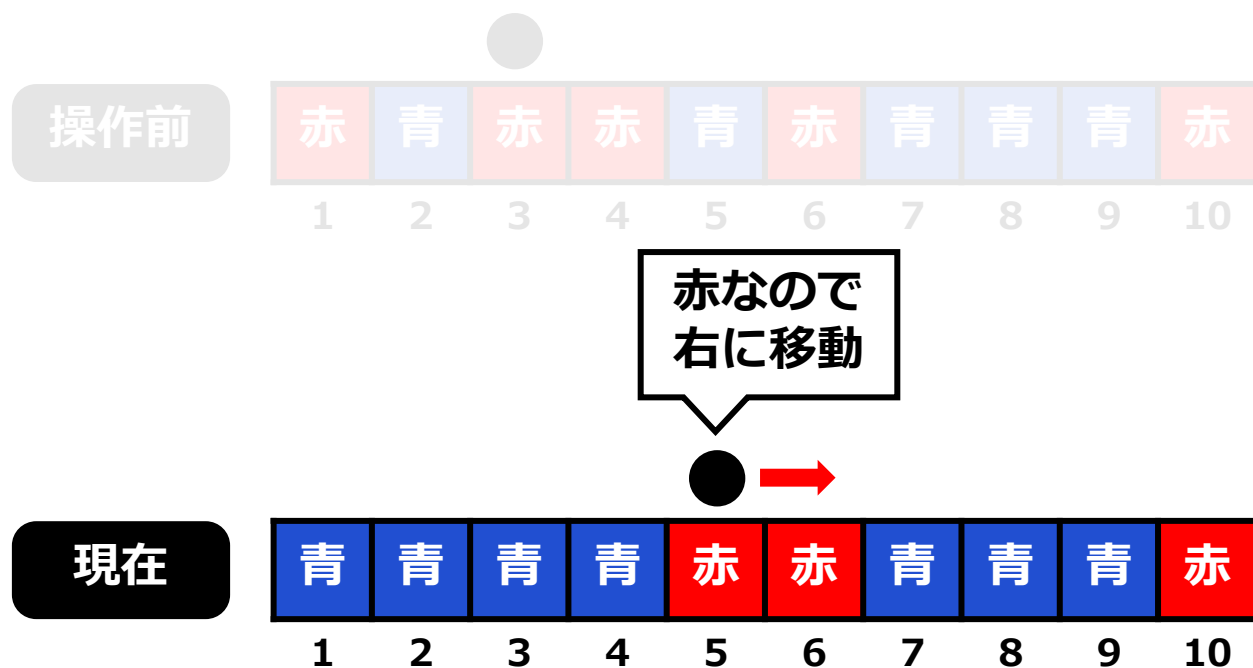


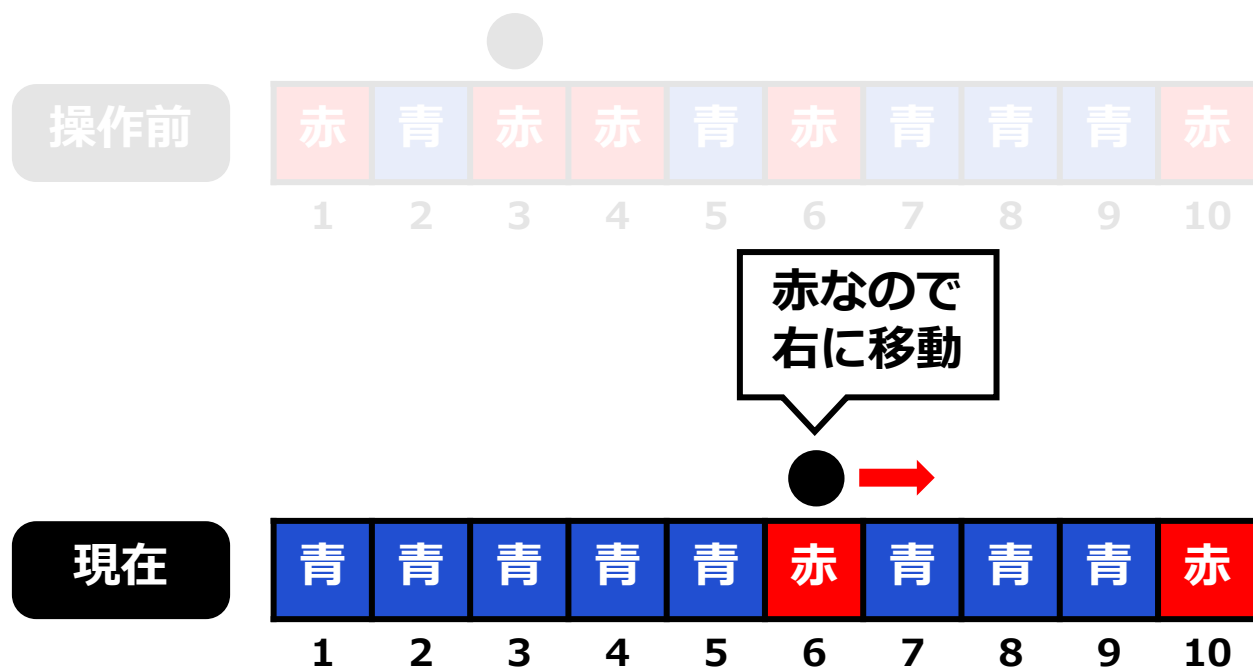


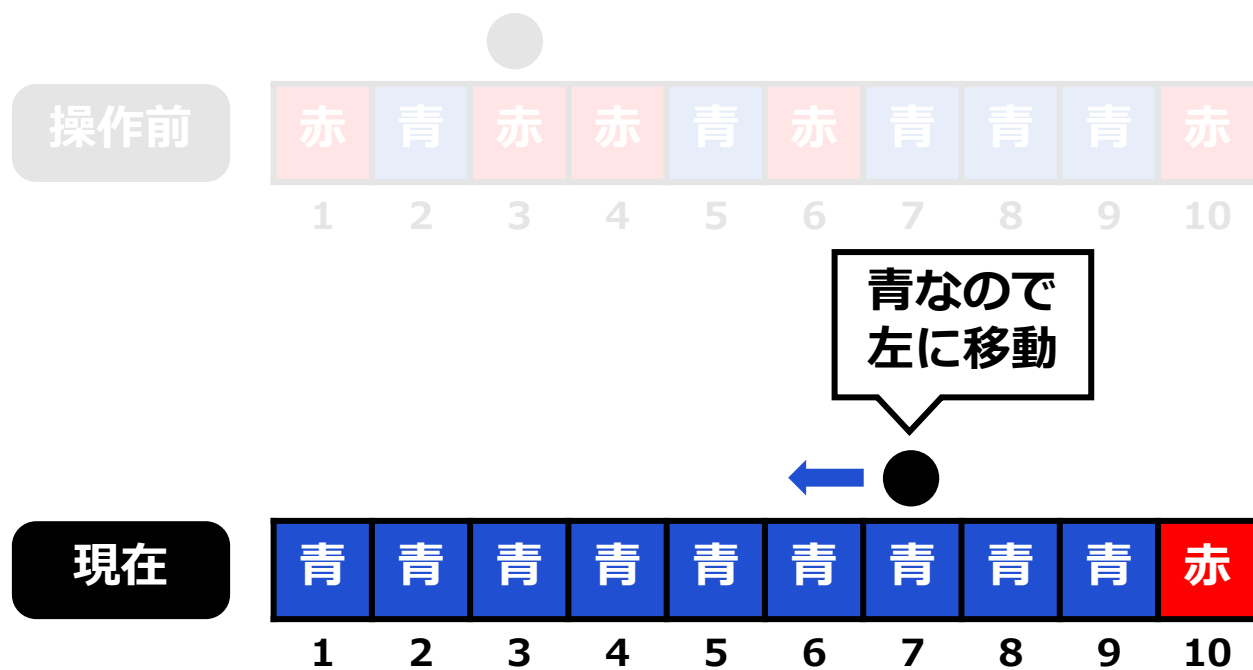


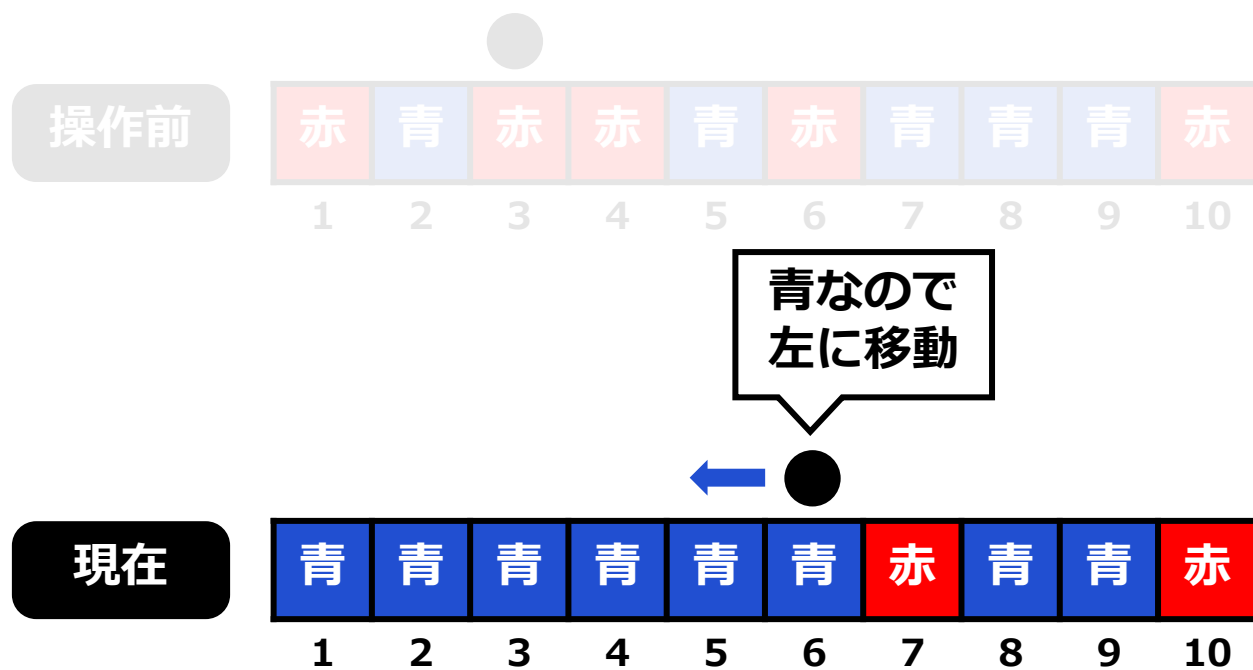


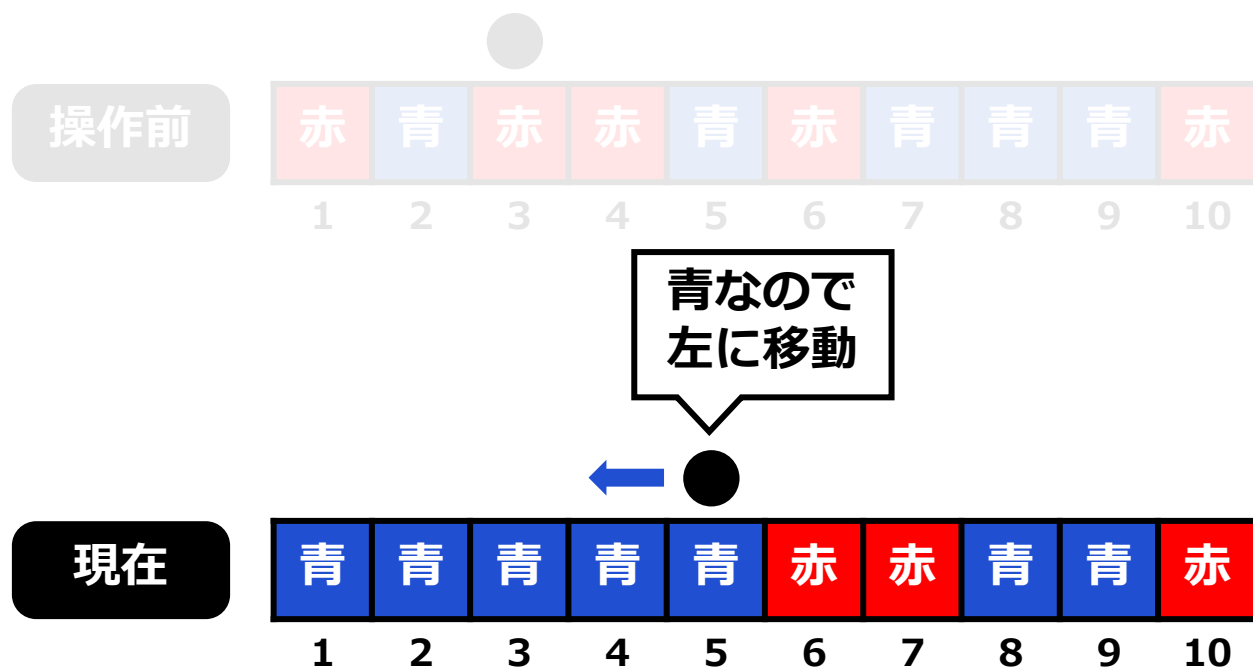


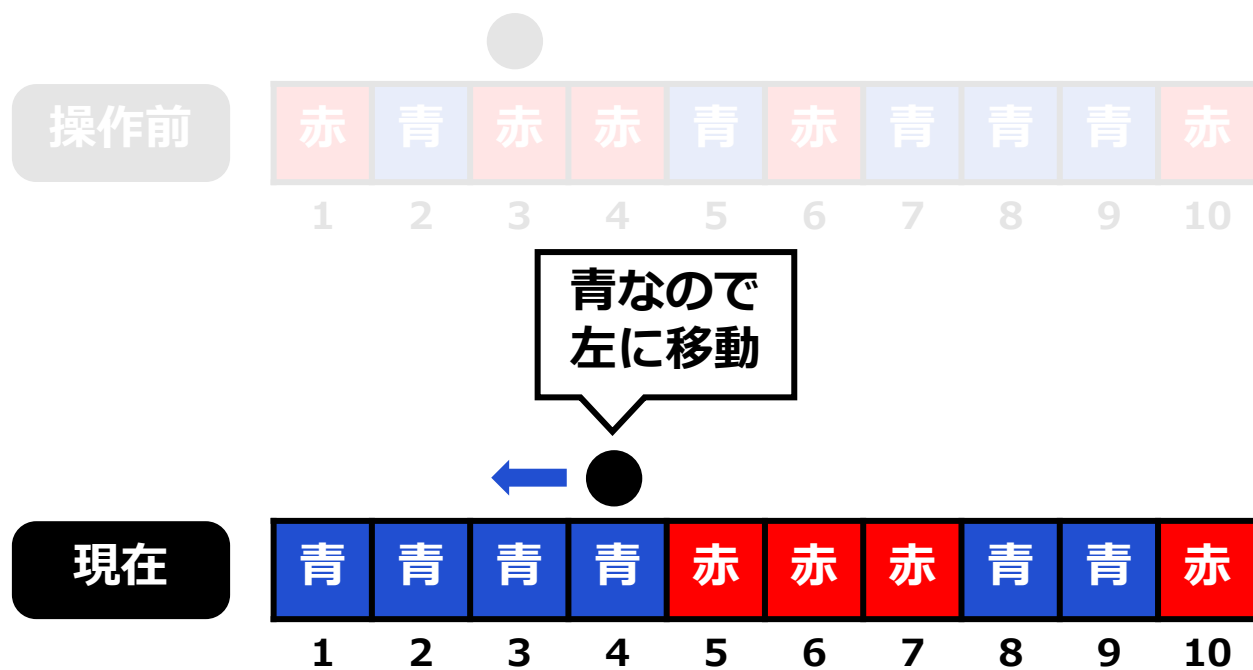


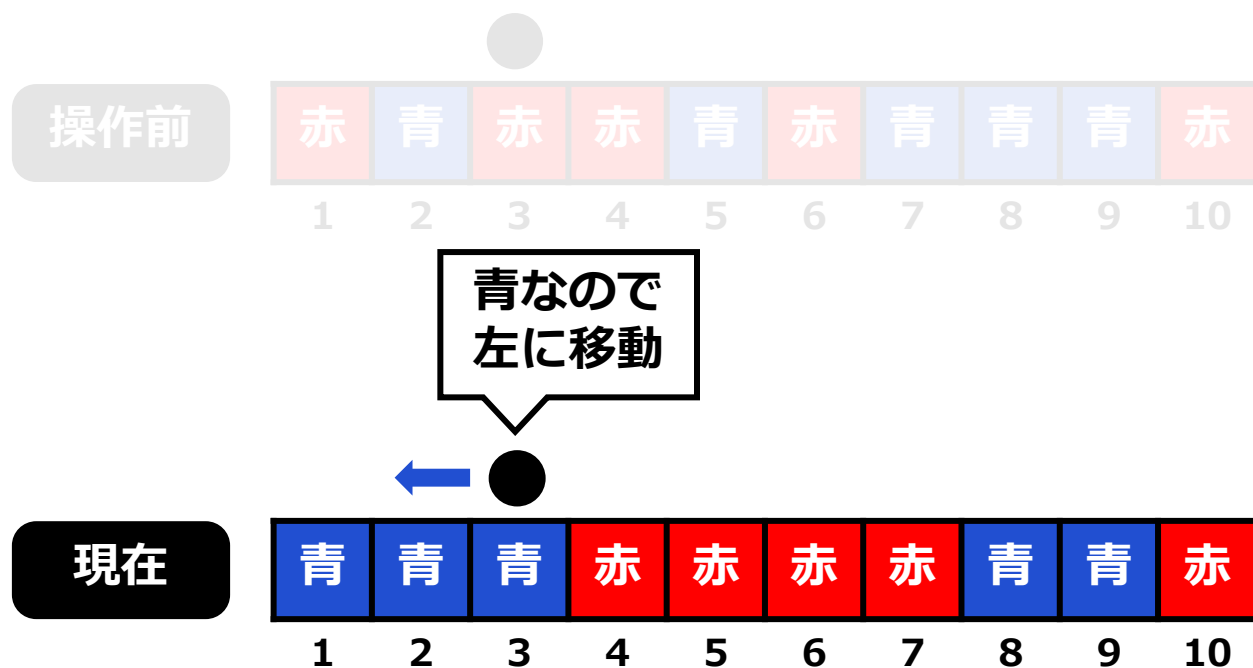


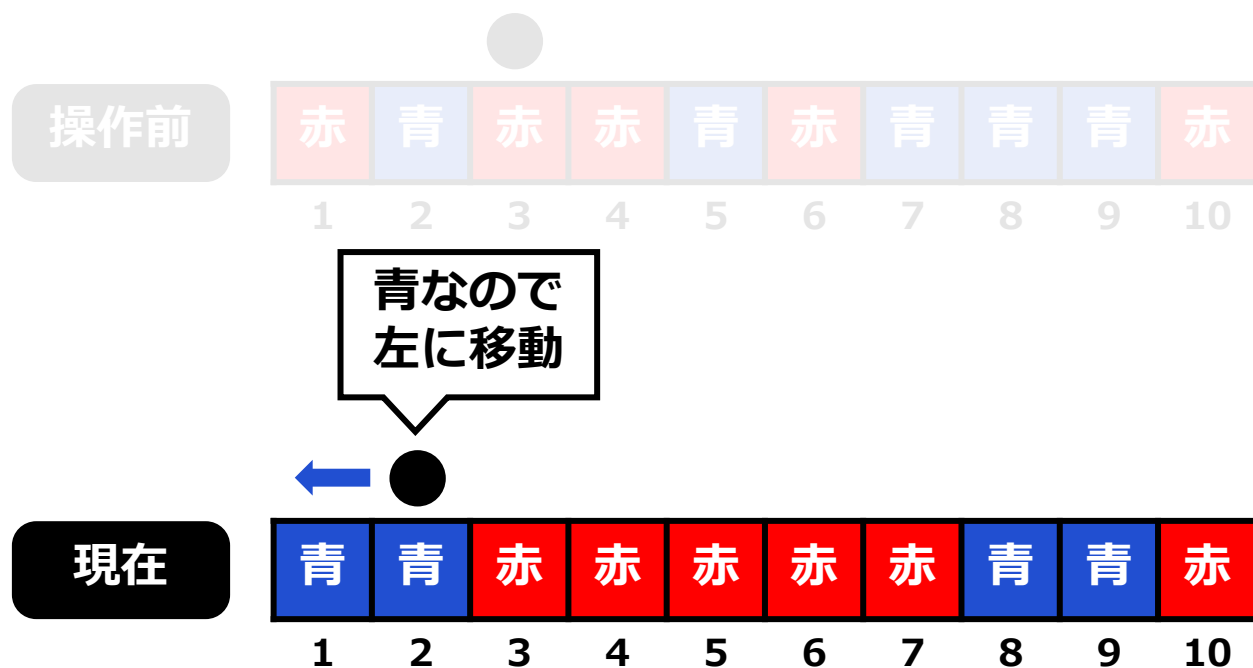


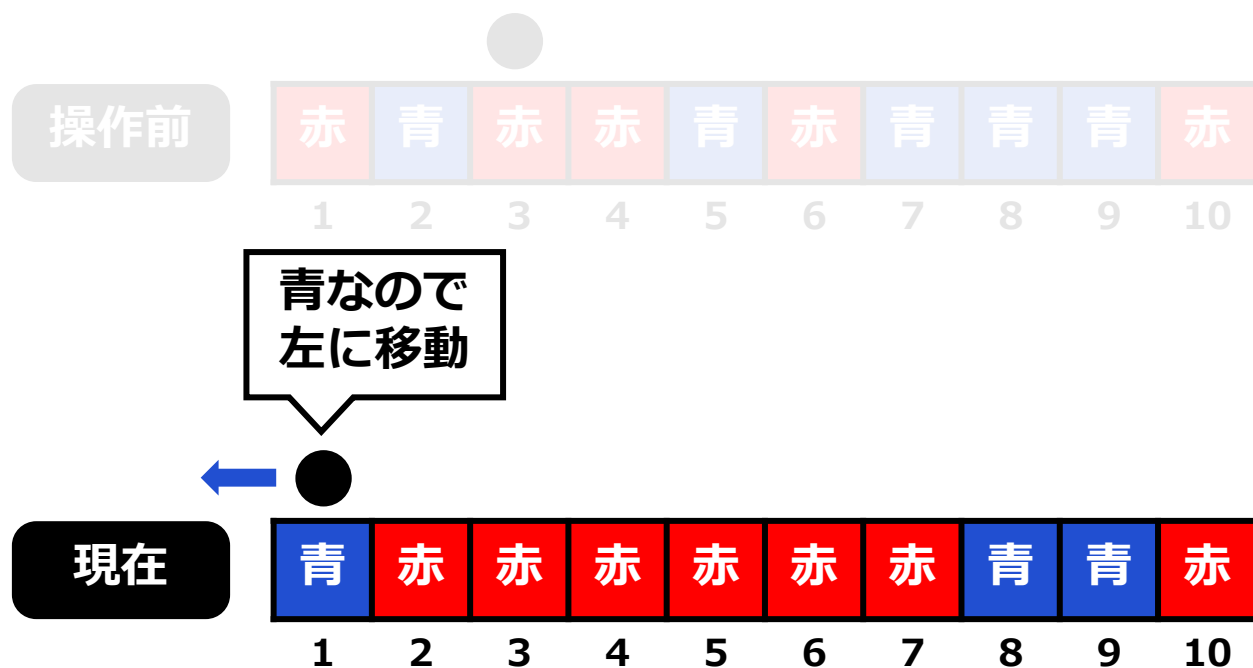




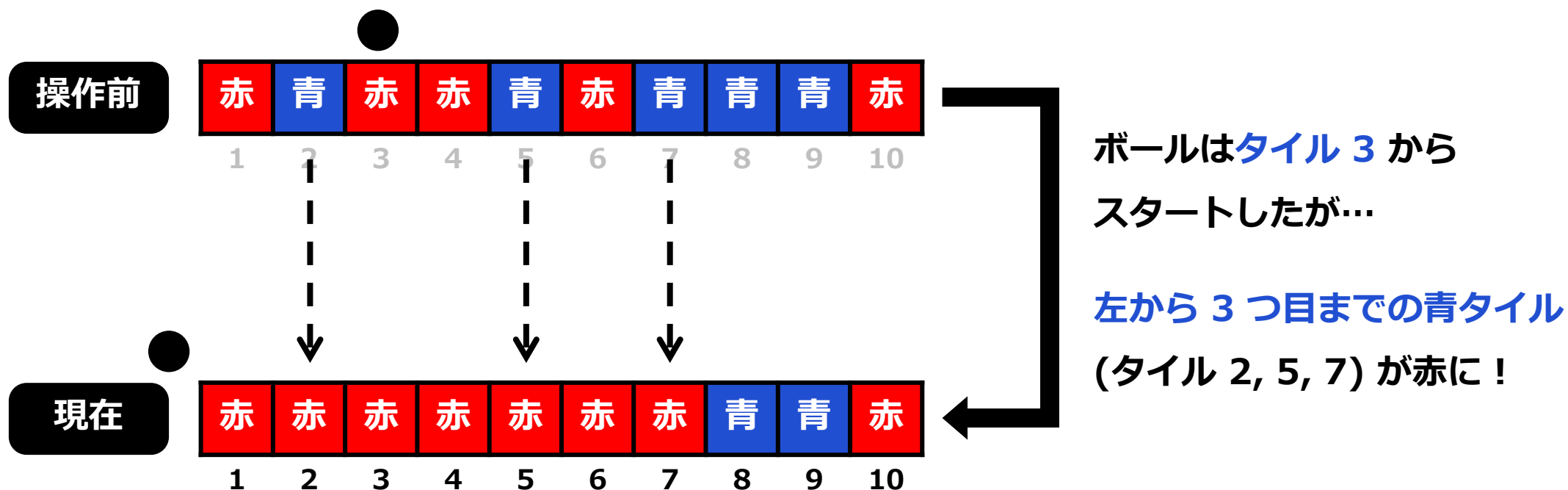








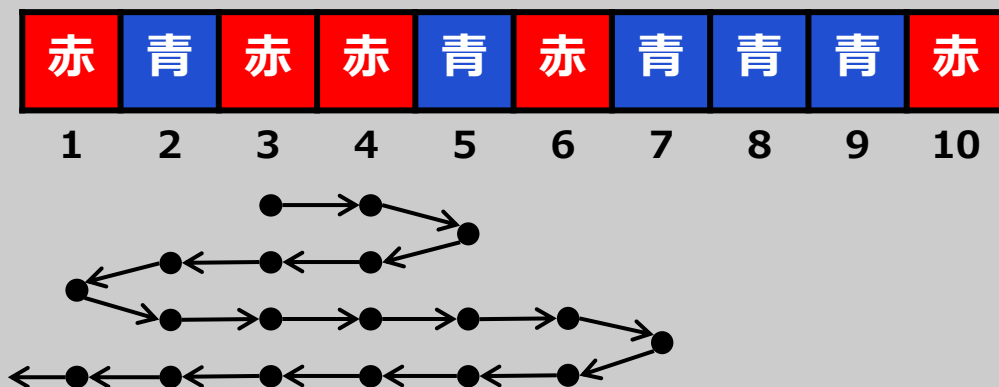




どうしてこんなことになるのか？

まずはボールが左から出る場合を証明しましょう

まず、タイル x にあったボールが左から出た場合、「右に移動する回数」より「左に移動する回数」の方が x 回だけ多い



たとえば先程の例の場合

ボールはタイル 3 から出発するが...

- ・ 右移動：8回
- ・ 左移動：11回 ← **3回多い**

つまり、「青→赤になったタイル」が「赤→青になったタイル」より x 個だけ多いことになる

なぜか？

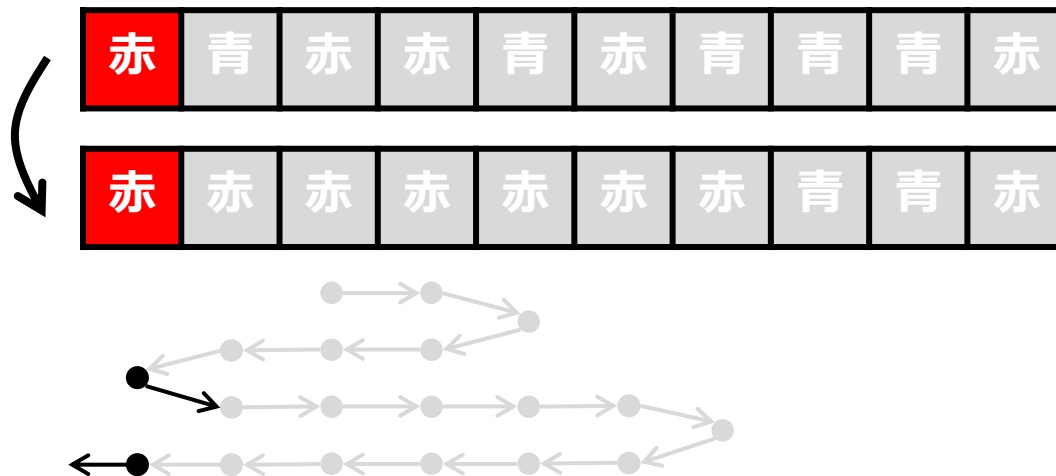
- 青→赤になったタイルについては右移動の方が 1 回多い
- 赤→青になったタイルについては左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

つまり、「青→赤になったタイル」が「赤→青になったタイル」より
 x 個だけ多いことになる

なぜか？

- 青→赤になったタイルについては
右移動の方が 1 回多い
- 赤→青になったタイルについては
左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

タイル1：赤→赤
・ 左移動 1 回
・ 右移動 1 回



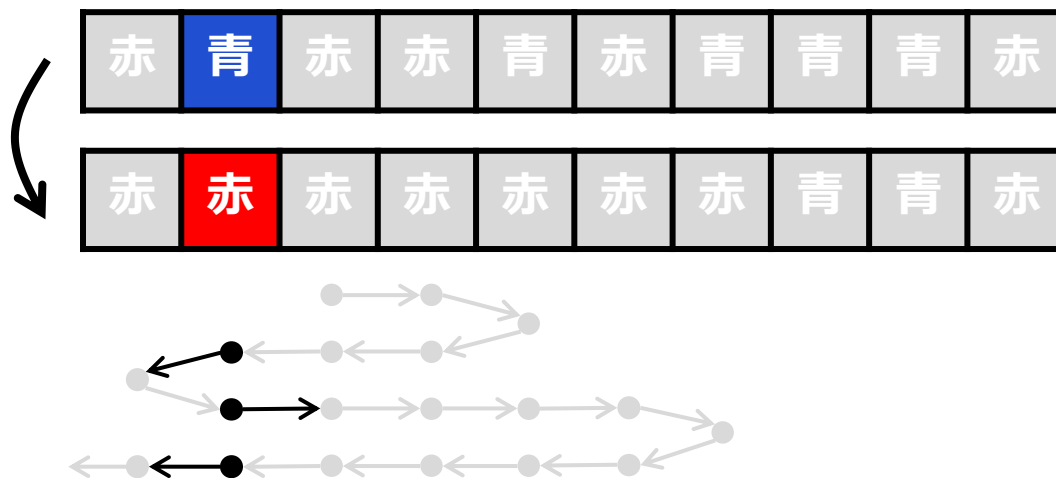
つまり、「青→赤になったタイル」が「赤→青になったタイル」より
 x 個だけ多いことになる

なぜか？

- 青→赤になったタイルについては
右移動の方が 1 回多い
- 赤→青になったタイルについては
左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

タイル2：青→赤

- 左移動 2 回
- 右移動 1 回



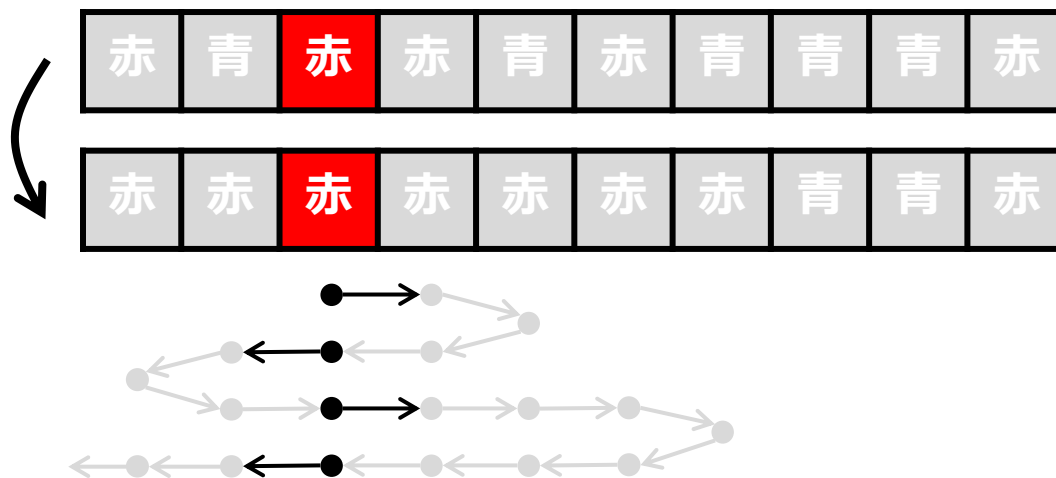
つまり、「青→赤になったタイル」が「赤→青になったタイル」より
 x 個だけ多いことになる

なぜか？

- 青→赤になったタイルについては
右移動の方が 1 回多い
- 赤→青になったタイルについては
左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

タイル3：赤→赤

- 左移動 2 回
- 右移動 2 回



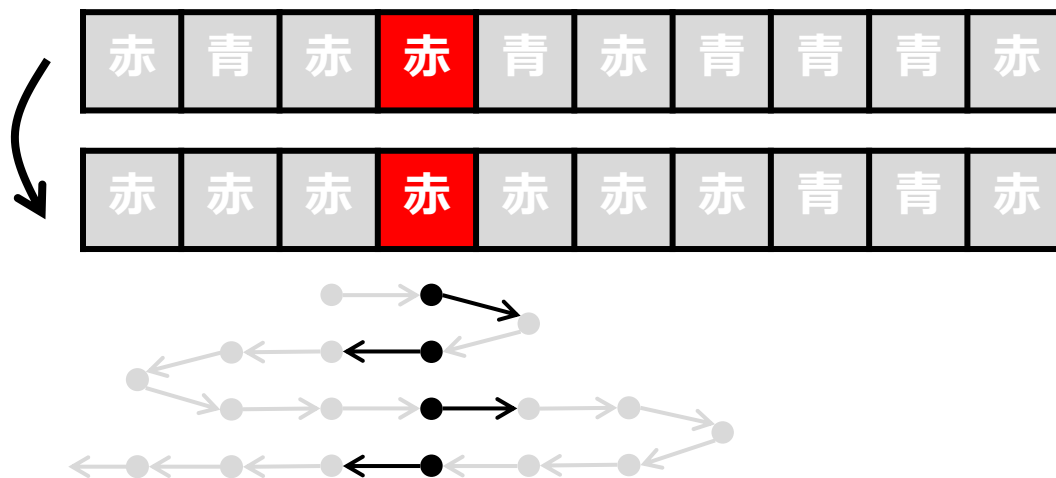
つまり、「青→赤になったタイル」が「赤→青になったタイル」より
 x 個だけ多いことになる

なぜか？

- 青→赤になったタイルについては
右移動の方が 1 回多い
- 赤→青になったタイルについては
左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

タイル4：赤→赤

- 左移動 2 回
- 右移動 2 回



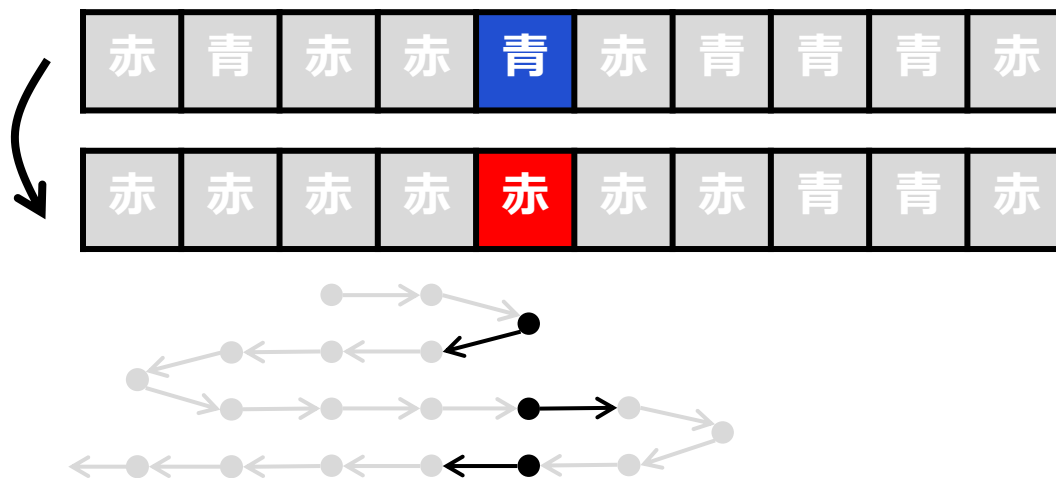
つまり、「青→赤になったタイル」が「赤→青になったタイル」より
 x 個だけ多いことになる

なぜか？

- 青→赤になったタイルについては
右移動の方が 1 回多い
- 赤→青になったタイルについては
左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

タイル5：青→赤

- 左移動 2 回
- 右移動 1 回



つまり、「青→赤になったタイル」が「赤→青になったタイル」より x 個だけ多いことになる

なぜか？

- 青→赤になったタイルについては右移動の方が 1 回多い
- 赤→青になったタイルについては左移動の方が 1 回多い
- それ以外の場合は左移動 = 右移動

このようになる理由

タイルが青→赤→青→赤→青→赤→青→…と変化するため

そこで、ボールが通った地点は最終的にはすべて赤になる

➡ 左から x 番目までの青タイルが赤に変わるはず！※



1 2 3 4 5 6 7 8 9 10

ボールが通過する範囲

たとえば先程の例の場合

たしかにタイル 7 までが赤になっている

しかし、ボールが通った地点は最終的にはすべて赤になる

➡ 左から x 番目までの青タイルが赤に変わるはず！※

これが、ボールが左から出るとき

左から x 番目までの青だけが赤に変わる理由



たとえば先程の例の場合

に

※ボールが右から出るときも同じことがいえる

※赤→青になるタイルは存在しないため。そして、左の青タイルから順に青→赤になるため

この性質を使って、小課題 2 が解けないか？

まず、ボールがどちら側から出るかは、以下のように判定できる

- 青タイルの数が x 個以上の場合 ➡ 左から出る
- 青タイルの数が x 個未満の場合 ➡ 右から出る

まず、ボールがどちら側から出るかは、以下のように判定できる

- ・ 青タイルの数が x 個以上の場合 ➡ 左から出る
- ・ 青タイルの数が x 個未満の場合 ➡ 右から出る

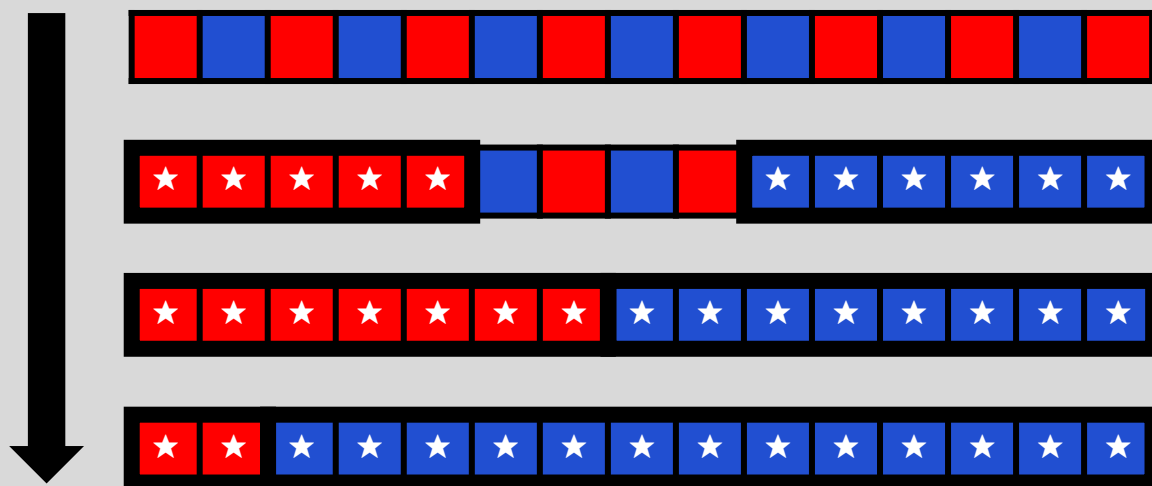


- これが分かれば、あとは左から x 個 (または右から $N + 1 - x$ 個) のタイルの色を愚直に変えていけば良い
- ボタン 1 回当たりの計算量は $O(N)$ なので、全体の計算量は $O(NM)$

小課題 3

$$Q \leq 5$$

ボタンを何回か押すと、盤面は次のように変化する



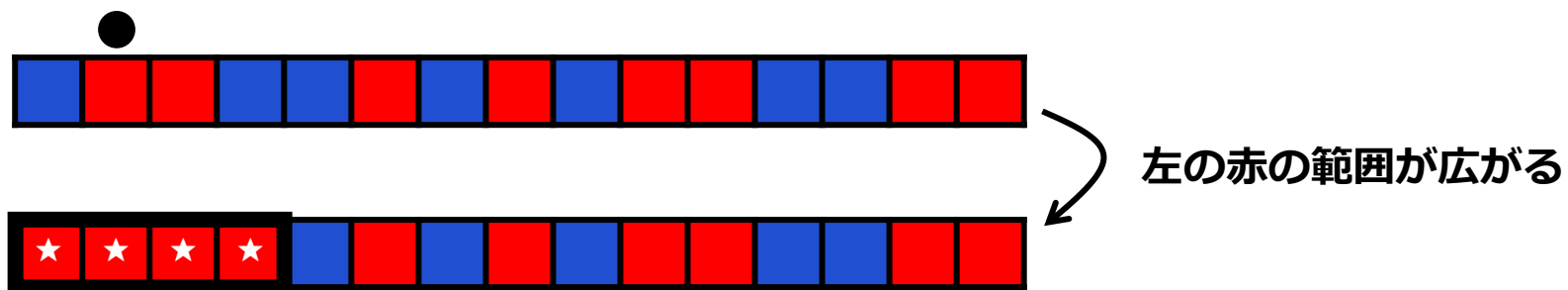
フェーズ1

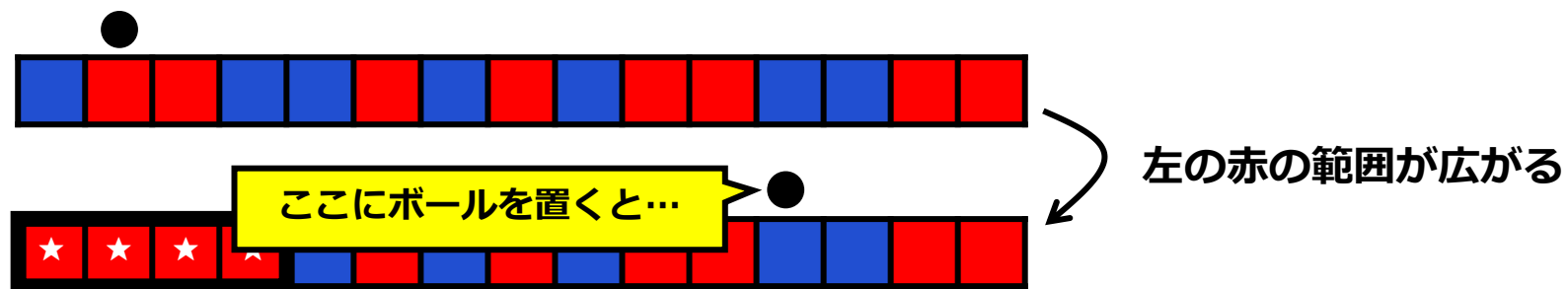
左の赤と右の青の範囲が
徐々に広がっていく

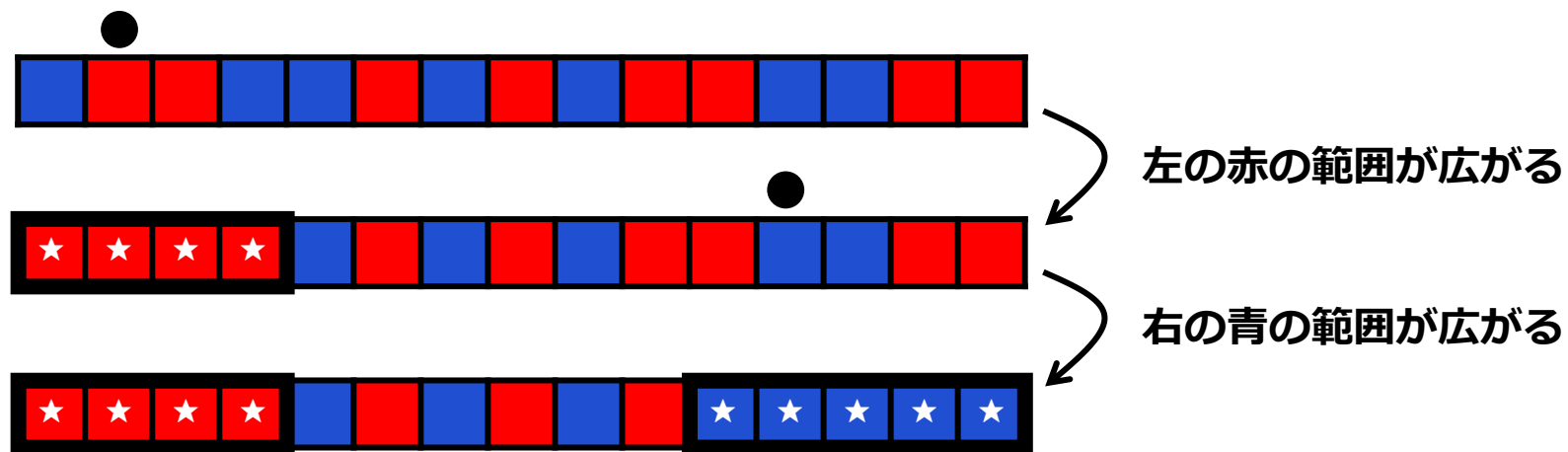
フェーズ2

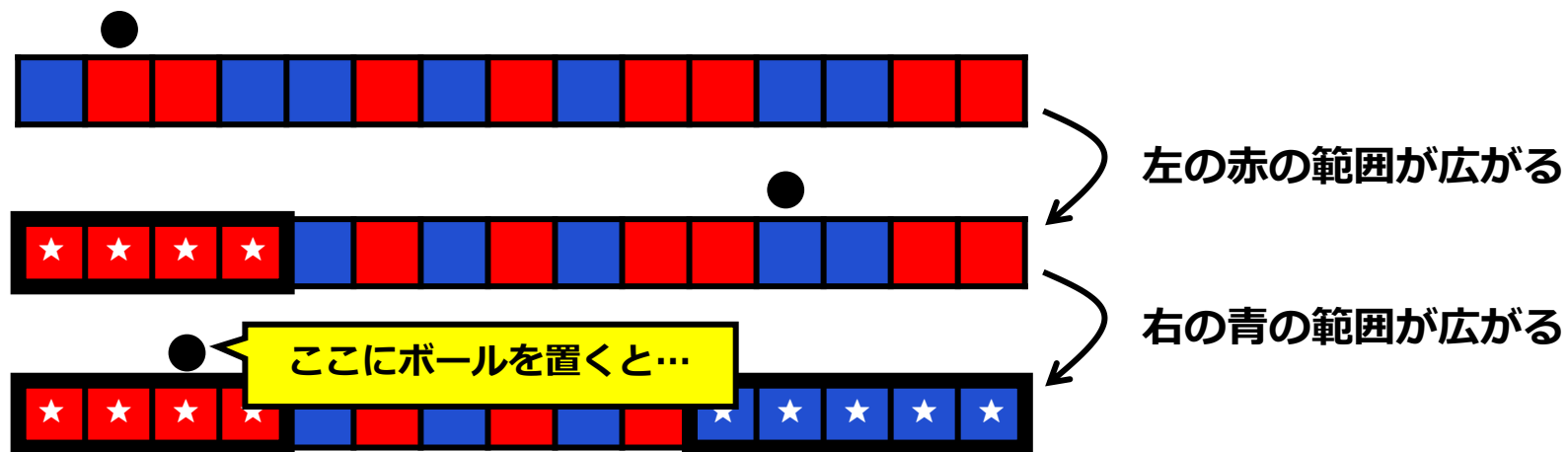
一度「左の赤」と「右の青」がぶつかると
それ以降はぶつかったままである

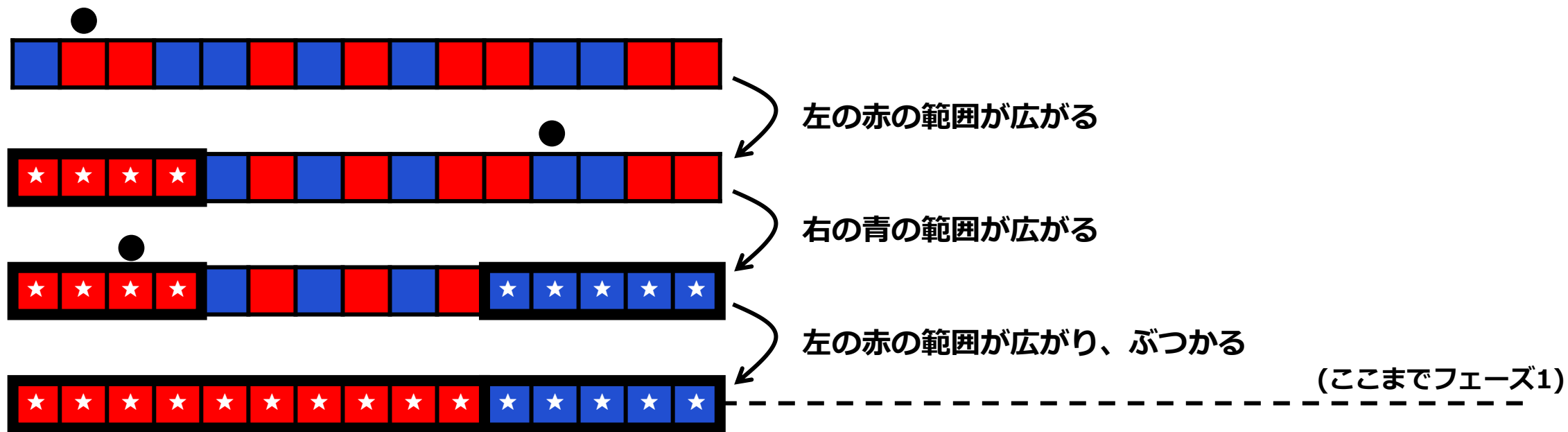


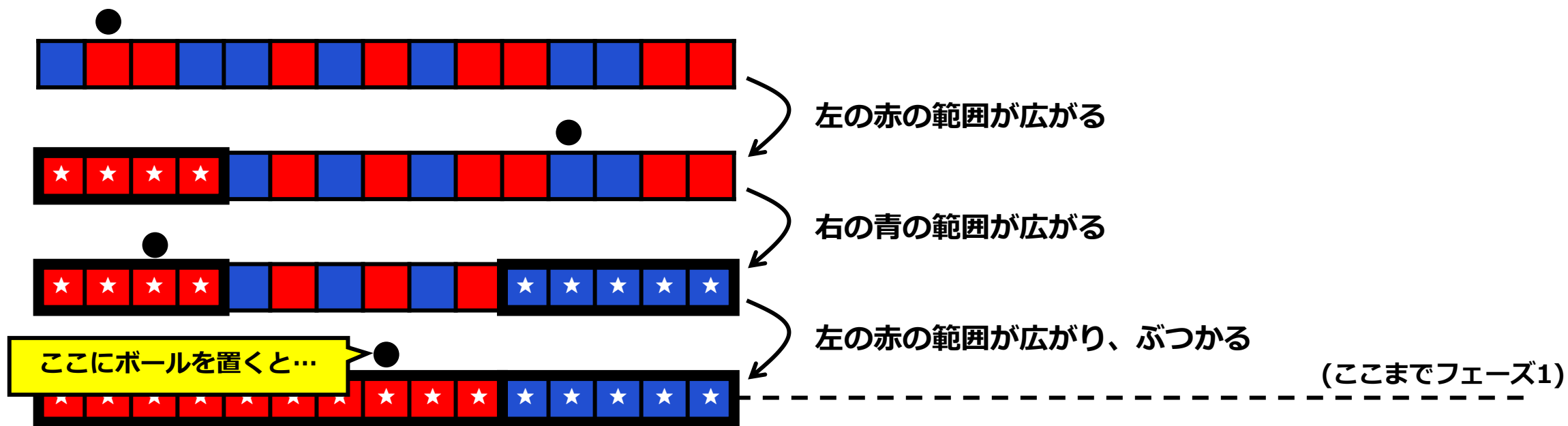


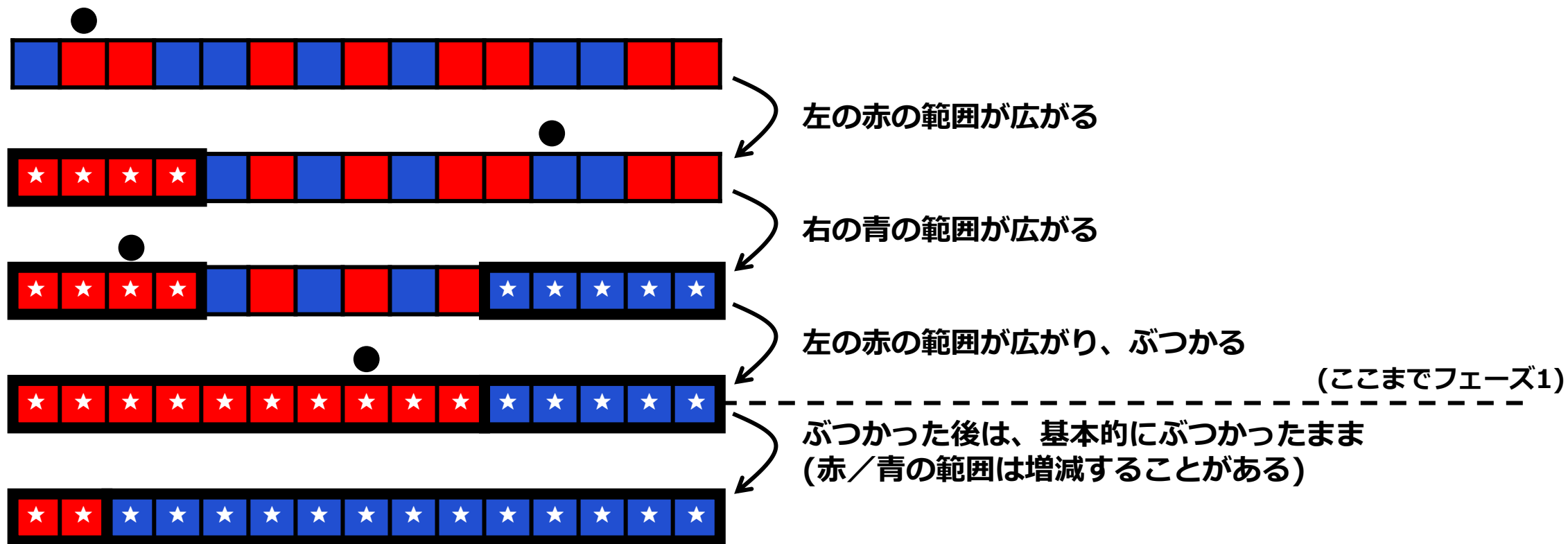












この考察を使って
小課題 2 を高速化できないか？

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

このフェーズは、以下の 2 つの変数を持つことで

計算量 $O(1)$ で処理できる

- **IndexL** : 左から何番目までの青タイルが赤になったか
- **IndexR** : 右から何番目までの赤タイルが青になったか

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

このフェーズは、以下の 2 つの変数を持つことで
計算量 $O(1)$ で処理できる

- **IndexL** : 左から何番目までの青タイルが赤になったか
- **IndexR** : 右から何番目までの赤タイルが青になったか

もしボールをタイル x の上に置いて
左から出た場合

IndexL を x だけ増やせばよい！

※右から出た場合は **IndexR** を増やす

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

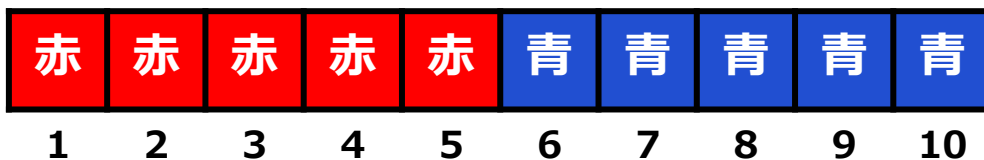
以下の「すごい性質」を使うと高速化できる

ボールを置いたタイルを赤にした後の赤タイルの枚数を y とすると...

実は、操作後の赤タイルは

$$(x + y) \bmod (N + 1) \text{ 枚}$$

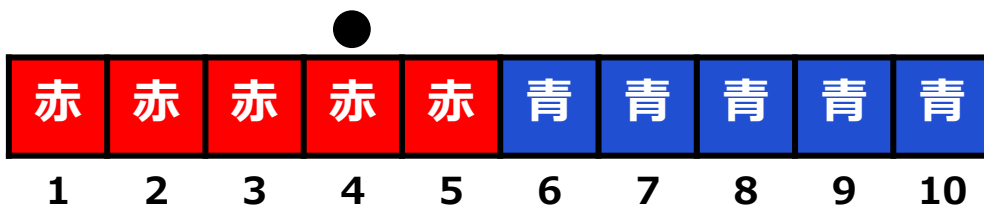
本当か？



この状態からタイル 4 にボールを置くと…



この状態からタイル 4 にボールを置くと...



タイル 4 の色は変わらないので、赤は 5 枚





この状態からタイル 8 にボールを置くと…

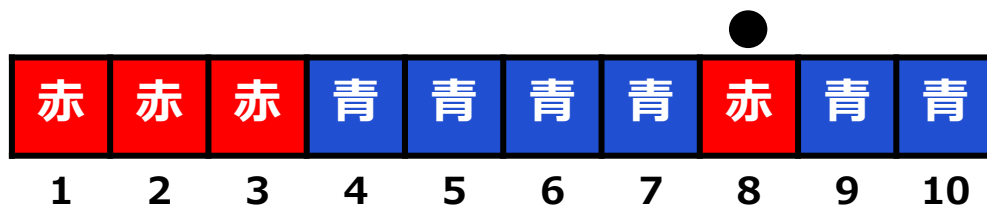
3

小課題3のフェーズ2：例B

90 / 191



この状態からタイル 8 にボールを置くと...



タイル 8 の色は赤になり、赤は 4 枚に

3

小課題3のフェーズ2：例B

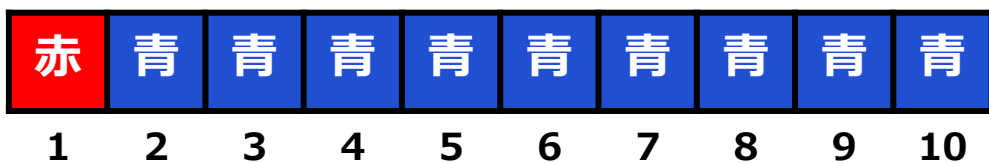
91 / 191



この状態からタイル 8 にボールを置くと...



タイル 8 の色は赤になり、赤は 4 枚に



3 番目までの赤が青になるので、赤は 1 枚
 $(4+8) \bmod 11 = 1$ と一致！

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

したがって、ボールの置き場所を x 、ボタンを押す前の赤のタイル数を y とするとき
操作後の赤のタイル数は次のようになる

- $x < y$ のとき : $(x + y + 1) \bmod (N + 1)$
- $x \geq y$ のとき : $(x + y) \bmod (N + 1)$

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

したがって、ボールの置き場所を x 、ボタンを押す前の赤のタイル数を y とするとき
操作後の赤のタイル数は次のようになる

- $x < y$ のとき : $(x + y + 1) \bmod (N + 1)$
- $x \geq y$ のとき : $(x + y) \bmod (N + 1)$

+1 が付く理由 :

ボールが青タイルの上に置かれるので、
ボールが動く前に赤が 1 つ増えるため

この解法では、ボタン 1 回当たり計算量 $O(1)$ で済む

- ・ ボタン M 個、クエリ Q 個なので全体の計算量は $O(MQ)$

小課題1

愚直にシミュレーションして解く

小課題2

次の性質を使って解く：

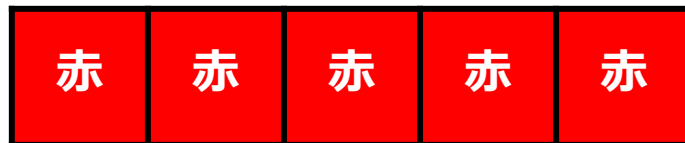
- ・ ボールが左から出る場合、左から x 番目までの青タイルが赤になる
- ・ ボールが右から出る場合、右から $N + 1 - x$ 番目までの赤タイルが青になる

小課題3

- ・ 前半パートは、左から何番目までの青が赤になったか IndexL 、右から何番目までの赤が青になったか IndexR を記録して解く
- ・ 後半パートは、赤タイルの枚数が $\text{mod } (N + 1)$ で簡単に計算できる性質を使う

小課題 4

$N = 10, C_i = 'R'$ である



最初は全部赤タイルなので、フェーズ 1 はない

➡ いきなり小課題 3 のフェーズ 2 の解法が使える

したがって、以下のような単純な解法で解くことができる

```
Answer = 5
for (int i = L; i <= R; i++) {
    if (Answer < A[i]) {
        Answer = (Answer + A[i] + 1) % (N + 1);
    }
    else {
        Answer = (Answer + A[i]) % (N + 1);
    }
}
```

したがって、以下のような単純な解法で解くことができる

```
Answer = 5
for (int i = L; i <= R; i++) {
    if (Answer < A[i]) {
        Answer = (Answer + A[i] + 1) % (N + 1);
    }
    else {
        Answer = (Answer + A[i]) % (N + 1);
    }
}
```

しかし、 $M \leq 120000, Q \leq 120000$
なので TLE してしまう！

セグメント木を使って高速化しよう！

$$(L, R) = (1, 4)$$

$$(L, R) = (1, 2)$$

$$(L, R) = (3, 4)$$

$$(L, R) = (1, 1)$$

$$(L, R) = (2, 2)$$

$$(L, R) = (3, 3)$$

$$(L, R) = (4, 4)$$

各ノードには、ボタン $L, L + 1, \dots, R$ を連続して押したときの
「操作前と操作後のタイルの枚数の関係」を持つ

ボタン 1 ではボールの位置が 2 なので…

- $x < 2$ の場合 : $(x + 3) \bmod 6$
- $x \geq 2$ の場合 : $(x + 2) \bmod 6$

例 : 操作前が 0 枚の場合、操作後は 3 枚

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
3	4	4	5	0	1

たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる

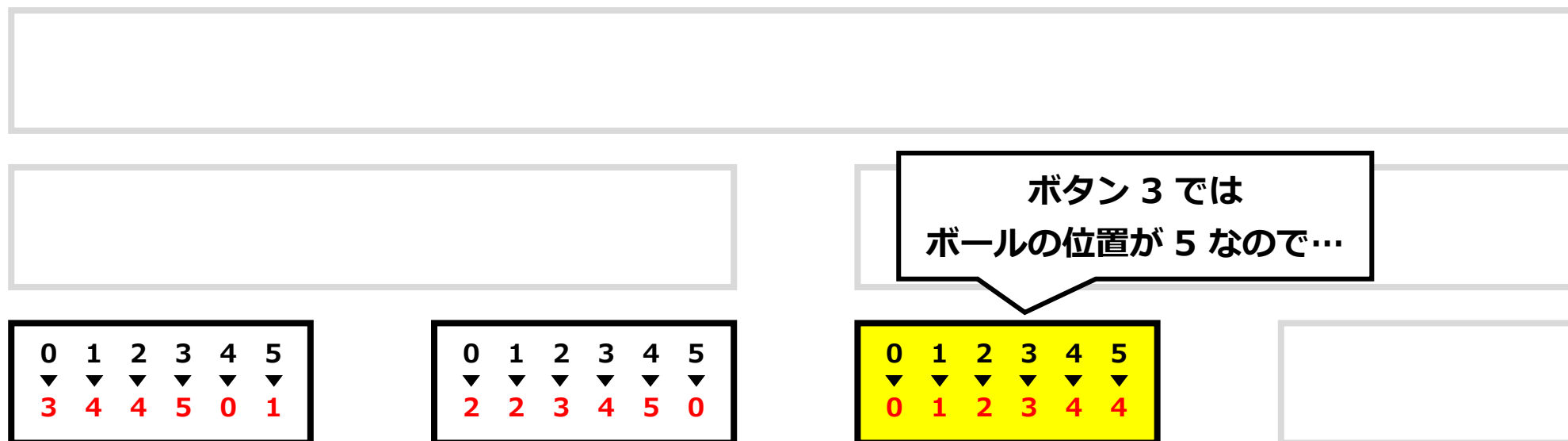
The diagram shows a machine with two buttons, represented by empty rectangular boxes at the top. Below the buttons are two rows of balls, each in a container with indices 0 to 5. The left container has a black border and contains balls with values 3, 4, 4, 5, 0, 1. The right container has a yellow background and a black border, and contains balls with values 2, 2, 3, 4, 5, 0. A speech bubble points to the right container, explaining the operation of button 2.

ボタン 2 ではボールの位置が 1 なので…

- $x < 1$ の場合 : $(x + 2) \bmod 6$
- $x \geq 1$ の場合 : $(x + 1) \bmod 6$

例 : 操作前が 0 枚の場合、操作後は 2 枚

たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる



たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる

ボタン 4 では
ボールの位置が 4 なので…

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
3	4	4	5	0	1

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
2	2	3	4	5	0

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
0	1	2	3	4	4

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
5	0	1	2	2	3

たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる



たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる

例：操作前が 0 個の場合

左側が $0 \rightarrow 0$ 、右側が $0 \rightarrow 5$ なので
操作後は 5 個

ボタン 1~2	0	1	2	3	4	5
	▼	▼	▼	▼	▼	▼
	4	5	5	0	2	2

ボタン 3~4	0	1	2	3	4	5
	▼	▼	▼	▼	▼	▼
	5	0	1	2	2	2

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
3	4	4	5	0	1

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
2	2	3	4	5	0

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
0	1	2	3	4	4

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
5	0	1	2	2	3

たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる

例：操作前が 0 個の場合

左側が $0 \rightarrow 4$ 、右側が $4 \rightarrow 2$ なので
操作後は 2 個

ボタン 1~4

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
2	2	2	5	1	1

ボタン 1~2

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
4	5	5	0	2	2

ボタン 3~4

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
5	0	1	2	2	2

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
3	4	4	5	0	1

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
2	2	3	4	5	0

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
0	1	2	3	4	4

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
5	0	1	2	2	3

たとえば、 $N = 5, A = [2, 1, 5, 4]$ の場合は上のようになる

**セグメント木の構成が終わった
次はどうやってクエリに答えるか？**

いくつかの例を考えよう



たとえば、 $(L, R) = (1, 3)$ の場合は $[1, 2]$ と $[3, 3]$ の和で表せるので
最初 5 個の場合、上図のように答えが「2 個」とわかる



たとえば、 $(L, R) = (2, 4)$ の場合は $[2, 2]$ と $[3, 4]$ の和で表せるので
最初 5 個の場合、上図のように答えが「5 個」とわかる

一般の場合でも、任意の区間 $[L, R]$ はセグメント木の $O(\log M)$ 個のノードで表せる

➡ クエリ当たり $O(\log M)$ で答えられる！

セグメント木の構成にかかる時間は $O(M)$ なので、全体の計算量は $O(M + Q \log M)$

小課題 5

C_i が RR...RRBB...BB の形になっている

前の小課題は $N = 10$ だったので
セグメント木を直接管理できた

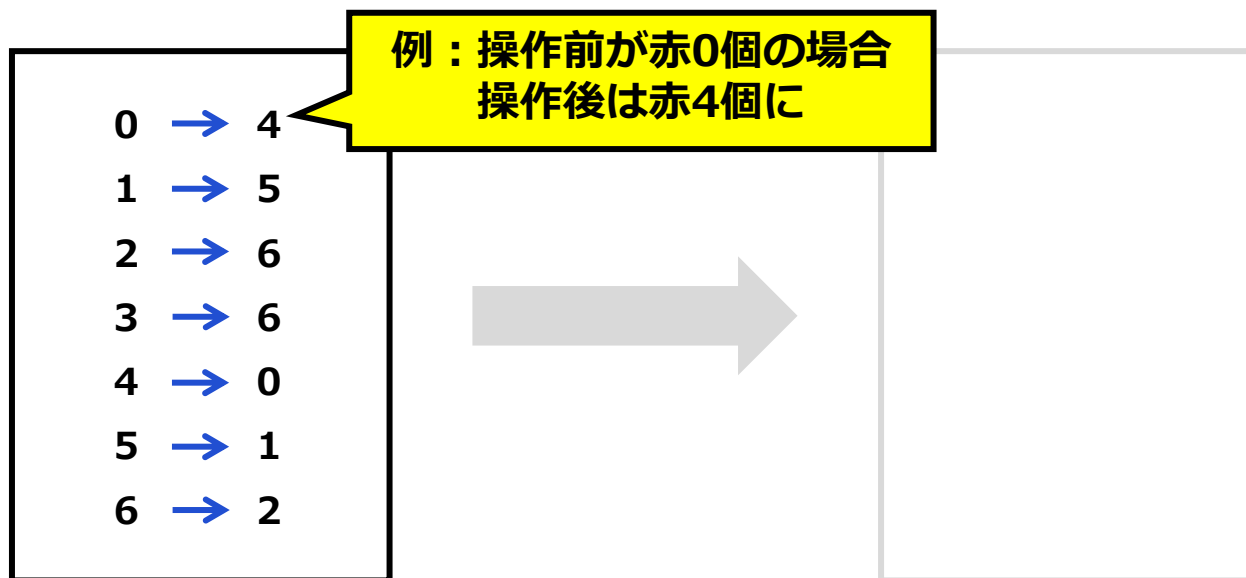
but $N \leq 120000$ だと MLE する !

前の小課題は $N = 10$ だったので

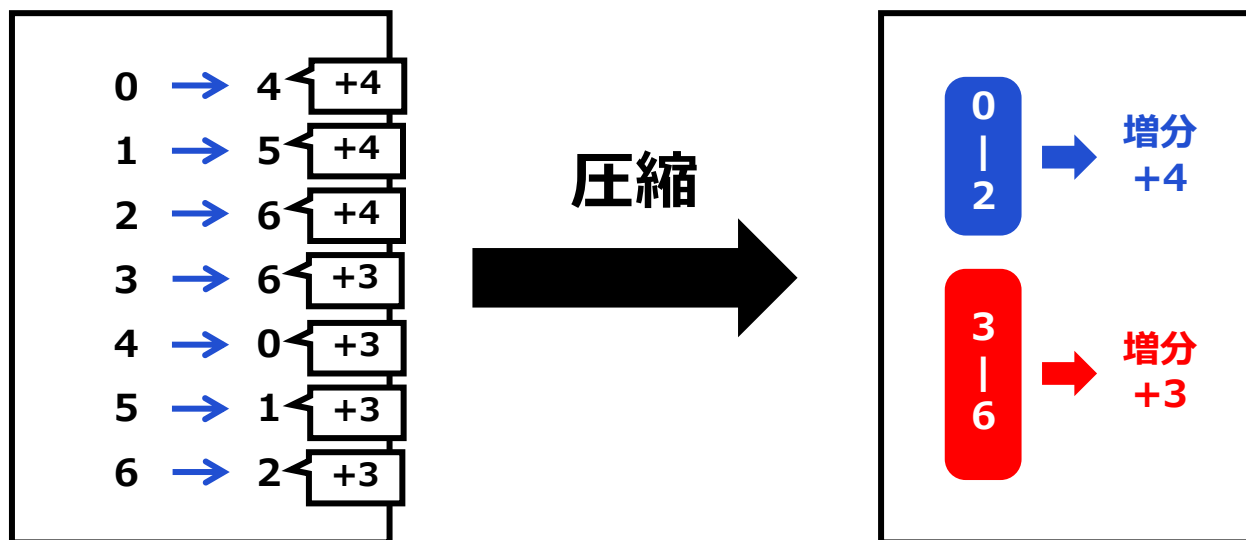
しかし、「増分が同じになる区間」を圧縮すれば

MLE を回避することができる！

but $N \leq 120000$ だと MLE する！

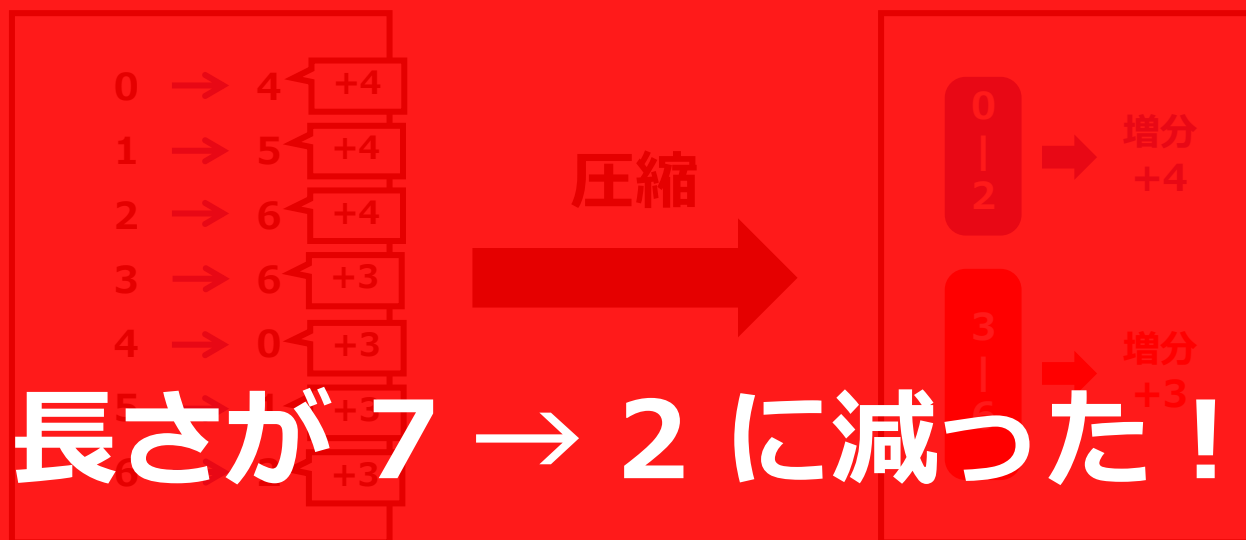


たとえば、ボールを位置 3 に押した場合の状態遷移は左図

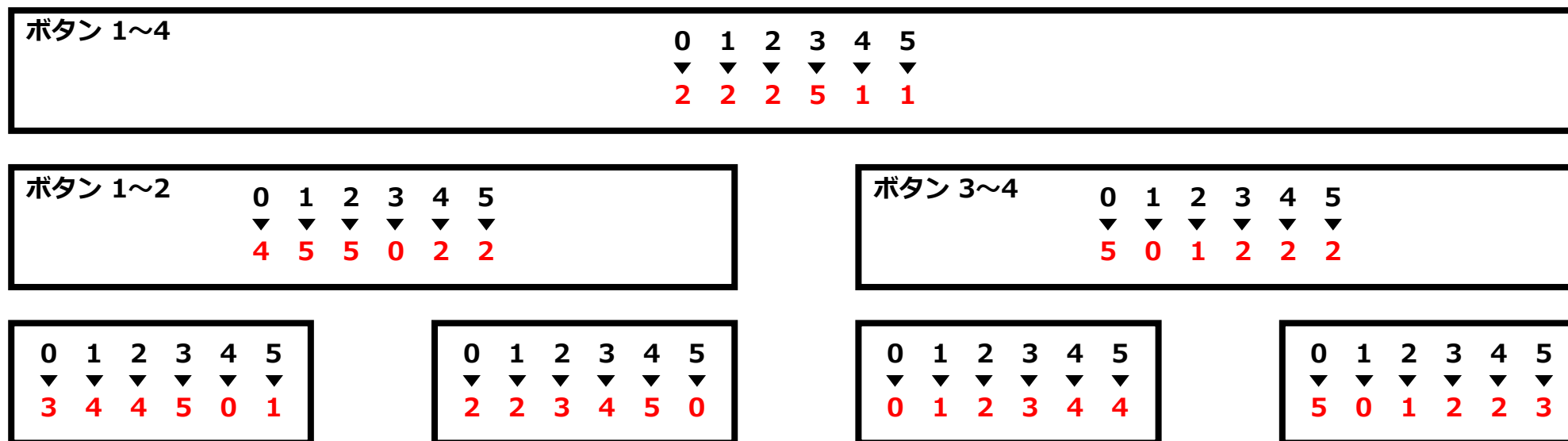


たとえば、ボールを位置 3 に押した場合の状態遷移は左図

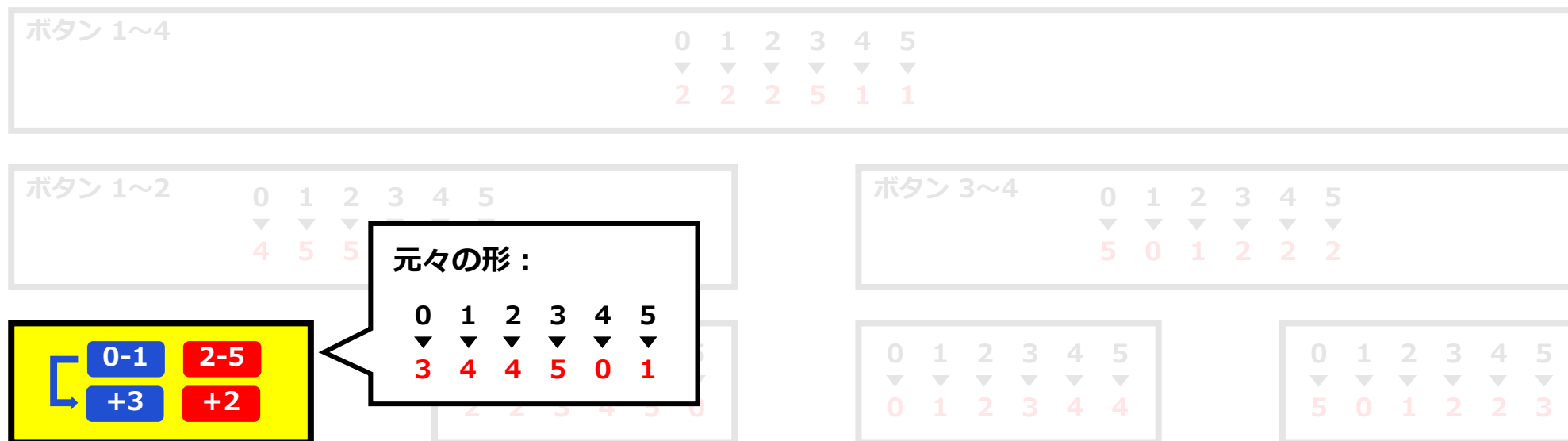
これを圧縮すると右図のようになる



たとえば、ボールを位置 3 に押した場合の状態遷移は左図
これを圧縮すると右図のようになる



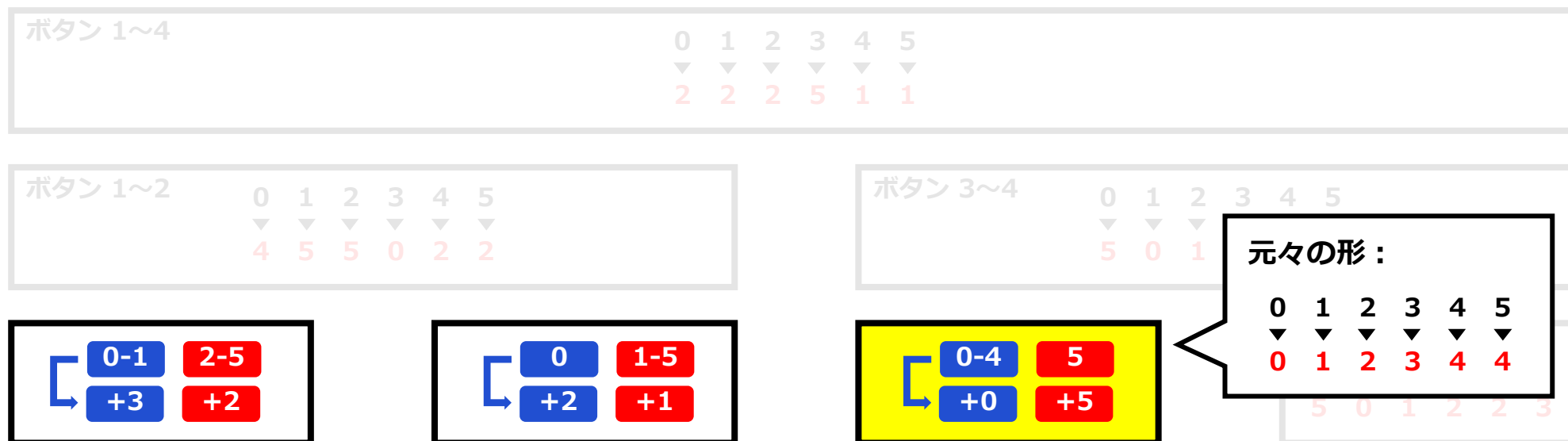
たとえば、 $N = 4, A = [2, 1, 5, 4]$ のときのセグメント木は上図の通り



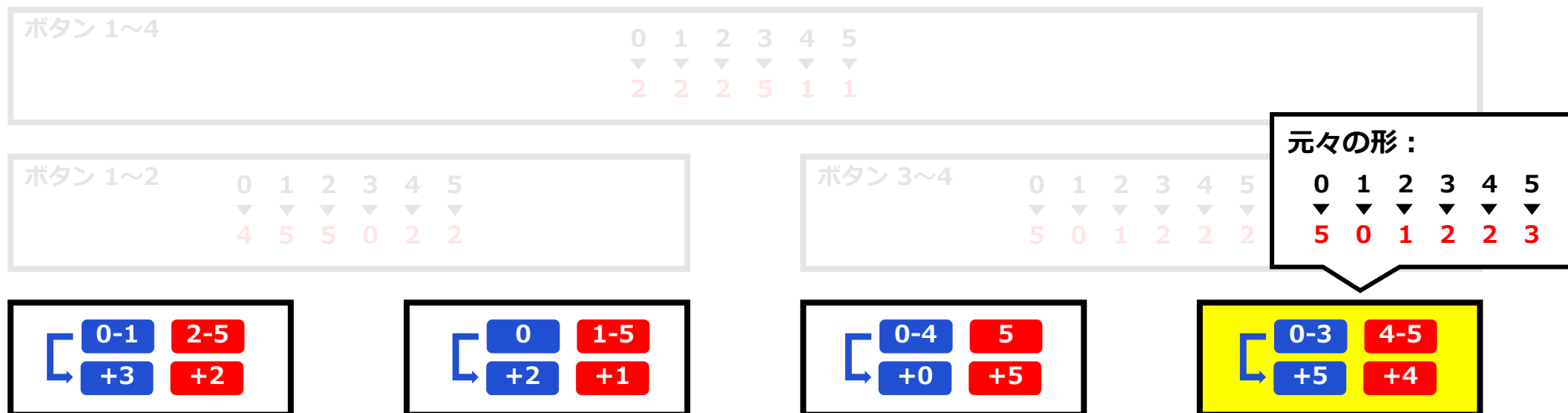
これを圧縮すると、このように変化する



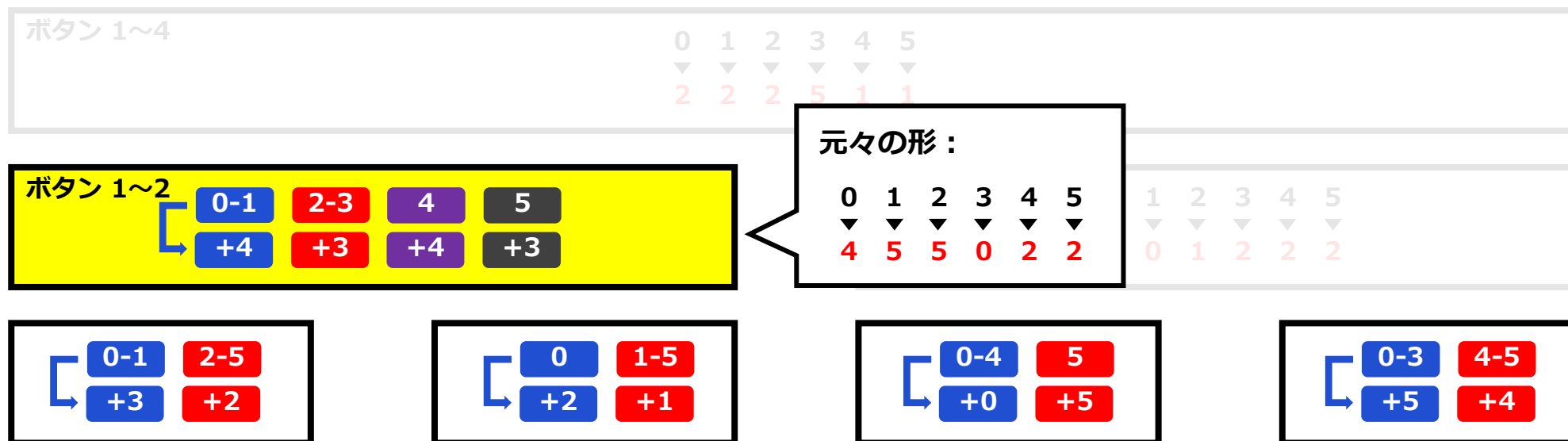
これを圧縮すると、このように変化する



これを圧縮すると、このように変化する



これを圧縮すると、このように変化する



これを圧縮すると、このように変化する

ボタン 1~4

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
2	2	2	5	1	1

元々の形：

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
5	0	1	2	2	2

ボタン 1~2

0-1	2-3	4	5
+4	+3	+4	+3

ボタン 3~4

0-3	4	5
+5	+3	+3

0-1	2-5
+3	+2

0	1-5
+2	+1

0-4	5
+0	+5

0-3	4-5
+5	+4

これを圧縮すると、このように変化する

元々の形：

0	1	2	3	4	5
▼	▼	▼	▼	▼	▼
2	2	2	5	1	1

ボタン 1~4

0	1	2	3	4	5
+2	+1	+0	+2	+3	+2

ボタン 1~2

0-1	2-3	4	5
+4	+3	+4	+3

ボタン 3~4

0-3	4	5
+5	+3	+3

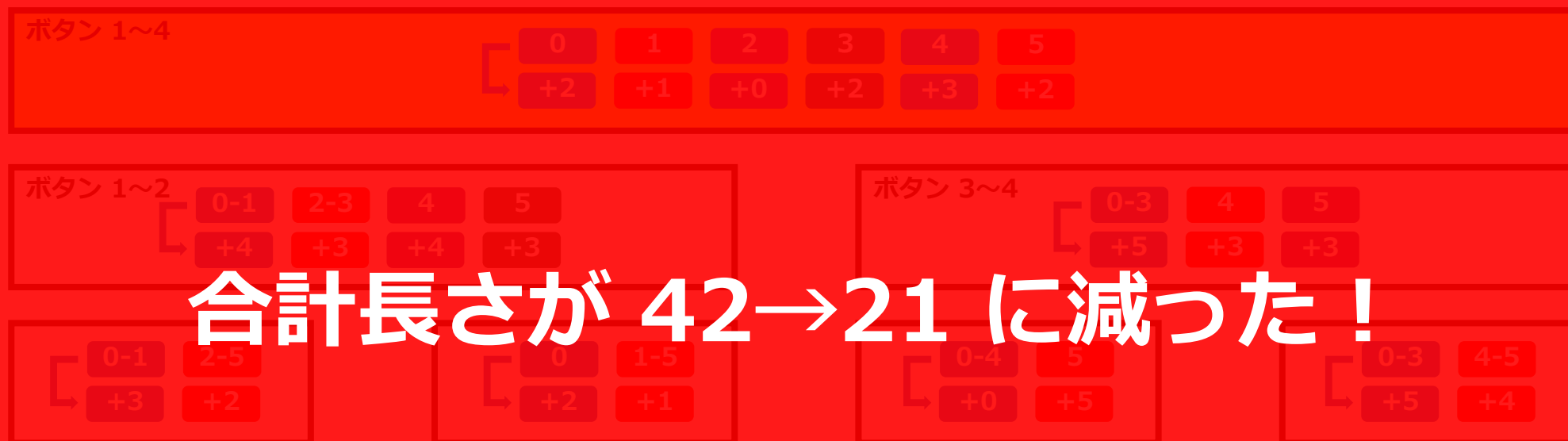
0-1	2-5
+3	+2

0	1-5
+2	+1

0-4	5
+0	+5

0-3	4-5
+5	+4

これを圧縮すると、このように変化する

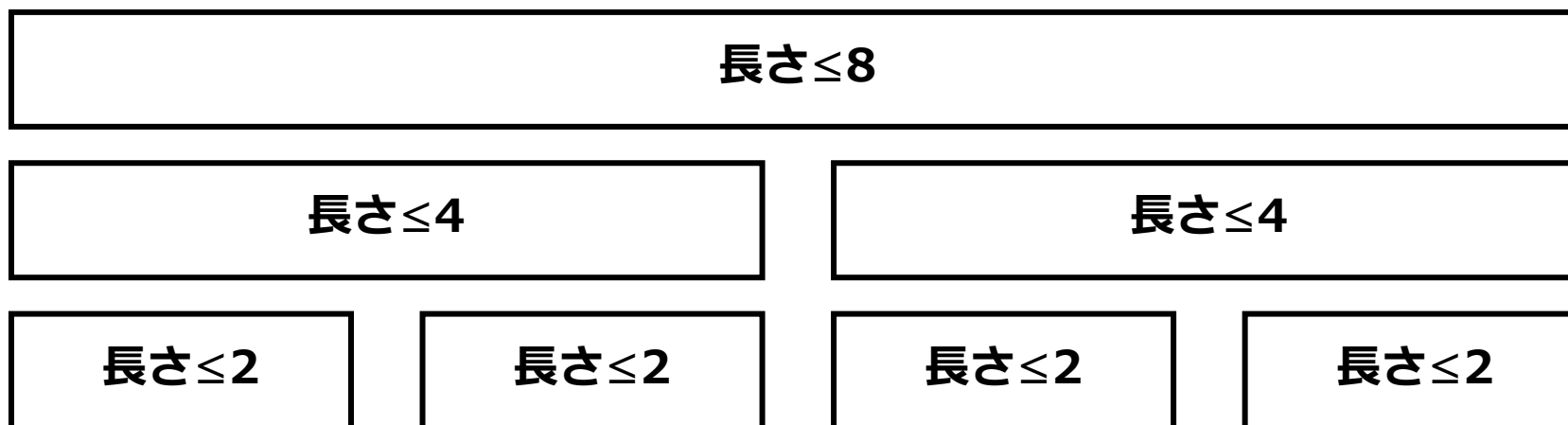


これを圧縮すると、このように変化する

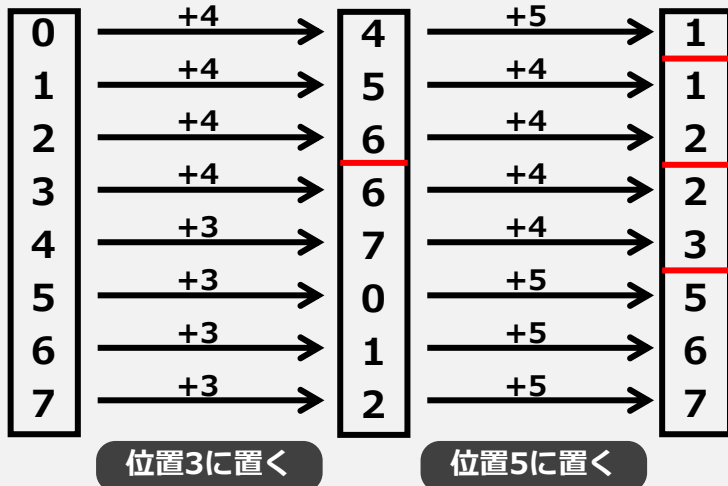
**Q. この単純な圧縮方法で
データサイズはどのくらい減るのか？**

実は、ボタンを k 回押すことに対応するノードは、長さ $2k$ 以内に圧縮可能

例： $M = 4$ の場合は下図の通り



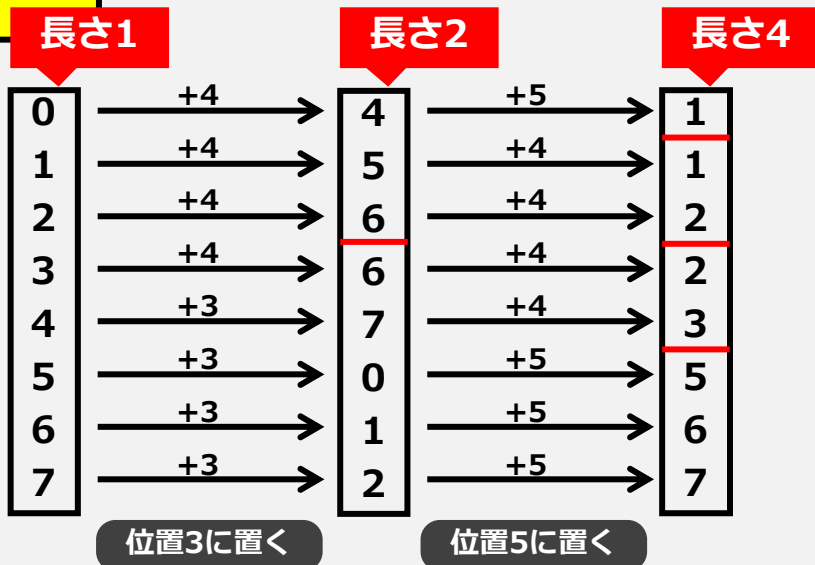
実は、ボタンを k 回押すことに対応するノードは、長さ $2k$ 以内に圧縮可能

証明

- 右図のように、増分が異なる位置に赤線を引いた図を考える

実は、ボタンを k 回押すことに対応するノードは、長さ $2k$ 以内に圧縮可能

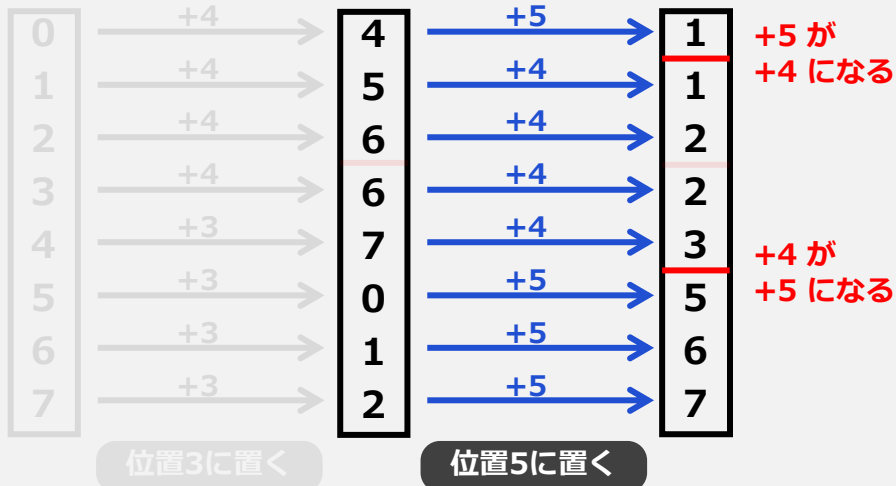
証明



- 右図のように、増分が異なる位置に赤線を引いた図を考える
- このとき、圧縮した後の長さは (赤線の数) + 1

実は、ボタンを k 回押すことに対応するノードは、長さ $2k$ 以内に圧縮可能

証明



- 右図のように、増分が異なる位置に赤線を引いた図を考える
- このとき、圧縮した後の長さは (赤線の数) + 1



1 回の操作を行うと、赤線はどこで追加されるのか？

- “操作が与える増分” が変わるところ
- 位置 p に置いたとき、変わるところは「0 の直前」と「 p の直前」の 2 箇所だけ

実は、ボタンを k 回押すことに対応するノードは、長さ $2k$ 以内に圧縮可能

つまり、ボタン 1 回で赤線は 2 個しか増えない

ボタン k 回のノードは、圧縮後の長さが $2k$ 以下！



位置5に置く

図を考える

- このとき、圧縮した後の長さは (赤線の数) + 1

1 回の操作を行うと、赤線はどこで追加されるのか？

- 位置 p に置いたとき、変わるところは「0 の直前」と「 p の直前」の 2 箇所だけ

したがって、セグメント木の各深さごとの長さの合計は $2M$ 以下になる

→合計長さは $2M \log M$ 以下

したがって、セグメント木の各深さごとの長さの合計は $2M$ 以下になる

→ 合計長さは $2M \log M$ 以下 (下図は $M = 4$ の場合)



このように、セグメント木を圧縮すれば小課題 5 が解ける

プログラムの計算量は以下の通り：

- ・ セグメント木の構成：二分探索が必要なので、計算量は $O(M \log^2 M)$
- ・ クエリ処理：これも二分探索が必要なので、計算量は $O(\log^2 M)$
- ・ したがって、全体の計算量は $O((M + Q) \log^2 M)$

62点

小課題 6

$A_i \leq 20$ または $A_i > N - 20$

まず、この問題では以下の 2 つのフェーズを解く必要がある

※小課題 3 の解説を参照

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

まず、この問題では以下の 2 つのフェーズを解く必要がある

※小課題 3 の解説を参照

フェーズ1 左の赤・右の青の範囲が徐々に広がっていくフェーズ

フェーズ2 左の赤と右の青がぶつかった後のフェーズ

➡ 実は、フェーズ 2 は小課題 5 で解けている
なのでフェーズ 1 だけを考えればよい

どうやってフェーズ 1 を解くか？

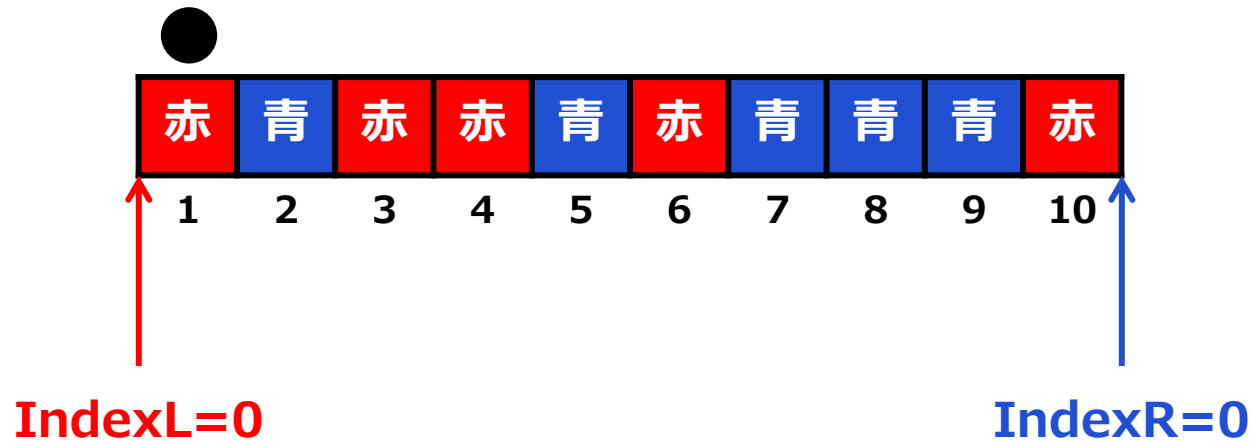
以下の変数を用意する

- IndexL : 左から何番目までの青タイルが赤になったか
- IndexR : 右から何番目までの赤タイルが青になったか

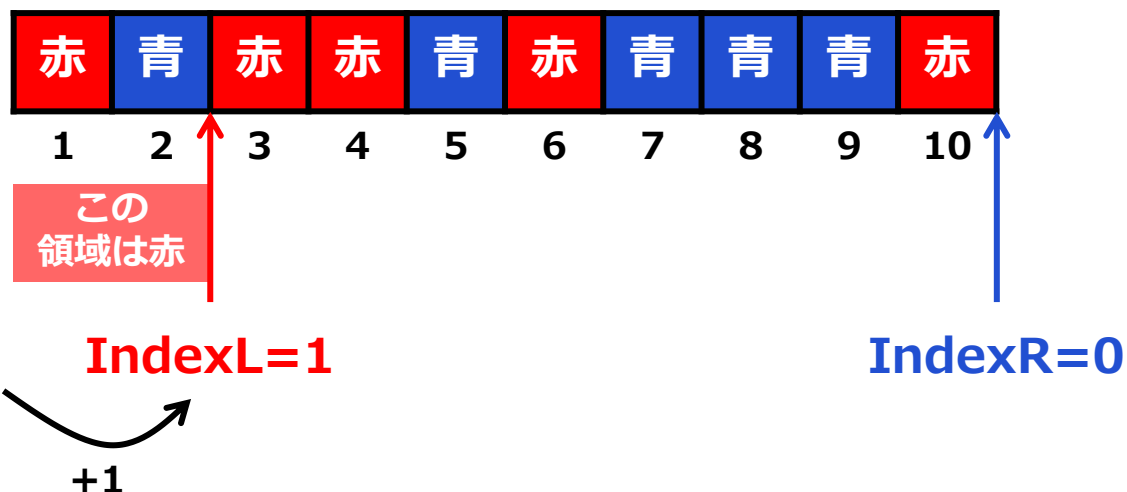
ボールが左から x_L 番目・右から x_R 番目の位置に置かれた場合…

- ボールが左から出るとき : IndexL に x_L を加算
- ボールが右から出るとき : IndexR に x_R を加算

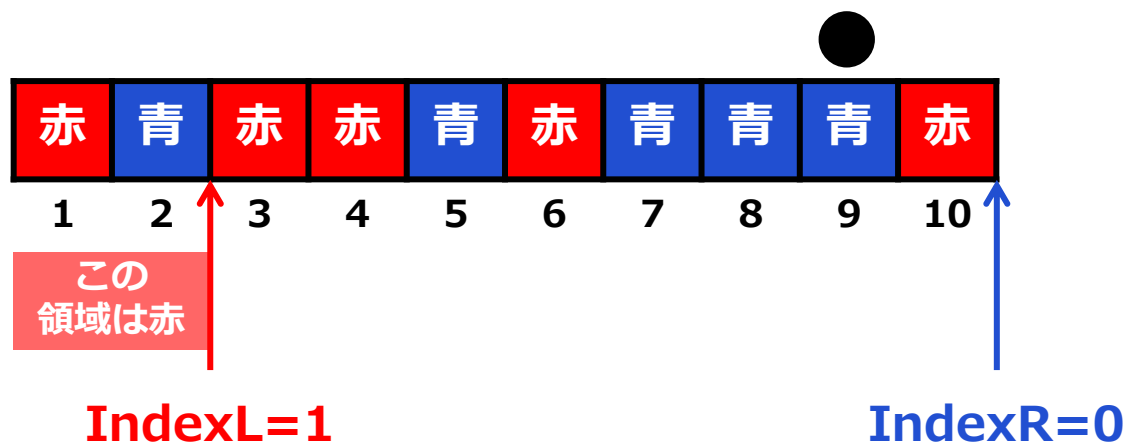
操作 ボールが位置 1 に置かれる



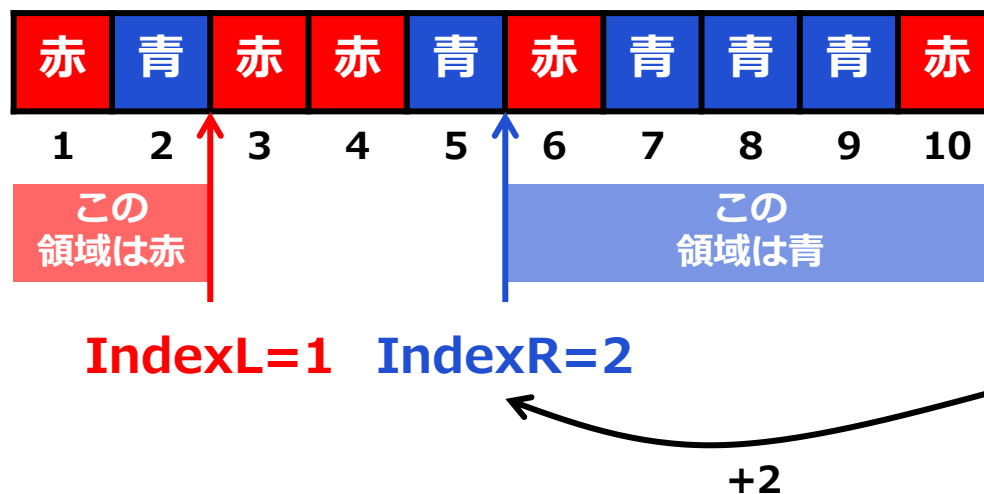
操作 ボールが位置 1 に置かれる



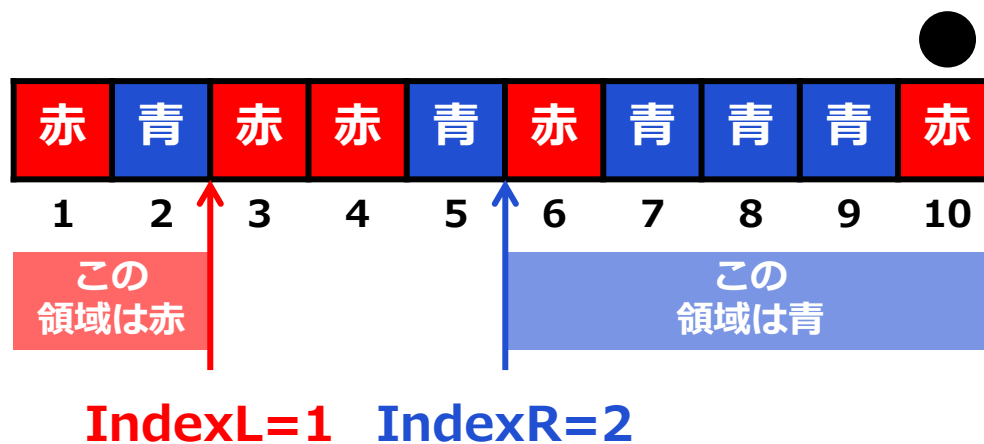
操作 ボールが位置 9 (右から 2 番目) に置かれる



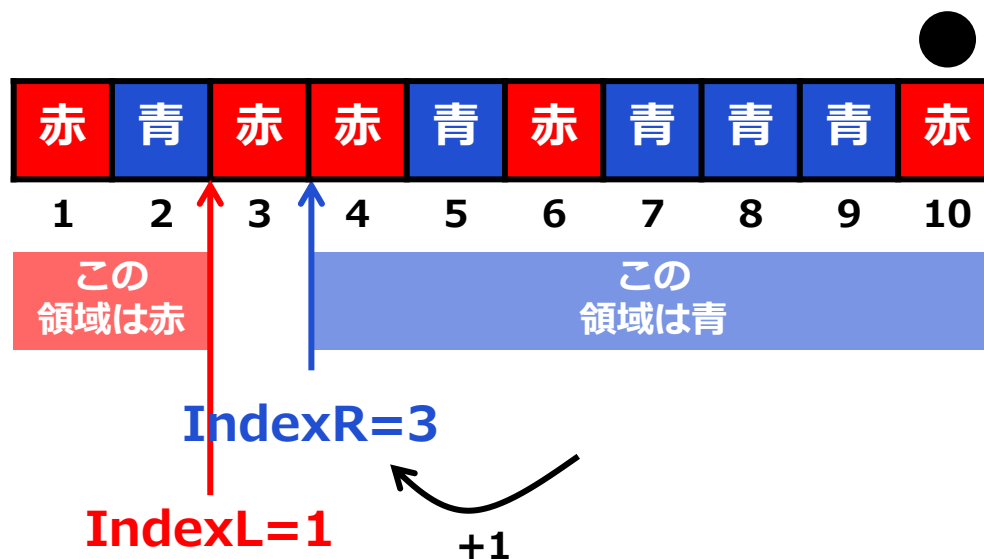
操作 ボールが位置 9 (右から 2 番目) に置かれる



操作 ボールが位置 10 (右から 1 番目) に置かれる



操作 ボールが位置 10 (右から 1 番目) に置かれる



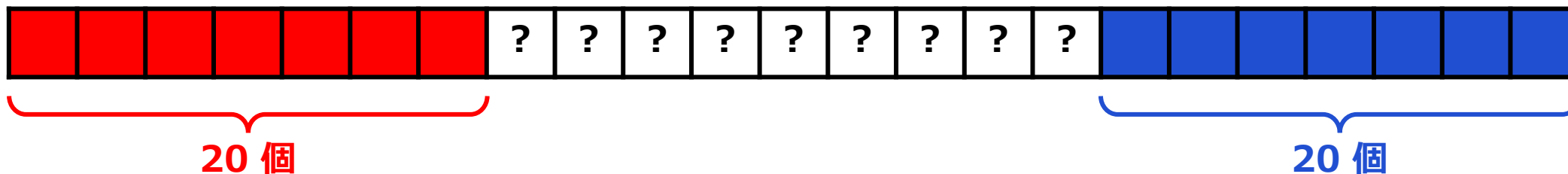
しかし、この解法ではボタン当たり $O(1)$

全体で計算量 $O(M)$ がかかってしまう



どうにかしていくつかのボタンをまとめて処理できないか？

高速化の方法



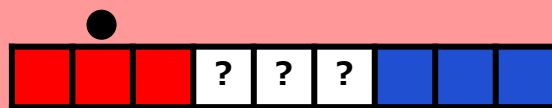
手始めに、タイルの初期状態が
左端 20 個が赤・右端 20 個が青のときを考えよう

重要

このとき、ボールを置いた後の動きとしては以下の 2 つがあり得る

 $x_L \leq 20$ の場合 $x_R \leq 20$ の場合

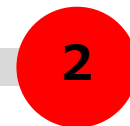
基本的にはボールが左から出るので
IndexL が x_L 増加



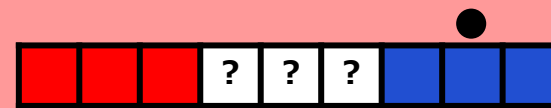
※左の赤と右の青がぶつかったときのみ、ボールが右から出ることがある

重要

このとき、ボールを置いた後の動きとしては以下の 2 つがあり得る

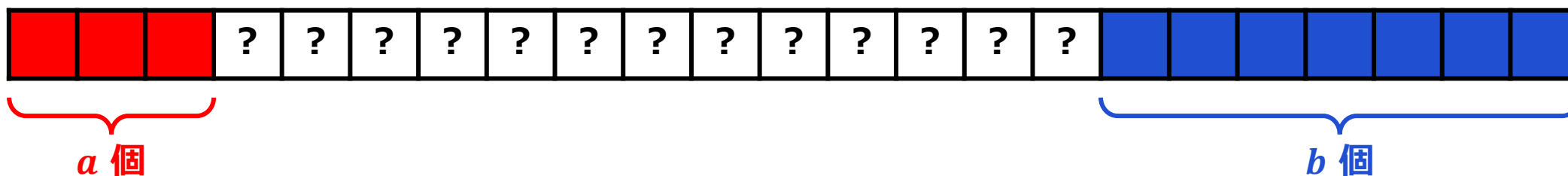
 $x_L \leq 20$ の場合 $x_R \leq 20$ の場合

基本的にはボールが右から出るので
IndexR が $x_R - 1$ 増加



したがって、ボタン $l, l+1, \dots, r$ を押した後の IndexL , IndexR の値は累積和によって計算できる！

➡ r で二分探索をすれば、左の青と右の赤がいつぶつかるかも $O(\log M)$ で計算できる

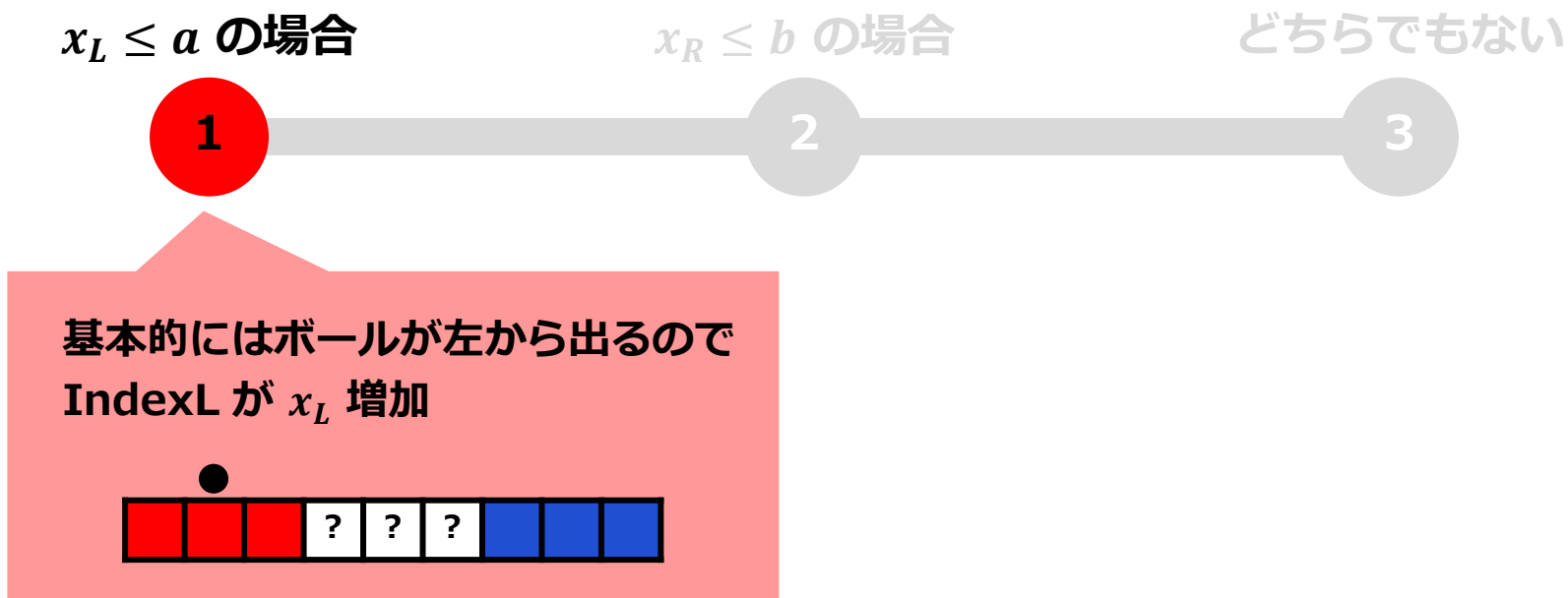


次に一般の場合を考えよう

現時点で、タイルの左端 a 個が赤、右端 b 個が青であるとする

重要

このとき、ボールを置いた後の動きとしては以下の 3 つがあり得る



※左の赤と右の青がぶつかったときのみ、ボールが右から出ることがある

重要

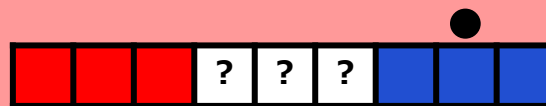
このとき、ボールを置いた後の動きとしては以下の 3 つがあり得る

 $x_L \leq a$ の場合 $x_R \leq b$ の場合

どちらでもない



基本的にはボールが左から出るので
IndexR が $x_R - 1$ 増加



重要

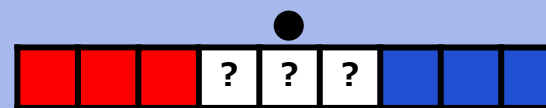
このとき、ボールを置いた後の動きとしては以下の 3 つがあり得る

 $x_L \leq a$ の場合 $x_R \leq b$ の場合

どちらでもない



どう動くかは分からない...



ここで、どちらでもない場合は
計算量 $O(1)$ かけて処理する必要がある

しかし、この小課題では $A_i \leq 20$ または $A_i > N - 20$ なので
“どちらでもない” になる可能性があるのは…

- 1 $1 \leq x_L \leq 20$ となる操作のうち最初の 20 個
- 2 $2 \leq x_R \leq 20$ となる操作のうち最初の 20 個
- 3 $x_R = 1$ となる操作のうち最初の 1 個

合計

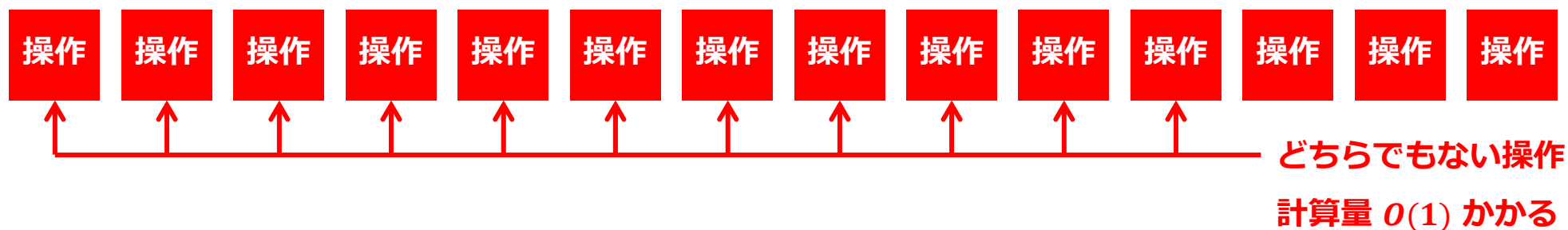
41個



したがって、「飛ばせる部分」は累積和 + 二分探索を使い
どうしようもない部分は計算量 $O(1)$ で処理すれば、**計算回数は約 $40 \log M$ に**

小課題 7

追加の制約はない



小課題 6 の解法を一般の制約に適用させると

「どちらでもない操作」を最大 M 回行うことになるかもしれない

しかし実際は、どちらでもない操作は $O(\log N)$ 回しか行われない

しかし実際は、どちらでもない操作は $O(\log N)$ 回しか行われない

なぜか？

左の赤が a 個、右の青が b 個の状態で「どうしようもない操作」を行うとき：

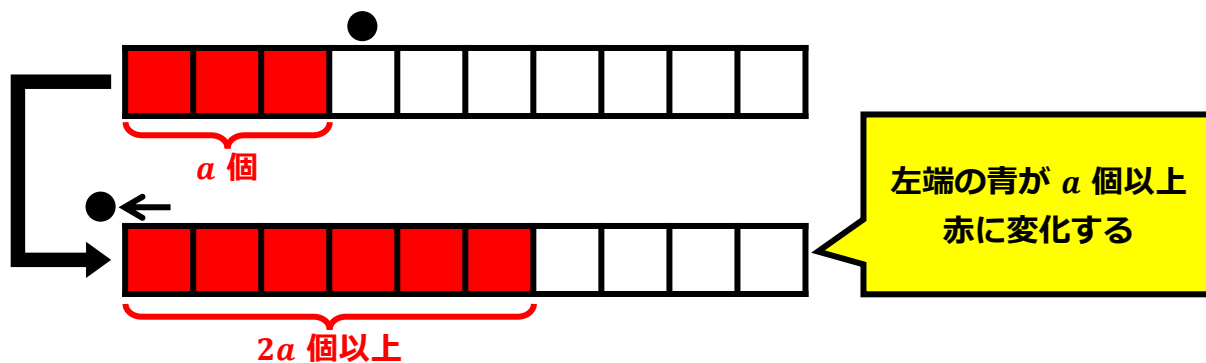
- ・ ボールが左から出るとき、操作後の左の赤は $2a$ 個以上に
- ・ ボールが右から出るとき、操作後の右の青は $2b$ 個以上に

しかし実際は、どうしてもない操作は $O(\log N)$ 回しか行われない

なぜか？

左の赤が a 個、右の青が b 個の状態です。「どうしてもない操作」を行うとき：

- ・ ボールが左から出るとき、操作後の左の赤は $2a$ 個以上に
- ・ ボールが右から出るとき、操作後の右の青は $2b$ 個以上に

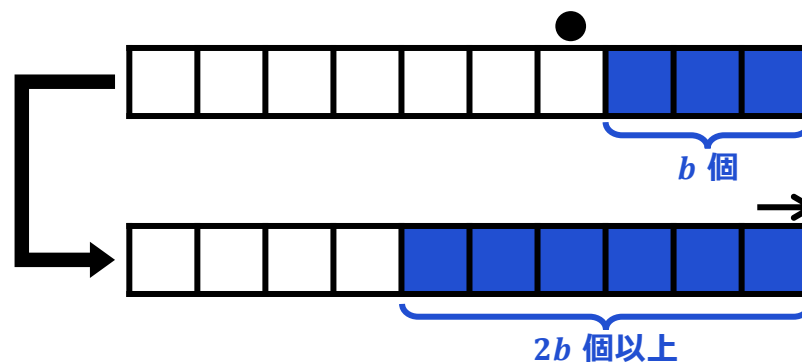
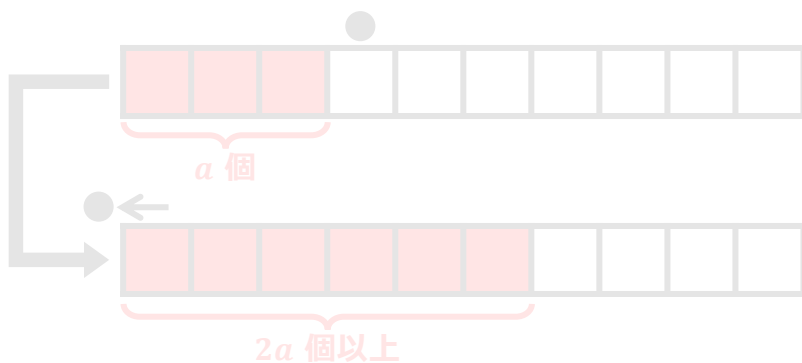


しかし実際は、どうしてもない操作は $O(\log N)$ 回しか行われない

なぜか？

左の赤が a 個、右の青が b 個の状態です「どうしてもない操作」を行うとき：

- ・ ボールが左から出るとき、操作後の左の赤は $2a$ 個以上に
- ・ ボールが右から出るとき、操作後の右の青は $2b$ 個以上に



右端の赤が b 個以上
青に変化する

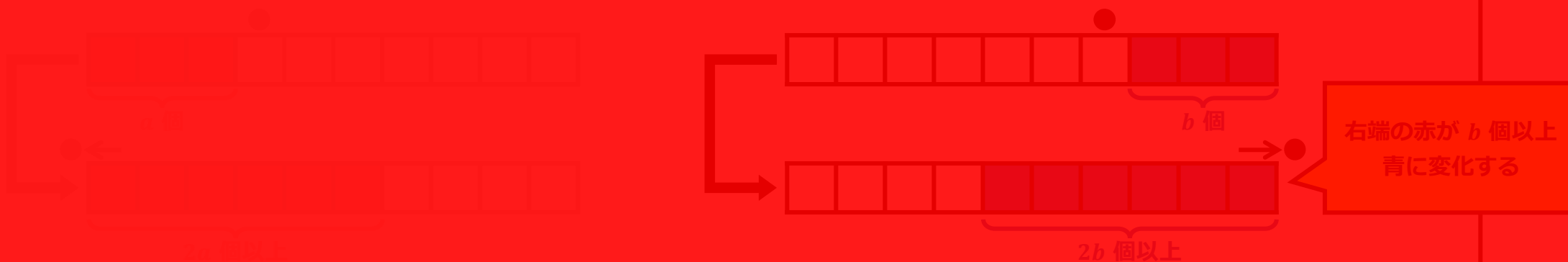
しかし実際は、どうしてもない操作は $O(\log N)$ 回しか行われない

なぜか？

左の赤が a 個、右の青が b 個の状態で「どうしてもない操作」を行うとき：

- ・ ボールが左から出るとき、操作後の左の赤は $2a$ 個以上に

小課題 6 と同じ方針で解けそう！



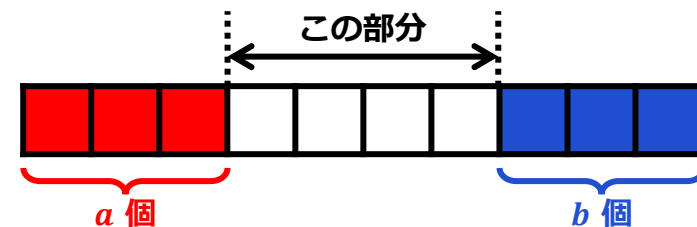
だが、まだ解けたわけではない

次の「どちらでもない操作」がいつ来るかを
高速に求める必要がある！

次の「どちらでもない操作」は
どうやって求めるか？

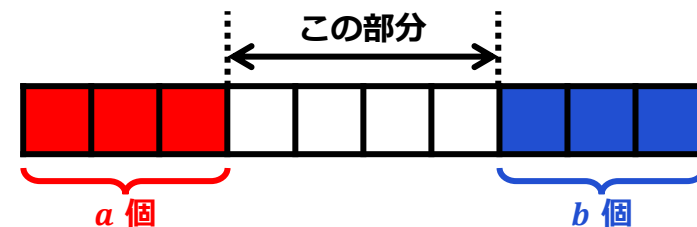
まず、現時点で左の赤が a 個、右の青が b 個であり、ボタン x までの処理が終わっている場合…

- 次のどちらでもない操作の番号は $a + 1 \leq A_i \leq N - b$ を満たす最小の $i (> x)$

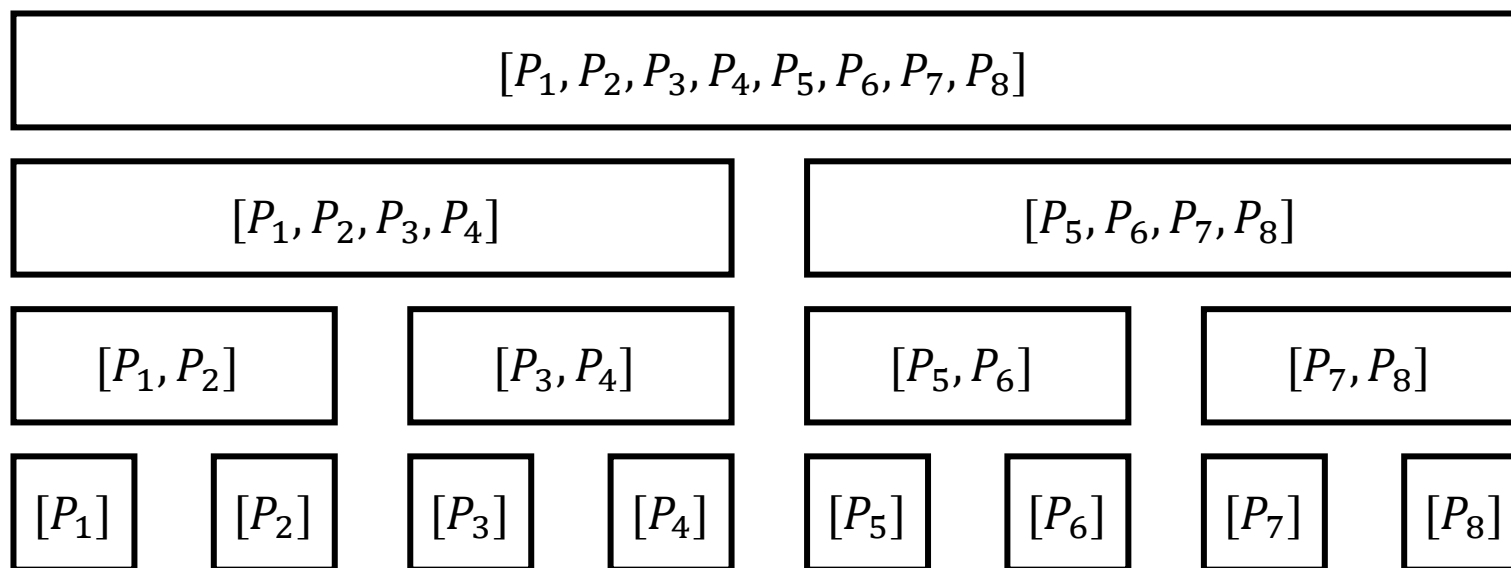


まず、現時点で左の赤が a 個、右の青が b 個であり、ボタン x までの処理が終わっている場合…

- 次のどちらでもない操作の番号は $a + 1 \leq A_i \leq N - b$ を満たす最小の $i (> x)$

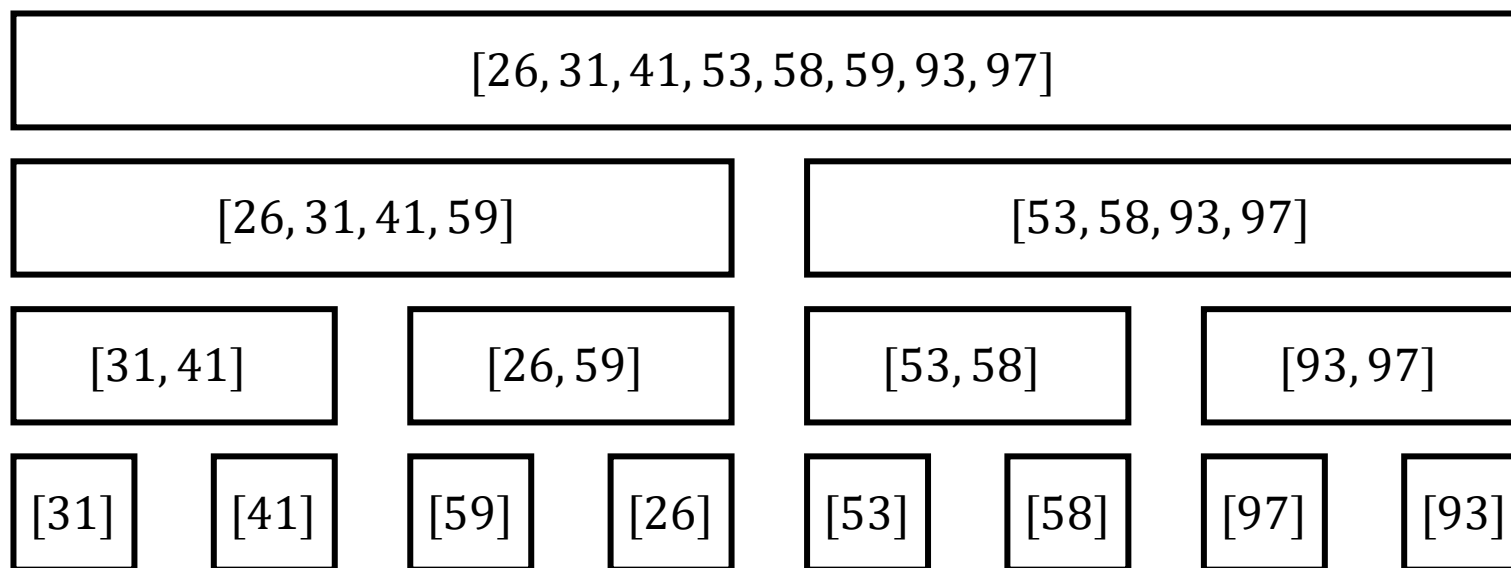


➡ これは、領域木というデータ構造を使えば高速



セグメント木の種類

区間 $[L, R]$ に対応するノードに $[P_L, P_{L+1}, \dots, P_R]$ をソートして格納



たとえば $X = [31, 41, 59, 26, 53, 58, 97, 93]$ の場合、領域木は上図の通り

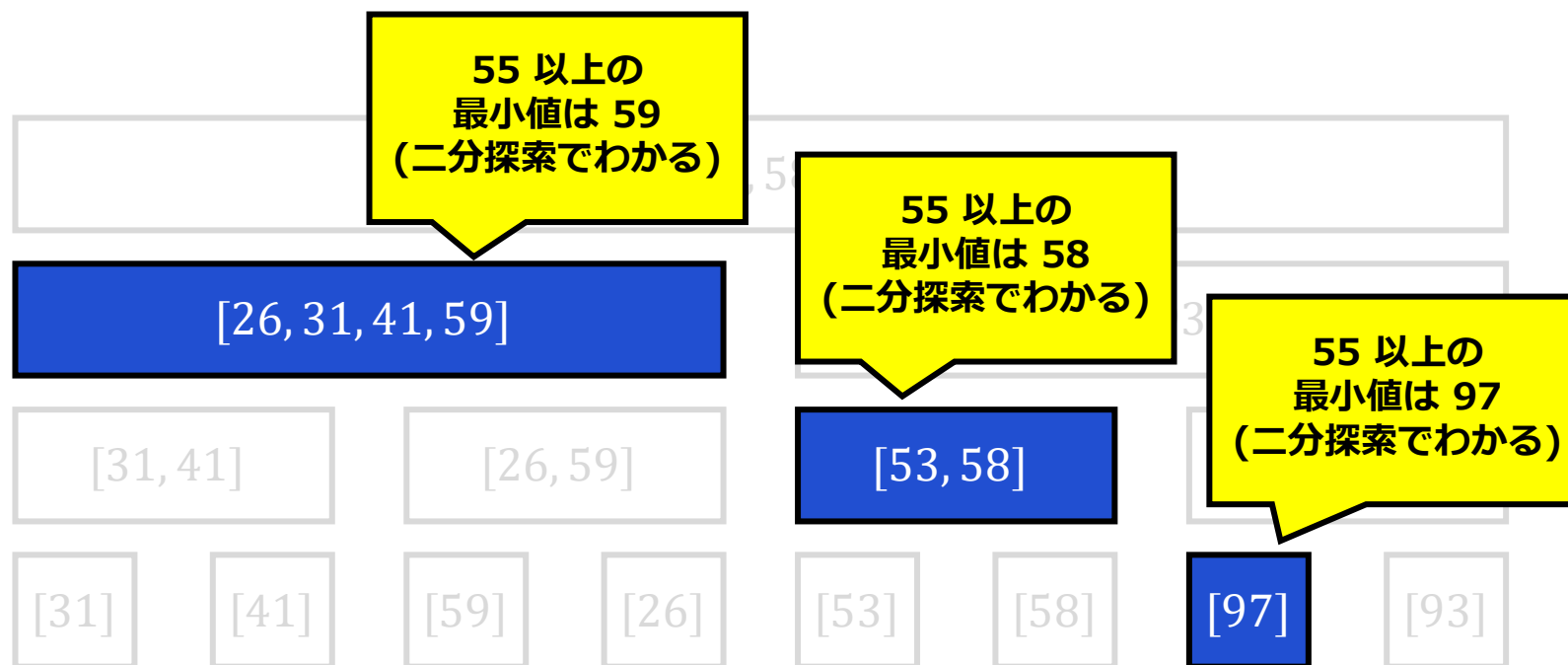
領域木上で二分探索をすると

$P_i \geq x \ (l \leq i \leq r)$ となる最小の P_i がわかる



たとえば $P_i \geq 40$ ($3 \leq i \leq 6$) となる最小の P_i はいくつか？

➡ 59 と 53 のうち小さい方「53」



たとえば $P_i \geq 55$ ($1 \leq i \leq 7$) となる最小の P_i はいくつか？

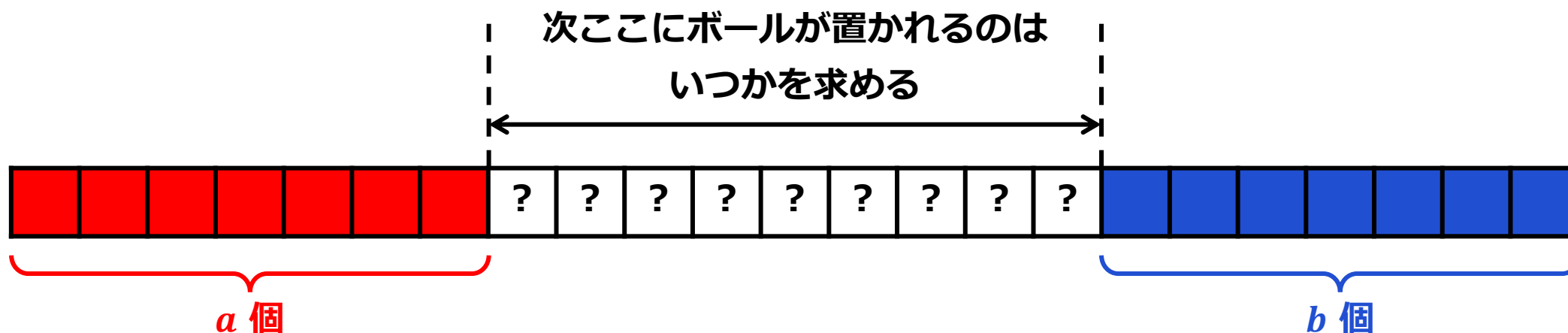
➡ 59, 58, 97 のうち小さい方「58」

このように領域木を使うと、次の「どちらでもない操作」を
計算量 $O(\log N \log M)$ で求めることができる

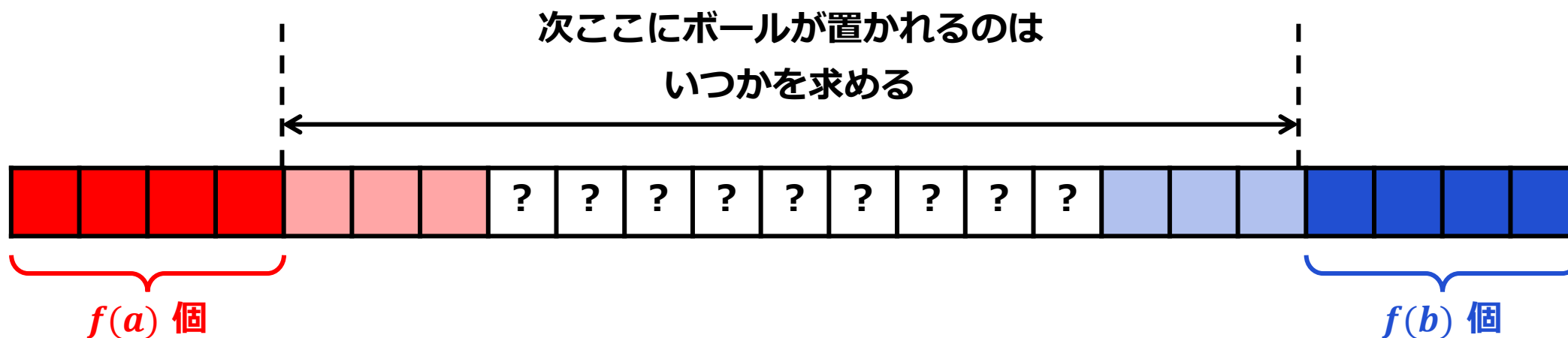


しかし、「どちらでもない操作」は全体を通して $O(Q \log N)$ 回
 $\log$ が 3 つとなり、間に合うかは微妙...

計算量を改善するには？



元々の方法では、 $a + 1 \leq A_i \leq N - b$ を満たす最小の i を
「次のどちらでもない操作」にしていた



しかし、 n 以下で最大の 2^k 数を $f(n)$ とするとき

$f(a) + 1 \leq A_i \leq N - f(b)$ を満たす i を

「次のどちらでもない操作」にすると上手くいく

なぜ上手くいくか？

- $f(a), f(b)$ の組は高々 $\log^2 N$ 個しかない
- したがって、すべての $f(a), f(b), x$ に対して「 $f(a) + 1 \leq A_i \leq N - f(b)$ を満たす最小の $i (\geq x)$ 」を前計算できる※



計算量 $O(1)$ で「次のどちらでもない操作」がわかる

さらに、範囲を $f(a) + 1$ 以上 $f(b)$ 以下に緩和させても
どちらでもない操作が起こる回数が $O(\log N)$ 回であることに変わりはない

なぜか？

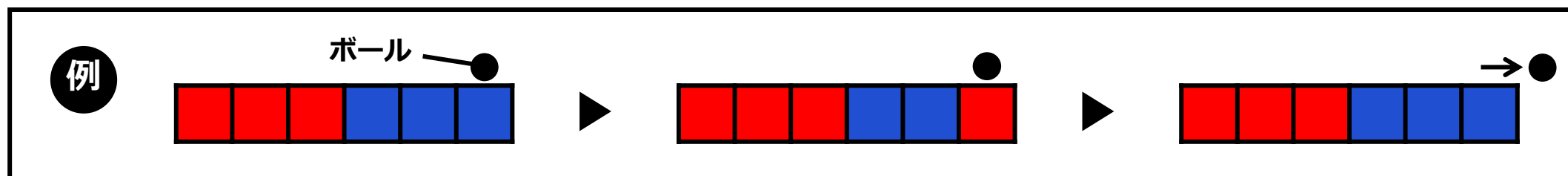
一度「どうしようもない操作」が起こると

$f(a), f(b)$ のうち少なくとも一方が 2 倍以上になるから

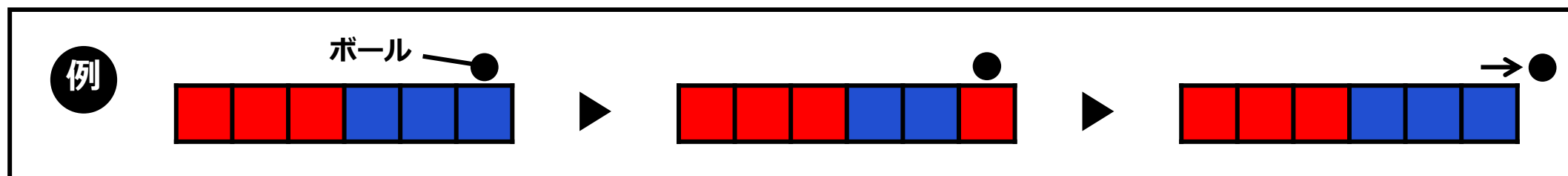
- このように 2 べきごとに累積 max を記録する方針を使うと、各クエリ $O(\log N (\log N + \log M))$ で答えを出すことができる
- $N, M, Q \leq 120000$ なので、実行時間制限の 2.5 秒には間に合うはず

実装上の注意点

ボールがタイル N に置かれた場合、盤面が変わらないことがある



ボールがタイル N に置かれた場合、盤面が変わらないことがある

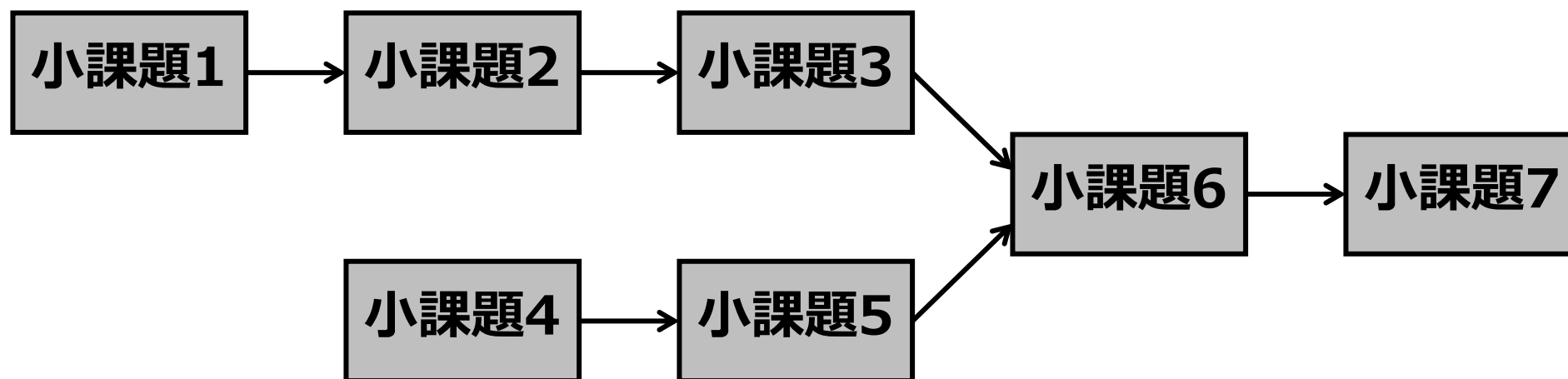


➡ 上手い実装をしなければ、WA や TLE になる！

たとえば、最初に「左の赤が $a = 0$ 個」「右の青が $b = 0$ 個」と設定した場合…
ボールが毎回タイル N に置かれたとき、全部「どちらでもない操作」になってしまう

解法のまとめ

小課題1	愚直にシミュレーション
小課題2	位置 x に置いて左から出た場合、左から x 個の青が赤になることに気づく
小課題3	一度 RRRRBBBB の形になったら $\text{mod } (N + 1)$ で計算できることに気づく
小課題4	圧縮無しのセグメント木
小課題5	圧縮ありのセグメント木
小課題6	累積和 + 二分探索で「左の赤と右の青」がいつぶつかるのかを求める この制約だと、どうしてもない操作※は高々 40 回以内
小課題7	どちらでもない操作が $O(\log N)$ 回以内であることに気づく + 領域木の代わりに「2べきのデータ構造」を使うと $\log$ が 2 つになる



本問題は満点までのステップ数が多いが、誘導も多い

JOI では、小課題の誘導に乗ることが重要になる場合もある

得点分布

191 / 191

